

Manual do Usuário do Painel 3X-UI

العربية · English · Español · فارسی · Bahasa Indonesia · 日本語 · Português · Русский ·
Türkçe · Українська · Tiếng Việt · 简体中文 · 繁體中文

Versão do 3X-UI: 3.5.0. O manual foi elaborado com base nesta versão e é válido para ela. Um resumo das alterações da versão 3.5.0 em relação à 3.4.2 encontra-se na seção «[O que há de novo na 3.5.0](#)».

Manual detalhado em português do painel web **3X-UI** (gerenciamento do Xray-core): funções, configuração e operação, com explicação de cada campo e chave na interface.

Os nomes e rótulos correspondem à interface do painel. As palavras *inbound* / *outbound* não são traduzidas.

Sumário

- O que há de novo na 3.5.0
- 1. Introdução, requisitos e instalação
 - 1.1. O que é o 3X-UI
 - 1.2. Sistemas operacionais e arquiteturas suportados
 - 1.3. Métodos de instalação
 - 1.4. Primeiro acesso e credenciais padrão
 - 1.5. Localização dos arquivos
 - 1.6. Comando de gerenciamento `x-ui` (menu do script)
 - 1.7. Subcomandos do `x-ui` (sem menu interativo)
 - 1.8. Migração de SQLite para PostgreSQL
- 2. Acesso ao painel e segurança
 - 2.1. Formulário de login
 - 2.2. Autenticação de dois fatores (2FA / TOTP)
 - 2.3. Limitador de tentativas de login (proteção contra força bruta)
 - 2.4. Alteração de login e senha do administrador
 - 2.5. Caminho secreto (URI-путь / webBasePath) e porta do painel
 - 2.6. Tempo de vida da sessão (timeout)
 - 2.7. LDAP (sincronização e autenticação)
- 3. Visão Geral / Dashboard
 - 3.1. Princípios gerais de coleta de dados
 - 3.2. CPU
 - 3.3. Memória (RAM)
 - 3.4. Swap
 - 3.5. Disco (Storage)
 - 3.6. Tempo de atividade do sistema (Uptime)
 - 3.7. Carga do sistema (Load average)
 - 3.8. Rede: velocidade e volume total de tráfego
 - 3.9. Endereços IP do servidor
 - 3.10. Conexões TCP/UDP
 - 3.11. Status do Xray e controle do processo
 - 3.12. Atualização do painel (3X-UI)
 - 3.13. Atualização dos arquivos de geolocalização (GeoIP / GeoSite)
 - 3.14. Backup e restauração do banco de dados
 - 3.15. Elementos adicionais da interface
- 4. Inbounds: criação e parâmetros gerais
 - 4.1. Campos gerais do formulário
 - 4.2. Sniffing (Sniffing)
 - 4.3. Allocate (estratégia de alocação de portas)
 - 4.4. External Proxy (Proxy externo)
 - 4.5. Fallbacks (Fallbacks)
 - 4.6. Redefinição periódica de tráfego
 - 4.7. JSON da entrada (avançado)
 - 4.8. Ações com inbound: QR / Edit / Reset / Delete e estatísticas

- 5. Protocolos
 - 5.1. Lista de protocolos suportados
 - 5.2. Quais protocolos suportam TLS / REALITY / transporte
 - 5.3. VLESS
 - 5.4. VMess
 - 5.5. Trojan
 - 5.6. Shadowsocks
 - 5.7. Dokodemo-door / Tunnel (encaminhador transparente)
 - 5.8. SOCKS / HTTP (protocolo `mixed`)
 - 5.9. WireGuard (inbound)
 - 5.10. Hysteria (padrão v2)
 - 5.11. MTPROTO (proxy para Telegram)
 - 5.12. Guia rápido para escolha de protocolo
- 6. Transporte (Stream Settings)
 - 6.1. Escolha da rede de transmissão
 - 6.2. RAW / TCP (`tcpSettings`)
 - 6.3. mKCP (`kcpSettings`)
 - 6.4. WebSocket (`wsSettings`)
 - 6.5. gRPC (`grpcSettings`)
 - 6.6. HTTPUpgrade (`httpupgradeSettings`)
 - 6.7. XHTTP / SplitHTTP (`xhttpSettings`)
 - 6.8. Transporte Hysteria (`hysteriaSettings`)
 - 6.9. Parâmetros complementares
- 7. Segurança da conexão: TLS, XTLS e REALITY
 - 7.1. Qual é a diferença: TLS vs XTLS vs REALITY
 - 7.2. Modo «Nenhum» (`none`)
 - 7.3. Modo TLS
 - 7.4. Modo REALITY
 - 7.5. Recomendações práticas de configuração
- 8. Clientes
 - 8.1. Campos do cliente
 - 8.2. Vinculação ao inbound
 - 8.3. Operações sobre o cliente
 - 8.4. Operações em massa
 - 8.5. Pesquisa, filtros e ordenação
 - 8.6. Ícones e status
- 9. Grupos de clientes
 - 9.1. O que é um grupo de clientes e para que serve
 - 9.2. Relação do grupo com clientes, inbound, nós e protocolos
 - 9.3. Cadastro de grupos e grupos "vazios"
 - 9.4. Campos e colunas do grupo
 - 9.5. Criação de grupo
 - 9.6. Renomear grupo
 - 9.7. Adição de clientes ao grupo
 - 9.8. Remoção de clientes do grupo (sem excluir os clientes em si)
 - 9.9. Zerar tráfego do grupo

- 9.10. Exclusão do grupo e exclusão dos clientes do grupo
- 9.11. Relação com a página "Clientes"
- 9.12. Resumo dos endpoints de API
- 9.13. Tráfego por grupo
- 10. Assinaturas (Subscription)
 - 10.1. O que é subld e como o link é formado
 - 10.2. Configurações do servidor de assinaturas
 - 10.3. Formatos de saída
 - 10.4. Página de informações da assinatura e QR-codes
 - 10.5. Modelos personalizados da página de assinatura
- 11. Xray: roteamento, outbounds, DNS e extensões
 - 11.1. Estrutura do editor: abas/modos
 - 11.2. Configurações gerais (General)
 - 11.3. Regras de roteamento (routing)
 - 11.4. Outbounds (conexões de saída)
 - 11.5. Balanceadores (Balancers)
 - 11.6. DNS
 - 11.7. Fake DNS
 - 11.8. WireGuard / WARP / NordVPN
 - 11.9. Reverse-proxy e TUN
 - 11.10. Logs e estatísticas (Stats, metrics)
 - 11.11. Salvamento, reinicialização e transformações automáticas
 - 11.12. Outbound de assinatura (com atualização automática)
 - 11.13. Rotação de IP no WARP
- 12. Nós (multipainel, master/slave)
 - 12.1. Resumo no topo da lista
 - 12.2. Adicionando e editando um nó
 - 12.3. Verificação TLS (para nós https)
 - 12.4. O que é exibido para cada nó
 - 12.5. Ações sobre um nó
 - 12.6. Histórico de métricas
 - 12.7. Como os inbounds e clientes são sincronizados
 - 12.8. Cadeias de nós (subnós / nós transitivos)
 - 12.9. Correções da 3.5.0 (estabilidade dos nós)
 - 12.10. Nós: novidades na versão 3.3.0
- 13. Configurações do Painel
 - 13.1. Salvar e reiniciar o painel
 - 13.2. Configurações gerais (aba "Painel" / *General*)
 - 13.3. Acesso ao painel: IP, porta, caminho, domínio, certificado
 - 13.4. Sessão, proxy do painel e proxies confiáveis (aba "Proxy e servidor" / *Proxy and Server*)
 - 13.5. Bot do Telegram (aba "Bot do Telegram" / *Telegram Bot*)
 - 13.6. Data e hora (aba "Data e hora" / *Date and Time*)
 - 13.7. Tráfego externo e comportamento do Xray (aba "Tráfego externo" / *External Traffic*)
 - 13.8. Outros: template de configuração do Xray e URL de verificação
 - 13.9. Conta do administrador e tokens de API
 - 13.10. Alterações de API na versão 3.3.0 (importante para integrações)

- 14. Bot do Telegram
 - 14.1. Ativação e configuração do bot
 - 14.2. Menu principal e botões
 - 14.3. Comandos do bot
 - 14.4. Gerenciamento de clientes (somente administrador)
 - 14.5. Notificações e relatórios
 - 14.6. Backup e logs
 - 14.7. Particularidades de funcionamento
- 15. Bases geográficas (geoip / geosite e personalizadas)
 - 15.1. O que são geoip.dat e geosite.dat
 - 15.2. Arquivos geo padrão e sua atualização
 - 15.3. Atualização automática de geo-dados pelo Xray (Geodata Auto-Update)
 - 15.4. Validação e restrições
 - 15.5. Verificação automática na inicialização do painel
 - 15.6. Uso das bases geográficas nas regras de roteamento
- 16. Operação: backups, logs, atualização, CLI
 - 16.1. Backup e restauração do banco de dados
 - 16.2. Visualização de logs
 - 16.3. Nível e configuração de logging do Xray
 - 16.4. Gerenciamento do Xray: parada e reinicialização
 - 16.5. Reinicialização e atualização do painel
 - 16.6. Tarefas periódicas (cron)
 - 16.7. Menu de console e CLI (`x-ui`)
 - 16.8. Remoção do painel
 - 16.9. Comando `x-ui migrateDB`

O que há de novo na 3.5.0

A versão 3.5.0 é um grande lançamento: o MTProto foi migrado para o modelo multicliente (motor mtg-multi, segredos pessoais, cotas e ad-tag), os hosts gerenciados tornaram-se grupais (vários inbound e endereços em um único registro), a restauração no painel PostgreSQL aceita backups SQLite, os outbound ganharam «Target Strategy», o teste «Real delay» e as colunas Egress/Country, e um balanceador pode usar outro balanceador como fallback. O núcleo Xray 26.7.11 vem incluído. A seguir, as alterações em relação à 3.4.2 pelas seções do manual.

Alterações na seção 1 – Introdução, requisitos e instalação

- O núcleo Xray foi atualizado para **26.7.11**. Migrações automáticas associadas: as cifras Shadowsocks `none / plain` e VMess `none / zero` foram removidas do núcleo (as configurações salvas são reescritas automaticamente), e um outbound VLESS/Trojan sem criptografia para um endereço público é rejeitado ao salvar.
- Novo comando `x-ui pgclient [versão]` e item **10. Install/Upgrade client tools (pg_dump/pg_restore)** no menu PostgreSQL – instalação/atualização das ferramentas de cliente do PostgreSQL.
- Correções de scripts: instalação do PostgreSQL e do fail2ban na família RHEL (EPEL), Arch sem o `pacman -Syu` completo, nome correto do binário do Xray no ARM de 32 bits (`xray-linux-arm32`), confirmação do IPv4 autodetectado antes da emissão do certificado de IP, e corrigido o falso «Your input is invalid» ao escolher a porta ACME padrão.

Alterações na seção 2 – Acesso ao painel e segurança

- Limite de IP: uma conexão «morta» agora é banida **uma única vez**, e não a cada varredura de 10 segundos – os contadores do fail2ban não são mais inflados, e não é preciso aumentar o `maxretry`.

Alterações na seção 4 – Inbounds: criação e parâmetros gerais

- A lista de inbound ganhou uma **pesquisa** (por observação, porta e protocolo), e as listas suspensas de nós («Implantar em», filtro «Nós») tornaram-se pesquisáveis.

Alterações na seção 5 – Protocolos

- **O MTProto foi migrado para o modelo multicliente** (motor mtg-multi): os usuários do MTProto agora são clientes comuns, com seu próprio segredo, cota, prazo, ad-tag e link pessoal `tg://proxy`. O campo «Secret» no nível do inbound foi removido (os inbound existentes são convertidos automaticamente), e o «FakeTLS domain» tornou-se o domínio padrão para os novos segredos. Novos campos do inbound: **Max connections** (limitação de conexões), **Public IPv4/IPv6** (para o ad-tag middle proxy). As alterações de clientes são aplicadas «a quente», sem derrubar as sessões do Telegram dos demais.
- WireGuard: o menu do inbound ganhou o conjunto completo de ações de clientes (Export All URLs, vincular/desvincular, grupos), a exportação foi dividida nas abas **Config** e **Links**, o campo «**IPs permitidos** **WireGuard**» tornou-se editável (várias entradas separadas por vírgula), e no config do cliente de um inbound de nó o `Endpoint` agora aponta para o endereço do nó.

Alterações na seção 7 – Segurança da conexão: TLS, XTLS e REALITY

- A combinação **Finalmask + REALITY** é **rejeitada** ao salvar (ela levava à queda do Xray-core na primeira conexão); o placeholder do minClientVer foi atualizado para 26.3.27.
- Novo tipo de máscara TCP do Finalmask – **XMC (Minecraft)**: mascaramento do fluxo como tráfego de Minecraft (Hostname, Usernames, Password obrigatório com geração automática).

Alterações na seção 8 – Clientes

- Nova coluna «**Velocidade**» – velocidade ao vivo de cada cliente (↑/↓, média móvel de ~5 segundos).
- A pesquisa de clientes volta a encontrar por **Telegram ID**; no formulário do cliente, os inbound desativados ficam ocultos da lista de vinculação; corrigido o acúmulo de duplicatas na janela «Desvincular».
- Os clientes MTPROTO têm campos próprios: «**MTPROTO secret**» (com regeneração) e «**Ad-tag (canal patrocinador)**» (32 caracteres hex); a cota e o prazo agora são realmente aplicados ao MTPROTO.

Alterações na seção 9 – Grupos de clientes

- A janela de informações do cliente agora mostra o seu **grupo**.

Alterações na seção 10 – Assinaturas (Subscription)

- **Os hosts gerenciados tornaram-se grupais**: um único registro abrange **vários inbound** (seleção múltipla) e **vários endereços** (tags, cada entrada pode ter seu próprio :porta ; autocompletar de endereços; vazio – herda o endereço do inbound). As colunas da lista mostram chips de endereços e de inbound (com «+N»), as ações e a API trabalham com grupos (groupId), e surgiu o endpoint em massa POST /panel/api/hosts/bulk/add . A ordenação dos hosts agora é global (pela ordem, depois pela observação).
- O texto do **aviso** (subAnnounce) agora é exibido como banner na página de informações da assinatura; nos modelos personalizados está disponível a variável announce .
- A página de informações no navegador agora abre também pelos **links JSON/Clash** (e não apenas pelo principal).
- As configurações de host **Final Mask** e **Allow insecure** agora atuam também nos links raw (fm=) e para o **Hysteria2** (insecure=1 / skip-cert-verify: true), respectivamente.
- O intervalo de «Intervalos de atualização» (subUpdates) foi corrigido para **0–525600** (o antigo limite de 168 da interface bloqueava o salvamento das configurações após o upgrade da 2.x).
- Os **clientes WireGuard nativos agora entram nas assinaturas Clash e JSON** (antes – apenas na raw).

Alterações na seção 11 – Xray: roteamento, outbounds, DNS e extensões

- Editor de outbound: novo campo «**Target Strategy**» (11 valores de AsIs a ForceIPv4), modo de teste «**Real delay**» (tempo completo com estabelecimento do túnel; o modo HTTP agora é medido em uma conexão «aquecida»), colunas **Egress** (IP de saída atrás do «olho») e **Country** (bandeira + país, rótulo WARP) após o teste HTTP/Real.
- **O fallback do balanceador pode apontar para outro balanceador**: o painel constrói por conta própria um objeto loopback oculto (_bl_...), protege contra ciclos e contra a exclusão de um balanceador em uso; o prefixo _bl_ é reservado.

- A aba «Roteamento básico» ganhou o seletor **«Default Outbound»** – qual outbound processa o tráfego que não corresponde a nenhuma regra (o selecionado é movido para a primeira posição).
- Servidores DNS em IPs privados não são mais bloqueados pela regra `geoip:private` – o painel mantém por conta própria uma regra allow gerenciada.
- O Happy Eyeballs nas configurações de dial (sockopt) agora é realmente ativado; o «Try delay» padrão é 250 ms, e um 0 explícito é preservado.
- Importação de assinaturas de outbound: nos links `ss://` com `?plugin=//` final, a porta é analisada corretamente.

Alterações na seção 12 – Nós (multipainel, master/slave)

- Pacote de correções: salvar um cliente sem alterações não derruba mais o tráfego ao vivo dos inbound de nó; as sobrescritas de Host do nó são aceitas no master na primeira aceitação; a renovação automática abre uma janela de cota nova; a exclusão de um cliente no master o exclui completamente nos nós; os inbound do nó não são varridos antes da primeira aceitação; um único inbound incorreto não interrompe a sincronização de tráfego do nó; a verificação de conflito de portas é limitada ao próprio nó.

Alterações na seção 14 – Bot do Telegram

- Ao menu de comandos do bot foram adicionados `/usage`, `/inbound`, `/restart` e o novo comando de administrador `/clearall` (reset de tráfego de todos os clientes, com confirmação).
- A lista de clientes online é rotulada como `email – observação do inbound`; as mensagens de backup e do log de banimentos contêm o nome do host; a busca por Telegram ID funciona independentemente da formatação das configurações.

Alterações na seção 16 – Operação: backups, logs, atualização, CLI

- **A restauração no painel PostgreSQL aceita arquivos SQLite:** um backup comum `.db` ou um `.dump` de migração é importado diretamente no PostgreSQL (em uma única transação, com verificações antes de parar o Xray). O diálogo de seleção de arquivo aceita `.dump`, `.db` em ambos os SGBDs; «Baixar arquivo de migração» permaneceu apenas nos painéis PostgreSQL.
- Antes de restaurar um arquivo `pg_dump`, o painel verifica a legibilidade do dump e, em caso de divergência de versões, sugere o comando exato `x-ui pgclient <versão>`.
- Autocorreções na inicialização: contadores de tráfego transbordados são limitados e restaurados; a restrição UNIQUE obsoleta na porta do inbound é removida (atrapalhava o multi-node).
- Logs do Xray: um novo job a cada 10 minutos trunca o access-log e o error-log quando excedem **64 MiB**; a limpeza diária agora limpa ambos.
- Docker: a renovação automática de certificados foi consertada (o crond é iniciado e o estado do acme.sh é preservado em um volume).

1. Introdução, requisitos e instalação

1.1. O que é o 3X-UI

3X-UI é um painel de controle web de código aberto para servidores [Xray-core](#). O painel oferece uma interface web unificada e multilíngue para implantação, configuração e monitoramento de uma ampla variedade de protocolos de proxy e VPN: desde um único VPS até configurações distribuídas com múltiplos nós.

O 3X-UI é um fork avançado do projeto original X-UI. Em relação a ele, foram adicionados suporte a um número maior de protocolos, maior estabilidade, contabilização de tráfego por cliente e diversas funcionalidades convenientes.

Principais funcionalidades:

- **Inbound com diferentes protocolos** – VLESS, VMess, Trojan, Shadowsocks, WireGuard, Hysteria2, HTTP, SOCKS (Mixed), Dokodemo-door / Tunnel, TUN e **MTProto** (proxy do Telegram, adicionado na versão 3.3.0).
- **Transportes modernos e criptografia** – TCP (Raw), mKCP, WebSocket, gRPC, HTTPUpgrade e XHTTP, protegidos por TLS, XTLS e REALITY.
- **Fallback** – atendimento de múltiplos protocolos na mesma porta (por exemplo, VLESS e Trojan na 443) por meio de fallback no Xray.
- **Gerenciamento por cliente** – cotas de tráfego, datas de expiração, limites de IP, exibição do status "online", links de convite com um clique, QR codes e assinaturas.
- **Estatísticas de tráfego** – por inbound, cliente e outbound, com possibilidade de reset.
- **Suporte a múltiplos nós** – gerenciamento e escalabilidade para vários servidores a partir de um único painel.
- **Outbound e roteamento** – WARP, NordVPN, regras de roteamento personalizadas, balanceadores de carga, encadeamento de proxies.
- **Servidor de assinaturas integrado** com múltiplos formatos de saída.
- **Bot do Telegram** para monitoramento e gerenciamento remotos.
- **REST API** com documentação Swagger integrada.
- **Armazenamento flexível** – SQLite (padrão) ou PostgreSQL.
- **13 idiomas de interface**, temas escuro e claro.
- **Integração com Fail2ban** para aplicação de limites de IP por cliente.

Importante: o projeto destina-se apenas a uso pessoal. Não é recomendado utilizá-lo para fins ilegais ou em ambientes de produção.

1.2. Sistemas operacionais e arquiteturas suportados

Sistemas operacionais

O script de instalação detecta a distribuição pelo campo `ID` do arquivo `/etc/os-release` (ou `/usr/lib/os-release`). Os sistemas oficialmente suportados são:

Ubuntu, Debian, Armbian, Fedora, CentOS, RHEL, AlmaLinux, Rocky Linux, Oracle Linux, Amazon Linux, Virtuozzo, Arch, Manjaro, Parch, openSUSE (Tumbleweed / Leap), Alpine e Windows.

Em sistemas da família Alpine, é utilizado o serviço OpenRC (`rc-service` / `rc-update`); nos demais, o `systemd`. No CentOS 7, os pacotes são instalados via `yum` ; em versões mais recentes, via `dnf` . Se a distribuição não for reconhecida, o script tenta usar por padrão o gerenciador de pacotes `apt-get` .

Arquiteturas de processador

A arquitetura é determinada pela saída de `uname -m` e mapeada para um dos valores suportados:

Valor de <code>uname -m</code>	Arquitetura do 3X-UI
<code>x86_64</code> , <code>x64</code> , <code>amd64</code>	<code>amd64</code>
<code>i*86</code> , <code>x86</code>	<code>386</code>
<code>armv8*</code> , <code>arm64</code> , <code>aarch64</code>	<code>arm64</code>
<code>armv7*</code> , <code>arm</code>	<code>armv7</code>
<code>armv6*</code>	<code>armv6</code>
<code>armv5*</code>	<code>armv5</code>
<code>s390x</code>	<code>s390x</code>

Se a arquitetura não constar nesta lista, o script exibe a mensagem "Unsupported CPU architecture!" e encerra a instalação.

Dependências básicas

Antes de instalar o painel, o script instala automaticamente um conjunto básico de pacotes (os nomes variam por distribuição): `cron` / `cronie` / `dcron` , `curl` , `tar` , `tzdata` / `timezone` , `socat` , `ca-certificates` , `openssl` .

1.3. Métodos de instalação

Método 1. Script de instalação (recomendado)

A instalação é realizada com um único comando como root:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh)
```

O script exige obrigatoriamente privilégios de root: se executado sem ser root, exibe "Please run this script with root privilege" e encerra com erro.

O que o instalador faz passo a passo:

1. Detecta o sistema operacional e a arquitetura.
2. Instala as dependências básicas.

3. Faz o download do arquivo de release `x-ui-linux-<arch>.tar.gz` e o extrai no diretório `/usr/local/x-ui`.
4. Faz o download do script de gerenciamento `x-ui.sh` e o instala como o comando `/usr/bin/x-ui`.
5. Cria o diretório de logs `/var/log/x-ui`.
6. Executa a configuração inicial: escolha do banco de dados, geração de credenciais, escolha da porta, configuração opcional de SSL.
7. Instala e inicia o serviço de inicialização automática (unit systemd `x-ui.service` ou script init OpenRC para Alpine).

Escolha do banco de dados durante a instalação. O instalador oferece:

- 1) SQLite (padrão, recomendado para menos de 500 clientes) – um único arquivo `/etc/x-ui/x-ui.db`, sem necessidade de configuração.
- 2) PostgreSQL (recomendado para grande número de clientes ou múltiplos nós). O PostgreSQL pode ser instalado localmente (cria um usuário e banco de dados dedicados com o nome `xui`) ou é possível informar um DSN para um servidor já existente. Os parâmetros de conexão são gravados no arquivo de ambiente do serviço (`/etc/default/x-ui`, `/etc/conf.d/x-ui` ou `/etc/sysconfig/x-ui` dependendo da distribuição) como variáveis `XUI_DB_TYPE` e `XUI_DB_DSN`.

Exemplo: registro dos parâmetros do PostgreSQL no arquivo de ambiente do serviço. Após escolher PostgreSQL e informar o DSN, o instalador adicionará ao arquivo de ambiente linhas semelhantes a estas:

```
XUI_DB_TYPE=postgres
XUI_DB_DSN=postgres://xui:S3cretPass@127.0.0.1:5432/xui?sslmode=disable
```

Aqui `xui` é o nome do usuário e do banco, `127.0.0.1:5432` é o endereço e a porta do servidor, e `sslmode=disable` é adequado para conexões locais (para servidor remoto, geralmente se usa `require`).

Instalação de uma versão específica (antiga). É possível especificar explicitamente uma tag de versão – o instalador fará o download do release correspondente:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/v2.4.0/install.sh)
v2.4.0
```

A versão mínima permitida para esse tipo de instalação é `v2.3.5`; ao especificar uma versão mais antiga, é exibida a mensagem "Please use a newer version (at least v2.3.5)".

Instalação da build de desenvolvimento. Além da tag de versão, o instalador aceita o argumento `dev-latest` (alias `dev`) – isso instala a build rolling de desenvolvimento com base no último commit da branch `main`:

```
bash <(curl -Ls https://raw.githubusercontent.com/mhsanaei/3x-ui/master/install.sh) dev-
latest
```

A build de desenvolvimento é um pré-release por commit (tag `dev-latest`), não uma versão estável, portanto a verificação de versão mínima não é executada para ela. Ao iniciar, é exibido o aviso "Installing the rolling dev build (tag: dev-latest). This is a per-commit pre-release, not a stable version.". Sem argumento, o instalador instala o último release estável. Faz sentido usar a build de desenvolvimento apenas para verificar correções ainda não publicadas; em uso normal, instale versões estáveis.

Método 2. Docker

Execução com banco de dados SQLite padrão:

```
docker compose up -d
```

Para execução com o serviço PostgreSQL integrado, é necessário descomentar as linhas `XUI_DB_*` no `docker-compose.yml` e iniciar com o perfil:

```
docker compose --profile postgres up -d
```

A imagem inclui o Fail2ban (ativo por padrão) para aplicação de limites de IP por cliente. O Fail2ban bloqueia infratores via `iptables`, o que requer a capability `NET_ADMIN`. No `docker-compose.yml` ela já está concedida via `cap_add`. A partir da 3.5.0, foi consertada no contêiner a **renovação automática de certificados** emitidos pelo menu SSL: o entrypoint inicia o `crond` e registra novamente a tarefa cron do `acme.sh`, e o `docker-compose.yml` ganhou um volume separado para o estado do `acme.sh` — a renovação sobrevive à recriação do contêiner (antes, os certificados expiravam silenciosamente após ~90 dias). Ao iniciar manualmente com `docker run`, as capabilities precisam ser adicionadas manualmente; caso contrário, os bloqueios serão apenas registrados em log, mas não aplicados:

Exemplo: comando completo `docker run`. Variante mínima com mapeamento da porta do painel, capabilities de rede e volume persistente para o banco de dados:

```
docker run -d \
  --name 3x-ui \
  --restart unless-stopped \
  --cap-add=NET_ADMIN --cap-add=NET_RAW \
  -v $PWD/db:/etc/x-ui \
  -v $PWD/cert:/root/cert \
  -p 2053:2053 \
  ghcr.io/mhsanaei/3x-ui:latest
```

O volume `/etc/x-ui` preserva o arquivo `x-ui.db` entre reinicializações do container; sem ele, as configurações e contas serão perdidas.

```
docker run -d --cap-add=NET_ADMIN --cap-add=NET_RAW ... ghcr.io/mhsanaei/3x-ui
```

No Docker, o painel é o processo principal do container: a inicialização automática é controlada pela política de reinicialização do container (por exemplo, `restart: unless-stopped`), e não por um serviço dentro do container.

1.4. Primeiro acesso e credenciais padrão

Na primeira instalação (quando ainda são usadas as credenciais padrão), o instalador **gera valores aleatórios** para nome de usuário, senha, caminho web e porta:

Parâmetro	Como é gerado na instalação	Observação
Nome de usuário (Username)	string aleatória de 10 caracteres	gerado automaticamente
Senha (Password)	string aleatória de 10 caracteres	gerada automaticamente
Caminho web do painel (WebBasePath)	string aleatória de 18 caracteres	protege o painel contra descoberta pela URL raiz
Porta do painel (Port)	por padrão, porta aleatória no intervalo 1024–62000; pode ser definida manualmente	o valor "de fábrica" de <code>webPort</code> é <code>2053</code> , mas o instalador o substitui

Ao final da instalação, o script exibe um resumo: nome de usuário, senha, porta, caminho web, token de API e o link de acesso (Access URL) no formato:

```
https://<domínio-ou-IP>:<porta>/<caminho-web>
```

Se o certificado SSL não estiver configurado, o link será via `http://` e o script exibirá um aviso sobre a necessidade de configurar o SSL (item de menu 19).

Troca obrigatória de credenciais. Como o login e a senha são gerados aleatoriamente, eles devem ser **salvos imediatamente após a instalação**. É possível alterá-los a qualquer momento pelo item de menu "Reset Username & Password" (veja abaixo) ou pela interface web nas configurações do painel. Após o reset, o script lembra: "Please use the new login username and password to access the X-UI panel. Also remember them!".

Após a instalação, use o comando `x-ui` para abrir o menu de gerenciamento (veja a seção 1.6).

1.5. Localização dos arquivos

Caminho	Finalidade
<code>/usr/local/x-ui/</code>	diretório de instalação do painel (binário <code>x-ui</code> , script <code>x-ui.sh</code>)
<code>/usr/local/x-ui/bin/xray-linux-<arch></code>	binário do Xray-core (em armv5/armv6/armv7 é renomeado para <code>xray-linux-arm</code>)
<code>/usr/bin/x-ui</code>	script de gerenciamento (comando <code>x-ui</code>)
<code>/etc/x-ui/x-ui.db</code>	arquivo do banco de dados SQLite (padrão)
<code>/var/log/x-ui/</code>	diretório de logs do painel
<code>/etc/systemd/system/x-ui.service</code>	unit systemd do serviço (não utilizado no Alpine)
<code>/etc/init.d/x-ui</code>	script init do OpenRC (somente Alpine)
<code>/etc/default/x-ui</code> · <code>/etc/conf.d/x-ui</code> · <code>/etc/sysconfig/x-ui</code>	arquivo de variáveis de ambiente do serviço (o caminho depende da distribuição); aqui são gravadas <code>XUI_DB_TYPE</code> / <code>XUI_DB_DSN</code>

O diretório do banco de dados pode ser substituído pela variável de ambiente `XUI_DB_FOLDER` (padrão `/etc/x-ui`), e o diretório dos binários do Xray pela variável `XUI_BIN_FOLDER` (padrão `bin` relativo ao diretório do painel). O nome do arquivo do banco de dados é `x-ui.db`.

Exemplo: mover o banco para um disco separado. Para armazenar `x-ui.db` não em `/etc/x-ui`, mas em um disco montado em `/data`, por exemplo, defina a variável no arquivo de ambiente do serviço e reinicie o painel:

```
echo 'XUI_DB_FOLDER=/data/x-ui' >> /etc/default/x-ui
mkdir -p /data/x-ui
systemctl restart x-ui
```

O caminho completo para o banco será `/data/x-ui/x-ui.db`.

Principais variáveis de ambiente

Variável	Finalidade	Padrão
<code>XUI_DB_TYPE</code>	backend do banco de dados: <code>sqlite</code> ou <code>postgres</code>	<code>sqlite</code>
<code>XUI_DB_DSN</code>	string de conexão PostgreSQL (quando <code>XUI_DB_TYPE=postgres</code>)	–
<code>XUI_DB_FOLDER</code>	diretório do arquivo do banco SQLite	<code>/etc/x-ui</code>
<code>XUI_INIT_WEB_BASE_PATH</code>	URI inicial do painel web (somente na primeira inicialização)	<code>/</code>
<code>XUI_DB_MAX_OPEN_CONNS</code>	número máximo de conexões abertas (pool PostgreSQL)	–
<code>XUI_DB_MAX_IDLE_CONNS</code>	número máximo de conexões ociosas (pool PostgreSQL)	–
<code>XUI_ENABLE_FAIL2BAN</code>	habilitar aplicação de limites de IP via Fail2ban	<code>true</code>
<code>XUI_LOG_LEVEL</code>	nível de log (<code>debug</code> , <code>info</code> , <code>warning</code> , <code>error</code>)	<code>info</code>
<code>XUI_DEBUG</code>	modo de depuração	<code>false</code>

Exemplo: habilitar log detalhado temporariamente. Para diagnosticar um problema, eleve o nível de log para `debug` e reinicie o serviço:

```
echo 'XUI_LOG_LEVEL=debug' >> /etc/default/x-ui
systemctl restart x-ui
x-ui log # visualizar o log de depuração
```

Após o diagnóstico, restaure o valor `info` para evitar que o log cresça excessivamente.

Caminho inicial do painel web via variável de ambiente. A variável `XUI_INIT_WEB_BASE_PATH` define o URI do painel web (`webBasePath`) na inicialização inicial das configurações. Isso é útil ao implantar via Docker ou systemd para fixar imediatamente o caminho de acesso ao painel. O valor é normalizado automaticamente – as barras inicial e final são adicionadas quando necessário, e um valor vazio ou composto apenas de espaços é ignorado (aplicando-se então o caminho padrão `/`). A variável afeta **somente a inicialização inicial**: se as configurações já foram criadas, o caminho é alterado pela interface web ou pelo item de menu "Reset Web Base Path".

1.6. Comando de gerenciamento `x-ui` (menu do script)

Após a instalação, o comando `x-ui` (executado como root) abre o menu interativo "3X-UI Panel Management Script". A seleção de um item é feita digitando seu número (intervalo 0–27). Muitos itens também estão disponíveis como subcomandos para uso em scripts (veja a seção 1.7).

O menu está dividido em blocos temáticos.

Instalação e atualização

- **1. Install** — instala o painel (executa `install.sh`). Antes da instalação, verifica se o painel ainda não está instalado.
- **2. Update** — atualiza todos os componentes do `x-ui` para a versão mais recente. Os dados não são perdidos; após a atualização, o painel é reiniciado automaticamente. Requer confirmação.
- **3. Update Menu** — atualiza apenas o script de gerenciamento (`x-ui.sh` / comando `x-ui`) para a versão atual sem reinstalar o painel.
- **4. Legacy Version** — instala uma versão específica (antiga) do painel. O script solicita o número da versão (por exemplo, `2.4.0`) e faz o download do release correspondente.
- **5. Uninstall** — remoção completa do painel **junto com o Xray**. O serviço é parado e desabilitado, os diretórios `/etc/x-ui/` e `/usr/local/x-ui/`, o arquivo de ambiente do serviço e o próprio script de gerenciamento são removidos. Requer confirmação (padrão "não").

Credenciais e configurações

- **6. Reset Username & Password** — redefine o nome de usuário e a senha do painel. É possível inserir valores próprios ou deixar em branco para geração aleatória (nome aleatório com 10 caracteres, senha aleatória com 18 caracteres). Também oferece a opção de desativar a autenticação de dois fatores (2FA), se estiver configurada. Após o reset, o painel é reiniciado.
- **7. Reset Web Base Path** — redefine o caminho web do painel: um novo caminho aleatório (18 caracteres) é gerado e o painel é reiniciado. Usado quando o caminho anterior foi comprometido ou esquecido.
- **8. Reset Settings** — redefine todas as configurações do painel para os valores padrão. **As credenciais (nome de usuário e senha) e os dados das contas não são perdidos.** Requer confirmação; após o reset, o painel é reiniciado.
- **9. Change Port** — altera a porta do painel web. É solicitado o número da porta (1–65535); após a definição, é necessário reiniciar para que a porta entre em vigor.
- **10. View Current Settings** — exibe as configurações atuais (`x-ui setting -show`). Mostra também o backend de banco de dados em uso (SQLite ou PostgreSQL com a senha mascarada no DSN) e o link de acesso (Access URL). Se o SSL não estiver configurado, oferece a emissão de um certificado Let's Encrypt para o endereço IP.

Gerenciamento do serviço

- **11. Start** — inicia o serviço do painel. Se o painel já estiver em execução, exibe uma mensagem informando que não é necessário iniciar novamente.
- **12. Stop** — para o serviço do painel.
- **13. Restart** — reinicia o serviço do painel.

- **14. Restart Xray** – reinicia apenas o núcleo Xray-core sem reiniciar o painel (via `systemctl reload x-ui`; no Docker, via sinal `USR1` ao processo do painel).
- **15. Check Status** – verifica o estado do serviço (`systemctl status x-ui` ou `rc-service x-ui status`).
- **16. Logs Management** – gerenciamento de logs: visualização do log de depuração (Debug Log, via `journalctl`) e, exceto no Alpine, limpeza de todos os logs (Clear All logs).

Inicialização automática

- **17. Enable Autostart** – habilita a inicialização automática do painel ao carregar o SO (`systemctl enable x-ui` ou `rc-update add`).
- **18. Disable Autostart** – desabilita a inicialização automática ao carregar o SO.

No Docker, a inicialização automática é controlada pela política de reinicialização do container, portanto esses itens apenas exibem a dica correspondente.

Segurança e rede

- **19. SSL Certificate Management** – gerenciamento de certificados SSL via acme.sh: emissão de certificado para domínio, revogação, renovação forçada, visualização de domínios existentes, especificação de caminhos para o certificado do painel, além de emissão de certificado de curta duração (~6 dias, com renovação automática) para endereço IP.
- **20. Cloudflare SSL Certificate** – emissão de certificado SSL via validação DNS do Cloudflare.
- **21. IP Limit Management** – gerenciamento de limites de número de IPs por cliente (baseado no Fail2ban): visualização e remoção de bloqueios, entre outros.
- **22. Firewall Management** – gerenciamento do firewall (abertura/fechamento de portas e visualização de regras).
- **23. SSH Port Forwarding Management** – configuração de encaminhamento de portas SSH para acessar o painel a partir da máquina local por meio de um túnel SSH.

Desempenho e manutenção

- **24. Enable BBR** – habilita/desabilita o algoritmo de controle de congestionamento TCP BBR (submenu com os itens Enable BBR / Disable BBR).
- **25. Update Geo Files** – atualiza as bases geo (arquivos `.dat`) com seleção da fonte: Loyalsoldier (`geoip.dat` , `geosite.dat`), chocolate4u (`geoip_IR.dat` , `geosite_IR.dat`), runetfreedom (`geoip_RU.dat` , `geosite_RU.dat`) ou All (todos de uma vez). Após a atualização, o painel é reiniciado.
- **26. Speedtest by Ookla** – executa o teste de velocidade de rede via Speedtest by Ookla.
- **27. PostgreSQL Management** – gerenciamento da instância PostgreSQL integrada/associada (ativação e operações relacionadas).
- **0. Exit Script** – sai do menu.

1.7. Subcomandos do `x-ui` (sem menu interativo)

Para uso em scripts, o comando `x-ui` suporta subcomandos diretos (executar `x-ui` sem argumentos abre o menu):

Comando	Ação
<code>x-ui</code>	abrir o menu de gerenciamento
<code>x-ui start</code>	iniciar o painel
<code>x-ui stop</code>	parar o painel
<code>x-ui restart</code>	reiniciar o painel
<code>x-ui restart-xray</code>	reiniciar o Xray
<code>x-ui status</code>	estado atual do serviço
<code>x-ui settings</code>	configurações atuais
<code>x-ui enable</code>	habilitar inicialização automática ao carregar o SO
<code>x-ui disable</code>	desabilitar inicialização automática
<code>x-ui log</code>	visualizar logs
<code>x-ui banlog</code>	visualizar logs de bloqueios do Fail2ban
<code>x-ui update</code>	atualizar o painel
<code>x-ui update-all-geofiles</code>	atualizar todos os arquivos geo
<code>x-ui migrateDB [file]</code>	conversão <code>.db</code>  <code>.dump</code> (SQLite)
<code>x-ui legacy</code>	instalar versão antiga
<code>x-ui install</code>	instalar o painel
<code>x-ui uninstall</code>	remover o painel

1.8. Migração de SQLite para PostgreSQL

Uma instalação existente em SQLite pode ser migrada para PostgreSQL:

```
x-ui migrate-db --dsn "postgres://xui:password@127.0.0.1:5432/xui?sslmode=disable"
# em seguida, defina XUI_DB_TYPE e XUI_DB_DSN em /etc/default/x-ui e reinicie:
systemctl restart x-ui
```

O arquivo SQLite original permanece intacto – remova-o manualmente somente após verificar o funcionamento do novo backend.

Exemplo: verificação da troca para PostgreSQL. Após a migração, confirme que o painel está realmente funcionando no novo backend com o comando de visualização de configurações – a saída deve indicar PostgreSQL (a senha no DSN é mascarada):

```
x-ui settings | grep -i -E 'db|dsn'
```

Se o painel abrir e as contas estiverem presentes, o arquivo `x-ui.db` original pode ser removido.

2. Acesso ao painel e segurança

Esta seção descreve tudo o que diz respeito à autenticação do administrador do painel 3X-UI: o formulário de login, a autenticação de dois fatores (TOTP), a proteção contra tentativas de força bruta, a alteração de credenciais, a mudança do caminho secreto e da porta do painel, o tempo de vida da sessão, bem como a sincronização/authenticação via LDAP.

2.1. Formulário de login

A página de login é servida na raiz do caminho secreto do painel (`webBasePath`). Se o usuário já estiver autenticado, ele é redirecionado automaticamente para `.../panel/` . A página conta com um seletor de tema, seleção de idioma da interface e o próprio formulário.

Campos do formulário:

Campo	Dica/título	Obrigatório	Descrição
Имя пользователя	«Имя пользователя»	Sim	Login do administrador. Um valor vazio é rejeitado no lado do cliente e, no servidor, com a mensagem «Введите имя пользователя».
Пароль	«Пароль»	Sim	Senha do administrador. Um valor vazio é rejeitado com a mensagem «Введите пароль».
Код 2FA	«Код 2FA»	Somente quando a 2FA está habilitada	O campo aparece somente se a autenticação de dois fatores estiver habilitada no painel. Código de 6 dígitos gerado pelo aplicativo autenticador.

O botão «**Войти**» envia o formulário para `POST /login` .

Comportamento e mensagens:

- Em caso de login bem-sucedido, é exibida a mensagem «Вход выполнен успешно» e ocorre o redirecionamento para `.../panel/` .
- Em qualquer erro de credenciais ou código 2FA inválido, o servidor retorna uma **única** mensagem: «Неверные данные учетной записи.» (em inglês: *Invalid username or password or two-factor code.*). Isso é intencional – o painel não indica o que está errado (login, senha ou código), para não facilitar ataques de força bruta.
- O campo «Код 2FA» é exibido ou ocultado pelo painel com base na requisição `POST /getTwoFactorEnable` , que retorna o status atual da 2FA antes mesmo da autenticação.
- Se a sessão do servidor expirou, na próxima requisição é exibida a mensagem «Сессия истекла. Войдите в систему снова», e o usuário é redirecionado para a página de login.

Observação sobre CSRF: antes do envio do formulário, o cliente obtém um token CSRF (`GET /csrf-token`); as requisições `/login` e `/logout` são protegidas por verificação CSRF.

Exemplo: login via API. Quando a 2FA está desativada, basta o login e a senha; com a 2FA habilitada, adiciona-se o campo `twoFactorCode` :

```
# Sem 2FA
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль'

# Com 2FA habilitada – adiciona-se o código de 6 dígitos
curl -i -X POST https://panel.example.com:2053/мой-секрет/login \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'username=admin&password=ВашПароль&twoFactorCode=123456'
```

Em caso de sucesso, o servidor retornará `Set-Cookie` com o cookie de sessão – que deve ser enviado nas requisições subsequentes a `/panel/api/...`.

2.2. Autenticação de dois fatores (2FA / TOTP)

A 2FA no 3X-UI é implementada segundo o padrão **TOTP** e é compatível com qualquer aplicativo autenticador (Google Authenticator, Aegis, FreeOTP, entre outros). Os parâmetros são definidos de forma fixa: algoritmo **SHA1**, 6 dígitos, período de **30** segundos, emissor (issuer) `3x-ui`, rótulo `Administrator`.

Exemplo: otpauth-URI codificado no QR-code. Se o aplicativo autenticador não conseguir escanear pela câmera, o token pode ser adicionado manualmente por este link (substitua seu segredo Base32 no lugar de `JBSWY3DPEHPK3PXP`):

```
otpauth://totp/3x-ui:Administrator?secret=JBSWY3DPEHPK3PXP&issuer=3x-
ui&algorithm=SHA1&digits=6&period=30
```

Os parâmetros `algorithm=SHA1`, `digits=6`, `period=30` correspondem aos valores fixos do painel – não é necessário alterá-los.

As configurações ficam em **Настройки** → **Учетная запись**, aba «**Двухфакторная аутентификация**».

Elemento	Texto	Descrição
Alternância	«Включить 2FA»	Habilita/desabilita a autenticação de dois fatores.
Descrição	«Добавляет дополнительный уровень аутентификации для повышения безопасности.»	Dica exibida abaixo da alternância.

Como habilitar a 2FA

Ao ativar a alternância, o painel **gera localmente um novo segredo** – uma string aleatória em codificação Base32 (alfabeto `A–Z` e `2–7`). Uma janela é aberta com o título «Включить двухфакторную аутентификацию» e instruções passo a passo:

1. «Отсканируйте этот QR-код в приложении для аутентификации или скопируйте токен рядом с QR-кодом и вставьте его в приложение». Abaixo do QR-code, o próprio segredo é exibido em formato de texto – ao clicar no QR-code, o segredo é copiado para a área de transferência (exibe «Скопировано»).
2. «Введите код из приложения» – é necessário digitar o código de 6 dígitos gerado pelo aplicativo. O código é verificado **no lado do navegador**: o painel calcula o TOTP atual com base no segredo recém-

gerado e o compara com o digitado. Se o código estiver incorreto – «Неверный код»; o campo aceita somente exatamente 6 dígitos.

Somente após a confirmação bem-sucedida o segredo e o sinalizador de habilitação são salvos. Ao salvar, é exibida a mensagem «Двухфакторная аутентификация была успешно установлена».

Importante: as alterações na seção de configurações são aplicadas pelo botão geral «**Сохранить**», após o qual normalmente é necessário reiniciar o painel («Сохраните изменения и перезапустите панель для их применения»). Na primeira habilitação da 2FA, o servidor adicionalmente **invalida todas as sessões ativas** (incrementa o «login epoch»), portanto, após aplicar a configuração, será necessário fazer login novamente – desta vez com o código 2FA.

Como desabilitar a 2FA

Pressionar a alternância novamente abre a janela «Отключить двухфакторную аутентификацию» com a dica «Введите код из приложения, чтобы отключить двухфакторную аутентификацию.». Após inserir o código correto (a partir da 3.4.2 ele é verificado **no servidor**), o sinalizador e o segredo são apagados, e é exibida a mensagem «Двухфакторная аутентификация была успешно удалена».

Verificação do código no login

Durante o login, o servidor recupera o segredo salvo e compara o TOTP atual com o código 2FA enviado. Uma divergência é tratada como login malsucedido, mas ao usuário é exibida a mesma mensagem unificada «Неверные данные учетной записи.».

Recuperação de acesso

Não existe um mecanismo separado de «códigos de recuperação» no 3X-UI. Se o acesso ao aplicativo autenticador for perdido, não é possível recuperar o login pela interface do painel. O único caminho é desabilitar a 2FA diretamente no banco de dados no servidor: redefinir a chave `twoFactorEnable` para `false` (e, se necessário, limpar `twoFactorToken`) na tabela de configurações, após o que reiniciar o painel. Por isso, recomenda-se guardar o segredo (token Base32) em um local seguro ao habilitar a 2FA.

Exemplo: desabilitação de emergência da 2FA no servidor. Obtendo acesso ao servidor via SSH, pare o painel, redefina as chaves na tabela de configurações e reinicie o painel:

```
x-ui stop
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='false' WHERE
key='twoFactorEnable';"
sqlite3 /etc/x-ui/x-ui.db "UPDATE settings SET value='' WHERE key='twoFactorToken';"
x-ui start
```

Após isso, o login é feito apenas com login e senha, e a 2FA pode ser configurada novamente, se desejado.

Relação com a alteração de credenciais: ao alterar o login/senha (ver 2.4), a 2FA é **desabilitada automaticamente** no servidor, para que o segredo antigo não bloqueie o acesso com a nova conta.

2.3. Limitador de tentativas de login (proteção contra força bruta)

O painel possui um limitador integrado de logins malsucedidos (semelhante ao fail2ban no nível da aplicação). Os parâmetros são definidos no código e **não são configuráveis** pela interface:

Parâmetro	Valor	Finalidade
Máximo de falhas	5	Quantas tentativas malsucedidas são permitidas dentro da janela.
Janela de contagem	5 minutos	Janela deslizante em que as falhas se acumulam (as mais antigas são descartadas).
Bloqueio (cooldown)	15 minutos	Por quanto tempo a chave fica bloqueada após ultrapassar o limite.

Como funciona:

- A chave de bloqueio é construída a partir da **combinação «IP + login»** (o login é convertido para minúsculas e os espaços são removidos). Ou seja, o bloqueio se aplica ao par específico «endereço + nome de usuário», e não ao painel inteiro.
- A cada tentativa malsucedida (login/senha incorretos ou código 2FA inválido), o contador aumenta. Ao atingir **5 falhas em 5 minutos**, a chave é bloqueada por **15 minutos**. Durante o bloqueio, qualquer tentativa desse par é imediatamente rejeitada com a mesma mensagem «Неверные данные учетной записи.», mesmo que os dados estejam corretos.
- **Um login bem-sucedido redefine imediatamente** o contador e remove o bloqueio para esse par.
- O endereço IP do cliente é determinado levando em conta proxies confiáveis (ver `trustedProxyCIDRs`): os cabeçalhos `X-Real-IP` e `X-Forwarded-For` são aceitos somente se a requisição veio de um endereço confiável. Caso contrário, é usado o endereço real da conexão; se não for possível extraí-lo, é usada a string `unknown`.

Todas as tentativas são registradas em log. Para as malsucedidas, é gravado um aviso no log do servidor com o nome de usuário, IP, motivo e, em caso de bloqueio, o horário de `blocked_until`. Se as notificações de login via bot do Telegram estiverem habilitadas (`tgNotifyLogin` — «Уведомление о входе»), o administrador também recebe o nome de usuário, IP e horário tanto das tentativas bem-sucedidas quanto das malsucedidas e bloqueadas.

Exemplo: notificação de login no Telegram. Com `tgNotifyLogin` habilitado, após cada tentativa o administrador recebe uma mensagem semelhante a esta:

```
Уведомление о входе
Пользователь: admin
IP: 203.0.113.45
Время: 2026-06-10 14:32:07
Статус: успешно
```

Para o par «IP + login» bloqueado, o status indicará que a tentativa foi rejeitada pelo limitador.

2.4. Alteração de login e senha do administrador

Seção **Настройки** → **Учетная запись**, aba «Учетные данные администратора». Campos:

Campo	Texto	Descrição
Текущий логин	«Текущий логин»	Nome de usuário atual. Deve corresponder ao login atual; caso contrário, a alteração é rejeitada.
Текущий пароль	«Текущий пароль»	Senha atual para confirmação de identidade.
Новый логин	«Новый логин»	Novo nome de usuário. Não pode estar vazio.
Новый пароль	«Новый пароль»	Nova senha. Não pode estar vazia.

A alteração é aplicada pelo botão **«Подтвердить»** e enviada para `POST /panel/setting/updateUser`.

Lógica e mensagens do servidor:

- Se «Текущий логин» não corresponder ao real ou «Текущий пароль» estiver incorreto – «Произошла ошибка при изменении учетных данных администратора.» com a explicação «Неверное имя пользователя или пароль».
- Se o novo login ou a nova senha estiver vazio – explicação «Новое имя пользователя и новый пароль должны быть заполнены».
- Em caso de sucesso – «Вы успешно изменили учетные данные администратора.». A senha é armazenada como hash bcrypt.

Confirmação com código 2FA (a partir da 3.4.2). Se a autenticação de dois fatores estiver ativada no painel, a alteração de login/senha exige adicionalmente **inserir o código atual** do aplicativo autenticador. Abre-se a janela **«Alteração de credenciais»** com a dica «Insira o código do aplicativo para alterar as credenciais do administrador.»; o código (`twoFactorCode`) é verificado no servidor e, se for incorreto ou estiver vazio, a alteração não é aplicada. A mesma confirmação agora também é exigida ao desativar a 2FA (veja 2.2).

Exemplo: alteração de credenciais via API. A requisição requer um cookie de sessão ativo (obtido no login) e a confirmação do login/senha atuais:

```
curl -X POST https://panel.example.com:2053/мой-секрет/panel/setting/updateUser \
  -b 'session=ВАША_СЕССИОННАЯ_COOKIE' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  --data 'oldUsername=admin&oldPassword=СтарыйПароль&newUsername=root&newPassword=НовыйСложныйПароль'
```

Após o sucesso, a sessão atual é invalidada – será necessário fazer login novamente com as novas credenciais.

Efeitos importantes da alteração de credenciais:

- **Todas as sessões existentes são invalidadas** (o contador `login_epoch` do usuário é incrementado), portanto, após a alteração, o painel realiza logout automaticamente e redireciona para a página de login – é necessário fazer login novamente.
- Se a **2FA estava habilitada** no momento da alteração, ela é **automaticamente desabilitada** (o sinalizador e o segredo são redefinidos). A autenticação de dois fatores precisará ser configurada novamente após a alteração do login/senha.

Se a 2FA estiver habilitada, antes do envio do formulário é aberta a janela «Изменить учетные данные» com a dica «Введите код из приложения, чтобы изменить учетные данные администратора.» — as credenciais só podem ser alteradas mediante confirmação do código 2FA atual.

2.5. Caminho secreto (URI-путь / webBasePath) e porta do painel

Esses parâmetros ficam em **Настройки** → **Панель** e afetam diretamente a «invisibilidade» e a acessibilidade do painel. São aplicados após salvar e **reiniciar o painel**.

Campo	Texto	Valor padrão	Descrição
Порт панели	«Порт панели» (<code>panelPort</code>), dica «Порт, на котором работает панель»	2053	Porta TCP da interface web.
URI-путь	«URI-путь» (<code>panelUrlPath</code>), dica «Должен начинаться с '/' и заканчиваться '/'»	/	Caminho base secreto (<code>webBasePath</code>). O painel só é acessível por ele (por exemplo, <code>/мой-секрет/</code>).
IP-адрес для управления панелью	«IP-адрес для управления панелью» (<code>panelListeningIP</code>), dica «Оставьте пустым для подключения с любого IP»	vazio	Endereço em que o painel escuta. Vazio = todas as interfaces.
Домен панели	«Домен панели» (<code>panelListeningDomain</code>), dica «Оставьте пустым для подключения с любых доменов и IP.»	vazio	Restrição de acesso por domínio (Host).
Путь к публичному ключу сертификата панели	<code>publicKeyPath</code> , dica «Введите полный путь, начинающийся с '/'»	vazio	Certificado TLS para acesso HTTPS ao painel.
Путь к приватному ключу сертификата панели	<code>privateKeyPath</code> , mesma dica	vazio	Chave privada TLS.

Comportamento do caminho base (`webBasePath`):

- O valor é normalizado automaticamente: se não começar com `/` , o caractere é adicionado no início; se não terminar com `/` , é adicionado no final. Ou seja, o caminho sempre tem a forma `/.../` .
- O caminho base se aplica ao próprio painel, aos assets e ao cookie de sessão (o cookie é emitido apenas para esse caminho).

Recomendações de segurança (seção «Предупреждения безопасности»): o painel exibe avisos quando a configuração está «excessivamente pública»:

- «Панель работает по обычному HTTP — настройте TLS для продакшна.»
- «Стандартный порт 2053 широко известен — измените его на случайный.»
- «Базовый путь по умолчанию "/" широко известен — измените его на случайный.»

Em outras palavras, para um servidor em produção, deve-se definir uma **porta não padrão**, um **URI-путь não trivial** e um **certificado TLS**.

Exemplo: configuração «discreta» do painel para produção. Em **Настройки** → **Панель**, defina valores semelhantes a estes:

Campo	Valor
Порт панели	34571 (aleatória, em vez de 2053)
URI-путь	/aXf9Qm2/ (não trivial, começa e termina com /)
Путь к публичному ключу сертификата панели	/etc/letsencrypt/live/panel.example.com/fullchain.pem
Путь к приватному ключу сертификата панели	/etc/letsencrypt/live/panel.example.com/privkey.pem

Após salvar e reiniciar, o painel estará acessível apenas em `https://panel.example.com:34571/aXf9Qm2/`, e os avisos de segurança desaparecerão.

2.6. Tempo de vida da sessão (timeout)

O campo **«Продолжительность сессии»** (`sessionMaxAge`) encontra-se entre as configurações do painel/ intervalos.

Campo	Texto	Valor padrão	Unidade	Descrição
Продолжительность сессии	«Продолжительность сессии», dica «Продолжительность сессии в системе (значение: минута)»	360	minutos	Tempo de vida do cookie de sessão do administrador.

Comportamento:

- O valor é definido em **minutos** (padrão: 360 minutos = 6 horas) e é convertido para segundos na configuração do cookie.
- Se o valor for **maior que 0**, o cookie de sessão recebe o `MaxAge` correspondente. Após esse prazo, o cookie deixa de ser válido e, na próxima requisição, o usuário recebe «Сессия истекла. Войдите в систему снова».
- A sessão também se torna inválida antecipadamente ao alterar as credenciais ou na primeira habilitação da 2FA (por meio do mecanismo `login_epoch`, ver 2.4 e 2.2) e no logout explícito (`POST /logout`).
- O cookie de sessão é marcado como `HttpOnly`, com política `SameSite=Lax`; o sinalizador `Secure` é definido no acesso HTTPS direto ao painel.

Além do próprio timeout, há uma notificação relacionada: **«Задержка уведомления об истечении сессии»** (`expireTimeDiff`, dica «Получение уведомления об истечении срока действия сессии до достижения порогового значения (значение: день)», padrão `0`) — permite receber um aviso com antecedência.

2.7. LDAP (sincronização e autenticação)

A seção LDAP oferece duas possibilidades: (1) autenticar o login do administrador via LDAP, caso a senha local não corresponda, e (2) sincronizar periodicamente o estado dos clientes (sinalizador VLESS habilitado/desabilitado) a partir do diretório.

Como é utilizado no login: o servidor primeiro verifica o hash bcrypt local da senha. Se ele **não corresponder** e o LDAP estiver habilitado, o painel tenta autenticar o usuário no diretório: se **Bind DN** estiver configurado, é realizado um bind de serviço, em seguida a entrada do usuário é buscada pelo filtro e atributo, e é feita uma tentativa de bind com o DN encontrado e a senha digitada. O sucesso implica login. (Após uma autenticação LDAP bem-sucedida, se a 2FA estiver habilitada, o código TOTP ainda é verificado.)

Campos da seção:

Campo	Texto	Valor padrão	Descrição
Включить LDAP-синхронизацию	«Включить LDAP-синхронизацию» (<code>enable</code>)	false	Interruptor principal da integração LDAP.
LDAP-хост	«LDAP-хост» (<code>host</code>)	vazio	Endereço do servidor LDAP.
Порт LDAP	«Порт LDAP» (<code>port</code>)	389	Porta. Para LDAPS, normalmente 636.
Использовать TLS (LDAPS)	«Использовать TLS (LDAPS)» (<code>useTls</code>)	false	Quando habilitado, utiliza o esquema <code>ldaps://</code> (por padrão – com verificação do certificado do servidor).
Pular verificação do certificado TLS	«Пропускать проверку TLS-сертификата» (<code>ldapInsecureSkipVerify</code>)	false	Adicionado na 3.4.2. Desativa a verificação do certificado do servidor no LDAPS – para CAs internas/autoassinadas. Dica: «Inseguro – desativa a verificação do certificado do servidor. Use apenas com CAs internas/não confiáveis». O interruptor está disponível apenas quando «Usar TLS (LDAPS)» está ativado.
Bind DN	«Bind DN» (<code>bindDn</code>)	vazio	DN da conta de serviço para bind/busca inicial. Se vazio – o bind não é realizado (busca anônima).
Пароль bind	dicas: «Настроено; оставьте пустым, чтобы сохранить текущий пароль.» / «Не настроено.» / «Настроено – введите новое значение для замены»	vazio	Senha para Bind DN . Armazenada separadamente; para manter a senha atual, deixe o campo vazio.

Campo	Texto	Valor padrão	Descrição
Base DN	«Base DN» (<code>baseDn</code>)	vazio	Raiz da subárvore em que a busca é realizada (busca recursiva em toda a subárvore).
Filtro usuário	«Filtro usuário» (<code>userFilter</code>)	(<code>objectClass=person</code>)	Filtro LDAP para seleção de contas. Na autenticação, o login é inserido no filtro com escape.
Atributo usuário (username/ email)	«Atributo usuário (username/ email)» (<code>userAttr</code>)	mail	Atributo mapeado ao login/ identificador do cliente (por exemplo, <code>mail</code> ou <code>uid</code>).
Atributo VLESS- bandeira	«Atributo VLESS-bandeira» (<code>vlessField</code>)	<code>vless_enabled</code>	Atributo que determina se o acesso VLESS do cliente deve estar habilitado.
Comum atributo bandeira (opc.)	«Comum atributo bandeira (opc.)» (<code>flagField</code>), diga «Se definido, redefine bandeira VLESS – por exemplo, <code>shadowInactive</code> .»	vazio	Se definido, é usado em vez de <code>vless_enabled</code> .
Truthy-valores	«Truthy-valores» (<code>truthyValues</code>), diga «Por exemplo; por padrão: <code>true,1,yes,on</code> »	<code>true,1,yes,on</code>	Lista de valores do atributo de sinalizador tratados como «habilitado».
Inverter bandeira	«Inverter bandeira» (<code>invertFlag</code>), diga «Ative, quando atributo significa «desativado» (por exemplo, <code>shadowInactive</code>).»	false	Inverte o significado do sinalizador.
Agendamento sincronização	«Agendamento sincronização» (<code>syncSchedule</code>), diga «Linha tipo cron, por exemplo, <code>@every 1m</code> »	<code>@every 1m</code>	Periodicidade da sincronização em formato semelhante a cron.
Tags de entrada	«Tags de entrada» (<code>inboundTags</code>), diga «Entradas, nas quais LDAP- sincronização pode auto- criar ou auto-deletar clientes.»	vazio	Limita em quais inbound as operações automáticas são permitidas. Se não houver inbound: «Entradas não encontradas. Primeiro crie entrada.»
Auto-criação clientes	«Auto-criação clientes» (<code>autoCreate</code>)	false	Criar o cliente nos inbound especificados se ele aparecer no diretório.
Auto-exclusão clientes	«Auto-exclusão clientes» (<code>autoDelete</code>)	false	Remover o cliente se ele desaparecer do diretório.
Limite por padrão (GB)	«Limite por padrão (GB)» (<code>defaultTotalGb</code>)	0	Limite de tráfego para clientes criados automaticamente (0 = sem limite).

Campo	Texto	Valor padrão	Descrição
Срок по умолчанию (дни)	«Срок по умолчанию (дни)» (defaultExpiryDays)	0	Prazo de validade para clientes criados automaticamente (0 = sem prazo).
Лимит IP по умолчанию	«Лимит IP по умолчанию» (defaultIpLimit)	0	Limite de IPs simultâneos (0 = sem restrição).

Particularidades da lógica do sinalizador de sincronização: ao ler o atributo de sinalizador (`flagField` , padrão `vless_enabled`), o valor é considerado «habilitado» se estiver na lista de `truthy-значения`; com a inversão habilitada, o resultado é invertido. O atributo do usuário (`userAttr`) é usado como chave de mapeamento (email/nome) – entradas sem valor desse atributo são ignoradas.

Segurança: recomenda-se habilitar **TLS (LDAPS)** para que as senhas de bind e as senhas verificadas não sejam transmitidas em texto claro, e usar para o `Bind DN` uma conta com permissões mínimas necessárias de leitura.

Exemplo: configuração típica de sincronização LDAP (Active Directory). Preenchimento dos campos da seção para um diretório onde o status de acesso é armazenado em um atributo semelhante a `userAccountControl` , com mapeamento por e-mail:

Campo	Valor
LDAP-хост	<code>ldap.example.com</code>
Порт LDAP	636
Использовать TLS (LDAPS)	habilitado
Bind DN	<code>CN=svc-3xui,OU=Service,DC=example,DC=com</code>
Base DN	<code>OU=Users,DC=example,DC=com</code>
Фильтр пользователя	<code>(objectClass=person)</code>
Атрибут пользователя (username/email)	<code>mail</code>
Атрибут VLESS-флага	<code>vless_enabled</code>
Truthy-значения	<code>true,1,yes,on</code>
Расписание синхронизации	<code>@every 5m</code>

Com essa configuração, a cada 5 minutos o painel percorrerá a subárvore `OU=Users` , mapeará os clientes por `mail` e habilitará/desabilitará o acesso VLESS com base no valor de `vless_enabled` .

3. Visão Geral / Dashboard

O Dashboard (*Overview* na interface em inglês) é a página inicial do painel. Ele exibe em tempo real o estado do servidor e do processo Xray. Todos os indicadores vêm do lado do servidor. Um agendador em segundo plano reconstrói o snapshot **a cada 2 segundos** e o distribui para todas as abas abertas via WebSocket; a cada minuto, as séries de métricas acumuladas são salvas em disco. O endpoint HTTP `GET /status` retorna o último snapshot em cache.

A seguir, cada indicador e cada elemento de controle da página são descritos em detalhes.

No menu lateral do painel (e na gaveta móvel), a partir da versão 3.4.2 há o botão «**Documentação**» (ícone de livro) – abre a documentação oficial <https://docs.sanaei.dev/>. Também na 3.4.2 foi feita uma melhoria transversal de **acessibilidade**: os ícones-botão e os elementos clicáveis ganharam rótulos aria e ativação por Enter/Space (para leitores de tela e navegação por teclado).

3.1. Princípios gerais de coleta de dados

- O snapshot é coletado pela biblioteca `gopsutil`. Se uma medição específica falhar, o campo permanece zerado e um aviso é registrado no log (`get cpu percent failed`, `get uptime failed`, etc.) – isso não derruba o dashboard inteiro; apenas o bloco correspondente exibirá 0/N-A.
- As velocidades "instantâneas" (CPU %, rede, disco I/O) são calculadas como a diferença entre o snapshot atual e o anterior, dividida pelo intervalo em segundos. Por isso, ao carregar a página pela primeira vez, os valores de velocidade podem ser zero até que a segunda medição seja acumulada.
- O histórico pode ser visualizado na seção "Histórico do sistema" (*System History*) – os gráficos são construídos a partir das mesmas séries de dados descritas abaixo (veja item 3.12).

3.2. CPU

O bloco "CPU" mostra a carga atual do processador em percentual, além dos parâmetros do processador em si.

Indicador	Descrição
Carga da CPU, %	Fração do tempo de processamento utilizado no último intervalo. Suavizado por média exponencial (EMA, coeficiente <code>alpha = 0.3</code>) para evitar que picos agitem o indicador. O valor é sempre limitado ao intervalo 0–100 %. Na primeira medição, retorna 0 (inicialização do ponto base).
Processadores lógicos	Número de núcleos lógicos – ou seja, incluindo Hyper-Threading.
Núcleos físicos	Número de núcleos físicos.
Frequência	Frequência base do processador em MHz. Consultada de forma lazy e armazenada em cache: a primeira medição bem-sucedida é salva, novas tentativas ocorrem no máximo a cada 5 minutos, e a consulta tem timeout de 1,5 s (em alguns sistemas a consulta de frequência responde lentamente).

O algoritmo de cálculo da carga de CPU funciona assim: quando há uma implementação nativa da plataforma, ela é usada; caso contrário, o cálculo é feito com base nos deltas dos contadores de tempo de processamento (busy / total). O tempo de Guest e GuestNice é excluído para evitar contagem dupla.

3.3. Memória (RAM)

O bloco "Memória" (RAM) mostra o uso atual e o total. É exibido como "usado / total" e/ou percentual de utilização. O histórico registra o percentual.

3.4. Swap

O bloco "Swap" mostra o uso atual e o total. Se o arquivo/partição de swap não estiver configurado (total = 0), o indicador será zero; na ausência de swap, o valor 0 é registrado na série histórica.

3.5. Disco (Storage)

O bloco "Disco" (Storage) mostra o uso atual e o total, considerando **apenas a partição raiz** /. O histórico "Uso de disco" (Disk Usage) registra o percentual de utilização. Separadamente, é coletado o I/O de disco (leitura / escrita, bytes/s) como delta dos contadores por intervalo — exibido na aba "Disco I/O" do histórico.

3.6. Tempo de atividade do sistema (Uptime)

O indicador "Tempo de atividade do sistema" (Uptime) representa o tempo desde a inicialização **de todo o servidor** (em segundos), e não o tempo de execução do painel ou do Xray. O uptime do processo Xray é armazenado separadamente (veja item 3.9), assim como o número de threads do painel (Threads).

Memória utilizada pelo painel

Junto com os indicadores do processo do painel, é exibido o volume de memória RAM ocupado pelo próprio processo 3X-UI. Esse valor é obtido do RSS real do processo (como visto pelo sistema operacional) e coincide com o que as ferramentas do sistema mostram. O número diminui conforme a memória é liberada. Anteriormente, o painel exibia um contador interno do Go que superestimava o consumo de memória (por exemplo, ~300 MB em um servidor ocioso com um único cliente) e nunca diminuía — esse artefato não existe mais. Adicionalmente, um processo periódico em segundo plano devolve memória não utilizada ao sistema operacional para que o indicador reflita o consumo real.

3.7. Carga do sistema (Load average)

O bloco "Carga do sistema" (System Load) é um array de três números [Load1, Load5, Load15]. Legenda: "Média de carga do sistema nos últimos 1, 5 e 15 minutos" (System load average for the past 1, 5, and 15 minutes). O gráfico de histórico é chamado "Média de carga do sistema (1 / 5 / 15 min)". As séries históricas armazenam os valores separadamente: load1, load5, load15.

Este é o indicador Unix padrão: o número médio de processos na fila de execução. O ponto de referência é comparar com o número de núcleos: uma carga persistentemente superior ao número de núcleos físicos indica sobrecarga.

3.8. Rede: velocidade e volume total de tráfego

Apenas **interfaces físicas** são consideradas. Interfaces virtuais e de túnel são excluídas: `lo` / `lo0`, e tudo que começa com `loopback`, `docker`, `br-`, `veth`, `virbr`, `tun`, `tap`, `wg`, `tailscale`, `zt`. Os valores são somados para todas as interfaces restantes.

Velocidade geral (*Overall Speed*) – velocidade instantânea, delta dos contadores por intervalo:

Indicador	Descrição
Upload (legenda "Upload")	Velocidade de saída, bytes/s.
Download (legenda "Download")	Velocidade de entrada, bytes/s.

Volume total de tráfego (*Total Data*) – contadores acumulados desde a inicialização do sistema:

Indicador	Descrição
Enviado (legenda "Sent")	Total de bytes enviados.
Recebido (legenda "Received")	Total de bytes recebidos.

Adicionalmente, são coletadas velocidades de pacotes (pacotes/s) e contadores totais de pacotes – exibidos na aba "Pacotes de rede" (*Network Packets*) do histórico. Séries históricas de rede: `netUp`, `netDown`, `pktUp`, `pktDown`.

3.9. Endereços IP do servidor

O bloco "Endereços IP do servidor" (*IP Addresses*) exibe `IPv4` e `IPv6`. Os endereços externos são determinados por serviços de terceiros (`api4.ipify.org`, `ipv4.icanhazip.com`, `v4.api.ipinfo.io/ip`, `ipv4.myexternalip.com/raw`, `4.ident.me`, `check-host.net/ip` para `IPv4` e equivalentes para `IPv6`). A lista é percorrida em ordem até a primeira resposta bem-sucedida; o timeout de cada requisição é de 3 s.

Particularidades:

- O resultado é **armazenado em cache** durante o tempo de vida do processo: um endereço determinado com sucesso não é consultado novamente.
- Se nenhum serviço responder, o campo permanece como `N/A`. Para `IPv6`, ao primeiro `N/A`, as requisições `IPv6` são completamente desativadas para não desperdiçar tempo em redes sem `IPv6`.
- Há um botão de "olho" para ocultar/exibir os endereços – dica "Alternar visibilidade dos endereços IP" (*Toggle visibility of the IP*). Isso é apenas um ocultamento visual na interface (por exemplo, para capturas de tela) e não afeta os endereços em si.

3.10. Conexões TCP/UDP

O bloco "Estatísticas de conexões" (*Connection Stats*) exibe o número total de conexões TCP e UDP ativas no servidor (em todo o sistema, não apenas do Xray). O gráfico de histórico é "Conexões ativas (TCP / UDP)" (*Active Connections*), séries `tcpCount`, `udpCount`.

3.11. Status do Xray e controle do processo

O cartão "Xray" exibe o estado do processo Xray-core e permite controlá-lo.

Estados

Valor	Legenda	Tradução	Quando é definido
running	"Rodando"	<i>Running</i>	O processo Xray está em execução.
stop	"Parado"	<i>Stopped</i>	O processo não está em execução e não há erro de inicialização registrado.
error	"Erro"	<i>Error</i>	O processo não está em execução, mas há um erro de inicialização registrado. O texto do erro é exibido em uma janela pop-up com o título "Ocorreu um erro ao executar o Xray" (<i>An error occurred while running Xray</i>).
–	"Desconhecido"	<i>Unknown</i>	Exibido enquanto o status ainda não foi recebido.

Ao lado do status é exibida a **versão do Xray**.

Botões de controle

- **Parar (Stop)**. Chama `POST /stopXrayService`. Em caso de sucesso, o painel envia via WebSocket o novo estado `stop` e a notificação "Xray parado com sucesso" (*Xray service has been stopped*); em caso de erro – o estado `error` com o texto do erro. Importante: se o painel estiver acessível *através* do próprio Xray, parar o Xray pode interromper a conexão com o painel – isso não é um problema ao conectar-se diretamente ao painel.
- **Reiniciar (Restart)**. Chama `POST /restartXrayService`. Antes da ação, é exibida uma confirmação "Reiniciar o xray?" com a explicação "Reinicia o serviço xray com a configuração salva". Em caso de sucesso – estado `running` e notificação "Xray reiniciado com sucesso" (*Xray service has been restarted successfully*). O reinício aplica a configuração salva atual – use-o após alterar as configurações.

Observação. Neste fork, o dashboard conta com controle completo de Start / Stop / Restart para todos os tipos de autenticação; na interface original do 3x-ui não há botão separado de "iniciar" – a inicialização é feita via reinício.

Botão de visualização dos logs do Xray

No cartão do Xray há um botão para visualizar os logs do Xray (*Logs*). Ele aparece somente quando o log de acesso está configurado na configuração do Xray: o visualizador embutido lê exatamente esse arquivo, portanto, sem o log de acesso o botão fica oculto. A visibilidade do botão está vinculada ao atributo separado `accessLogEnable` e não depende mais do limite de IP – a lista de clientes online e o limite de endereços IP continuam funcionando mesmo sem o log de acesso (veja item 8).

Seleção de versão do Xray

A seção "Seleção de versão" (*Version*) permite alternar o Xray-core para outro release. A lista de versões é carregada via `GET /getXrayVersion`:

- A fonte é a API do GitHub do repositório `XTLS/Xray-core` (`/releases`). As requisições são armazenadas em cache por **15 minutos**; em caso de falha do GitHub, a última lista obtida com sucesso é retornada para que o seletor não fique vazio.
- A lista inclui apenas releases no formato `X.Y.Z` e **não anteriores a 26.4.25**.

Dicas: "Selecione a versão desejada" (*Choose the version you want to switch to.*) e aviso "Importante: versões antigas podem não ser compatíveis com as configurações atuais" (*Choose carefully, as older versions may not be compatible with current configurations.*).

Troca de versão: `POST /installXray/:version`. Cenário:

Exemplo. Alternar para uma versão específica do Xray-core (o cookie de sessão deve ter sido obtido previamente com autenticação):

```
curl -X POST 'https://panel.example.com:2053/xpanel/installXray/v25.6.8' \
  -b cookie.txt
```

Aqui `v25.6.8` é a tag da lista retornada por `GET /getXrayVersion`. A versão deve estar presente nessa lista, caso contrário o painel retornará uma recusa. A partir da versão 3.4.2, a versão mínima do Xray permitida para instalação foi elevada para **26.6.27**, e a 3.5.0 vem com o núcleo **Xray 26.7.11** incluído. Alterações do núcleo que acompanham: as cifras Shadowsocks `none / plain` e VMess `none / zero` foram removidas (as configurações salvas são reescritas automaticamente: SS – para uma cifra suportada, VMess – para `auto`), e um outbound VLESS/Trojan sem criptografia para um endereço público é rejeitado ao salvar – o núcleo não iniciaria com tal configuração.

1. A versão selecionada é verificada na lista atual de releases (caso contrário – recusa).
2. O Xray é parado.
3. O arquivo `Xray-<os>-<arch>.zip` correspondente ao SO e arquitetura atuais é baixado do GitHub (suportados: amd64/64, arm64-v8a, arm32-v7a/v6/v5, 386/32, s390x; para Windows – `xray.exe`). O tamanho do arquivo e do binário é limitado a 200 MB.
4. O binário é substituído de forma atômica (via arquivo temporário + renomeação) e marcado como executável.
5. O Xray é reiniciado.

Antes da troca é exibido o diálogo "Alterar versão do Xray" (*Do you really want to change the Xray version?*) com a descrição "Isso alterará a versão do Xray para #version#". Em caso de sucesso – notificação "Xray atualizado com sucesso" (*Xray updated successfully*).

3.12. Atualização do painel (3X-UI)

Bloco de verificação de atualizações do painel. Os dados chegam via `GET /getPanelUpdateInfo`:

Campo	Descrição
Versão atual do painel	Versão do painel instalado.
Última versão do painel	Último release do 3x-ui obtido do GitHub.
Atualização disponível	Indicador de que a última versão é mais recente que a atual. Se não houver atualização – é exibido "Painel atualizado" / "Atualizado".

O botão **"Atualizar painel"** (*Update Panel*) inicia `POST /updatePanel`. Dica: "Isso atualizará o 3X-UI para o último release e reiniciará o serviço do painel". Antes da execução – confirmação "Você realmente deseja atualizar o painel?" com o texto "Isso atualizará o 3X-UI para a versão #version# e reiniciará o serviço do painel".

Particularidades e limitações:

- A auto-atualização é suportada **apenas no Linux** (em outros sistemas operacionais é retornado um erro).
- O script de atualização é baixado do repositório oficial (`raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh` , limite de 2 MB) e executado via `bash` , preferencialmente de forma isolada via `systemd-run` .
- Em caso de início bem-sucedido, é exibido "Atualização do painel iniciada" (*Panel update started*); se a verificação de atualização falhou – "Falha na verificação de atualização do painel". Durante a instalação é exibido o aviso "Instalação em andamento. Não atualize a página".

3.13. Atualização dos arquivos de geolocalização (GeoIP / GeoSite)

O botão/diálogo de atualização das bases geográficas chama `POST /updateGeofile` (todos os arquivos) ou `POST /updateGeofile/:fileName` (um arquivo específico). A atualização funciona com uma lista branca estrita de nomes e fontes:

Arquivo	Fonte
<code>geoip.dat</code> , <code>geosite.dat</code>	<code>Loyalsoldier/v2ray-rules-dat</code> (latest)
<code>geoip_IR.dat</code> , <code>geosite_IR.dat</code>	<code>chocolate4u/Iran-v2ray-rules</code> (latest)
<code>geoip_RU.dat</code> , <code>geosite_RU.dat</code>	<code>runetfreedom/russia-v2ray-rules-dat</code> (latest)

Comportamento:

- O nome do arquivo é validado: `..` , barras e caminhos absolutos são proibidos; apenas `[a-zA-Z0-9._-]+.dat` é permitido. Arquivos fora da lista branca não são baixados.
- É utilizada a requisição condicional `If-Modified-Since` : se o arquivo no servidor de origem não foi alterado (HTTP 304), ele não é baixado novamente, apenas o timestamp é atualizado.
- Após o download, o Xray é **reiniciado** (para carregar as novas bases).

Exemplo. Atualizar apenas as bases geográficas russas, sem tocar nos outros arquivos:

```
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geoip_RU.dat' -b
cookie.txt
curl -X POST 'https://panel.example.com:2053/xpanel/updateGeofile/geosite_RU.dat' -b
cookie.txt
```

Para atualizar todos os arquivos da lista branca de uma só vez – chame `POST /updateGeofile` sem nome de arquivo.

- Diálogos: "Você realmente deseja atualizar o arquivo geográfico?" com "Isso atualizará o arquivo #filename#" para um arquivo específico e "Isso atualizará todos os arquivos geográficos" para o botão "Atualizar todos". Sucesso – "Arquivos geográficos atualizados com sucesso".

3.14. Backup e restauração do banco de dados

Bloco "Backup & Restore". O comportamento depende do SGBD utilizado (SQLite por padrão ou PostgreSQL).

Exportação do banco (Backup)

O botão "Exportar banco de dados" / "Backup" (*Back Up*) chama `GET /getDb`. O arquivo é entregue como anexo:

- **SQLite**: primeiro é realizado um checkpoint (liberação do WAL), depois o arquivo `x-ui.db` é baixado. Dica: "Clique para baixar o arquivo .db contendo o backup do seu banco de dados atual..."
- **PostgreSQL**: é baixado o dump `x-ui.dump` em formato personalizado (`pg_dump --format=custom --no-owner --no-privileges`). As ferramentas cliente do PostgreSQL devem estar instaladas no servidor; caso contrário – erro sobre ausência do `pg_dump`.

Importação do banco (Restauração)

O botão "Importar banco de dados" / "Restauração" (*Restore*) faz upload do arquivo via `POST /importDB` (campo de formulário `db`). Dica: "Clique para selecionar e carregar o arquivo .db... para restaurar o banco de dados a partir do backup".

Cenário para **SQLite** é seguro, com rollback:

1. O arquivo é verificado quanto ao formato SQLite e salvo em um arquivo temporário, depois sua integridade é verificada.
2. O Xray é parado, o banco de dados atual é fechado e renomeado para `*.backup` (fallback).
3. O novo arquivo ocupa o lugar do banco de dados ativo, e a inicialização e migração são realizadas. Se algo der errado – o fallback é restaurado.
4. O Xray é reiniciado.

Para **PostgreSQL**, a partir da 3.5.0 a «Restauração» aceita **três tipos de arquivo** (o tipo é detectado pelo conteúdo, não pela extensão): um arquivo `pg_dump` (PGDMP) – aplicado via `pg_restore --clean --if-exists --single-transaction ...`; um **banco SQLite .db** (backup comum) e um **.dump de migração SQLite** – estes são verificados, reconstruídos se necessário e importados no PostgreSQL **em uma única transação** pelo mesmo motor do `x-ui migrate-db --dsn` (em caso de erro, o PostgreSQL permanece intocado). O diálogo de seleção de arquivo aceita `.dump, .db` em ambos os SGBDs; o envio de um arquivo `pg_dump` a um painel SQLite gera um erro claro. A dica adverte explicitamente: "Isso substituirá todos os dados atuais".

Mensagens: "Banco de dados importado com sucesso", "Ocorreu um erro ao importar o banco de dados", "...ao ler o banco de dados", "...ao obter o banco de dados".

Arquivo de migração (entre SQLite e PostgreSQL)

O botão "Baixar arquivo de migração" (*Download Migration*) chama `GET /getMigration`. A partir da 3.5.0, ele é exibido **apenas em painéis PostgreSQL** e baixa `x-ui.db` – um banco SQLite pronto, construído a partir dos dados do PostgreSQL (o caminho «de volta ao SQLite»). Nos painéis SQLite, o botão foi removido por ser redundante: um backup comum `.db` agora já é restaurado diretamente em um painel PostgreSQL.

3.15. Elementos adicionais da interface

- **Indicador de clientes online.** O dashboard mantém a série `online` (*Online Clients* / "Clientes online") – número de clientes com conexão ativa. É calculado quando o Xray está em execução (caso contrário 0) e registrado no histórico no mesmo ciclo de 2 segundos. Gráfico – aba "Online".
- **Histórico do sistema (gráficos).** Botão/seção "Gráficos" → "Histórico do sistema" com abas: "Largura de banda", "Pacotes", "Disco I/O", "Online", "Carga", "Conexões", "Uso de disco". Os dados são obtidos via `GET /history/:metric/:bucket`; intervalos de agregação permitidos (bucket, seg): **2, 30, 60, 180, 360, 720, 1440, 2880, 10080**, até 60 pontos por aba. No seletor de intervalo da página estão disponíveis os botões **2m, 1h, 3h, 6h, 12h, 24h, 2d, 7d** (buckets **2, 60, 180, 360, 720, 1440, 2880, 10080** respectivamente). Para intervalos longos **2d** e **7d**, os rótulos de tempo no eixo incluem a data no formato `MM-DD HH:MM`. O armazenamento é organizado com decimação em três níveis (rollup): dados recentes são mantidos com passo de 2 s pela última **hora**, depois são mediados para passo de 1 min por **48 horas** e para passo de 10 min por **7 dias**. Portanto, os gráficos (CPU, RAM, tráfego, pacotes, conexões, disco, online, carga) podem ser visualizados por um período **de até 7 dias** (antes era até 48 horas), sendo que quanto mais no passado, menor a granularidade. Métricas permitidas: `cpu, mem, swap, netUp, netDown, pktUp, pktDown, diskRead, diskWrite, diskUsage, tcpCount, udpCount, online, load1, load5, load15`. A legenda "Últimos 2 minutos" corresponde a bucket = 2 (modo tempo real).

Exemplo. Obter a série de carga da CPU dos últimos ~2 minutos (bucket = 2 s, até 60 pontos) e a mesma série agregada por 5 minutos (bucket = 300 s):

```
curl 'https://panel.example.com:2053/xpanel/history/cpu/2' -b cookie.txt
curl 'https://panel.example.com:2053/xpanel/history/cpu/300' -b cookie.txt
```

A métrica pode ser substituída por qualquer uma permitida (`mem, netUp, tcpCount, load1`, etc.). Um bucket fora da lista branca **2, 30, 60, 180, 360, 720, 1440, 2880, 10080** será rejeitado.

- **Métricas do Xray** – bloco separado com consumo de memória e coleta de lixo do Xray (séries `xrAlloc, xrSys, xrHeapObjects, xrNumGC, xrPauseNs`) e "Observatório" (estado das conexões de saída). Funcionam apenas se o bloco `metrics` estiver configurado na configuração do Xray (`listen 127.0.0.1:11111, tag metrics_out`); caso contrário, é exibido "O endpoint de métricas do Xray não está configurado". Na janela de métricas do Xray há um seletor de intervalo próprio com os botões **2m, 1h, 3h, 6h, 12h** (buckets **2, 60, 180, 360, 720**).

Exemplo de bloco que ativa o painel de métricas do Xray. Na seção de configurações do Xray devem estar presentes simultaneamente `metrics` (com tag) e um inbound que escute essa tag:

```
{
  "metrics": {
    "tag": "metrics_out"
  },
  "inbounds": [
    {
      "listen": "127.0.0.1",
      "port": 11111,
      "protocol": "dokodemo-door",
      "settings": { "address": "127.0.0.1" },
      "tag": "metrics_out"
    }
  ]
}
```

O endereço `127.0.0.1:11111` é intencionalmente não exposto externamente – o painel o consulta localmente.

- **Alternador de tema escuro.** Localizado no menu geral/cabeçalho, e não no dashboard em si. Opções: "Tema" (*Theme*) com as variantes "Escuro" e "Ultra escuro" (*Ultra Dark*). Esta é uma configuração puramente visual de aparência e não afeta o funcionamento do painel.
 - **Outros links** no entorno do dashboard (no menu/rodapé): "Logs", "Configuração" – visualização do JSON final do Xray (GET `/getConfigJson`), "Documentação".
-

4. Inbounds: criação e parâmetros gerais

A seção «**Entradas**» (ing. *Inbounds*) é a lista de todos os pontos de entrada do Xray pelos quais os clientes se conectam. Cada inbound armazena tanto campos do painel (observação, limite de tráfego, agendamento de redefinição) quanto blocos JSON brutos da configuração do Xray (`settings` , `streamSettings` , `sniffing`).

A criação é feita pelo botão «**Criar conexão**» (*Add Inbound*), e a edição pelo «**Modificar conexão**» (*Modify Inbound*). Ambas as operações são enviadas para os endpoints de API `POST /add` e `POST /update/:id`.

A seguir, são descritos todos os campos do formulário que **não** se referem às configurações de um protocolo específico (clientes, criptografia, REALITY/TLS) e **não** se referem ao transporte/fluxo (abas «**Fluxo**», «**Segurança**») — esses são temas de seções separadas.

4.1. Campos gerais do formulário

Remark (Observação)

Parâmetro	Valor
Campo	<code>remark</code>
Tipo	<code>string</code>
Padrão	vazio

Nome legível por humanos do inbound, exibido na lista e nos cabeçalhos dos diálogos («Excluir conexão "{remark}"?» etc.). O rótulo do campo é «**Observação**». Não afeta o funcionamento do Xray, serve apenas para facilitar a administração; recomenda-se definir nomes únicos e significativos, pois eles são inseridos nos nomes dos arquivos exportados e nas confirmações de operações em massa.

Protocol (Protocolo)

Parâmetro	Valor
Campo	<code>protocol</code>
Rótulo	« Protocolo »
Validação	<code>required,oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun</code>

Lista suspensa do protocolo do inbound. Valores permitidos:

Valor	Observação
<code>vmess</code>	
<code>vless</code>	
<code>trojan</code>	

Valor	Observação
shadowsocks	
wireguard	
hysteria	Hysteria v2 – é hysteria com streamSettings.version = 2 ; não existe um protocolo separado
http	
mixed	socks/http na mesma porta
tunnel	
tun	aceito pelo validador; não existe uma constante de protocolo separada

O campo é obrigatório (required). A escolha do protocolo determina quais campos de configuração de clientes e qual transporte estarão disponíveis (consulte as seções específicas de cada protocolo).

Importante: ao salvar, o serviço normaliza o streamSettings . As configurações de transporte são mantidas apenas para os protocolos vmess , vless , trojan , shadowsocks , hysteria ; para os demais (http , mixed , tunnel , wireguard , tun), o campo streamSettings é **forçosamente limpo**.

Para inbounds do tipo tunnel /TPProxy cujo bloco streamSettings não contém a chave security (variante sem transporte), o formulário abre e salva sem o erro de validação streamSettings.security Invalid input .

Listen IP (IP de escuta)

Parâmetro	Valor
Campo	listen
Tipo	string
Padrão	vazio → o Xray escuta em 0.0.0.0 (todos os IPs)

Endereço IP no qual o inbound aceita conexões. Dica do campo:

«Deixe vazio para escutar em todos os endereços IP».

Ao gerar a configuração do Xray, o valor vazio é substituído por 0.0.0.0 . Além de um IP, o campo também aceita um **caminho de socket Unix** – dica:

«Você também pode especificar o caminho de um socket Unix (por exemplo, /run/xray/in.sock) ou o nome de um socket abstrato com o prefixo @ (por exemplo, @xray/in.sock) para escutar em um socket em vez de uma porta TCP – nesse caso, defina a porta como 0».

Assim, o campo aceita duas formas de socket Unix: caminho no sistema de arquivos (`/run/xray/in.sock`) e nome de socket abstrato com prefixo `@` (`@xray/in.sock`). Em ambos os casos, defina `Port` como `0` .

Este campo é alterado quando é necessário restringir o inbound a uma única interface (por exemplo, `127.0.0.1` para um inbound que funciona apenas como destino de fallback atrás do Nginx) ou quando o inbound escuta em um socket Unix.

Exemplo. Inbound que escuta apenas na interface local (destino de fallback típico atrás do Nginx) e socket Unix:

```
listen = 127.0.0.1  porta = 8443
listen = /run/xray/in.sock  porta = 0
```

Port (Porta)

Parâmetro	Valor
Campo	<code>port</code>
Rótulo	«Porta»
Validação	<code>gte=0,lte=65535</code>
Padrão	— (definido pelo usuário)

Porta TCP/UDP de escuta. Valores permitidos de `0` a `65535` . O valor `0` é usado apenas em combinação com a escuta em socket Unix (veja acima).

Ao salvar, o serviço verifica conflito de porta: dois inbounds não podem ocupar simultaneamente o mesmo `listen:port` para o mesmo transporte (TCP/UDP). O transporte é determinado a partir do protocolo e do `streamSettings / settings` : por exemplo, `hysteria` e `wireguard` sempre ocupam UDP, `kcp / quic` — UDP, e a maioria dos demais — TCP. Em caso de conflito, o salvamento é rejeitado com um erro.

Além disso, o painel não permite ocupar a **porta reservada da API interna do Xray** (tag `api` , padrão `62789` em `127.0.0.1`): um inbound TCP local cujo endereço de escuta coincida com essa porta no loopback é rejeitado com o mesmo erro de conflito de porta. A porta real da API é lida a partir do modelo de configuração do Xray (com valor de fallback `62789`). Em nós (nodes), essa restrição não se aplica — eles têm seu próprio Xray.

A tag Xray (Tag , única) é gerada automaticamente a partir da porta e do transporte no formato `in-<porta>-<tcp|udp|tcpudp|any>` ; para um inbound implantado em um nó, é adicionado o prefixo `n<nodeId>-` . Em caso de colisão, são acrescentados `-2` , `-3` etc. ao final da tag. Normalmente o usuário não edita a tag.

Total traffic (Total de tráfego, GB)

Parâmetro	Valor
Campo	<code>total</code> (em bytes)

Parâmetro	Valor
Rótulo	«Consumo total»
Padrão	0

Limite total de tráfego do inbound. No formulário, o valor é inserido em gigabytes; no banco de dados, é armazenado em bytes. Dica do campo:

«= Sem limite. (unidade: GB)».

Ou seja, **0 significa sem limite**. Este é o limite no nível do inbound inteiro (e não de clientes individuais); o tráfego realmente consumido é armazenado nos campos `up` (enviado) e `down` (recebido) e comparado com `total`.

Expiry date / Duration (Data de expiração / prazo)

Parâmetro	Valor
Campo	<code>expiryTime</code> (timestamp Unix)
Rótulo	«Data de expiração» (ing. <i>Duration</i>)
Padrão	vazio / 0

Período de validade do inbound. Dica:

«Deixe vazio para que seja ilimitado».

O valor vazio (0) significa um inbound sem prazo de expiração. O valor é armazenado como timestamp Unix; o formulário permite definir tanto uma data específica quanto um prazo em dias (contagem relativa a partir do momento atual – rótulo em inglês *Duration*).

Enabled (Ativar)

Parâmetro	Valor
Campo	<code>enable</code>
Rótulo	«Ativar» (ing. <i>Enabled</i>)
Padrão	definido na criação

Indicador de atividade do inbound. A alternância desse sinalizador na lista é tratada por um endpoint «leve» separado `POST /setEnable/:id`, e não por uma atualização completa – isso foi feito propositalmente para evitar a re-serialização de todo o bloco `settings` (de todos os clientes) a cada clique no alternador de um inbound com milhares de clientes. Ao desativar, o inbound é removido do Xray em execução; ao ativar, é adicionado de volta.

Node / Deploy to (Nó / Implantar em)

Parâmetro	Valor
Campo	nodeId
Rótulo	«Implantar em», «Painel local»
Padrão	vazio (painel local)

Seleção de onde o inbound opera fisicamente: no painel local ou em um dos nós registrados. A partir da 3.5.0, a lista «Implantar em» (assim como o filtro «Nós» acima da lista de inbound) suporta **pesquisa por digitação**, e acima da própria lista de inbound apareceu um **campo de pesquisa** (por observação, porta e protocolo; funciona em conjunto com o filtro de nós). Detalhe de implementação: `nodeId = 0` é normalizado para `nil`, pois `0` não é um id de nó válido, mas sim um artefato do binding do formulário; `nil / 0` significa painel local. Ao salvar um inbound em um nó offline, pode aparecer um toast «a alteração será sincronizada quando o nó se reconectar». A partir da versão 3.4.2, esse toast só é exibido quando o nó está realmente offline ou desligado (antes, ele podia aparecer também ao salvar em um nó online).

Estratégia de endereço para links (Share address strategy)

Parâmetro	Valor
Campo	estratégia + (opcionalmente) endereço personalizado
Rótulo	«Estratégia de endereço para links» (ing. <i>Share address strategy</i>)
Padrão	«Endereço de escuta do inbound» (<i>Inbound listen</i>)

A lista suspensa determina qual endereço é inserido nos **links de compartilhamento e QR codes exportados** deste inbound. Valores:

Valor	Rótulo	O que é inserido
node	«Endereço do nó» (<i>Node address</i>)	endereço do nó no qual o inbound opera
listen	«Endereço de escuta do inbound» (<i>Inbound listen</i>)	endereço de escuta do próprio inbound
custom	«Personalizado» (<i>Custom</i>)	endereço próprio do campo «Endereço de compartilhamento personalizado» (<i>Custom share address</i>)

Ao selecionar «**Personalizado**», aparece o campo «**Endereço de compartilhamento personalizado**»; nele é inserido um host ou IP **sem esquema e sem porta** (o valor é validado). A opção «**Endereço do nó**» é exibida na lista apenas se existir um nó ativo no qual este inbound possa operar; caso contrário, fica oculta e o valor é revertido para «**Endereço de escuta do inbound**».

Essa estratégia afeta **apenas** os links de compartilhamento diretos e os QR codes. Ela **não** afeta a geração de assinaturas — lá o endereço ainda é determinado pela lógica habitual do painel.

4.2. Sniffing (Sniffing)

A aba «**Sniffing**» edita o bloco `sniffing` da configuração do Xray, que é armazenado como JSON bruto. O Sniffing permite que o Xray «inspecione» o nome de domínio/protocolo real dentro de uma conexão para fins de roteamento.

Subcampo	Rótulo	Finalidade
<code>enabled</code>	(alternador da aba)	Ativa/desativa o sniffing para o inbound
<code>destOverride</code>	–	Lista de protocolos para os quais o endereço de destino é interceptado: <code>http</code> , <code>tls</code> , <code>quic</code> , <code>fakedns</code>
<code>metadataOnly</code>	«Somente metadados»	Usar apenas metadados da conexão, sem leitura do payload
<code>routeOnly</code>	«Somente roteamento»	Aplicar o resultado do sniffing apenas para roteamento, sem reescrever o endereço de destino
<code>domainsExcluded</code>	«Domínios excluídos»	Domínios excluídos do sniffing
(IPs excluídos)	«IPs excluídos»	Endereços IP excluídos do sniffing

- **`destOverride`** – conjunto de sniffers: `http` (determina o domínio a partir do cabeçalho HTTP Host), `tls` (a partir do SNI), `quic` (a partir do QUIC ClientHello), `fakedns` (correspondência com o pool FakeDNS). Normalmente, `http` e `tls` são ativados para determinar o domínio.

Exemplo do bloco `sniffing` (determinar domínio por HTTP e TLS, usar o resultado apenas para roteamento, sem tocar na rede local):

```
{
  "enabled": true,
  "destOverride": ["http", "tls"],
  "routeOnly": true,
  "domainsExcluded": ["courier.push.apple.com"]
}
```

- **`metadataOnly`** – quando ativado, o Xray não lê o conteúdo do primeiro pacote e se baseia apenas nos metadados; útil para não interferir em protocolos cujos dados não devem ser «inspecionados».
- **`routeOnly`** – o resultado do sniffing é usado apenas pelas regras de roteamento; o endereço da conexão no outbound não é reescrito para o domínio identificado.

Observação: o painel armazena o `sniffing` como um bloco JSON opaco e não acrescenta nada a ele ao salvar – todos os valores padrão para essas caixas de seleção são formados no lado do aplicativo cliente. Em sua forma bruta, o bloco pode ser editado pela seção «JSON da entrada» (veja abaixo).

4.3. Allocate (estratégia de alocação de portas)

O bloco `allocate` em `streamSettings` controla como o Xray distribui as portas de escuta. Faz parte da configuração do Xray; o painel o armazena e transmite como parte do `streamSettings` /JSON do inbound. Parâmetros (conforme a terminologia do Xray-core):

Subcampo	Finalidade	Valores / padrão
<code>strategy</code>	Estratégia de alocação de portas	<code>always</code> – sempre escutar na porta definida (padrão); <code>random</code> – alterar periodicamente as portas de escuta dentro de um intervalo
<code>refresh</code>	Intervalo de troca de portas (minutos) com <code>random</code>	número inteiro de minutos (recomendado 5; mínimo 2)
<code>concurrency</code>	Quantas portas manter abertas simultaneamente com <code>random</code>	inteiro (padrão 3; no máximo um terço da largura do intervalo de portas)

`strategy: always` mantém o inbound em uma porta fixa (modo padrão). `strategy: random` é usado em cenários anti-bloqueio, quando o inbound «salta» periodicamente entre as portas de um intervalo; nesse caso, `refresh` e `concurrency` fazem sentido. Esses valores devem ser alterados apenas ao usar conscientemente o modo de portas aleatórias.

Exemplo do bloco `allocate` em `streamSettings` (modo de portas aleatórias: manter 3 portas abertas, trocar a cada 5 minutos):

```
{
  "allocate": {
    "strategy": "random",
    "refresh": 5,
    "concurrency": 3
  }
}
```

Para que isso funcione, a `port` do inbound deve ser definida como um intervalo (por exemplo, `20000–20100`).

4.4. External Proxy (Proxy externo)

O campo «**External Proxy**» pertence às configurações de geração de links de convite e é armazenado em `streamSettings` do inbound. Ele define uma lista de endereços externos alternativos (host/porta, opcionalmente com TLS forçado – «**TLS forçado**») que são inseridos nos links dos clientes em vez do `listen:port` real do inbound.

É usado quando os clientes devem se conectar não diretamente ao servidor, mas por meio de um proxy externo/reverso/CDN: nesse caso, os links compartilhados especificam o endereço público desse frontend. Isso não afeta o processo de recepção de conexões pelo Xray – é apenas «cosmético» nos links gerados. Campos relacionados no formulário: «**TLS forçado**», «**Fingerprint**», rótulos de cada registro.

4.5. Fallbacks (Fallbacks)

A seção «**Fallbacks**» define as regras de redirecionamento para conexões que não correspondem a nenhum cliente do inbound. Está disponível para o inbound master com transporte TLS (VLESS/Trojan TCP-TLS). É gerenciado pelos endpoints `GET /:id/fallbacks` / `POST /:id/fallbacks`.

Dica da seção:

«Quando uma conexão neste inbound não corresponde a nenhum cliente, ela é redirecionada para outro destino. Selecione um inbound filho abaixo para que os campos de roteamento (SNI / ALPN / Path / xver) sejam preenchidos automaticamente a partir do seu transporte, ou deixe a seleção vazia e defina Dest diretamente (por exemplo, 8080 ou 127.0.0.1:8080) para redirecionar para um servidor externo, como o Nginx. Cada inbound filho deve escutar em 127.0.0.1 com security=none».

A seção de fallbacks é exibida apenas para inbounds VLESS/Trojan sobre RAW (TCP) com segurança TLS ou REALITY. Um novo inbound começa com `security=none`, portanto a seção pode parecer ausente no início. Nesse estado (VLESS/Trojan, RAW/TCP, segurança ainda não configurada), em vez da seção é exibida uma dica integrada: os fallbacks estarão disponíveis após selecionar TLS ou Reality na aba «**Segurança**».

Campos de uma linha de fallback

Campo	Padrão	Descrição
(inbound filho)	—	Seleção do inbound filho (rótulo « Selecionar inbound »). Se selecionado, os campos Name/Alpn/Path/Dest podem ser preenchidos automaticamente a partir do seu transporte
Name	vazio (= qualquer)	Condição de correspondência por nome (SNI/nome). Rótulo «qualquer» — « qualquer »
Alpn	vazio	Condição de correspondência por ALPN
Path	vazio	Condição de correspondência por caminho (para transportes WS/HTTP do inbound filho)
Dest	automático	Para onde redirecionar. Placeholder « automático (listen:porta do filho) ». Pode ser uma porta (8080) ou <code>host:port</code> (127.0.0.1:8080)
Xver	0	Versão do protocolo PROXY (« Xver »): 0 — desativado, 1 ou 2 — versão correspondente do PROXY protocol
(ordem)	por posição	Ordem de aplicação das regras; definida pelos botões « Acima »/« Abaixo »

Lógica de salvamento: toda a lista de fallbacks do master é substituída atômica. Uma linha que não tem nem um inbound filho selecionado (`childId <= 0`) nem um `Dest` definido é **ignorada**. Se o inbound filho selecionado for igual ao id do próprio master, ele é zerado. Na geração do JSON final: se `Dest` estiver vazio, ele é calculado a partir do inbound filho como `listen:port`, onde `0.0.0.0 / :: / ::0` são substituídos por `127.0.0.1`; campos vazios `name / alpn / path` não são incluídos no JSON de saída; `xver` é adicionado apenas se for maior que 0.

Exemplo do `settings.fallbacks` final (tráfego com `alpn=h2` vai para o destino WS pelo caminho `/ws`, todo o restante vai para o Nginx local na porta 8080):

```
{
  "fallbacks": [
    { "alpn": "h2", "path": "/ws", "dest": "127.0.0.1:2001", "xver": 1 },
    { "dest": 8080 }
  ]
}
```

A última linha sem `name / alpn / path` é a regra «padrão», que captura todo o restante.

Botões e dicas da seção fallbacks

- «Adicionar fallback» – adiciona uma linha; «Sem fallbacks ainda» – estado vazio.
- «Adicionar rapidamente todos os adequados» / «Adicionar todos» – adiciona uma linha de fallback para cada inbound adequado ainda não conectado. Resultado: «Adicionado(s) {n} fallback(s)» ou «Nenhum inbound adequado novo».
- «Preencher a partir do filho» – reaplicar os campos de roteamento (SNI/ALPN/Path/xver) a partir do transporte do inbound filho selecionado; após a execução – «Preenchido a partir do filho».
- «Modificar campos de roteamento» / «Ocultar avançados» – mostrar/ocultar campos detalhados da linha.
- Os rótulos «Roteia quando» e «Padrão – captura todo o restante» explicam a condição de ativação de cada linha.

Após salvar os fallbacks, o servidor chama a reinicialização do Xray para que os novos `settings.fallbacks` entrem em vigor.

4.6. Redefinição periódica de tráfego

O bloco «Redefinição de tráfego» configura a redefinição automática dos contadores de tráfego do inbound por agendamento. Descrição:

«Redefinição automática do contador de tráfego nos intervalos especificados».

Parâmetro	Valor
Campo	<code>trafficReset</code>
Validação	<code>omitempty,oneof=never hourly daily weekly monthly</code>
Padrão	<code>never</code>
Campo associado	<code>lastTrafficResetTime</code> – timestamp da última redefinição (rótulo «Última redefinição»)

Lista suspensa:

Valor	Rótulo
<code>never</code>	«Nunca»
<code>hourly</code>	«A cada hora»
<code>daily</code>	«Diariamente»

Valor	Rótulo
weekly	«Semanalmente»
monthly	«Mensalmente»

Para cada período, está registrado um cron job que é executado no agendamento correspondente (@hourly , @daily , @weekly , @monthly). O job seleciona todos os inbounds com o trafficReset definido e, para cada um, redefine os contadores do próprio inbound (up=0 , down=0) e o tráfego de todos os seus clientes. Ou seja, a redefinição periódica afeta tanto o inbound quanto seus clientes.

Exemplo do valor do campo. Para que os contadores sejam zerados no primeiro dia de cada mês, seleciona-se «Mensalmente» no formulário, o que é salvo como:

```
{ "trafficReset": "monthly" }
```

O valor never (padrão) desativa completamente a redefinição automática.

4.7. JSON da entrada (avançado)

A seção «Seções JSON da entrada» fornece acesso direto aos blocos JSON brutos do inbound. Descrição:

«JSON completo da entrada e editores separados para settings, sniffing e streamSettings».

Editores disponíveis:

Aba	Rótulo	O que edita
Tudo	«Objeto completo da entrada com todos os campos em um único editor»	o objeto Inbound inteiro
Configurações	«Wrapper do bloco settings do Xray»	campo settings
Sniffing	«Wrapper do bloco sniffing do Xray»	campo sniffing
Stream	«Wrapper do bloco stream do Xray»	campo streamSettings

Esses campos são serializados como objetos JSON aninhados: blocos vazios são retornados como null , e um texto que não é JSON válido é encapsulado em uma string para que os dados não sejam perdidos. Erros de parse ao salvar são exibidos com o prefixo «JSON avançado».

A janela de visualização «JSON da entrada», assim como a janela de importação de inbound, usa um editor de código completo com realce de sintaxe JSON (em vez de um campo de texto comum): a visualização de configuração – no modo somente leitura com realce, e a importação – no modo editável, o que facilita a leitura e edição.

4.8. Ações com inbound: QR / Edit / Reset / Delete e estatísticas

Na lista e no cartão do inbound estão disponíveis as seguintes ações (menu «Menu»):

Estatísticas de tráfego

É exibido o tráfego agregado do inbound: «Enviado/recebido» (campos `up` / `down`), «Total de tráfego», «Total de conexões». No cartão também – «Criado», «Atualizado», «Data de expiração».

Na lista de inbounds há uma coluna **Speed** com a velocidade atual do tráfego por inbound (upload/download), calculada a partir dos incrementos dos contadores entre pesquisas; a mesma velocidade ao vivo é exibida na janela de estatísticas do inbound. Quando a próxima pesquisa não produz incremento, o valor da velocidade é zerado.

No resumo de clientes na página de inbounds, o status é determinado pela prioridade «esgotado/encerrado»: clientes cujo prazo expirou ou cujo tráfego foi esgotado (e nos quais a tarefa automática removeu o `enable`) pertencem ao status «Esgotado/Encerrado» (*Depleted/Ended*), e não ao cinza «Desativado» (*Disabled*), e não são contados duas vezes. A classificação coincide com a exibida no cartão do próprio cliente e contabiliza corretamente os clientes vinculados a múltiplos inbounds.

QR code e cópia de links

- «Detalhes» – expande os links de conexão e de assinatura.
- QR code do cliente: dica «Clique no QR code para copiar».
- «Copiar link» (ing. *Copy URL*), «Exportar links». A partir da versão 3.4.2, a ação de página «Exportar todos os links» (`GET /panel/api/inbounds/allLinks` ; janela «Exportar todos os links do inbound», arquivo «All-Inbounds») gera os links pelo motor de assinaturas – aplica o modelo de remark a cada cliente e prefere os endpoints Host gerenciados (antes era usado um remark fixo `inbound-email`).

Edit (Modificar)

«Modificar conexão» – abre o formulário de edição (`POST /update/:id`). Ao atualizar, o serviço relê a linha existente, transfere os campos alterados, regenera a tag se necessário (caso a tag anterior tenha sido gerada automaticamente) e sincroniza o runtime do Xray. Sucesso – toast «Conexão atualizada com sucesso».

Reset Traffic (Redefinir tráfego)

«Redefinir tráfego» – zera os contadores `up` / `down` especificamente deste inbound (`POST /:id/resetTraffic` , define `up=0` , `down=0`). Confirmação:

«Redefinir o tráfego de "{remark}"?» / «Redefine os contadores de envio/recebimento desta conexão para 0».

A redefinição de tráfego do inbound **não** afeta os contadores de seus clientes (para eles existem ações separadas «Redefinir tráfego dos clientes»). Após a redefinição, é iniciada a reinicialização do Xray. Sucesso – toast «Tráfego de entrada redefinido». Existe também a variante em massa – «Redefinir tráfego de todas as conexões» (`POST /resetAllTraffics`).

Delete (Excluir)

«Excluir conexão» (`POST /del/:id`). Confirmação:

«Excluir conexão "{remark}"?» / «A conexão e todos os seus clientes serão excluídos. Esta ação não pode ser desfeita».

A exclusão remove o inbound do Xray em execução (com reinicialização se necessário). Sucesso – toast **«Conexão excluída com sucesso»**. Exclusão em massa – `POST /bulkDel` , com relatório por elemento e no máximo uma reinicialização do Xray.

Outras ações com clientes do inbound

No menu também estão disponíveis: **«Clonar»** (cópia do inbound com nova porta e lista de clientes vazia), **«Excluir todos os clientes»** (`POST /:id/deleteAllClients` – exclui todos os clientes, o próprio inbound é mantido), **«Excluir clientes desativados»**, **«Vincular/Desvincular clientes»**, **«Importar»/«Exportar conexões»** (`POST /import`). Os detalhes das operações com clientes pertencem à seção sobre clientes.

5. Protocolos

Ao criar uma conexão de entrada (inbound), o primeiro campo a ser escolhido é o **Protocolo** («Protocol»). O protocolo define qual método de autenticação e criptografia de tráfego o Xray-core aplicará a esse inbound, quais campos em `settings` precisarão ser preenchidos, e quais transportes (`network`) e tipos de segurança (TLS / REALITY) estão disponíveis para ele.

O campo de protocolo é definido uma única vez na criação do inbound e **não pode ser alterado na edição** (na tela de edição, o menu suspenso fica bloqueado). Para mudar de protocolo, é necessário criar um novo inbound.

5.1. Lista de protocolos suportados

O servidor aceita os seguintes valores para o campo `Protocol` :

```
oneof=vmess vless trojan shadowsocks wireguard hysteria http mixed tunnel tun mtproto
```

A partir da versão **3.3.0**, o valor `mtproto` (proxy do Telegram) foi adicionado à lista.

Valor no config	Finalidade	Modelo de cliente
<code>vless</code>	Protocolo proxy principal (padrão ao criar inbound)	Clientes com UUID, suporte a flow e criptografia pós-quântica
<code>vmess</code>	Protocolo proxy clássico do Xray	Clientes com UUID e parâmetro <code>security</code>
<code>trojan</code>	Proxy que se disfarça de HTTPS comum	Clientes com senha
<code>shadowsocks</code>	Proxy Shadowsocks (incluindo SIP022 / 2022-blake3)	Um ou vários usuários (2022)
<code>wireguard</code>	Inbound WireGuard	Peers (e não clientes)
<code>hysteria</code>	Inbound Hysteria (padrão versão 2)	Clientes com token <code>auth</code>
<code>http</code>	Proxy HTTP clássico (forward proxy)	Contas user/pass, sem contabilização de tráfego
<code>mixed</code>	Proxy SOCKS + HTTP combinado	Contas user/pass
<code>tunnel</code>	Encaminhador transparente (xray <code>dokodemo-door</code>)	Sem clientes
<code>tun</code>	Interface TUN (apenas renderização de existentes)	Sem clientes
<code>mtproto</code>	Proxy do Telegram (MTProto), adicionado na 3.3.0; gerenciado por um processo separado <code>mtg</code> , não pelo Xray	Sem clientes (acesso por segredo)

Observação sobre `tun` : o valor foi mantido na lista por compatibilidade e para **exibição** de inbounds salvos anteriormente, mas na versão atual do backend sua criação não é recomendada — o suporte foi marcado como obsoleto. Criar novos inbounds desse tipo não faz sentido.

Observação sobre Hysteria 2: não existe um protocolo separado «hysteria2». É o protocolo `hysteria` com o campo `streamSettings.version = 2`. O esquema de link `hysteria2://` é escolhido automaticamente ao gerar links de compartilhamento quando a versão do stream é igual a 2.

Nem todos os protocolos suportam distribuição por nós (nodes). Apenas os seguintes podem ser implantados em nós: `vless`, `vmess`, `trojan`, `shadowsocks`, `hysteria`, `wireguard`. Os protocolos `http`, `mixed`, `tunnel`, `tun`, `mtproto` funcionam apenas no painel local.

5.2. Quais protocolos suportam TLS / REALITY / transporte

A possibilidade de habilitar uma determinada camada de segurança ou transporte depende do protocolo e da rede selecionada (`streamSettings.network`):

Recurso	Disponível para protocolos	Redes permitidas (<code>network</code>)
TLS	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> (e sempre para <code>hysteria</code>)	<code>tcp</code> , <code>ws</code> , <code>http</code> , <code>grpc</code> , <code>httpupgrade</code> , <code>xhttp</code>
REALITY	<code>vless</code> , <code>trojan</code>	<code>tcp</code> , <code>http</code> , <code>grpc</code> , <code>xhttp</code>
flow (<code>xtls-rprx-vision</code>)	apenas <code>vless</code>	apenas <code>tcp</code> , com <code>security = tls</code> ou <code>reality</code>
Stream / transporte (aba «Fluxo»)	<code>vmess</code> , <code>vless</code> , <code>trojan</code> , <code>shadowsocks</code> , <code>hysteria</code>	—

Para os protocolos `http`, `mixed`, `tunnel`, `tun`, `wireguard` a aba de transporte não está disponível – eles não possuem configurações de stream do Xray.

5.3. VLESS

Finalidade: protocolo proxy moderno principal. Suporta XTLS-Vision (`flow`), REALITY e criptografia pós-quântica no nível do próprio VLESS (campos `decryption` / `encryption`). Usado por padrão para novos inbounds.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>clients</code>	<code>[]</code>	Lista de clientes. Cada um possui: <code>id</code> (UUID), <code>email</code> (obrigatório), <code>flow</code> , limites (<code>limitIp</code> , <code>totalGB</code> , <code>expiryTime</code>), <code>enable</code> , <code>tgId</code> , <code>subId</code> , <code>comment</code> , <code>reset</code>
<code>decryption</code>	<code>none</code>	Parâmetro de decriptografia no lado do servidor. Rótulo no UI: «Decriptografia» (em inglês «Decryption»)
<code>encryption</code>	<code>none</code>	Parâmetro de criptografia correspondente (incluído no link do cliente). Rótulo: «Criptografia» (em inglês «Encryption»)

Campo	Valor padrão	Descrição
fallbacks	[]	Lista de fallbacks (consulte a seção sobre fallbacks); disponível quando <code>network = tcp</code> e <code>security = TLS</code> ou <code>REALITY</code>
testseed	(4 números: 900, 500, 900, 256)	«Vision testseed» – 4 inteiros positivos para padding do XTLS-Vision. Aplicado apenas aos clientes com flow <code>xtls-rprx-vision</code> ; caso contrário, ignorado

flow (xtls-rprx-vision)

O `flow` é definido **no cliente**, não no inbound, e aceita um dos três valores:

Valor	Significado
`` (vazio)	Sem XTLS-flow (padrão)
xtls-rprx-vision	XTLS-Vision – modo recomendado para VLESS sobre TCP+TLS/REALITY
xtls-rprx-vision-udp443	O mesmo Vision, mas com tratamento de UDP/443 (QUIC)

O campo `flow` fica disponível para seleção apenas quando todas as condições são atendidas: protocolo `vless`, `network = tcp` e `security = tls` ou `reality`. O campo **Vision testseed** no formulário é exibido apenas nas mesmas condições.

Exceção para XHTTP: com VLESS sobre `network = xhttp` com autenticação pós-quântica VLESS habilitada (`encryption / decryption`, `vlessenc`), o flow `xtls-rprx-vision` também é permitido – independentemente da camada de segurança, inclusive com `REALITY`. Nesse caso, o painel transmite corretamente `xtls-rprx-vision` nos links de compartilhamento e nas assinaturas (incluindo o formato Clash/Mihomo), de modo que o cliente recebe a configuração com o Vision.

Decriptografia / Criptografia (autenticação pós-quântica VLESS)

Os campos `decryption` e `encryption` são autenticação no nível do próprio VLESS (separadamente do TLS/REALITY de transporte). Por padrão, ambos são `none`. No formulário, abaixo desses campos está o bloco «**Geração de chaves**» – um menu suspenso de modo e um botão «**Gerar**» (ao lado – botão «**Limpar**»). O menu suspenso contém seis opções: **X25519 (native)**, **X25519 (xorpub)**, **X25519 (random)**, **ML-KEM-768 (native)**, **ML-KEM-768 (xorpub)**, **ML-KEM-768 (random)** – ou seja, dois tipos de chave (clássica X25519 e pós-quântica ML-KEM-768), cada uma em três modos:

- **native** – par de chaves base do tipo selecionado;
- **xorpub** – modo derivado com processamento adicional da parte pública;
- **random** – modo derivado com componente aleatório.

Selecione o modo desejado no menu e clique em «**Gerar**»: o painel preencherá **ambos** os campos (`decryption` e `encryption`) com o par de valores pronto para esse modo. O botão «**Limpar**» redefine os dois campos para `none`.

Abaixo do bloco é exibida a linha de status «**Selecionado: ...**», que reconhece a partir da string gerada tanto o tipo de chave (X25519 ou ML-KEM-768) quanto o modo (native / xorpub / random) e os exibe. Campos vazios ou `none` são exibidos como «None».

Tecnicamente, os botões fazem chamadas a `GET /panel/api/server/getNewVlessEnc` (geração de chaves via `xray vlessenc`) e preenchem **ambos** os campos com valores pareados no formato

`mlkem768x25519plus.native.<rtt>.<role>` (por exemplo, `decryption = mlkem768x25519plus.native.600s.server-x25519`, `encryption = mlkem768x25519plus.native.0rtt.client-x25519`). O parâmetro `decryption` permanece no servidor, e `encryption` vai para o link do cliente.

Importante: ao gerar a configuração do inbound para o Xray, o painel remove o excedente: se `encryption` permanecer em `settings` (que pertence ao lado do cliente), ele **é removido** da configuração do servidor. No servidor fica apenas `decryption`.

Quando escolher VLESS: é a opção padrão recomendada para um novo inbound, especialmente em combinação com REALITY (sem certificado próprio) ou com TLS + XTLS-Vision.

Exemplo: bloco `settings` do VLESS-inbound com um cliente e XTLS-Vision. O campo `flow` está no cliente, `decryption` permanece no servidor:

```
{
  "clients": [
    {
      "id": "d342d11e-d424-4583-b36e-524ab1f0afa4",
      "email": "user1",
      "flow": "xtls-rprx-vision",
      "limitIp": 2,
      "totalGB": 0,
      "expiryTime": 0,
      "enable": true
    }
  ],
  "decryption": "none"
}
```

Para a combinação com REALITY, o bloco `streamSettings` correspondente (aba «Transport» → Security: REALITY) fica assim:

```
{
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "dest": "www.microsoft.com:443",
    "serverNames": ["www.microsoft.com"],
    "privateKey": "<chave privada X25519>",
    "shortIds": ["", "6ba85179e30d4fc2"]
  }
}
```

5.4. VMess

Finalidade: protocolo proxy clássico do Xray. Autenticação por UUID; no cliente, o método de criptografia da carga útil (`security`) é configurado adicionalmente.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>clients</code>	<code>[]</code>	Lista de clientes

Cada cliente VMess (além dos campos comuns `email` , `limites` , `enable` , `tgId` , `subId` , `comment` , `reset`) possui:

Campo do cliente	Valor padrão	Descrição
<code>id</code>	—	UUID do cliente
<code>security</code>	<code>auto</code>	Método de criptografia da carga útil VMess. Valores permitidos: <code>aes-128-gcm</code> , <code>chacha20-poly1305</code> , <code>auto</code> , <code>none</code> , <code>zero</code>

Valores de `security` :

- `auto` — o Xray escolhe o cifrado automaticamente dependendo da plataforma (recomendado);
- `aes-128-gcm` , `chacha20-poly1305` — cifras AEAD fixas;
- `none` — sem criptografia da carga útil (faz sentido apenas sobre TLS);
- `zero` — sem criptografia e sem autenticação da carga útil.

Compatibilidade histórica: registros antigos podiam armazenar `security: ""` — ao ler, a string vazia é convertida para `auto` . Ao gerar a configuração do servidor, o campo `security` dos clientes VMess é **removido** de `settings` , pois não é necessário para o inbound.

Quando escolher VMess: para compatibilidade com clientes antigos ou configurações existentes. Para novos deployments, geralmente o VLESS é preferível.

5.5. Trojan

Finalidade: proxy que imita tráfego HTTPS comum. Autenticação por senha. Como o VLESS, suporta fallbacks e (com `network = tcp`) REALITY/TLS.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>clients</code>	<code>[]</code>	Lista de clientes
<code>fallbacks</code>	<code>[]</code>	Lista de fallbacks (disponível com <code>network = tcp</code> e TLS/REALITY)

O campo principal de cada cliente Trojan:

Campo do cliente	Valor padrão	Descrição
password	–	Senha do cliente (obrigatória, mínimo 1 caractere)
email	–	Identificador único do cliente

Os demais campos do cliente são comuns (`limitIp`, `totalGB`, `expiryTime`, `enable`, `tgId`, `subId`, `comment`, `reset`).

Quando escolher Trojan: quando é necessário disfarce de HTTPS na porta 443, incluindo com fallbacks para um servidor web (Nginx) para conexões não solicitadas.

Exemplo: bloco settings do Trojan com fallback para servidor web local. Conexões não solicitadas (sem senha válida) são direcionadas ao Nginx, que escuta em `127.0.0.1:8080` :

```
{
  "clients": [
    { "password": "S3cret-Pass-1", "email": "user1" }
  ],
  "fallbacks": [
    { "dest": "127.0.0.1:8080" }
  ]
}
```

Para o fallback são necessários `network = tcp` e `Security = TLS` ou `REALITY`; caso contrário, o campo `fallbacks` não está disponível.

5.6. Shadowsocks

Finalidade: proxy Shadowsocks leve. Suporta tanto cifras AEAD legadas quanto os novos métodos SIP022 (`2022-blake3-*`). Pode funcionar em modo de usuário único ou multi-usuário.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
method	<code>2022-blake3-aes-256-gcm</code>	Método de criptografia do inbound. Rótulo no UI: «Método de criptografia» (em inglês «Encryption method»)
password	``	Senha do inbound (para métodos 2022, gerada automaticamente com o comprimento adequado ao método selecionado)
network	<code>tcp,udp</code>	Transporte. Rótulo: «Rede» (em inglês «Network»). Opções: <code>tcp,udp</code> (TCP, UDP), <code>tcp</code> , <code>udp</code>
clients	<code>[]</code>	Lista de clientes
ivCheck	<code>false</code> (desligado)	Chave «ivCheck» – proteção contra reutilização de IV

Métodos de criptografia (method)

Conjunto permitido:

Método	Categoria
aes-256-gcm	AEAD legado
chacha20-poly1305	AEAD legado
chacha20-ietf-poly1305	AEAD legado
xchacha20-ietf-poly1305	AEAD legado
2022-blake3-aes-128-gcm	SS 2022 (recomendado)
2022-blake3-aes-256-gcm	SS 2022 (padrão)
2022-blake3-chacha20-poly1305	SS 2022, usuário único

Lógica do painel em relação aos métodos:

- **Métodos 2022** (2022-blake3-*) são considerados «SS 2022». O método 2022-blake3-chacha20-poly1305 é **de usuário único** (multi-usuário não é suportado); os demais métodos 2022 permitem vários clientes. O campo de senha (com botão de geração que ajusta o comprimento da chave ao método) é exibido no formulário especificamente para métodos 2022.
- **Cifras legadas** (aes-* , chacha20-*) funcionam pelo esquema clássico «um método + uma senha».

Normalização antes de iniciar o Xray: para cifras legadas, cada cliente deve ter o `method` coincidindo com o método do inbound (caso contrário, o Xray falha com «unsupported cipher method:»). Para métodos 2022, ao contrário — o campo `method` do cliente deve estar **vazio** (caso contrário, o Xray rejeita o inbound com «users must have empty method»). O painel ajusta os dados automaticamente ao trocar o método.

Regeneração de chaves de cliente ao mudar o tamanho da chave: para Shadowsocks-2022, ao mudar o método de criptografia para um método com tamanho de chave diferente (por exemplo, entre 2022-blake3-aes-256-gcm e 2022-blake3-aes-128-gcm), o painel regenera automaticamente as PSK dos clientes para o novo comprimento ao salvar o inbound. Caso contrário, as chaves antigas permaneceriam com o comprimento anterior e o Xray as rejeitaria. Consequência: os clientes afetados precisam obter a assinatura novamente — os links anteriores deixarão de funcionar.

Quando escolher Shadowsocks: para deployments simples sem disfarce TLS; a escolha moderna são os métodos 2022-blake3.

Exemplo: bloco settings do Shadowsocks para método 2022-blake3 (modo multi-usuário). O inbound tem sua própria senha (chave base64 do comprimento necessário), cada cliente tem sua própria senha, e o campo `method` do cliente está **vazio**:

```
{
  "method": "2022-blake3-aes-256-gcm",
  "password": "d2hhdGV2ZXItMzItYnl0ZS1iYXNlNjQta2V5LWhlcU=",
  "network": "tcp,udp",
  "clients": [
    {
      "email": "user1",
      "password": "Y2xpZW50LWtleS0zMilieXRlcy1iYXNlNjQtaGVyZQ==",
      "method": ""
    }
  ]
}
```

Para cifras legadas (`aes-256-gcm` etc.) – ao contrário: a senha é única por inbound, e o `method` do cliente deve coincidir com o método do inbound.

5.7. Dokodemo-door / Tunnel (encaminhador transparente)

Finalidade: encaminhador transparente (no painel – protocolo `tunnel`, que implementa o comportamento `dokodemo-door`). Recebe tráfego e o redireciona para um endereço/porta especificado, sem autenticação e sem clientes.

Campos do bloco `settings`:

Campo	Valor padrão	Descrição
<code>rewriteAddress</code>	(nenhum)	«Reescrever endereço» (em inglês «Rewrite address») – endereço de destino para o qual o tráfego é redirecionado
<code>rewritePort</code>	(nenhum)	«Reescrever porta» (em inglês «Rewrite port») – porta de destino (0–65535)
<code>allowedNetwork</code>	<code>tcp,udp</code>	«Rede permitida» (em inglês «Allowed network»). Opções: <code>tcp,udp</code> , <code>tcp</code> , <code>udp</code>
<code>portMap</code>	<code>{}</code>	«Mapeamento de portas» – mapa porta→porta (Record<string,string>)
<code>followRedirect</code>	<code>false</code> (desligado)	«Seguir redirect» (em inglês «Follow redirect») – usar o endereço de destino original da conexão interceptada

Aba «Transport» para Tunnel: em inbounds desse tipo a aba «**Transport**» está disponível, limitada à configuração `sockopt` – suficiente para o modo **TProxy** (proxy transparente/redirect via `sockopt.tproxy`). O menu suspenso de seleção de transporte (`network`) e a aba «Security» para Tunnel estão ocultos, pois TLS/REALITY não são suportados por esse tipo.

Quando escolher: para proxy transparente/redirecionamento de portas para serviços internos.

O campo «Reescrever porta» (`rewritePort`) pode ser deixado vazio: ao limpar o valor, ele simplesmente é excluído das configurações do inbound, sem causar erro ao salvar. (Anteriormente, limpar esse campo causava um erro de validação `settings.rewritePort` e bloqueava o salvamento, inclusive pela aba JSON.)

5.8. SOCKS / HTTP (protocolo `mixed`)

Nesta compilação não existe um protocolo `socks` separado – SOCKS e proxy HTTP estão unidos no protocolo `mixed` (SOCKS + HTTP combinado). Além disso, existe um proxy `http` puro separado.

5.8.1. Mixed (SOCKS + HTTP)

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>auth</code>	<code>password</code>	«Auth» – modo de autenticação. Opções: <code>password</code> (por login/senha) ou <code>noauth</code> (sem autorização)
<code>accounts</code>	(opcional)	«Contas» – lista de pares user/pass. Com <code>auth = noauth</code> , o campo não é gravado no config
<code>udp</code>	<code>false</code> (desligado)	Chave «UDP» – suporte a UDP via SOCKS
<code>ip</code>	<code>127.0.0.1</code>	«UDP IP» – endereço local para associações UDP. O campo é exibido apenas quando <code>udp</code> está habilitado

As contas são adicionadas pelo botão «Adicionar»; ao adicionar, são gerados login aleatório (8 caracteres) e senha aleatória (12 caracteres), que podem ser editados.

5.8.2. HTTP (proxy puro)

Finalidade: forward proxy HTTP clássico. No nível do Xray, não rastreia clientes como «de faturamento» (sem email/limites) – existe apenas uma lista de contas.

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>accounts</code>	<code>[]</code>	«Contas» – lista de pares user/pass (ambos os campos são obrigatórios)
<code>allowTransparent</code>	<code>false</code> (desligado)	«Permitir transparente» (em inglês «Allow transparent») – encaminhar requisições com o cabeçalho Host original

Quando escolher SOCKS/HTTP: para acesso proxy local ou de serviço sem disfarce complexo. `mixed` é conveniente pois uma única porta atende tanto clientes SOCKS quanto HTTP.

5.9. WireGuard (inbound)

Finalidade: inbound WireGuard – um túnel VPN WireGuard, e não um proxy com disfarce. Transporte e TLS/REALITY não se aplicam a ele.

A partir da versão 3.4.2 o WireGuard foi migrado para o modelo multicliente (como VLESS/VMess/Trojan/Shadowsocks/Hysteria). O próprio inbound armazena apenas a parte do servidor; os dispositivos-peers agora

são adicionados como **clientes** comuns (veja a seção 8) — com atribuição automática de endereço no túnel, suporte a assinaturas, limites de tráfego/prazo e grupos. A antiga lista inline «Peers / Adicionar peer» do formulário do inbound foi removida: um novo inbound é criado sem peers, e cada cliente recebe um endereço automaticamente.

Campos do inbound (bloco `settings`):

Campo	Valor padrão	Descrição
<code>secretKey</code>	—	Chave privada do servidor (obrigatória). Ao lado há botão de geração; a chave pública (<code>Public Key</code>) é derivada dela automaticamente (somente leitura; o <code>pubKey</code> armazenado separadamente foi removido)
<code>mtu</code>	1420	MTU da interface
<code>dns</code>	1.1.1.1, 1.0.0.1	Novo na 3.4.2. DNS gravado na linha <code>DNS = do .conf</code> do cliente
<code>noKernelTun</code>	false (desligado)	«TUN sem kernel» — TUN em userspace em vez do kernel
<code>domainStrategy</code>	(opcional)	«Domain Strategy»: <code>ForceIP</code> , <code>ForceIPv4</code> , <code>ForceIPv4v6</code> , <code>ForceIPv6</code> , <code>ForceIPv6v4</code>

Parâmetros WireGuard do cliente — aba «**Credenciais**» na janela de adição/edição do cliente (exibidos quando o inbound vinculado é WireGuard):

Campo	Descrição
«Chave privada WireGuard»	Chave privada do cliente (editável, há botão «Regenerar»); ao digitar, a chave pública é derivada dela automaticamente
«Chave pública WireGuard»	Somente leitura; calculada a partir da privada
«Chave compartilhada WireGuard» (PSK)	Chave compartilhada adicional opcional
«IPs permitidos WireGuard»	A partir da 3.5.0 — editável (placeholder <code>10.0.0.2/32</code> , dica «Deixe vazio para atribuição automática; separe as entradas por vírgulas»). Vazio — o endereço é atribuído automaticamente; o servidor valida cada entrada (IP ou CIDR), normaliza endereços únicos para <code>/32</code> e rejeita um endereço ocupado por outro cliente deste inbound

O endereço no túnel é atribuído pelo servidor automaticamente a partir da sub-rede do inbound (padrão `10.0.0.0/24` : o servidor ocupa o `.1` , os clientes — a partir do `.2`); se os clientes existentes usam outra `/24` , os novos endereços são atribuídos na mesma sub-rede.

Domain Strategy e aba Transport

Além dos listados, o inbound WireGuard tem o campo **Domain Strategy** (estratégia de resolução de domínios: `ForceIP` , `ForceIPv4` , `ForceIPv4v6` , `ForceIPv6` , `ForceIPv6v4`). O campo é opcional e é gravado no config apenas se definido.

O campo **Workers** (workers , número de threads de trabalho) foi removido dos formulários WireGuard (tanto inbound quanto outbound): a partir do xray-core v26.6.22, o motor não o utiliza mais e se baseia no mecanismo interno do wireguard-go. Configs salvos anteriormente funcionam sem alterações – ao analisar, o campo é simplesmente descartado, sem necessidade de migração.

Para WireGuard também está disponível a aba «**Transport**» – mas em forma reduzida: nela são configurados apenas `sockopt` e a ofuscação **Finalmask**. O menu suspenso de seleção de transporte (network) está oculto, pois WireGuard sempre escuta via UDP. Nos registros de ruído (noise) do Finalmask, um campo separado define o **Rand Range** (faixa de bytes 0–255, com validação), e como método de ofuscação para WireGuard e Hysteria está disponível o **Salamander**.

Ações do inbound e exportação (a partir da 3.5.0)

O menu da linha do inbound WireGuard agora contém o conjunto «multicliente» completo de ações – «**Exportar todos os links**», exportação por assinatura, **vinculação/desvinculação de clientes**, adição a grupo e exclusão de todos os clientes (antes havia apenas exportar/reset/clonar/excluir), e a lista exibe o contador de clientes. Para o WireGuard, a janela «Exportar todos os links» é dividida em duas abas: **Config** (blocos `.conf` concatenados, um por cliente) e **Links** (links pessoais `wireguard://`); «Copiar» e «Baixar» atuam na aba aberta. Para um inbound hospedado em um nó, o `Endpoint` no `.conf` /QR do cliente agora aponta para o **endereço do nó**, e não o do painel principal.

Configuração e compartilhamento do cliente WireGuard

No cliente WireGuard, nas janelas «Informações»/QR, está disponível um **cartão de configuração recolhível**: o `.conf` completo ([Interface] com `PrivateKey / Address / DNS / MTU` e [Peer] com `PublicKey / PresharedKey / AllowedIPs = 0.0.0.0/0, :::/0 / Endpoint / PersistentKeepalive`) com os botões **Copiar**, **Baixar** e **QR**. Também é suportado um link no formato `wireguard:// / wg://`, que o aplicativo cliente interpreta como um `.conf` pronto.

Quando escolher WireGuard: quando é necessário exatamente um túnel VPN WireGuard, e não um proxy com disfarce.

5.10. Hysteria (padrão v2)

Finalidade: inbound Hysteria sobre QUIC. O painel trabalha com a versão 2 por padrão. Cada cliente é autenticado pelo token `auth` em vez de UUID/senha. TLS para Hysteria está sempre disponível (consulte a tabela de recursos em 5.2).

Campos do bloco `settings` :

Campo	Valor padrão	Descrição
<code>version</code>	2	Versão do protocolo (mínimo 1; padrão do painel é 2)
<code>clients</code>	[]	Lista de clientes

O campo principal de cada cliente é `auth` (token, obrigatório), mais os campos comuns (`email` , limites, `enable` , `tgId` , `subId` , `comment` , `reset`).

Parâmetros adicionais são definidos em `streamSettings.hysteriaSettings` :

Campo	Valor / opções	Descrição
<code>version</code>	fixado em 2 (campo bloqueado)	«Versão» (em inglês «Version»)
<code>udpIdleTimeout</code>	(inteiro ≥ 1, seg.)	«UDP idle timeout (s)» – tempo limite de inatividade UDP
<code>masquerade</code>	desligado por padrão	«Masquerade» – disfarce como servidor web comum para requisições «não solicitadas»

Ao habilitar `masquerade` , fica disponível a seleção do tipo (`type`):

- `` – default (página 404);
- `proxy` – proxy reverso (campos «Upstream URL», «Reescrever Host», «Ignorar TLS verify»);
- `file` – servir diretório (campo «Diretório», por exemplo `/var/www/html`);
- `string` – resposta fixa (campos «Código de status», «Body», «Cabeçalhos»).

Quando escolher Hysteria: quando é necessário transporte QUIC e resiliência em canais instáveis/móveis; o `masquerade` aumenta o sigilo do ponto de entrada.

5.11. MTProto (proxy para Telegram)

Adicionado na versão **3.3.0**. Valor do protocolo – `mtproto` .

MTProto é o protocolo do proxy nativo do Telegram. No 3X-UI, esse inbound **é gerenciado não pelo Xray, mas por um processo separado `mtg`** , controlado pelo próprio painel. O painel verifica periodicamente os inbounds MTProto habilitados em relação aos processos `mtg` em execução: inicializa os ausentes, para os excedentes e coleta os contadores de tráfego das métricas do `mtg` . Por isso, a **contabilização de tráfego** por esse inbound funciona como nos protocolos comuns.

A partir da versão 3.5.0, o MTProto é um protocolo multicliente (o motor foi trocado pelo fork `mtg-multi`): um único inbound atende muitos usuários, cada um deles um **cliente** comum (veja a seção 8) com seu próprio segredo FakeTLS, cota de tráfego, prazo, chave de ativação, ad-tag opcional e link pessoal `tg://proxy` . O modelo anterior «um inbound = um segredo» foi abolido: o campo «Secret» no nível do inbound foi removido do formulário, e ao atualizar o painel o segredo único existente é automaticamente convertido no primeiro cliente desse inbound (nenhuma ação manual é necessária).

Consequências:

- A aba «**Transport**» (**Stream Settings**) **não se aplica a este inbound**; já os clientes agora são gerenciados como nos demais protocolos – vinculação/desvinculação, limites, grupos, assinaturas.
- As alterações de clientes (adição, exclusão, troca de segredo, ativação/desativação, ad-tag) são aplicadas «**a quente**» pela management-API do `mtg` , sem romper as sessões do Telegram dos demais usuários; apenas

alterações estruturais do inbound (endereço, domain fronting, limite de conexões, IPs públicos) reiniciam o processo.

- O inbound MTProto é iniciado **apenas no painel principal**; não é implantado em nós filhos (nodes) (inbounds com `NodeID` definido são ignorados).
- A aba «**Sniffing**» para MTProto está oculta — esse protocolo é gerenciado pelo processo `mtg`, não pelo Xray, portanto o sniffing não se aplica a ele.

Campos. Armazenados em `settings` do inbound:

Campo no UI	Chave	Descrição
Remark	<code>remark</code>	Rótulo do inbound.
Listen IP	<code>listen</code>	IP de escuta (vazio = todas as interfaces).
Port	<code>port</code>	Porta do proxy.
Domínio de disfarce (FakeTLS)	<code>settings.fakeTlsDomain</code>	Padrão <code>www.cloudflare.com</code> . A partir da 3.5.0 — domínio padrão para a geração dos segredos de novos clientes (dica: «Default FakeTLS domain used to generate a new client's secret. Each client can front its own domain»); cada cliente pode se disfarçar com seu próprio domínio por meio do seu segredo.
Max connections	<code>settings.throttleMaxConnections</code>	Novo na 3.5.0. Limitação de conexões simultâneas para todos os usuários, com divisão justa; <code>0</code> — sem limitação.
Public IPv4 / Public IPv6	<code>settings.publicIpv4</code> / <code>settings.publicIpv6</code>	Novo na 3.5.0. Endereço público do servidor para o ad-tag middle proxy (placeholders <code>1.2.3.4</code> / <code>2001:db8::1</code>); vazio — o <code>mtg</code> detecta sozinho.

Formato do segredo (FakeTLS). O segredo agora vive **em cada cliente** (campo «**MTProto secret**» no formulário do cliente, com botão de regeneração; ao lado — o opcional «**Ad-tag (canal patrocinador)**», exatamente 32 caracteres hex). O formato do segredo é o mesmo: `ee` + 32 caracteres hex + código hex do domínio de disfarce (`ee<hex32><hex(domínio)>`); ao vincular um novo cliente a um inbound MTProto, o segredo é gerado automaticamente a partir do domínio padrão do inbound. A cota de tráfego e o prazo de validade do cliente, a partir da 3.5.0, **são realmente aplicados** (pelos limites do `mtg-multi`): um cliente esgotado ou expirado é desativado, e o reset de tráfego devolve o acesso imediatamente.

Domain-fronting e opções avançadas do `mtg`

O inbound MTProto possui parâmetros adicionais do processo `mtg`. Os campos **Domain fronting IP**, **Domain fronting port** e **Domain fronting PROXY protocol** definem para onde o `mtg` envia tráfego não-Telegram (por exemplo, para um site NGINX falso); se o IP for deixado vazio, o domínio FakeTLS é usado via DNS, e a porta padrão é `443`. Também estão disponíveis **Accept PROXY protocol** (para o listener), **IP preference** (`prefer-ipv6` / `prefer-ipv4` / `only-ipv6` / `only-ipv4`) e **Debug logging**. Cada valor é gravado no arquivo `mtg-<id>.toml` apenas se estiver definido.

Roteamento do tráfego do Telegram via Xray

O botão «**Route through Xray**» (desligado por padrão) e o campo opcional **Outbound** permitem subordinar o egresso do Telegram ao roteador Xray. Ao habilitar, o painel insere na configuração do Xray uma ponte SOCKS local com a tag do próprio inbound, e o `mtg` envia o tráfego do Telegram por ela. Após isso, o tráfego pode ser correspondido por regras na aba «Routing» ou direcionado forçosamente ao outbound ou balanceador selecionado pelo campo **Outbound** (se o campo estiver vazio, as regras de roteamento decidem).

Como distribuir para o usuário. Para o inbound MTProto, o painel gera um link de convite:

Exemplo: segredo FakeTLS e link pronto. Se o domínio de disfarce é `www.cloudflare.com`, o segredo é montado como `ee` + 32 caracteres hex + código hex do domínio, por exemplo:

```
secret = ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

Link de convite pronto (enviado ao usuário no Telegram junto com o QR code):

```
tg://proxy?
server=203.0.113.10&port=443&secret=ee1a2b3c4d5e6f70819293a4b5c6d7e8f7777772e636c6f7564666c6172652e636f6d
```

```
tg://proxy?server=<endereço>&port=<porta>&secret=<segredo>
```

(equivalente — `https://t.me/proxy?server=...&port=...&secret=...`). A partir da 3.5.0, o link é **pessoal** — construído a partir do segredo do cliente específico, e o fragmento `#remark` foi removido dele (parsers pouco rigorosos do Telegram o colavam ao segredo e quebravam a importação); a observação é exibida como um rótulo separado na janela de informações. Esse link e o QR code devem ser enviados ao usuário no Telegram — ao abrir, o proxy é adicionado imediatamente ao aplicativo. O link também é fornecido pelo servidor de assinaturas.

Quando usar. Método padrão para contornar bloqueios do Telegram; o disfarce FakeTLS (domínio de disfarce) torna o tráfego semelhante a uma visita comum ao site indicado.

5.12. Guia rápido para escolha de protocolo

- **VLESS** — escolha padrão; melhor opção com REALITY ou TLS + XTLS-Vision, suporta autenticação pós-quântica.
- **Trojan** — disfarce de HTTPS com fallbacks para servidor web.
- **VMess** — compatibilidade com clientes antigos.
- **Shadowsocks** — proxy simples sem TLS; a escolha moderna são os métodos `2022-blake3-*`.
- **Hysteria** — QUIC, resiliência em canais ruins.
- **mixed / http** — proxies SOCKS/HTTP de serviço.
- **WireGuard** — túnel VPN completo.
- **tunnel** — redirecionamento transparente de portas.
- **MTProto** — proxy para contornar bloqueios do Telegram (FakeTLS); processo separado `mtg`.

6. Transporte (Stream Settings)

O transporte (no campo «**Transporte**» da interface do painel, em inglês *Transmission*) define o modo como o Xray-core transmite os dados dentro de um inbound: qual protocolo de rede é utilizado sobre TLS/Reality e como o tráfego é enquadrado. Esses parâmetros são armazenados no objeto `streamSettings` da configuração do Xray e definidos na aba de transporte do editor de inbound. A criptografia (TLS / Reality) é tratada em uma seção separada – aqui descrevemos apenas a escolha da rede e seus parâmetros.

6.1. Escolha da rede de transmissão

A rede é selecionada na lista suspensa «**Transporte**» (`streamSettings.network`). O valor padrão é `tcp` (exibido na lista como **RAW**). As opções disponíveis são:

Valor na lista	Campo <code>network</code>	Transporte
RAW	<code>tcp</code>	TCP simples (renomeado para RAW nas versões mais recentes do Xray), opcionalmente com ofuscação HTTP
mKCP	<code>kcp</code>	Transporte UDP confiável mKCP
WebSocket	<code>ws</code>	WebSocket sobre HTTP(S)
gRPC	<code>grpc</code>	Túnel gRPC (HTTP/2)
HTTPUpgrade	<code>httpupgrade</code>	HTTP Upgrade
XHTTP	<code>xhttp</code>	XHTTP / SplitHTTP – transporte moderno multiplexado

Ao alterar o valor, o painel limpa o bloco de configurações da rede anterior e preenche o bloco da nova rede com os valores padrão de seu esquema, de modo que cada campo do subformulário sempre possui um valor inicial coerente.

Importante. Nesta versão do painel, **o transporte HTTP/2 (`h2`) não está disponível na lista** – ele foi removido do conjunto de redes; para um túnel bidirecional semelhante ao HTTP/2 use gRPC, e para o transporte moderno mascarado por HTTP use XHTTP. O transporte **Hysteria** (`hysteria`) não é selecionado por esta lista: ele é vinculado diretamente ao protocolo Hysteria e aparece automaticamente quando o próprio inbound é criado com o protocolo Hysteria (veja o item 6.8).

Abaixo, cada rede e cada um de seus campos são descritos individualmente.

6.2. RAW / TCP (`tcpSettings`)

Transporte TCP básico. Por padrão, o tráfego é transmitido «como está»; opcionalmente, pode ser mascarado como uma troca HTTP/1.1 comum.

Campo	Valor padrão	Descrição
Proxy Protocol (<code>acceptProxyProtocol</code>)	<code>false</code> (desativado)	Aceitar o cabeçalho PROXY protocol de um balanceador/ proxy upstream
Ofuscação HTTP (<code>header.type</code>)	<code>none</code> (desativado)	Ativa o mascaramento do tráfego como HTTP/1.1

Proxy Protocol

Alternador «**Proxy Protocol**» (`acceptProxyProtocol`). Quando ativado, o Xray aguarda o cabeçalho PROXY protocol na conexão de entrada e extrai dele o IP real do cliente. Ative apenas se houver um proxy reverso/ balanceador à frente do painel (por exemplo, HAProxy ou nginx com `send-proxy`) que adicione esse cabeçalho. Desativado por padrão.

Ofuscação HTTP (camouflage)

Alternador «**HTTP Ofuscação**». Controla o campo `header` :

- **Desativado** → `header.type = "none"` (o campo `header` simplesmente não está presente no fio). TCP puro.
- **Ativado** → `header.type = "http"` . O tráfego é enquadrado sob a aparência de uma requisição e resposta HTTP/1.1. Ao ativar, o painel preenche imediatamente os sub-objetos `request` e `response` com os valores padrão.

Quando a ofuscação HTTP está ativada, aparecem os campos de configuração da requisição e da resposta simuladas.

Cabeçalho da requisição (`header.request`):

Campo	Chave	Valor padrão	Descrição
Versão da requisição	<code>request.version</code>	<code>1.1</code>	Versão HTTP na linha de início da requisição
Método da requisição	<code>request.method</code>	<code>GET</code>	Método HTTP da requisição simulada
Caminho da requisição	<code>request.path</code>	<code>/</code>	Caminho(s). Inserido como lista de valores separados por vírgula; no fio é um array de strings. Se deixado em branco, substitui-se por <code>/</code>
Cabeçalhos da requisição	<code>request.headers</code>	<code>{}</code> (vazio)	Tabela «Nome/Valor» de cabeçalhos HTTP. Armazenado como mapa <code>nome → [valores]</code> (um nome pode ter múltiplos valores)

Cabeçalho da resposta (`header.response`):

Campo	Chave	Valor padrão	Descrição
Versão da resposta	<code>response.version</code>	1.1	Versão HTTP na linha de início da resposta
Status da resposta	<code>response.status</code>	200	Código de status HTTP da resposta simulada
Razão da resposta	<code>response.reason</code>	OK	Descrição textual do status (reason-phrase)
Cabeçalhos da resposta	<code>response.headers</code>	{} (vazio)	Tabela «Nome/Valor» dos cabeçalhos da resposta (mapa nome → [valores])

Os campos de cabeçalho são editados linha a linha – cada linha define o nome do cabeçalho (Nome) e seu valor (Valor). Esses parâmetros servem apenas para mascarar a aparência do tráfego; não afetam a criptografia. Os valores padrão (GET / HTTP/1.1 , resposta 200 OK) são adequados para a maioria dos cenários – altere-os apenas se precisar imitar um site/serviço específico.

Exemplo de `streamSettings` para RAW com ofuscação HTTP:

```
{
  "network": "tcp",
  "tcpSettings": {
    "acceptProxyProtocol": false,
    "header": {
      "type": "http",
      "request": {
        "version": "1.1",
        "method": "GET",
        "path": ["/"],
        "headers": {
          "Host": ["www.example.com"]
        }
      },
      "response": {
        "version": "1.1",
        "status": "200",
        "reason": "OK"
      }
    }
  }
}
```

Observe que `path` no fio é um array de strings, e cada cabeçalho é um array de valores (`Host` → `["www.example.com"]`).

6.3. mKCP (`kcpSettings`)

mKCP é um transporte confiável sobre UDP. Útil em canais com perda de pacotes e alta latência, mas gera tráfego de controle elevado. Todos os valores padrão coincidem com os recomendados no xray-core.

Campo	Chave	Padrão	Permitido	Descrição
MTU	<code>mtu</code>	1350	576–1460	Tamanho máximo do pacote (bytes). Reduza em caso de problemas de fragmentação
TTI (ms)	<code>tti</code>	20	10–100	Intervalo de transmissão (ms). Menor = menor latência, mas maior overhead
Uplink (MB/s)	<code>uplinkCapacity</code>	5	≥ 0	Capacidade estimada de envio (MB/s)
Downlink (MB/s)	<code>downlinkCapacity</code>	20	≥ 0	Capacidade estimada de recebimento (MB/s)
Multiplicador CWND	<code>cwndMultiplier</code>	1	≥ 1	Multiplicador da janela de congestionamento (congestion window)
Tamanho máx. janela de envio	<code>maxSendingWindow</code>	2097152	≥ 0	Tamanho máximo da janela de envio

Observações sobre os campos:

- **Uplink / Downlink capacity** definem quão agressivamente o mKCP ocupa o canal. Ajuste conforme a largura de banda real: valores muito altos geram tráfego desnecessário, muito baixos subutilizam o canal.
- **TTI** afeta diretamente o compromisso «latência [?] overhead»: valores menores reduzem a latência, mas aumentam o volume de pacotes de controle.
- **MTU** limita o tamanho de um pacote mKCP; reduzi-lo ajuda em canais onde pacotes UDP grandes são fragmentados ou perdidos.

Nesta versão do painel, o campo «seed» (senha de ofuscação do mKCP) e a lista suspensa de **tipo de cabeçalho/ofuscação** (`none` , `srtplib` , `utp` , `wechat-video` , `dtls` , `wireguard`) no subformulário mKCP **não estão disponíveis como campos separados** — a ofuscação de transporte foi movida para o mecanismo geral «FinalMask» (incluindo o modo `mkcp-legacy`), descrito na seção correspondente. O parâmetro «congestion» como alternador independente também não está exposto; o controle de congestionamento é configurado via `cwndMultiplier` e `maxSendingWindow`.

6.4. WebSocket (`wsSettings`)

Transporte WebSocket sobre HTTP(S). Passa bem por CDNs e proxies reversos, mascarando-se como tráfego web comum.

Campo	Chave	Padrão	Descrição
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Aceitar o cabeçalho PROXY protocol de um proxy upstream (veja o item 6.2)
Host	<code>host</code>	<code>""</code> (vazio)	Valor do cabeçalho HTTP <code>Host</code> . Indique ao usar CDN/ domain fronting
Caminho	<code>path</code>	<code>/</code>	Caminho na requisição do handshake WebSocket
Período de heartbeat	<code>heartbeatPeriod</code>	0	Intervalo de envio de quadros heartbeat (em segundos). 0 desativa o heartbeat

Campo	Chave	Padrão	Descrição
Cabeçalhos	headers	{ } (vazio)	Cabeçalhos HTTP adicionais do handshake. Mapa «Nome → Valor» (apenas valores em string, sem arrays)

Observações:

- **Caminho** deve ser igual no servidor (inbound) e no cliente. Com frequência, esse caminho mascara o ponto de entrada no lado do servidor web.
- **Host** faz sentido definir quando o inbound está atrás de uma CDN ou utiliza domain fronting; caso contrário, pode ficar em branco.
- **Período de heartbeat** mantém a conexão «viva» ao passar por proxies/CDNs que encerram sessões inativas agressivamente. Por padrão (0) o heartbeat está desativado.
- Ao contrário do RAW, a tabela de cabeçalhos do WebSocket usa o formato «plano» nome → valor (um valor por cabeçalho).

Exemplo de streamSettings para WebSocket atrás de CDN:

```
{
  "network": "ws",
  "wsSettings": {
    "acceptProxyProtocol": false,
    "host": "cdn.example.com",
    "path": "/ray",
    "heartbeatPeriod": 0,
    "headers": {
      "User-Agent": "Mozilla/5.0"
    }
  }
}
```

Os valores de host e path devem coincidir no cliente; diferentemente do RAW, o valor do cabeçalho aqui é uma string simples, não um array.

6.5. gRPC (grpcSettings)

O transporte com menos parâmetros. Tuneliza o tráfego dentro de chamadas gRPC (sobre HTTP/2); tem boa compatibilidade com CDNs que suportam gRPC. Não há ofuscação de cabeçalhos.

Campo	Chave	Padrão	Descrição
Nome do serviço (Service Name)	serviceName	"" (vazio)	Nome do serviço gRPC (efetivamente o «caminho» do túnel). Deve ser igual no servidor e no cliente
Authority	authority	"" (vazio)	Valor do pseudo-cabeçalho :authority (análogo ao Host para HTTP/2). Indique ao usar CDN/domínio
Multi Mode	multiMode	false (desativado)	Ativa a multiplexação de múltiplos fluxos gRPC paralelos dentro de uma única conexão

Observações:

- **Service Name** é o identificador principal do canal gRPC; deve ser o mesmo em ambos os lados. Valor vazio é permitido, mas normalmente define-se uma string não óbvia para mascaramento.
- **Authority** afeta qual `:authority` é enviado nos quadros HTTP/2; é necessário principalmente ao fazer proxy via CDN.
- **Multi Mode** permite que múltiplos fluxos lógicos passem por uma única conexão física; ative para melhorar o desempenho quando tanto o servidor quanto o cliente suportam isso.

Exemplo de `streamSettings` para gRPC:

```
{
  "network": "grpc",
  "grpcSettings": {
    "serviceName": "GunService",
    "authority": "grpc.example.com",
    "multiMode": false
  }
}
```

`serviceName` (aqui `GunService`) funciona como o «caminho» do túnel e deve ser igual no servidor e no cliente.

6.6. HTTPUpgrade (`httpupgradeSettings`)

Transporte baseado no mecanismo HTTP Upgrade (similar ao WebSocket, mas sem o protocolo WebSocket em si). Também passa bem por proxies e CDNs. O conjunto de campos é o mesmo do WebSocket, mas **sem** o período de heartbeat.

Campo	Chave	Padrão	Descrição
Proxy Protocol	<code>acceptProxyProtocol</code>	<code>false</code>	Aceitar o cabeçalho PROXY protocol de um proxy upstream
Host	<code>host</code>	<code>""</code> (vazio)	Valor do cabeçalho HTTP <code>Host</code>
Caminho	<code>path</code>	<code>/</code>	Caminho da requisição HTTP com o cabeçalho <code>Upgrade</code>
Cabeçalhos	<code>headers</code>	<code>{}</code> (vazio)	Cabeçalhos HTTP adicionais. Mapa «plano» nome → valor (igual ao WebSocket)

O propósito dos campos **Host**, **Caminho** e **Cabeçalhos** é idêntico ao do WebSocket (item 6.4). O heartbeat não está previsto para HTTPUpgrade – essa é uma característica específica do WebSocket.

6.7. XHTTP / SplitHTTP (`xhttpSettings`)

XHTTP (também chamado de SplitHTTP) é um transporte HTTP multiplexado moderno do xray-core. Divide os fluxos de upload e download em requisições HTTP separadas, o que é ideal para CDNs e ambientes com

restrições de duração de conexão. Nem todos os campos são exibidos de uma vez no editor: alguns aparecem dependendo do modo selecionado (`mode`) e dos alternadores ativados.

Campos básicos (sempre visíveis)

Campo	Chave	Padrão	Descrição
Host	<code>host</code>	<code>""</code> (vazio)	Valor do cabeçalho HTTP <code>Host</code>
Caminho	<code>path</code>	<code>/</code>	Caminho base das requisições HTTP
Modo (<code>Mode</code>)	<code>mode</code>	<code>auto</code>	Modo de transmissão (veja abaixo)
Server Max Header Bytes	<code>serverMaxHeaderBytes</code>	<code>0</code>	Limite do tamanho dos cabeçalhos de requisição no servidor (bytes). <code>0</code> – valor padrão do xray-core
Padding Bytes	<code>xPaddingBytes</code>	<code>100–1000</code>	Faixa de padding aleatório (em bytes, formato <code>mín–máx</code>) para dificultar a análise de tamanhos
Cabeçalhos	<code>headers</code>	<code>{}</code> (vazio)	Cabeçalhos HTTP adicionais. Mapa «plano» <code>nome → valor</code>
Método HTTP Uplink	<code>uplinkHTTPMethod</code>	<code>""</code> (Padrão = <code>POST</code>)	Método HTTP das requisições de upload. Opções: vazio (padrão <code>POST</code>), <code>POST</code> , <code>PUT</code> , <code>GET</code> (o último disponível apenas no modo <code>packet-up</code>)
Padding Obfs Mode	<code>xPaddingObfsMode</code>	<code>false</code> (desativado)	Ativa a ofuscação avançada de padding e exibe campos adicionais (veja abaixo)
No SSE Header	<code>noSSEHeader</code>	<code>false</code> (desativado)	Não enviar o cabeçalho <code>Content-Type: text/event-stream</code> (SSE). Ative se ele interferir na passagem por nós intermediários

Campo «Modo» (`mode`)

Lista suspensa com os valores:

Valor	Descrição
<code>auto</code>	Seleção automática de modo (padrão)
<code>packet-up</code>	O fluxo de upload é dividido em requisições HTTP separadas (um pacote por requisição)
<code>stream-up</code>	O fluxo de upload é transmitido em uma única requisição de streaming contínua
<code>stream-one</code>	Uma única requisição de streaming bidirecional compartilhada

A escolha do modo determina quais campos adicionais se tornam visíveis.

Campos visíveis apenas com `mode = packet-up` :

Campo	Chave	Padrão	Descrição
Máx. posts bufferizados no upload	scMaxBufferedPosts	30	Número máximo de requisições POST de upload bufferizadas simultaneamente
Tamanho máx. de upload (bytes)	scMaxEachPostBytes	1000000	Tamanho máximo de uma requisição POST de upload (bytes)
Uplink Data Placement	uplinkDataPlacement	"" (Padrão = body)	Onde posicionar os dados do fluxo de upload: <code>body</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Uplink Data Key	uplinkDataKey	""	Nome da chave/cabeçalho para os dados de uplink. Aparece apenas se <code>uplinkDataPlacement</code> estiver definido e não for <code>body</code>

Campo visível apenas com `mode = stream-up` :

Campo	Chave	Padrão	Descrição
Stream-Up Server	scStreamUpServerSecs	20-80	Faixa de tempo de manutenção da conexão de streaming no servidor (em segundos, formato <code>mín-máx</code>)

Campos de ofuscação de padding (visíveis com `xPaddingObfsMode = ativado`)

Campo	Chave	Padrão	Descrição
Padding Key	xPaddingKey	"" (placeholder <code>x_padding</code>)	Nome da chave para o padding
Padding Header	xPaddingHeader	"" (placeholder <code>X-Padding</code>)	Nome do cabeçalho HTTP pelo qual o padding é transmitido
Padding Placement	xPaddingPlacement	"" (Padrão = <code>queryInHeader</code>)	Onde posicionar o padding: <code>queryInHeader</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Padding Method	xPaddingMethod	"" (Padrão = <code>repeat-x</code>)	Método de geração de padding: <code>repeat-x</code> ou <code>tokenish</code>

Posicionamento de sessão e sequência (sempre visíveis)

Campo	Chave	Padrão	Descrição
Session ID Placement	sessionIDPlacement	"" (Padrão = <code>path</code>)	Onde transmitir o identificador de sessão: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Session ID Key	sessionIDKey	"" (placeholder <code>x_session</code>)	Nome da chave de sessão. Aparece apenas se <code>sessionIDPlacement</code> estiver definido e não for <code>path</code>

Campo	Chave	Padrão	Descrição
Session ID Table	<code>sessionIDTable</code>	<code>""</code> (placeholder Base62)	Conjunto de caracteres para geração de identificadores de sessão. É possível escolher um predefinido na lista com autocompletar (<code>ALPHABET</code> , <code>Alphabet</code> , <code>BASE36</code> , <code>Base62</code> , <code>HEX</code> , <code>alphabet</code> , <code>base36</code> , <code>hex</code> , <code>number</code>) ou inserir uma string ASCII personalizada. Vazio – valor padrão do xray-core
Session ID Length	<code>sessionIDLength</code>	<code>""</code> (vazio)	Comprimento ou faixa (por exemplo <code>8-16</code>) dos identificadores gerados. Exibido apenas quando <code>Session ID Table</code> está definido; o mínimo deve ser maior que 0
Sequence Placement	<code>seqPlacement</code>	<code>""</code> (Padrão = <code>path</code>)	Onde transmitir o número de sequência do pacote: <code>path</code> , <code>header</code> , <code>cookie</code> , <code>query</code>
Sequence Key	<code>seqKey</code>	<code>""</code> (placeholder <code>x_seq</code>)	Nome da chave de sequência. Aparece apenas se <code>seqPlacement</code> estiver definido e não for <code>path</code>

Os campos de sessão foram renomeados no xray-core v26.6.22: anteriormente chamados **Session Placement** / **Session Key** (`sessionPlacement` / `sessionKey`) – agora são **Session ID Placement** / **Session ID Key** (`sessionIDPlacement` / `sessionIDKey`); o núcleo não reconhece mais os nomes antigos. Inbounds salvos antes da atualização são migrados para as novas chaves automaticamente – não é necessário salvá-los novamente.

Recomendações:

- Para a maioria das instalações, basta manter **Modo = auto** , definir **Caminho/Host** e (ao usar CDN) sincronizá-los com o cliente.
- Os campos de posicionamento (`*Placement` / `*Key`) e de ofuscação de padding são necessários apenas para ajuste fino em cenários específicos de anti-DPI/CDN; quando vazios, são usados os valores padrão do xray-core indicados entre parênteses.
- Parâmetros relativos ao lado do cliente/outbound (por exemplo, intervalos de reenvio de POST, tamanhos de chunk) não são exibidos no formulário de inbound – o servidor ouvinte os ignora. O multiplexador XMUX, por outro lado, está disponível no formulário de inbound (veja abaixo).
- **Valores padrão internos não são gravados.** O painel não grava mais nos arquivos de configuração XHTTP os valores padrão internos `scMaxEachPostBytes` e `scMinPostsIntervals` – os valores internos do xray-core são aplicados. Isso elimina a assinatura DPI constante (literal `scMinPostsIntervals=30`) pela qual o tráfego poderia ser bloqueado anteriormente. Para inbounds já salvos, valores que coincidem com os padrões do xray-core não são incluídos nos links e assinaturas (não é necessário salvar os inbounds novamente); valores definidos manualmente são preservados.

Exemplo de `streamSettings` para XHTTP (modo `auto`):

```
{
  "network": "xhttp",
  "xhttpSettings": {
    "host": "xhttp.example.com",
    "path": "/yourpath",
    "mode": "auto",
    "xPaddingBytes": "100-1000"
  }
}
```

Para a maioria das instalações, esses quatro campos são suficientes; os campos de posicionamento de sessão/sequência e de ofuscação de padding são deixados em branco – então os valores padrão do xray-core são usados.

XMUX (multiplexação de conexões)

O alternador **XMUX** (`enableXmux`) ativa uma camada de multiplexação que distribui requisições paralelas por um pequeno pool de conexões físicas. Quando ativado, seis campos configuráveis são exibidos (armazenados em `xhttpSettings.xmux`):

Campo	Chave	Padrão	Descrição
Max Concurrency	<code>maxConcurrency</code>	16–32	Máximo de requisições simultâneas por conexão (faixa mín–máx)
Max Connections	<code>maxConnections</code>	6	Máximo de conexões físicas (0 – sem limite). A partir da versão 3.4.2, novos inbounds são criados com o valor 6 ; os criados anteriormente mantêm o seu.
Max Reuse Times	<code>cMaxReuseTimes</code>	"" (vazio)	Quantas vezes reutilizar uma conexão
Max Request Times	<code>hMaxRequestTimes</code>	600–900	Máximo de requisições por conexão (faixa)
Max Reusable Secs	<code>hMaxReusableSecs</code>	1800–3000	Tempo durante o qual a conexão pode ser reutilizada (segundos, faixa)
Keep Alive Period	<code>hKeepAlivePeriod</code>	"" (vazio)	Período de keep-alive para manter a conexão

Importante. Não é possível definir **Max Connections** e **Max Concurrency** simultaneamente – o xray-core rejeitará essa configuração. Por padrão, ao ativar o XMUX, o painel define `Max Concurrency = 16–32` ; se você definir **Max Connections** (valor maior que 0), o painel removerá o valor padrão de `Max Concurrency` para evitar conflito.

6.8. Transporte Hysteria (`hysteriaSettings`)

O transporte **Hysteria** não é selecionado na lista «Transporte»: ele é ativado automaticamente quando o inbound é criado com o protocolo Hysteria, e fica oculto para outros protocolos (ao sair do protocolo Hysteria, a rede é forçada de volta para `tcp`). Parâmetros:

Campo	Chave	Padrão	Descrição
Versão	<code>version</code>	2 (fixo, campo bloqueado)	Versão do Hysteria. Apenas Hysteria 2 é suportado
UDP idle timeout (s)	<code>udpIdleTimeout</code>	60	Timeout de ociosidade da sessão UDP (segundos). Faixa válida: 2–600; o xray-core rejeita valores fora desse intervalo ao iniciar
Masquerade	<code>masquerade</code>	desativado (ausente)	Ativa o mascaramento do ouvinte como um servidor HTTP/3 durante sondagens

Quando **Masquerade** está ativado, aparece a seleção do tipo (`type`) e os campos dependentes:

- `""` – **default (404 page)**: retorna uma página 404 padrão (sem campos adicionais).
- **proxy (reverse proxy)**: proxy reverso para um site externo.
 - `url` (**Upstream URL**, placeholder `https://www.example.com`) – endereço de destino;
 - `rewriteHost` (**Reescrever Host**, padrão `false`) – substituir o cabeçalho `Host` ;
 - `insecure` (**Ignorar TLS verify**, padrão `false`) – não verificar o certificado TLS do upstream.
- **file (serve directory)**: servir arquivos de um diretório.
 - `dir` (**Diretório**, placeholder `/var/www/html`).
- **string (fixed body)**: resposta HTTP fixa.
 - `statusCode` (**Código de status**, padrão `0` , faixa 0–599);
 - `content` (**Body**) – corpo da resposta;
 - `headers` (**Cabeçalhos**) – mapa `nome` → `valor` .

O Masquerade permite que o inbound baseado em Hysteria pareça um servidor HTTP/3 comum durante sondagens ativas, aumentando o sigilo. Por padrão, o mascaramento está desativado.

Exemplo de `hysteriaSettings` com proxy reverso (`masquerade` → `proxy`):

```
{
  "version": 2,
  "udpIdleTimeout": 60,
  "masquerade": {
    "type": "proxy",
    "url": "https://www.example.com",
    "rewriteHost": true,
    "insecure": false
  }
}
```

Aqui, durante sondagens, o ouvinte retorna a resposta de `https://www.example.com` , mascarando-se como um site HTTP/3 comum.

6.9. Parâmetros complementares

Além da seleção de rede, na mesma aba estão disponíveis dois blocos gerais independentes do transporte específico (detalhados nas seções correspondentes):

- **External Proxy** (`externalProxy`) – lista de endereços/portas externos que substituem o endereço do próprio painel nos links de assinatura.
- **Socketopt** (`socketopt`) – opções de socket de baixo nível (TCP Fast Open, mark, estratégia de domínio, proxy transparente etc.).

IP real do cliente (identificação do IP real atrás de CDN/relay)

Quando o inbound está atrás de um intermediário (CDN como Cloudflare, túnel/relay L4 ou outro painel), o Xray enxerga o endereço do intermediário, não o do visitante real. Esse endereço aparece na lista de clientes online e é usado para contar o limite de IPs por cliente, tornando ambos inúteis atrás de um proxy. Para restaurar o IP real, na seção **Socketopt** do formulário de inbound há a seleção de preset **Real client IP**, que combina as configurações `acceptProxyProtocol` e `trustedXForwardedFor`:

Preset	O que faz	Quando usar
Off / direct	Limpa os dois campos.	Inbound acessível diretamente pelos clientes
Cloudflare CDN	Define <code>socketopt.trustedXForwardedFor = ["CF-Connecting-IP"]</code> .	WebSocket / HTTPUpgrade / XHTTP / gRPC atrás da CDN Cloudflare (nuvem laranja)
L4 relay / Spectrum (PROXY)	Ativa <code>acceptProxyProtocol = true</code> .	Túnel/relay L4 à frente do inbound ou Cloudflare Spectrum

Os presets são mutuamente exclusivos: selecionar um limpa o campo do outro, de modo que um `trustedXForwardedFor` desatualizado não sobrescreva um IP restaurado pelo PROXY protocol. Abaixo do preset permanecem visíveis o alternador «raw» **Proxy Protocol** e a lista **Trusted X-Forwarded-For** – o preset apenas os preenche por você e, se necessário, podem ser editados manualmente. Se o preset selecionado não for compatível com o transporte atual (por exemplo, PROXY protocol no mKCP), o formulário exibe um aviso. Esses campos são exclusivos do lado do servidor e **nunca são enviados aos clientes nas assinaturas**.

Use apenas um. `acceptProxyProtocol` lê o IP real do cabeçalho L4 do PROXY protocol, enquanto `trustedXForwardedFor` lê do cabeçalho HTTP da requisição; combine-os manualmente apenas se a sua cadeia de intermediários exigir isso.

- **FinalMask** (`finalmask`) – mecanismo geral de ofuscação de transporte (incluindo a ofuscação legada do mKCP), que substituiu os campos separados «seed»/«header type» dentro dos subformulários de rede.

7. Segurança da conexão: TLS, XTLS e REALITY

Cada inbound que suporta transmissão por fluxo de transporte (VMess, VLESS, Trojan, Shadowsocks, Hysteria) possui a aba «**Segurança**» no editor. Nela é configurado como o canal de transporte é criptografado e mascarado. Há três modos disponíveis, alternados por botões de rádio:

Modo	Rótulo na UI	Quando disponível
none	Nenhum	Sempre (exceto Hysteria, onde TLS é obrigatório)
tls	TLS	Para VMess/VLESS/Trojan/Shadowsocks nas redes tcp , ws , http , grpc , httpupgrade , xhttp ; para Hysteria – sempre
reality	Reality	Apenas para VLESS/Trojan nas redes tcp , http , grpc , xhttp

O botão **Nenhum** não é exibido quando o protocolo é Hysteria (para ele, TLS é obrigatório). O botão **Reality** aparece somente com uma combinação válida de protocolo e rede (ver tabela acima).

Ao mudar o modo, o painel reconstrói completamente o bloco `streamSettings`: os `tlsSettings` e `realitySettings` do modo anterior são removidos e os valores padrão do modo selecionado são inseridos. Em particular, ao selecionar **Reality**, o painel automaticamente: substitui por um par aleatório de `target` + `serverNames` (SN1) da lista interna de domínios populares, gera `shortIds` aleatórios e faz uma requisição ao servidor para obter um par de chaves X25519 (privateKey/publicKey) atualizado.

7.1. Qual é a diferença: TLS vs XTLS vs REALITY

- **TLS** – criptografia clássica de transporte pelo protocolo TLS 1.2/1.3. O servidor deve ter um certificado válido (domínio próprio + cadeia). O tráfego parece um HTTPS comum, mas para um censor ativo o handshake TLS para o seu domínio é reconhecível; quando bloqueado por SN1 ou na ausência de certificado confiável, a conexão é bloqueada/exibe erro.
- **XTLS (Vision)** – não é um modo separado na lista «Segurança», mas um mecanismo de *flow* sobre TLS ou Reality. É ativado no lado do cliente do inbound pelo campo **Flow** = `xtls-rprx-vision` (ou `xtls-rprx-vision-udp443`). Vision está disponível para VLESS na rede `tcp` com `security = tls` ou `security = reality`, bem como para VLESS sobre o transporte `xhttp` com criptografia VLESS ativada (vlessenc / ML-KEM) – nesse caso, o campo **Flow** também pode ser configurado como `xtls-rprx-vision`, e o valor é corretamente incluído no link `vless://` (`flow=xtls-rprx-vision`). Vision reduz a «criptografia dupla» (TLS-in-TLS), entregando a carga útil diretamente após o handshake, o que acelera a transmissão e melhora o mascaramento. Por isso, a combinação **VLESS + Reality + Flow xtls-rprx-vision** é considerada a configuração moderna recomendada.

Restauração automática do flow Vision. Se a criptografia de um inbound VLESS/XHTTP (ML-KEM, campos decryption/encryption) for ativada após os clientes já terem sido adicionados, o inbound passa a ser elegível para flow. Nessa situação, o painel restaura automaticamente `flow = xtls-rprx-vision` nos clientes que deveriam tê-lo, mas cujo campo **Flow** estava vazio. Anteriormente, nesse cenário, o Vision desaparecia silenciosamente das configurações, links de convite e assinaturas (especialmente perceptível em inbounds de nó central). Nenhuma ação manual é necessária: a correção é aplicada automaticamente

ao salvar o inbound e uma única vez durante a atualização do painel. O comportamento é conservador – o painel não inventa flow nem sobreescreve um valor definido explicitamente pelo cliente.

- **REALITY** – mecanismo de mascaramento sem certificado próprio. O servidor «toma emprestado» o handshake TLS de um site externo real (`target / serverNames`), tornando a conexão indistinguível de um acesso a esse site para um observador, sem necessidade de certificado. A autenticação é baseada em um par de chaves X25519 e um conjunto de `shortIds` . REALITY é resistente a sondagem ativa (`active probing`) e bloqueio por SNI, pois o SNI aponta para um domínio externo real. O custo é a configuração mais rigorosa (um `target` correto com porta, sincronização de chaves com o cliente).

Regra rápida de escolha:

- tem domínio próprio e certificado válido, precisa de aparência HTTPS simples → **TLS** (se possível com Vision);
- não tem domínio/certificado ou precisa de máxima invisibilidade para DPI → **REALITY** (com Vision para VLESS/TCP).

7.2. Modo «Nenhum» (`none`)

O transporte é transmitido sem encapsulamento TLS: os blocos `tlsSettings` e `realitySettings` são excluídos de `streamSettings` . O modo não tem campos adicionais. É adequado quando:

- o inbound escuta apenas em `127.0.0.1` e serve como destino de fallback (segundo a regra do painel, o inbound filho para fallback deve escutar em `127.0.0.1` com `security=none`);
- a criptografia/mascaramento é fornecida por uma camada externa (por exemplo, proxy reverso Nginx na frente do painel);
- é usado um protocolo com criptografia própria (Shadowsocks) em uma rede interna.

Para inbounds acessíveis externamente, o modo «Nenhum» não é recomendado.

Exemplo: bloco `streamSettings` para TLS na rede `tcp` (VLESS/Trojan/VMess). É assim que fica o resultado após selecionar o modo **TLS** e preencher o SNI e os caminhos para o certificado:

```

"streamSettings": {
  "network": "tcp",
  "security": "tls",
  "tlsSettings": {
    "serverName": "vpn.example.com",
    "minVersion": "1.2",
    "maxVersion": "1.3",
    "alpn": ["h2", "http/1.1"],
    "settings": { "fingerprint": "chrome" },
    "certificates": [
      {
        "certificateFile": "/root/cert/vpn.example.com.crt",
        "keyFile": "/root/cert/vpn.example.com.key",
        "ocspStapling": 3600,
        "usage": "encipherment"
      }
    ]
  }
}

```

7.3. Modo TLS

Campos do bloco `tlsSettings`. Os valores padrão são obtidos do esquema do painel.

Parâmetros principais

Campo (rótulo)	Valor padrão	Descrição
SNi (<code>serverName</code>)	<code>""</code> (vazio)	Server Name Indication – nome de domínio apresentado no handshake TLS. Deve corresponder ao domínio do certificado. Placeholder em inglês: «Server Name Indication».
Cipher Suites (<code>cipherSuites</code>)	<code>""</code> → Auto	Lista de conjuntos de cifras permitidos. Por padrão está vazio – a escolha fica a critério do Xray/Go (opção Auto). Altere apenas quando for necessário restringir explicitamente as cifras.
Versão Mín/Máx (<code>minMaxVersion</code>)	min = <code>1.2</code> , max = <code>1.3</code>	Versões mínima e máxima do TLS. Valores disponíveis: <code>1.0</code> , <code>1.1</code> , <code>1.2</code> , <code>1.3</code> . Recomenda-se manter <code>1.2</code> – <code>1.3</code> ; reduzir o mínimo para 1.0/1.1 não é aconselhável (versões obsoletas e inseguras).
uTLS (<code>settings.fingerprint</code>)	<code>chrome</code> (no formulário – o item None = <code>""</code> está disponível)	Impressão digital TLS imitada do client hello (uTLS fingerprint), para que o handshake pareça o de um navegador popular. Ver lista abaixo. Em TLS, o primeiro item da lista é None (<code>""</code>), que desativa a imitação.
ALPN (<code>alpn</code>)	<code>["h2", "http/1.1"]</code>	Lista de protocolos de camada de aplicação negociados no TLS (seleção múltipla). Valores válidos: <code>h3</code> , <code>h2</code> , <code>http/1.1</code> . Por padrão são oferecidos <code>h2</code> e <code>http/1.1</code> .

Valores possíveis para **uTLS fingerprint** (iguais para TLS e REALITY): `chrome`, `firefox`, `safari`, `ios`, `android`, `edge`, `360`, `qq`, `random`, `randomized`, `randomizednoalpn`, `unsafe`. No formulário TLS, também está disponível a opção vazia **None** (a imitação de impressão digital não é aplicada).

Valores disponíveis para **Cipher Suites** (TLS 1.3 e conjuntos ECDHE): `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384`, `TLS_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA`, `TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA`, `TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256`, `TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384`, `TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256`, `TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256`.

Alternadores TLS

Alternador	Padrão	Descrição
Rejeitar SNI desconhecido (<code>rejectUnknownSni</code>)	desativado (<code>false</code>)	Se ativado, o servidor encerra a conexão quando o SNI apresentado pelo cliente não corresponde ao nome no certificado. Aumenta a invisibilidade (o servidor não responde a requisições «estranhas»), mas exige correspondência exata do SNI no cliente.
Desativar System Root (<code>disableSystemRoot</code>)	desativado (<code>false</code>)	Desativa o uso do repositório de certificados raiz confiáveis do sistema.
Retomada de sessão (<code>enableSessionResumption</code>)	desativado (<code>false</code>)	Ativa a retomada de sessão TLS (session resumption / session tickets).

Parâmetros adicionais do TLS (vcn, curvas, log de chaves, ECH Sockopt)

Abaixo das configurações principais do TLS há campos adicionais disponíveis.

Campo (rótulo)	Padrão	Descrição
Verify Peer Cert By Name (<code>settings.verifyPeerCertByName</code>)	""	Nomes (separados por vírgula) pelos quais o cliente verifica o certificado do servidor em vez do SNI. Esta é a substituição moderna do campo <code>allowInsecure</code> removido do Xray após 06/06/2026. Valor apenas para o painel: não é escrito no config do xray no servidor, mas é incluído nos links de convite e assinaturas (<code>vcn=...</code>) para que o cliente o aplique. Placeholder: <code>example.com</code> .
Curve Preferences (<code>curvePreferences</code>)	""	Restrição e ordem das curvas de troca de chaves TLS, em ordem de preferência (por exemplo, <code>X25519MLKEM768</code> , <code>X25519</code>). Vazio — os padrões do xray-core são usados.
Master Key Log (<code>masterKeyLog</code>)	""	Caminho para gravação das master keys TLS no formato <code>SSLKEYLOGFILE</code> (para descriptografar tráfego no Wireshark durante depuração). Placeholder: <code>/path/to/sslkeylog.txt</code> . Em produção, deixar vazio — o arquivo permite descriptografar todo o tráfego.

Campo (rótulo)	Padrão	Descrição
ECH Sockopt (echSockopt)	desativado	Alternador com parâmetros de socket para a conexão pela qual o Xray requisita a lista de configurações ECH. Quando ativado, ficam disponíveis: Dialer Proxy (dialerProxy – roteie a requisição pelo outbound especificado por tag), Domain Strategy (domainStrategy), TCP Fast Open (tcpFastOpen), Multipath TCP (tcpMptcp). Deixe desativado se não for necessário.

Os campos `verifyPeerCertByName`, `curvePreferences`, `masterKeyLog` e `echSockopt` estão no nível superior de `tlsSettings` e sobrevivem ao «recorte» dos campos do painel ao salvar a configuração.

Certificados

A seção **Certificado SSL** (no UI com o título «Certificado SSL») é configurada como uma lista: o botão **+** adiciona uma nova entrada de certificado, e o botão **– Remover** a exclui (o botão de remoção está disponível apenas quando há mais de uma entrada). Por padrão, ao ativar o TLS, uma entrada vazia é criada.

Para cada entrada, há um alternador de modo de entrada (`useFile`):

- **Caminho do certificado** (valor `useFile = true`, padrão) – especifica os caminhos para os arquivos no servidor:
 - **Chave pública** (`certificateFile`) – caminho para o arquivo de certificado (`.crt` / `.pem`);
 - **Chave privada** (`keyFile`) – caminho para o arquivo de chave privada (`.key`).
- **Conteúdo do certificado** (valor `useFile = false`) – o conteúdo é inserido diretamente nos campos (áreas de texto multilinha):
 - **Chave pública** (`certificate`) – conteúdo PEM do certificado;
 - **Chave privada** (`key`) – conteúdo PEM da chave.

Abaixo dos campos do modo «Caminho do certificado» há dois botões:

- **Usar certificado do painel** – preenche os campos com os caminhos do próprio certificado SSL do painel. Para um inbound no painel central, usa o certificado do painel (`POST /panel/setting/all → webCertFile / webKeyFile`); para um inbound atribuído a um nó, usa o certificado do próprio nó (`GET /panel/api/nodes/webCert/{nodeId}`), pois os caminhos do painel central não existem no nó. Se o certificado não estiver configurado, é exibido um aviso: «*Nenhum certificado configurado para o painel. Configure-o primeiro em Configurações.*» (o próprio certificado do painel é definido na seção «Configurações → Segurança»).
- **Limpar** – apaga ambos os caminhos.

Campos adicionais de cada entrada de certificado:

Campo	Padrão	Descrição
OCSP Stapling (ocsStapling)	0 (desativado)	Intervalo de atualização do OCSP stapling em segundos (mínimo 0). Para novos inbounds está desativado por padrão (0): isso elimina erros nos logs do xray para certificados sem OCSP responder (por exemplo, Let's Encrypt, que descontinuou o OCSP). Ative apenas para certificados que suportam stapling.

Campo	Padrão	Descrição
Carregamento único (<code>oneTimeLoading</code>)	desativado (<code>false</code>)	Se ativado, o certificado é lido do disco uma única vez na inicialização e não é relido quando o arquivo é alterado.
Opção de uso (<code>usage</code>)	<code>encipherment</code>	Finalidade do certificado. Valores válidos: <code>encipherment</code> (criptografia – certificado de servidor comum), <code>verify</code> (verificação), <code>issue</code> (emissão – o servidor assina/emite certificados).
Build Chain (<code>buildChain</code>)	desativado (<code>false</code>)	Exibido apenas quando <code>usage = issue</code> . Constrói a cadeia de certificados.

Não há botão separado para certificado autoassinado no editor de inbound: o painel não gera um certificado autoassinado dinamicamente para o inbound. O certificado é especificado por caminho/ conteúdo ou importado das configurações do painel pelo botão «Usar certificado do painel». A emissão/ obtenção do certificado SSL do próprio painel (incluindo upload de arquivos e vinculação ao domínio) é realizada na seção **Configurações** → **Segurança**; não há endpoints ACME/Let's Encrypt para inbounds aqui.

ECH e fixação de certificado (campos avançados do TLS)

Campo	Padrão	Descrição
ECH key (<code>echServerKeys</code>)	<code>""</code>	Chaves de servidor do Encrypted Client Hello.
ECH config (<code>settings.echConfigList</code>)	<code>""</code>	Lista de configurações ECH (parte do cliente, incluída no link).
SHA-256 do certificado do par (<code>settings.pinnedPeerCertSha256</code>)	<code>[]</code>	Hashes SHA-256 do certificado do par (strings hexadecimais, separadas por vírgula). Dica literal: « <i>Hashes SHA-256 do certificado do par como string hexadecimal (ex.: e8e2d3...), separados por vírgula. Apenas para o painel – não é gravado no config do xray no servidor, mas é incluído nos links de convite para que os clientes possam fixar o certificado.</i> »

Botões: Ao lado do campo **SHA-256 do certificado do par** há dois botões de preenchimento automático:

- **Fill from this inbound's certificate** (ícone de escudo) – preenche com o hash SHA-256 do certificado deste próprio inbound (obtido localmente pelo endpoint `getCertHash`).
- **Fetch the hash by pinging the SNI (xray tls ping)** (ícone de download) – obtém o hash do certificado ao vivo do servidor realizando uma conexão TLS com o SNI especificado (no servidor é chamado `getRemoteCertHash`). O campo **SNI** (`serverName`) deve estar preenchido – caso contrário, é exibida a dica «*Set the SNI (serverName) first to ping the remote certificate.*»

Os hashes obtidos são adicionados ao campo (separados por vírgula) e incluídos nos links de convite para que o cliente possa fixar o certificado.

- **Obter novo certificado ECH** – solicita ao servidor um novo par ECH para o SNI atual (`POST /panel/api/server/getNewEchCert`, no servidor executa `xray tls ech --serverName <SNI>`); preenche os campos **ECH key** e **ECH config**.
- **Limpar** – zera ambos os campos ECH.

7.4. Modo REALITY

Campos do bloco `realitySettings`. REALITY não usa certificado SSL: em vez disso, usa um handshake TLS emprestado de um domínio externo e um par de chaves X25519.

Parâmetros de mascaramento

Campo (rótulo)	Valor padrão	Descrição
Mostrar (<code>show</code>)	desativado (<code>false</code>)	Saída de depuração do REALITY nos logs do Xray. Normalmente mantido desativado.
Xver (<code>xver</code>)	0	Versão do protocolo PROXY transmitida ao backend (0 – desativado). Mínimo 0.
uTLS (<code>settings.fingerprint</code>)	chrome	Impressão digital TLS imitada (mesma lista do TLS, mas sem a opção vazia None).
Destino (<code>target</code>)	"" (a partir da 3.4.2 permanece vazio ao ativar o REALITY)	Campo obrigatório. Domínio real cujo handshake TLS o REALITY toma emprestado. Dica literal: «Obrigatório. Deve conter a porta (ex.: <i>example.com:443</i>). Sem a porta, o Xray-core não inicia.» A validação do painel verifica a presença e validade da porta; caso contrário, são exibidos os erros «Destino REALITY é obrigatório» / «Destino REALITY deve conter a porta...» / «O destino REALITY tem uma porta inválida». Ao lado do campo há os botões «Escanear» (verificar o destino atual «ao vivo») e «Buscar destinos» (abrir o scanner de destinos REALITY); veja abaixo.
SNI (<code>serverNames</code>)	[] (preenchido junto com o destino)	Lista de SNIs permitidos (entrada múltipla por tags). Deve corresponder ao domínio em Destino . Ao escanear o destino com sucesso, o SNI é preenchido pelo certificado dele.
Diferença de tempo máxima (ms) (<code>maxTimediff</code>)	0	Diferença máxima permitida de relógio entre cliente e servidor em milissegundos (0 – sem restrição). Mínimo 0.
Versão mínima do cliente (<code>minClientVer</code>)	""	Versão mínima do cliente Xray (placeholder a partir da 3.5.0 – 26.3.27). Vazio – sem restrição.
Versão máxima do cliente (<code>maxClientVer</code>)	""	Versão máxima do cliente Xray. Vazio – sem restrição.
Short IDs (<code>shortIds</code>)	[] (gerados ao ativar)	Lista de identificadores curtos (hex) que diferenciam os clientes. Entrada múltipla por tags; o botão de atualização gera um conjunto aleatório.
SpiderX (<code>settings.spiderX</code>)	/	Caminho do «spider» (parte do cliente do REALITY), usado ao imitar o acesso ao site externo. É incluído no link de convite.

A partir da versão 3.4.2, **Destino** (`target`) e **SNI** (`serverNames`) ao ativar o REALITY **não** são mais preenchidos automaticamente – ambos os campos permanecem vazios, e o destino é selecionado pelo scanner ao vivo (veja abaixo). Escolha um site HTTPS externo «robusto» e estável com suporte a TLS 1.3 e HTTP/2 que não esteja hospedado no seu próprio servidor.

Busca de destino REALITY (scanner ao vivo)

A partir da versão 3.4.2, o antigo botão «destino aleatório» (com a lista interna estática) foi substituído por duas ações ao lado do campo **Destino**:

- **«Escanear»** – verifica «ao vivo» o destino atual: estabelece uma conexão TLS e, se o destino for adequado, preenche o SNI pelo certificado dele. Toasts: «Destino adequado – destino e SNI preenchidos.» / «Destino acessível, mas não adequado para REALITY.» / «Não foi possível escanear o destino REALITY.».
- **«Buscar destinos»** – abre a janela **«Scanner de destinos REALITY»**: é possível indicar um domínio, **IP ou faixa CIDR** (então o painel encontra destinos pelos certificados dos hosts ativos) ou deixar o campo vazio (são verificados os candidatos internos). Os resultados são classificados; as colunas são «Status»/«Adequado», «Troca de chaves», «Certificado», TLS, ALPN, «Latência». Com o botão **«Usar»**, a linha selecionada é preenchida nos campos do inbound.

Um destino é considerado **adequado** se o servidor negociar **TLS 1.3, HTTP/2 (ALPN h2)**, troca de chaves **X25519/X25519MLKEM768** e apresentar um certificado **confiável** (não wildcard). O scan é executado do lado do servidor do painel (`POST /panel/api/server/scanRealityTarget` e `.../scanRealityTargets`).

Exemplo: bloco `streamSettings` para REALITY na rede `tcp` (VLESS). Nenhum certificado é necessário – em vez disso, um domínio emprestado e um par de chaves X25519:

```
"streamSettings": {
  "network": "tcp",
  "security": "reality",
  "realitySettings": {
    "show": false,
    "xver": 0,
    "dest": "www.nvidia.com:443",
    "serverNames": ["www.nvidia.com"],
    "privateKey": "YOUR_X25519_PRIVATE_KEY",
    "shortIds": [ "", "6ba85179e30d4fc2" ],
    "settings": {
      "publicKey": "YOUR_X25519_PUBLIC_KEY",
      "fingerprint": "chrome",
      "spiderX": "/"
    }
  }
}
```

Aqui, o campo **Destino** (`target`) do painel corresponde a `dest` no config gerado do Xray. Se um inbound REALITY foi criado com o destino na chave `dest` (por versões antigas do painel, via API ou ferramentas externas), o painel ao analisar normaliza `dest` → `target` quando `target` está vazio – portanto, esse inbound é carregado corretamente, o campo **Destino** não fica vazio, e salvar novamente não quebra o REALITY em funcionamento.

Chaves REALITY (X25519)

Campo	Padrão	Descrição
Chave pública (<code>settings.publicKey</code>)	""	Chave pública X25519 (inserida pelo cliente em sua configuração/link).
Chave privada (<code>privateKey</code>)	""	Chave privada X25519 (armazenada apenas no servidor).

Botões abaixo das chaves:

- **Obter novo certificado** – solicita ao servidor um novo par de chaves (GET `/panel/api/server/getNewX25519Cert` ; no servidor executa `xray x25519`), preenche a **Chave privada** e a **Chave pública**. Esse par também é gerado automaticamente na primeira ativação do modo REALITY.

Exemplo: obter um par de chaves X25519 via API (fora do formulário, por exemplo, em um script). A requisição retorna a chave privada e a chave pública:

```
curl -s -b cookie.txt https://your-panel:2053/panel/api/server/getNewX25519Cert
# Resposta:
# {"success":true,"obj":{"privateKey":"...", "publicKey":"..."}}
```

`cookie.txt` – arquivo de cookie de sessão obtido após o login via `POST /login`.

- **Limpar** – zera ambas as chaves.

Assinatura pós-quântica ML-DSA-65 (mlDSA65)

Camada adicional (opcional) de autenticação pós-quântica do REALITY:

Campo	Padrão	Descrição
mlDSA65 Seed (<code>mlDSA65Seed</code>)	""	Seed da chave ML-DSA-65 do servidor.
mlDSA65 Verify (<code>settings.mlDSA65Verify</code>)	""	Valor de verificação (parte do cliente, incluído no link).

Botões:

- **Obter novo Seed** – solicita um novo par (GET `/panel/api/server/getNewmlDSA65` ; no servidor executa `xray mlDSA65`), preenche **mlDSA65 Seed** e **mlDSA65 Verify**.
- **Limpar** – zera ambos os campos.

Limitação de velocidade do fallback e log de chaves do REALITY

Nas configurações do REALITY está disponível a limitação de velocidade do tráfego de fallback – ela impede que sondas ativas usem o servidor como canal gratuito para o domínio emprestado. A configuração é definida separadamente para dois sentidos – **Limit Fallback Upload** e **Limit Fallback Download** (`limitFallbackUpload` / `limitFallbackDownload`), cada um com o mesmo conjunto de campos:

Campo (rótulo)	Padrão	Descrição
After Bytes (afterBytes)	0	Quantos bytes permitir em velocidade máxima antes de iniciar a limitação. 0 – limitar desde o primeiro byte.
Bytes Per Sec (bytesPerSec)	0	Limite de velocidade do tráfego de fallback em bytes por segundo após o limiar. 0 – sem limite (desativa este sentido).
Burst Bytes Per Sec (burstBytesPerSec)	0	Reserva para picos breves acima da velocidade constante (tamanho do token-bucket). Se for menor que Bytes Per Sec , é elevado ao seu valor.

No mesmo local é adicionado o campo **Master Key Log** (masterKeyLog) – caminho para gravação das master keys TLS no formato `SSLKEYLOGFILE` para depuração no Wireshark; em produção, deixar vazio.

7.5. Recomendações práticas de configuração

1. **VLESS + Reality (recomendado):** crie um inbound VLESS na rede `tcp`, na aba «Segurança» selecione **Reality** – o painel preencherá automaticamente `target` /SNI aleatórios, `shortIds` e gerará as chaves X25519. Se necessário, clique em «Obter novo certificado» para gerar seu próprio par de chaves. Para clientes VLESS, ative **Flow** = `xtls-rprx-vision` (XTLS Vision) – isso proporcionará máximo desempenho e invisibilidade.

Exemplo: link de cliente final VLESS + Reality + Vision. É assim que fica o link de convite que o painel gera para esse inbound (os valores de chaves/ID são ilustrativos):

```
vless://uuid-клиента@1.2.3.4:443?
type=tcp&security=reality&pbk=ПУБЛИЧНЫЙ_КЛЮЧ&fp=chrome&sni=www.nvidia.com&sid=6ba85179e3
0d4fc2&spx=%2F&flow=xtls-rprx-vision#my-reality
```

Aqui `pbk` – chave pública X25519, `sni` – domínio emprestado de **Destino**, `sid` – um dos **Short IDs**, `flow=xtls-rprx-vision` – XTLS Vision ativado. 2. **TLS com domínio próprio:** selecione **TLS**, preencha **SNI** com o nome do domínio, adicione o certificado (por caminho para os arquivos ou por conteúdo) ou clique em «Usar certificado do painel» se o domínio e o certificado já estiverem configurados em «Configurações → Segurança». Mantenha **Versão Mín/Máx** = `1.2 – 1.3` e **uTLS** = `chrome` para imitar um navegador comum. 3. Não deixe o modo **Nenhum** para inbounds expostos externamente – use-o apenas para destinos de fallback locais (`127.0.0.1`) ou quando o TLS é fornecido por um proxy externo. 4. Dica da interface: para a maioria dos campos avançados, a dica é «*Recomenda-se manter as configurações padrão*» – altere-os apenas quando entender as consequências.

8. Clientes

Cliente é uma conta de usuário VPN: um conjunto de credenciais (UUID ou senha) vinculado a um ou mais inbound, com cota de tráfego própria, prazo de validade e limite de conexões simultâneas. Neste fork, o cliente é uma entidade independente (tabela `clients`): um mesmo cliente pode ser vinculado a vários inbound ao mesmo tempo, mantendo UUID/senha em comum e um contador de tráfego compartilhado. A seção **Clientes** exibe todas as contas do painel independentemente do inbound, com pesquisa, filtros, ordenação e operações em massa.

8.1. Campos do cliente

A seguir são detalhados todos os campos do editor **Adicionar cliente** / **Editar cliente**.

O formulário do cliente é dividido em duas abas: **Principal** (email, vinculação ao inbound, limites, prazo, grupo, comentário, reverse tag) e **Credenciais** (UUID/senha/auth, Flow, VMess Security). Nas etiquetas dos campos, a cota aparece como **Limite de tráfego (GB)** e os prazos como **Duração (dias)** e **Renovação automática (dias)**; os campos **Limite de tráfego (GB)** e **Limite de IP** exibem dicas explicando que `0` significa "sem restrições". Ao editar um cliente existente, o botão de geração de email aleatório fica oculto, e o botão de log de IP aparece diretamente ao lado do campo **Limite de IP**, mostrando o número de endereços registrados.

Campo	Chave JSON	Padrão	Descrição
Email	<code>email</code>	— (obrigatório)	Identificador único do cliente
UUID	<code>id</code>	gerado	Identificador para VMess/VLESS
Senha	<code>password</code>	gerada	Senha para Trojan/Shadowsocks
Autorização	<code>auth</code>	gerada	Senha para Hysteria
Flow	<code>flow</code>	vazio	Flow control (XTLS), somente VLESS
VMess Security	<code>security</code>	<code>auto</code>	Método de criptografia VMess
Limite de IP	<code>limitIp</code>	<code>0</code> (sem limite)	Máximo de IPs simultâneos
Total enviado/recebido (GB)	<code>totalGB</code>	<code>0</code> (sem limite)	Cota de tráfego
Prazo de validade	<code>expiryTime</code>	<code>0</code> (sem validade)	Data de expiração
Renovação automática	<code>reset</code>	<code>0</code> (desativado)	Período de reinício do tráfego, dias
ID de usuário do Telegram	<code>tgId</code>	<code>0</code> (nenhum)	ID numérico do Telegram
ID de assinatura	<code>subId</code>	gerado	Identificador de assinatura
Grupo	<code>group</code>	vazio	Rótulo lógico de agrupamento
Comentário	<code>comment</code>	vazio	Nota livre
Ativado	<code>enable</code>	<code>true</code>	Se a conta está ativa

Email (identificador)

O campo **Email** é o identificador principal e obrigatório do cliente. Apesar do nome, não precisa ser um endereço de e-mail: qualquer rótulo textual serve (nome de usuário, número). O valor deve ser **único** dentro do painel; tentar criar um segundo cliente com um email já em uso é rejeitado (`email already in use`), exceto quando o `subId` também coincide (o que é interpretado como vinculação do mesmo cliente).

O Email **não pode ficar vazio** (`client email is required`) e **não pode conter espaços, os caracteres / , \ ou caracteres de controle** ("Email não pode conter espaços, '/', ' ou caracteres de controle"). O email participa da contabilização de tráfego, do log de IP, da lista de online e nos nomes das operações — não é recomendado alterá-lo retroativamente.

UUID / Senha / Autorização (credenciais)

O campo de credenciais específico depende do protocolo do inbound ao qual o cliente está vinculado. Os valores são preenchidos automaticamente se o campo for deixado vazio:

- **UUID** (campo `id`) — para os protocolos **VMess** e **VLESS**. Se não definido, um UUID v4 aleatório é gerado.
- **Senha** (campo `password`) — para **Trojan** e **Shadowsocks**. Para Trojan, por padrão é gerado um UUID sem hífen. Para Shadowsocks é gerada uma chave em Base64 do comprimento adequado de acordo com o método de criptografia do inbound: 16 bytes para `2022-blake3-aes-128-gcm`, 32 bytes para `2022-blake3-aes-256-gcm` e `2022-blake3-chacha20-poly1305`; para outros métodos — UUID sem hífen. Se uma chave inserida manualmente não for compatível com o método `2022-blake3`, ela será substituída por uma gerada.
- **Autorização** (campo `auth`) — senha para **Hysteria**. Por padrão, UUID sem hífen.

Como um cliente pode ser vinculado a inbound de protocolos diferentes, o registro do cliente pode ter simultaneamente UUID, senha e auth — cada protocolo usa seu próprio campo.

Exemplo: como as credenciais do cliente aparecem em settings de diferentes inbound. O mesmo cliente em um inbound VLESS é identificado por `id`, em Trojan — por `password`, em Shadowsocks — por `password` (chave Base64):

```
// fragmento de settings.clients no inbound VLESS
{ "id": "b831381d-6324-4d53-ad4f-8cda48b30811", "email": "user-a", "flow": "xtls-rprx-vision" }

// o mesmo cliente no inbound Trojan
{ "password": "b831381d63244d53ad4f8cda48b30811", "email": "user-a" }

// o mesmo cliente no inbound Shadowsocks (método 2022-blake3-aes-256-gcm)
{ "password": "GPy0aA3f7C00az53eaQ8eqMfRDjmBl0h7v1u3+Z+pHk=", "email": "user-a" }
```

Flow

Flow (campo `flow`) — controle de fluxo XTLS. Aplicável **somente ao VLESS** e apenas quando o inbound está configurado para XTLS Vision: VLESS sobre transporte **TCP** com security **tls** ou **reality**. O valor permitido é

`xtls-rprx-vision` (bem como o histórico `xtls-rprx-vision-udp443`); um valor vazio indica ausência de flow.

Se o inbound não suporta XTLS-flow, o flow definido é **silenciosamente zerado** ao salvar o cliente: para o mesmo cliente vinculado a vários inbound, o flow é aplicado apenas onde é permitido. Altere somente se estiver usando intencionalmente VLESS-Vision.

VMess Security

VMess Security (campo `security`) – método de criptografia da carga útil para VMess. O valor padrão é `auto` (o Xray escolhe a cifra automaticamente). Os valores permitidos são os padrões para VMess: `auto`, `aes-128-gcm`, `chacha20-poly1305`, `none`, `zero`. Para outros protocolos, o campo não é utilizado.

Limite de IP (conexões simultâneas)

Limite de IP (campo `limitIp`) – número máximo de **endereços IP diferentes** a partir dos quais o cliente pode estar conectado simultaneamente. O valor padrão é `0`, que significa **sem restrição**. Com um valor positivo, o painel rastreia os IPs ativos do cliente e, ao ultrapassar o limite, desativa a conta por meio de uma tarefa em segundo plano. (A partir da **3.3.1** a contagem de IPs ocorre via API de online-stats do núcleo Xray e **não requer** log de acesso; em versões mais antigas do núcleo, o painel retorna à leitura do log de acesso, que deve estar ativado.) Use para impedir o compartilhamento de uma assinatura entre muitos dispositivos: por exemplo, `2` – permite dois dispositivos.

O Limite de IP é aplicado via **Fail2ban**, portanto o campo **Limite de IP** só está ativo quando o Fail2ban está instalado e em funcionamento (o painel verifica seu status via `GET /panel/api/server/fail2banStatus`). Se o Fail2ban não estiver instalado, o campo no editor do cliente (e no formulário de adição em massa) é bloqueado, e ao passar o cursor aparece uma dica sugerindo instalar o Fail2ban pelo menu bash `x-ui` ("Fail2ban is not installed, so the IP limit cannot be enforced. Install Fail2ban from the x-ui bash menu to enable this option."); no Windows a dica informa que o Fail2ban não está disponível ("Fail2ban is not available on Windows, so the IP limit cannot be enforced."), e se o recurso estiver desativado no servidor – "The IP limit feature is disabled on this server.". Ao atualizar o painel, o limite de IP salvo dos clientes em servidores sem Fail2ban é zerado por uma migração única, pois de qualquer forma ele não era aplicado ali.

Exemplo de valores. `limitIp: 0` – sem restrição; `limitIp: 1` – estritamente um dispositivo por vez; `limitIp: 3` – até três IPs diferentes. Com o quarto IP ativo, a tarefa em segundo plano desativará o cliente (`enable = false`) até que você execute **Reiniciar limite de IP**.

Operações relacionadas: **Log de IP** exibe a lista de IPs registrados do cliente; cada registro contém, além do próprio IP, o horário do último acesso e o rótulo do nó (`@ nome_do_nó`) pelo qual a conexão foi registrada – em uma configuração multipainel é possível ver por qual nó o cliente se conectou. **Reiniciar limite de IP** limpa o log de IP acumulado para que o cliente possa conectar-se novamente sem aguardar a expiração natural dos registros.

Total enviado/recebido (GB) – cota de tráfego

Total enviado/recebido (GB) (campo `totalGB`) – cota total de tráfego (envio + recebimento). O valor padrão – `0` – significa **sem limite**. Ao atingir a cota (`up + down >= total`), o cliente é considerado **esgotado**.

(depleted) e é desativado. Na interface, geralmente é inserido em gigabytes; no banco de dados é armazenado em bytes.

Na lista de clientes, a coluna **Tráfego** exibe uma barra colorida de uso: volume de tráfego consumido, rótulo do limite (ou símbolo ∞ quando sem limite) e uma dica ao passar o cursor com o detalhamento de enviado/recebido e o saldo restante. O mesmo indicador compacto é exibido nos cards de clientes no celular.

Prazo de validade (Expiry)

Prazo de validade (campo `expiryTime`) define o momento de expiração da conta. O campo tem três modos:

- **Sem validade** — `0`. O cliente nunca expira por tempo.
- **Data específica** — Unix-timestamp positivo (em milissegundos). Ao chegar o momento (`expiryTime <= agora`), o cliente é considerado expirado e desativado. Na interface, geralmente é definido escolhendo uma data ou uma duração em dias (**Duração**, unidade — **Dias**).
- **Início após o primeiro uso** — valor **negativo**, codificando a duração. Enquanto o cliente não transmitir nenhum byte, o prazo permanece negativo ("início adiado"). No primeiro ciclo de contagem de tráfego, o painel o converte em uma data absoluta: `agora + |duração|`. Isso permite vender, por exemplo, "30 dias a partir da primeira conexão", sem saber de antemão quando o cliente será ativado. A conversão é realizada uma vez por email, para que todos os inbound vinculados recebam o mesmo prazo.

Exemplo de codificação do prazo. Data fixa 1º de março de 2026, 00:00 UTC → `expiryTime: 1772323200000` (timestamp positivo em milissegundos). "30 dias a partir da primeira conexão" → `expiryTime: -2592000000` (valor negativo, $30 \times 24 \times 60 \times 60 \times 1000$); no primeiro byte de tráfego o painel o substituirá por `agora + 2592000000`. Sem validade → `expiryTime: 0`.

Renovação automática (período de reinício do tráfego do cliente)

O campo **Renovação automática** (campo `reset`) é o período de renovação/reinício automático em dias. Dica: "Renovação automática após o término. (0 = desativado) (unidade: dia)".

- `0` — renovação automática **desativada** (valor padrão). Após o término do prazo, o cliente simplesmente torna-se esgotado.
- `> 0` — a tarefa em segundo plano, ao expirar o prazo, **zera os contadores de tráfego** (`up = down = 0`), **avança o prazo de validade** em `reset` dias (se necessário, por vários períodos, até que o novo prazo fique no futuro) e, se necessário, **reativa** o cliente. Isso implementa uma assinatura periódica (por exemplo, mensal). A renovação automática **não se aplica a inbound em nós remotos** (`node_id IS NOT NULL`).

Consequência importante: clientes com `reset > 0` são **excluídos** do conceito de "esgotado" nas operações de exclusão em massa — o tráfego/prazo deles é zerado pela renovação automática como esperado, e não os torna candidatos à exclusão.

ID de usuário do Telegram

ID de usuário do Telegram (campo `tgId`) — identificador numérico do Telegram do usuário para vinculação ao bot Telegram integrado ao painel (notificações, visualização autônoma de estatísticas). Dica: "ID numérico do usuário Telegram (0 = nenhum)". O valor padrão `0` — sem vinculação. Este campo está disponível para filtragem (**Com / Sem**).

ID de assinatura (subId)

ID de assinatura (campo `subId`) – identificador pelo qual o cliente é incluído em uma **assinatura** (subscription). Todos os clientes com o mesmo `subId` são entregues por um único link de assinatura. Se o campo for deixado vazio ao criar, o painel **gera automaticamente um** `subId` aleatório (UUID). O valor deve ser **único** entre clientes com email diferente (`subId already in use`) e está sujeito às mesmas restrições de caracteres que o email ("O ID de assinatura não pode conter espaços, '/', " ou caracteres de controle").

Sem `subId`, o link de assinatura do cliente não está disponível ("Este cliente não tem subId, o link de compartilhamento não está disponível").

Aba Links (links externos e assinaturas)

Além das abas **Principal** e **Credenciais**, o editor do cliente possui uma terceira aba **Links** (dica: "Add third-party share links and remote subscription URLs to include in this client's subscription."). Nela, o botão **Add External Link** adiciona links de compartilhamento de terceiros (`vless://`, `vmess://`, `trojan://`, `ss://`, `hysteria2://`, `wireguard://`), e o botão **Add External Subscription** – URLs de assinaturas remotas (por exemplo, `https://provider.example/sub/...`).

Tudo isso é mesclado na saída da assinatura deste cliente (formatos raw, JSON e Clash): os links são adicionados como estão, e as assinaturas remotas são baixadas periodicamente pelo painel (com cache e timeout curto) e suas configurações são combinadas com as do próprio painel. Assim, em um único link de assinatura do cliente, é possível fornecer tanto os servidores próprios quanto configurações externas.

Grupo

Grupo (campo `group`) – rótulo lógico para agrupar clientes relacionados. Dica: "Rótulo lógico para agrupar clientes relacionados (por exemplo, equipe, cliente, região). Filtrável pela barra de ferramentas.", placeholder – "por exemplo, customer-a". O campo é opcional (vazio por padrão). É possível filtrar a lista por grupo e realizar operações em massa; para remover o rótulo de um cliente, use a ação **Desagrupar**.

Também é possível remover o grupo diretamente no editor de um único cliente: se o campo **Grupo** for limpo e salvo, o rótulo é corretamente removido e o cliente deixa de aparecer no grupo anterior.

Comentário

Comentário (campo `comment`) – nota textual livre para o administrador (vazio por padrão). O conteúdo é incluído na pesquisa e está disponível para filtragem (**Com** / **Sem** comentário).

Ativado

Ativado (campo `enable`) – flag de atividade da conta. Por padrão **ativado** (`true`); ao criar, mesmo que o flag não seja passado, o painel define `true` obrigatoriamente. Um cliente desativado (`enable = false`) não pode se conectar e na visão geral pertence à categoria **inativos** (deactive). O painel desativa automaticamente os clientes que esgotaram a cota, expiraram ou ultrapassaram o limite de IP.

Campos somente leitura

No card do cliente também são exibidos campos de serviço: **Criado** (`created_at`) e **Atualizado** (`updated_at`) – marcas de tempo de criação e da última alteração, preenchidas automaticamente e não editáveis. O campo **Reverse tag** (`reverse`) – reverse tag opcional para proxy reverso simples VLESS ("Reverse tag opcional").

8.2. Vinculação ao inbound

Cada cliente deve estar vinculado a pelo menos um inbound – ao criar é exigido no mínimo um (`at least one inbound is required`). No editor, este campo é chamado **Entradas vinculadas** com a dica **Selecione uma ou mais entradas**.

- **Vincular** – adicionar o cliente aos inbound selecionados (mesmo UUID/senha e tráfego compartilhado). As vinculações existentes são preservadas.
- **Desvincular** – remover o cliente dos inbound selecionados. O registro do cliente é preservado (para exclusão completa use **Excluir**). Pares nos quais o cliente não estava vinculado são ignorados silenciosamente.

Ao salvar um cliente vinculado a vários inbound, os campos incompatíveis com o protocolo/transporte específico (por exemplo, Flow fora do VLESS-Vision) são automaticamente ajustados para valores permitidos em cada inbound.

Acima da lista de seleção de inbound (no formulário do cliente, ao adicionar clientes em massa e nas janelas de vinculação/desvinculação em massa) há botões **Selecionar todos** e **Limpar**. Nessas listas, cada inbound é identificado pelo seu remark (se definido), caso contrário – pela tag do inbound.

8.3. Operações sobre o cliente

Para um cliente individual (via card **Informações do cliente** ou menu de contexto **Ações**) estão disponíveis:

Visualização de informações, QR code e link

- **Informações do cliente** – card com todos os campos, tráfego usado/restante (**Saldo**), prazo de validade e inbound vinculados. A partir da 3.5.0, o card também mostra o **grupo** do cliente (linha «Grupo» com um rótulo, acima de «Comentário»), quando um grupo está definido.

A consulta do cliente via API (`GET /panel/api/clients/get/:email`) além dos campos `client` e `inboundIds` retorna adicionalmente `usedTraffic` – o tráfego efetivamente consumido (enviado + recebido, incluindo dados dos nós), o que facilita a comparação do consumo com a cota `totalGB`.

- **QR code e Link** – link de configuração do cliente para importar em um aplicativo cliente. Gerado para todos os inbound vinculados com protocolo suportado (`GET /links/:email`). Se não houver links adequados: "Não há links de compartilhamento – primeiro vincule o cliente a uma entrada com protocolo suportado".
- **Link de assinatura** – URL de assinatura pelo `subId` (`GET /subLinks/:subId`). Disponível somente se o cliente tiver `subId` e o serviço de assinatura estiver habilitado em **Configurações do painel** → **Assinatura** (caso contrário "Serviço de assinatura desativado."). Adicionalmente é fornecida a **URL de assinatura JSON**.

Reiniciar tráfego

Reiniciar tráfego (`POST /resetTraffic/:email`) zera os contadores de envio/recebimento (`up` , `down`) do cliente específico. A cota (`totalGB`) e o prazo de validade **não são afetados** – apenas o volume consumido é zerado. Toast: "Tráfego reiniciado". Se o cliente não estiver vinculado a nenhum inbound: "Primeiro vincule este cliente a uma entrada."

O botão **Reiniciar tráfego** também está disponível no formulário de edição do cliente – no painel inferior, ao lado de **Cancelar** / **Salvar** (é solicitada confirmação antes de reiniciar). Se o cliente foi desativado por esgotamento de tráfego, o reinício (individual ou em massa) automaticamente o **reativa** (`enable = true`) e envia essa alteração imediatamente para os nós remotos – não é mais necessário reativar o cliente manualmente no master e nos nós.

Reiniciar limite de IP

Limpa o log de IP acumulado do cliente (`POST /clearIps/:email`) para remover o bloqueio temporário por exceder o limite de conexões simultâneas. Toast: "O log foi limpo".

Excluir

Excluir (`POST /del/:email`) – exclusão completa do cliente. Diálogo de confirmação: título "Excluir cliente {email}?", texto "O cliente será removido de todas as entradas vinculadas e seu registro de tráfego será destruído. Esta ação não pode ser desfeita.". A exclusão remove o cliente de **todos** os inbound e destrói seu registro de tráfego. Toast: "Cliente excluído".

8.4. Operações em massa

Na lista de clientes é possível marcar vários registros (**Selecionar todos**, **Limpar todos**); contador – "{count} selecionados". Sobre os selecionados estão disponíveis:

- **Excluir ({count})** (`POST /bulkDel`) – exclusão em grupo. Confirmação: "Excluir {count} clientes?", "Cada cliente selecionado é removido de todas as entradas vinculadas, seu registro de tráfego é destruído. Esta ação não pode ser desfeita.". Toast: "Clientes excluídos: {count}", em caso de falha parcial – "Excluídos: {ok}, falhas: {failed}".
- **Editar ({count}) / Ajuste** (`POST /bulkAdjust`) – alteração em massa do prazo e/ou cota. Diálogo "Editar {count} clientes" com a dica "Valores positivos adicionam, negativos reduzem. Clientes com prazo ou tráfego ilimitado são ignorados para o campo correspondente.". Campos: **Adicionar dias**, **Adicionar tráfego (GB)** e **Set flow**. Lógica:
 - **Prazo:** clientes com prazo ilimitado (`expiryTime == 0`) são ignorados ("unlimited expiry"); para clientes com data, o prazo é deslocado pelo número de dias informado; para clientes no modo "após o primeiro uso" (prazo negativo), a duração de espera é ajustada. Reduções que ultrapassem o saldo restante são ignoradas ("reduction exceeds remaining time/delay window").
 - **Tráfego:** clientes com tráfego ilimitado (`totalGB == 0`) são ignorados ("unlimited traffic"); caso contrário, a cota é alterada pelo volume informado, sem descer abaixo de zero.
 - **Flow:** a lista suspensa **Set flow** permite definir ou remover o XTLS flow de todos os clientes selecionados de uma vez. Por padrão está selecionado **No change** (sem alterações). A opção **Disable (clear flow)** remove o flow, e os valores `xtls-rprx-vision` e `xtls-rprx-vision-udp443` definem

o vision-flow correspondente. A definição do vision-flow é aplicada somente aos inbound que suportam flow; os inbound incompatíveis permanecem inalterados e são marcados como ignorados, enquanto a remoção do flow é sempre permitida.

- **Reativação automática (a partir da 3.4.2):** um cliente desativado **apenas por esgotamento** (prazo vencido ou cota excedida), ao ser estendido, é reativado automaticamente – localmente e no seu nó – se o ajuste o devolver aos limites. Os desativados manualmente ou ainda esgotados permanecem desligados (o campo `enable` agora é gravado explicitamente, por isso o estado é preservado mesmo em clientes sem inbound).
- Se não forem informados nem dias, nem tráfego, nem flow: "Informe dias, tráfego ou flow antes de aplicar.". Toast: "Editados: {count}" / "Editados: {ok}, ignorados: {skipped}"/>.

Exemplo: estender os clientes selecionados por 30 dias e adicionar 50 GB. No diálogo **Editar**, informe **Adicionar dias** = `30`, **Adicionar tráfego (GB)** = `50`. Para, ao contrário, subtrair uma semana e reduzir a cota em 10 GB, insira valores negativos: **Adicionar dias** = `-7`, **Adicionar tráfego (GB)** = `-10` (clientes com prazo ilimitado ou sem limite de tráfego no campo correspondente serão ignorados).

- **Vincular ({count}) / Desvincular ({count})** (`POST /bulkAttach / bulkDetach`) – vinculação/desvinculação em massa dos clientes selecionados aos inbound selecionados. Os destinos são apenas inbound multiusuário. Resultado da desvinculação: "Desvinculados {detached}, ignorados {skipped}.". A partir da 3.5.0, o seletor de inbound no formulário do cliente **oculta os inbound desativados** (exceto aqueles aos quais o cliente já está vinculado), e na janela «Desvincular» foi eliminado o acúmulo de registros duplicados de um mesmo cliente (os dados são corrigidos automaticamente na inicialização do painel).
- **Links de assinatura ({count})** – tabela resumida de URLs de assinatura e assinatura JSON dos clientes selecionados com o botão **Copiar todos**. Se nenhum tiver subId: "Nenhum dos clientes selecionados possui ID de assinatura."
- **Adicionar ao grupo e Desagrupar** – atribuição e remoção do rótulo de grupo.
- **Ativar ({count}) / Desativar ({count})** (`POST /bulkEnable / bulkDisable`) – ativação e desativação em massa dos clientes selecionados. **Ativar** ativa cada cliente selecionado em todos os inbound vinculados; clientes com cota de tráfego esgotada ou prazo expirado serão automaticamente desativados novamente. **Desativar** revoga imediatamente o acesso dos clientes, mas seus registros e o tráfego acumulado são preservados. Antes da execução, o painel solicita confirmação e, após a operação, exibe uma notificação com a quantidade de clientes processados e, se houver, com a quantidade para os quais a ação falhou.

Reinício de tráfego e exclusão por status

- **Reiniciar tráfego de todos os clientes** (`POST /resetAllTraffics`) – zera os contadores `up / down` de **todos** os clientes do painel. Confirmação: "Reiniciar tráfego de todos os clientes?" e "Os contadores de envio/recebimento de todos os clientes são zerados. Cotas e prazos de validade não são afetados. Esta ação não pode ser desfeita.". Toast: "Tráfego de todos os clientes reiniciado".
- **Excluir esgotados** (`POST /delDepleted`) – exclui todos os clientes cuja **cota está esgotada** (`total > 0 and up + down >= total`) **ou o prazo expirou** (`expiry_time > 0 and expiry_time <= agora`), com a condição `reset = 0` (clientes com renovação automática não são afetados). Confirmação: "Excluir clientes esgotados?", "Todos os clientes com cota de tráfego esgotada ou prazo expirado são excluídos. Esta ação não pode ser desfeita.". Toast: "Clientes esgotados excluídos: {count}"/>.

Exportação, importação e exclusão de clientes não vinculados

Quando nada está selecionado, no menu **Mais** da página **Clientes** estão disponíveis três operações.

Exportar clientes (GET /clients/export) abre um visualizador com a lista JSON de todos os clientes no formato {client, inboundIds} com botões de cópia e download (arquivo clients-export.json).

Importar clientes (POST /clients/import) abre um editor no qual é colado o mesmo JSON e se clica em **Import**: clientes com inboundIds são criados e vinculados aos inbound, clientes sem vinculações são restaurados como registros "soltos" independentes, e emails já existentes **nunca são sobrescritos** — eles entram na lista de ignorados. Toasts: "{count} clients imported", "{ok} imported, {failed} skipped".

Excluir clientes não vinculados (POST /clients/deleteorphans) — operação perigosa: exclui todos os clientes não vinculados a nenhum inbound, junto com seus registros de tráfego, log de IP e links externos. Confirmação: "Delete clients without an inbound?", "Removes every client that is not attached to any inbound, along with its traffic record. This cannot be undone.". Toast: "{count} unattached clients deleted". A ação é irreversível.

8.5. Pesquisa, filtros e ordenação

Acima da lista há uma barra de pesquisa ("Pesquisar email, comentário, sub ID, UUID, senha, auth, Telegram ID...") — ela pesquisa por email, comentário, subId, UUID, senha, auth e (novamente a partir da 3.5.0) por **Telegram ID**. Contador de resultados: "Exibindo {shown} de {total}".

A lista de clientes é atualizada automaticamente: o painel busca a página atual a cada poucos segundos, portanto clientes recém-conectados e a ordem de classificação alterada aparecem sem atualização manual (o indicador de carregamento não pisca durante a consulta em segundo plano).

A partir da 3.5.0, a tabela tem a coluna **«Velocidade»** (entre «Tráfego» e «Saldo»): a velocidade ao vivo do cliente — um rótulo azul ↑ envio / ↓ recebimento (média móvel de ~5 segundos, atualizada a cada 5 segundos); sem tráfego — um traço cinza. Em dispositivos móveis, a velocidade é exibida como uma linha no card do cliente.

O painel **Filtrar clientes** permite selecionar por status (categorias), protocolo, inbound vinculado, intervalo de prazo de validade, intervalo de tráfego utilizado, presença de renovação automática (**Com/Sem**), presença de ID do Telegram e comentário, além de grupo. Em painéis com nós aparece um multisseletor **Nós**: é possível restringir a lista a clientes dos nós selecionados; um item separado **Painel local** seleciona clientes de inbound sem vinculação a nó (o filtro é visível somente quando há nós). Ordenação: **Mais antigos/mais novos primeiro**, **Atualizados recentemente**, **Online recentemente**, **Email A→Z / Z→A**, **Mais tráfego**, **Mais saldo**, **Expirando em breve**.

8.6. Ícones e status

Prioridade de status: esgotado/expirado → inativo → expirando em breve → ativo.

- **Online / Offline** — cliente com conexão ativa (presente na lista online atual) e **ativado**. A lista online é atualizada por requisições separadas (/onlines , /onlinesByGuid).
- **Esgotado** (depleted) — cota consumida (up + down >= totalGB) **ou** prazo expirado (expiryTime <= agora). Esse cliente é desativado automaticamente e fica sujeito à ação **Excluir esgotados**.

- **Expirando em breve** (expiring) – cliente ativado com menos do que o intervalo limite até a expiração do prazo **ou** com menos do que o volume limite até o esgotamento da cota (os limites são definidos nas configurações do painel). Não se aplica se o cliente já estiver esgotado/desativado.
 - **Inativo** (deactive) – cliente com `enable = false` (desativado manualmente ou pela tarefa em segundo plano).
 - **Ativo** (active) – ativado, não esgotado, prazo não expirado e ainda longe dos limites.
-

9. Grupos de clientes

Esta é uma funcionalidade exclusiva deste fork do 3X-UI. No projeto original 3x-ui (MHSanaei) não existe o conceito de "grupo de clientes" – aqui foram adicionados uma tabela separada de grupos, a página **Grupos** no menu do painel e os métodos de API correspondentes. Se você migrar a configuração para o 3x-ui original, o rótulo de grupo simplesmente não será considerado em nenhum lugar.

9.1. O que é um grupo de clientes e para que serve

Grupo é um rótulo lógico nomeado (label) que pode ser atribuído a um ou mais clientes. Ele não cria uma nova forma de conexão e não é um inbound nem um nó – trata-se de um marcador puramente organizacional, útil para filtrar e processar clientes em massa.

A ideia central do modelo de clientes neste fork: **o cliente é uma entidade de nível superior, identificada pelo email** (o campo `email` na tabela `clients` possui índice único). Um mesmo cliente (um email com as mesmas credenciais) pode estar associado simultaneamente a vários inbound e até a vários nós, inclusive com protocolos diferentes. O rótulo de grupo é armazenado **uma única vez por cliente** e, portanto, é propagado automaticamente para todas as suas associações a inbound de uma só vez.

O rótulo de grupo é um marcador lógico de agrupamento:

Camada	Onde é armazenado	Campo
Registro do cliente (BD)	tabela <code>clients</code>	<code>group_name</code> (por padrão string vazia <code>' '</code>)
Cadastro de grupos (BD)	tabela <code>client_groups</code>	<code>name</code> (índice único, não vazio)
Configurações do inbound (Xray)	JSON <code>settings.clients[].group</code>	duplicado em cada objeto de cliente de cada inbound ao qual o cliente pertence

Para que serve na prática:

- **Um cliente em vários inbound/nós.** Se um cliente é "vendido" como acesso a vários inbound simultaneamente (por exemplo, protocolos diferentes ou nós diferentes), o grupo permite gerenciá-lo como uma unidade: zerar o tráfego, excluir, renomear o rótulo – em uma única operação em todos os seus inbound.
- **Operações em massa e filtragem.** Na página **Clientes**, a lista pode ser filtrada por grupo; na página **Grupos** estão disponíveis ações em massa sobre todos os membros do grupo.
- **Organização de um grande número de clientes.** Rótulos como `vip`, `trial`, `team-A` ajudam a distribuir milhares de clientes em categorias lógicas sem precisar criar inbound separados.

9.2. Relação do grupo com clientes, inbound, nós e protocolos

Esta é a subseção mais importante para compreensão, pois a sincronização do rótulo não é trivial.

O grupo descreve o cliente, não o inbound. O rótulo reside no registro do cliente (`clients.group_name`). Quando um cliente está associado a vários inbound, a qualquer mudança de grupo o painel percorre **todos** os

inbound nos quais esse cliente está presente e insere/remove o campo `group` dentro das configurações Xray deles (`settings.clients[]`). Tecnicamente isso funciona assim: pelo email do cliente são localizados todos os inbound dos quais ele faz parte, depois o objeto de cliente com esse email é editado no JSON de configurações de cada um desses inbound. Portanto:

- O grupo **não depende do protocolo**. Um mesmo email pode ser cliente VLESS em um inbound e cliente Hysteria em outro – o rótulo de grupo continuará sendo um só e será aplicado a ambos (as credenciais de cada protocolo são mantidas separadamente).
- O grupo **abrange os nós**. Os inbound pertencentes a nós se diferenciam dos inbound do painel principal pelo campo `nodeId` (nos inbound do painel principal ele é `null / 0`). O rótulo de grupo é propagado para os objetos de cliente nos inbound independentemente de ser um inbound principal ou de nó – desde que o cliente com esse email esteja presente nele.

O rótulo de grupo é resistente à sincronização com nós e à reconstrução de configurações. Esse comportamento foi projetado intencionalmente:

- Quando um nó envia um snapshot de tráfego, seus dados **não sobrescrevem** os campos locais `group_name` e `comment` do cliente no BD do painel. O grupo e o comentário são considerados campos "locais do painel" – o nó não os gerencia.
- Na reconstrução das configurações do inbound, um valor vazio de `group` nos dados recebidos **não redefine** o rótulo já salvo. O grupo é gerenciado exclusivamente pela página **Grupos** (e não pela edição das configurações Xray do inbound), portanto "grupo vazio" em uma reconstrução normal é interpretado como "não alterar", e não como "limpar".

Conclusão prática: as únicas operações que **intencionalmente limpam** o rótulo são a exclusão do grupo e a remoção explícita do cliente do grupo (veja abaixo). Edições comuns do inbound ou sincronização em segundo plano com o nó não perderão o grupo.

9.3. Cadastro de grupos e grupos "vazios"

A lista de grupos na página é formada pela união de duas fontes:

1. **Grupos derivados (derived)** – todos os valores não vazios de `group_name` efetivamente presentes nos clientes, com contagem do número de clientes.
2. **Grupos salvos (stored)** – registros da tabela `client_groups` .

Essa união gera um efeito importante: um grupo pode existir **sem nenhum cliente**. Esse grupo é criado pelo botão "Adicionar grupo" (registro em `client_groups`) e é exibido na lista com o contador `0` . Esses registros são os chamados **grupos vazios**. A lista é sempre ordenada por nome sem distinção de maiúsculas/minúsculas.

Contadores de resumo na página:

Campo	O que mostra
Total de grupos	Número total de grupos (salvos e derivados juntos).
Clientes com grupo	Quantos clientes possuem um rótulo de grupo não vazio.
Grupos vazios	Quantos grupos existem sem clientes (contador <code>0</code>).
Clientes no grupo	Número de clientes em um grupo específico (coluna da tabela).

9.4. Campos e colunas do grupo

O registro do grupo na tabela `client_groups` contém:

Campo	Tipo	Padrão	Descrição
<code>Id</code>	int	auto-incremento	Chave primária do registro do grupo.
<code>Name</code>	string	— (obrigatório)	Nome do grupo. Índice único, não pode ser vazio. Na UI — coluna Nome .
<code>CreatedAt</code>	int64 (ms)	hora de criação	Momento de criação do registro do grupo.
<code>UpdatedAt</code>	int64 (ms)	hora de alteração	Momento da última modificação.

Na tabela da página são exibidas no mínimo as colunas **Nome** e **Clientes no grupo**, além dos botões de ação (veja abaixo).

9.5. Criação de grupo

Botão **Adicionar grupo**.

Passos:

1. Clique em **Adicionar grupo**.
2. Insira o nome do grupo.
3. Confirme.

Comportamento do backend (`POST /panel/api/clients/groups/create` , corpo `{"name": "..."}`):

- O nome é cortado nos espaços das extremidades. Um nome vazio é rejeitado com o erro «group name is required».
- Se já existir um grupo com esse nome — erro «group already exists».
- Em caso de sucesso, é criado um registro em `client_groups` (inicialmente sem clientes — trata-se de um grupo vazio).

Mensagem de sucesso: «Grupo "{name}" criado.»

Exemplo: criar um grupo vazio via API. Prepare um conjunto de rótulos com antecedência, antes de preenchê-los com clientes:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/create' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"name": "vip"}'
```

Resposta em caso de sucesso:

```
{ "success": true, "msg": "Группа «vip» создана.", "obj": null }
```

Uma chamada repetida com o mesmo nome retornará `"success": false` e a mensagem `group already exists`.

Criar um grupo vazio com antecedência é conveniente quando você quer preparar um conjunto de rótulos e depois preenchê-los com clientes usando "Adicionar clientes..."

9.6. Renomear grupo

Botão **Renomear**, título do diálogo — «**Renomear {name}**».

Comportamento (`POST /panel/api/clients/groups/rename` , corpo `{"oldName": "...", "newName": "..."} :`

- Ambos os nomes são cortados nos espaços. Nome antigo vazio — erro «old group name is required», nome novo vazio — «new group name is required».
- Se o nome novo for igual ao antigo — nada é feito (0 clientes afetados).
- Caso contrário, a renomeação é executada atômicaamente:
 - o registro em `client_groups` é renomeado;
 - em todos os clientes com `group_name = oldName` o campo é atualizado para `newName` ;
 - em **todos os inbound** dos quais os clientes afetados fazem parte (incluindo os de nós), o valor de `group` nas configurações Xray é corrigido do antigo para o novo.
- Após a renomeação, o painel marca o Xray como necessitando de reinicialização e envia notificação sobre alteração de clientes.

Mensagens:

- Sucesso: «**Grupo renomeado para {count} cliente(s).**»
- Conflito de nomes na UI: «**Já existe um grupo com o nome "{name}"**».

9.7. Adição de clientes ao grupo

Botão **Adicionar clientes...**, título — «**Adicionar clientes ao grupo "{name}"**».

Dica literal no diálogo:

«Selecione os clientes para adicionar a este grupo. As associações existentes a inbound são mantidas; somente o rótulo de grupo é alterado. Clientes já pertencentes a este grupo não são exibidos.»

Se não houver ninguém a adicionar, é exibida a mensagem «**Não há outros clientes para adicionar.**»

Comportamento (`POST /panel/api/clients/groups/bulkAdd` , corpo `{"emails": [...], "group": "..."} :`

- O nome do grupo é obrigatório (caso contrário, erro «group name is required»); lista de emails vazia — a operação não faz nada.
- Se esse grupo ainda não existir em `client_groups` nem entre os derivados — ele será criado automaticamente.

- Para os emails selecionados, o campo `group_name` dos clientes é definido como `group`; **as associações dos clientes a inbound não são alteradas** – somente o rótulo é afetado. Em seguida, o campo `group` é definido em todos os inbound desses clientes.
- É retornado o número de registros de clientes afetados; o Xray é marcado para reinicialização.

Mensagem de sucesso: «**{count} cliente(s) adicionado(s) a {name}.**»

Exemplo: marcar vários clientes com um grupo em uma única requisição. Os clientes são especificados por email e as associações a inbound não são alteradas:

```
curl -s 'https://panel.example.com:2053/panel/api/clients/groups/bulkAdd' \
-H 'Content-Type: application/json' \
-b cookie.txt \
-d '{"emails": ["alice@example.com", "bob@example.com"], "group": "vip"}'
```

Se o grupo `vip` ainda não existir, ele será criado automaticamente. Após a requisição, o campo `group_name` = `"vip"` será definido no registro desses clientes, e o objeto de cliente em cada um dos seus inbound nas configurações Xray receberá o campo `"group": "vip"`:

```
{ "id": "6f1b...", "email": "alice@example.com", "group": "vip", "enable": true }
```

9.8. Remoção de clientes do grupo (sem excluir os clientes em si)

Botão **Remover clientes...**, título – «**Remover clientes do grupo "{name}"**».

Dica literal:

«Selecione os membros para remover deste grupo. Os próprios clientes são mantidos (use "Excluir clientes do grupo" para exclusão completa).»

Comportamento (`POST /panel/api/clients/groups/bulkRemove`, corpo `{"emails": [...]}`): tecnicamente é o mesmo que "Adicionar ao grupo" com um grupo vazio. O campo `group_name` dos clientes selecionados é limpo e o campo `group` é removido das configurações Xray dos seus inbound. Os próprios clientes e suas associações a inbound são mantidos.

Mensagem de sucesso: «**{count} cliente(s) removido(s) de {name}.**»

9.9. Zerar tráfego do grupo

Botão **Zerar tráfego**.

Diálogo de confirmação:

- Título: «**Zerar tráfego do grupo {name}?**»
- Texto: «**Apenas o contador de tráfego do grupo é zerado. Os contadores dos clientes individuais não são afetados.**»

Comportamento: é zerado **apenas o contador exibido do próprio grupo** — o painel memoriza a soma atual de `up/down` do grupo como marca de base e, na lista de grupos, passa a exibir `max(0, soma - marca de base)`. Os contadores `up / down`, as cotas e o campo `enable` dos clientes individuais **não são alterados**, e não é necessário reiniciar o Xray. Esta é uma mudança de comportamento em relação às versões anteriores, em que o reset zerava o tráfego de todos os clientes do grupo.

Requisição: `POST /panel/api/clients/groups/resetTraffic` com o corpo `{"name": "<nome do grupo>"}`.

Mensagem de sucesso: «Tráfego do grupo {name} zerado.»

9.10. Exclusão do grupo e exclusão dos clientes do grupo

Na página existem **duas operações de exclusão fundamentalmente diferentes** — é fácil confundi-las, por isso a distinção é crítica.

9.10.1. Excluir grupo (manter clientes)

Botão «**Excluir grupo (manter clientes)**».

Diálogo:

- Título: «**Excluir grupo {name}?**»
- Texto: «**Isso exclui o grupo e limpa seu rótulo em {count} cliente(s). Os próprios clientes não são excluídos.**»

Comportamento (`POST /panel/api/clients/groups/delete`, corpo `{"name": "..."}`): o registro do grupo é excluído de `client_groups`, o campo `group_name` de todos os seus clientes é limpo e o campo `group` é removido dos seus inbound. **Os clientes, suas conexões e o tráfego são mantidos.** O Xray é marcado para reinicialização.

Mensagem de sucesso: «**Grupo limpo em {count} cliente(s).**»

9.10.2. Excluir clientes do grupo (exclusão completa)

Botão «**Excluir clientes do grupo**».

Diálogo:

- Título: «**Excluir todos os clientes em {name}?**»
- Texto: «**Isso exclui permanentemente {count} cliente(s) junto com seus registros de tráfego. O rótulo do grupo também é limpo. Essa ação não pode ser desfeita.**»

Esta é uma operação destrutiva: ela exclui os próprios clientes (por meio de exclusão em massa por email, endpoint `POST /panel/api/clients/bulkDel`), incluindo seus registros de tráfego, removendo-os assim de todos os inbound.

Mensagens:

- Sucesso: «**{count} cliente(s) excluído(s).**»

- Resultado parcial: «{ok} excluído(s), {failed} ignorado(s)»

Se o grupo estiver vazio, as ações sobre seus membros não estão disponíveis – é exibida a mensagem «Este grupo ainda não possui clientes.»

9.11. Relação com a página "Clientes"

O rótulo de grupo é visível e utilizado também fora da página **Grupos**:

- No registro compacto do cliente existe o campo `group`, portanto a pertinência ao grupo é exibida na lista de clientes.
- A lista de clientes (`/panel/api/clients/list/paged`) aceita o parâmetro de filtro `group`: é possível passar um nome ou vários separados por vírgula. A correspondência é feita com lógica "OU" dentro do campo, sem distinção de maiúsculas/minúsculas. Caso especial: um elemento vazio na lista de grupos do filtro significa "clientes sem grupo" (cujo `group` está vazio).
- Na resposta da página de clientes é retornado o array `groups` – lista completa dos nomes de grupos existentes, para que a UI possa construir a lista suspensa de filtro.

Exemplo: filtrar clientes por grupos. A requisição retorna apenas clientes com os rótulos `vip` ou `trial` (vários nomes – separados por vírgula, lógica "OU"):

```
GET /panel/api/clients/list/paged?group=vip,trial
```

Para obter clientes **sem** grupo, passe um elemento vazio na lista – por exemplo, o valor de filtro `group=` (string vazia) ou `group=vip,` (rótulo `vip` mais clientes sem grupo).

9.12. Resumo dos endpoints de API

Todas as rotas de grupos estão montadas sob `/panel/api/clients`:

Método e caminho	Finalidade	Corpo da requisição
GET <code>/panel/api/clients/groups</code>	Lista de grupos com contadores de clientes	–
GET <code>/panel/api/clients/groups/:name/emails</code>	Emails de todos os membros do grupo (ordenados por email)	–
POST <code>/panel/api/clients/groups/create</code>	Criar grupo vazio	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/rename</code>	Renomear grupo	<code>{"oldName", "newName"}</code>
POST <code>/panel/api/clients/groups/delete</code>	Excluir grupo mantendo clientes (limpeza do rótulo)	<code>{"name"}</code>
POST <code>/panel/api/clients/groups/bulkAdd</code>	Adicionar clientes ao grupo (por email)	<code>{"emails": [...], "group"}</code>

Método e caminho	Finalidade	Corpo da requisição
POST /panel/api/clients/groups/bulkRemove	Remover clientes do grupo (limpeza do rótulo)	<code>{"emails": [...]}</code>
POST /panel/api/clients/bulkDel	Exclusão completa de clientes (usada por "Excluir clientes do grupo")	<code>{"emails": [...], "keepTraffic"}</code>

Exemplo: cenário típico de ciclo de vida de um grupo via API.

```
# 1. Criar o rótulo trial
curl -s ../panel/api/clients/groups/create -d '{"name":"trial"}'

# 2. Atribuí-lo a dois clientes
curl -s ../panel/api/clients/groups/bulkAdd -d '{"emails": ["u1@example.com", "u2@example.com"], "group":"trial"}'

# 3. Zerar o tráfego de todos os membros (por email de /groups/trial/emails)
curl -s ../panel/api/clients/groups/bulkRemove -d '{"emails":["u2@example.com"]}'

# 4. Excluir o grupo, mas manter os clientes (somente limpeza do rótulo)
curl -s ../panel/api/clients/groups/delete -d '{"name":"trial"}'
```

O passo 4 exclui o registro do grupo e limpa `group_name` nos seus clientes, mas os próprios clientes, suas conexões e o tráfego são mantidos. Para a exclusão permanente dos próprios clientes, usa-se `bulkDel` em vez disso.

As operações que alteram o rótulo nos clientes (`rename` , `delete` , `bulkAdd` , `bulkRemove`) marcam o Xray como necessitando de reinicialização e enviam notificação sobre alteração de clientes.

9.13. Tráfego por grupo

Novidade da versão 3.3.0: na seção **Grupos** (página "Clientes", aba de gerenciamento de grupos) a tabela de grupos agora exibe não apenas o número de clientes em cada grupo, mas também o tráfego total consumido pelo grupo. A coluna é intitulada «**Tráfego utilizado**».

O que a coluna mostra

Para cada linha de grupo é exibida a soma do tráfego de todos os clientes pertencentes a esse grupo – ou seja, `up + down` (tráfego enviado + recebido) somados de todos os seus membros. Isso fornece uma resposta rápida à pergunta "quanto o grupo todo baixou/enviou no total", sem precisar abrir os clientes um por um e somar manualmente.

Ao lado na tabela de grupos são exibidas:

Coluna	O que significa
Nome	Nome do grupo
Clientes	Quantos clientes estão marcados com este grupo (antes a coluna se chamava "Clientes no grupo")
Enviado	Total de <code>up</code> (tráfego enviado) de todos os clientes do grupo

Coluna	O que significa
Recebido	Total de <code>down</code> (tráfego recebido) de todos os clientes do grupo
Tráfego utilizado	Total de <code>up + down</code> de todos os clientes do grupo

O tráfego enviado e recebido são exibidos em colunas separadas **Enviado** e **Recebido**, enquanto a coluna **Tráfego utilizado** mostra a soma dos dois. A coluna de número de clientes se chama simplesmente **Clientes**.

O resumo acima da tabela exibe adicionalmente agregados de todos os grupos – «**Total de grupos**» e «**Clientes com grupo**», e o tráfego total é dividido em dois cartões: «**Total enviado / recebido**» (com setas para cima/baixo – tráfego enviado e recebido separadamente de todos os grupos) e «**Tráfego total**» (com ícone de diagrama – soma total dos dois).

Como é calculado

O cálculo é feito por uma única consulta SQL à tabela de clientes com junção (`LEFT JOIN`) da tabela de contabilização de tráfego:

- pelo campo de rótulo do grupo (`group_name`) os clientes são agrupados e seu número é contado – isso é o "Clientes no grupo";
- o tráfego é obtido como a soma de `up + down` da tabela `client_traffics` anexada. Ou seja, são somados os bytes enviados (`up`) e recebidos (`down`) de cada cliente;
- como o email é único tanto na tabela de clientes quanto na tabela de tráfego, a junção não duplica o tráfego de um único cliente.

Particularidades dos valores:

- **Clientes sem registro de tráfego** são contados no contador de membros, mas adicionam 0 à soma, portanto um grupo recém-criado exibe tráfego `0`.
- **Grupos vazios** (criados, mas sem clientes) também estão presentes na lista com contador zero e tráfego zero: além dos grupos "derivados" dos rótulos dos clientes, os grupos explicitamente salvos são mesclados ao resultado, após o que a lista é ordenada por nome sem distinção de maiúsculas/minúsculas.
- Clientes sem rótulo de grupo (`group_name` vazio) não entram no cálculo.

Ações relacionadas

A partir da tabela de grupos ainda estão disponíveis ações sobre o grupo inteiro, incluindo «**Zerar tráfego**» – zera **apenas o contador do próprio grupo** (a marca de base), sem tocar no tráfego dos clientes individuais (veja [9.9](#)). Após esse reset, a coluna "Tráfego utilizado" para esse grupo exibe `0`.

10. Assinaturas (Subscription)

Uma assinatura (subscription) é um mecanismo que permite fornecer ao cliente um único link permanente (URL), pelo qual o cliente VPN baixa e atualiza periodicamente o conjunto completo de configurações. Em vez de enviar manualmente ao usuário um link separado para cada inbound, é transmitido um único endereço no formato `https://domínio:porta/sub/<subId>`. Por esse endereço, o painel monta em tempo real todas as configurações vinculadas ao cliente e as entrega no formato desejado. Quando as configurações do servidor mudam (novo endereço, rotação de chaves Reality, adição de inbound), o cliente recebe a configuração atualizada na próxima atualização automática, sem exigir nenhuma ação do usuário.

A assinatura é servida por um servidor HTTP/HTTPS separado dentro do painel, que é iniciado independentemente do painel web e escuta em sua própria porta. Isso é feito por razões de segurança: a porta de assinatura pode ser aberta para o exterior sem precisar abrir a porta do painel em si.

10.1. O que é subId e como o link é formado

Cada cliente em um inbound possui o campo `subId` (na interface – «ID da assinatura»). É exatamente esse valor que funciona como chave da assinatura: o painel busca em todos os inbounds os clientes cujo `subId` corresponde ao solicitado e combina suas configurações em uma única resposta.

- Se vários clientes (em um ou em diferentes inbounds) tiverem o mesmo `subId`, suas configurações serão incluídas em uma única assinatura. Essa é a forma padrão de fornecer a um usuário vários servidores/protocolos por um único link.

Exemplo: um usuário – dois servidores em um único link. Suponha que existam dois inbounds (VLESS no servidor A e Trojan no servidor B). Para fornecer ao usuário ambas as configurações com um único link, atribua o mesmo `subId` a ambos os seus clientes:

```
Inbound 1 (VLESS): email = ivan@vpn, subId = ivan2025
Inbound 2 (Trojan): email = ivan@vpn, subId = ivan2025
```

Então, pelo endereço `https://sub.example.com:2096/sub/ivan2025`, o painel entregará ambas as configurações de uma vez. Se você adicionar posteriormente um terceiro inbound com o mesmo `subId`, ele aparecerá para o usuário na próxima atualização automática da assinatura, sem necessidade de enviar um novo link.

- Se o campo `subId` do cliente estiver vazio, não é possível compartilhar o link de acesso geral. A interface indica isso com a dica: «Este cliente não tem subId, o link de acesso compartilhado não está disponível.»

Links externos e assinaturas do cliente (aba «Links»)

No formulário do cliente há uma aba «**Links**», onde para um cliente específico é possível anexar fontes adicionais de configurações que são incorporadas à assinatura dele (formatos RAW, JSON e Clash):

- **Add External Link** – link de compartilhamento externo (`vless://`, `trojan://`, `ss://`, etc.). É adicionado à saída como está, e para JSON/Clash é adicionalmente analisado em configuração.

- **Add External Subscription** — endereço de assinatura externa. O painel baixa a assinatura automaticamente (com cache e um curto tempo limite) e incorpora as configurações obtidas na lista geral do cliente.

Isso é conveniente para fornecer ao cliente servidores adicionais além dos seus inbounds pelo mesmo link único. Se a resposta da assinatura remota for muito grande, ela não é mais truncada silenciosamente: o painel retorna um erro e continua usando o último valor armazenado em cache com sucesso.

- O valor de `subId` não pode ser definido arbitrariamente: ao salvar, é verificado que ele não contém espaços, caracteres `/`, `,`, `\` nem caracteres de controle. A dica de validação correspondente é: «O ID da assinatura não pode conter espaços, '/', ',' ou caracteres de controle».

O link final é construído como `<base>/<subPath>/<subId>` (consulte a seção sobre configurações do servidor de assinaturas e o campo «URI de proxy reverso»). Se nenhum cliente for encontrado pelo `subId` (o cliente foi excluído, o `subId` não existe), o servidor retorna HTTP 404 sem corpo. Em caso de erro interno — HTTP 500. Os clientes VPN se orientam apenas pelo código de resposta, portanto o corpo do erro é intencionalmente vazio.

Ordem dos links de inbound na assinatura

Cada inbound possui o campo «**Ordem na assinatura**» (`subSortIndex`) — um número a partir de 1, que define a posição dos links desse inbound na saída da assinatura. Valores menores vêm primeiro; em caso de valores iguais, a ordem original de criação (por id) é preservada. A ordem se aplica a todos os formatos de saída — texto bruto, página de assinatura, JSON e Clash. Esse campo não afeta a ordem dos inbounds no próprio painel.

O campo é editado no formulário do inbound, ao lado das configurações de endereço no link (share address), e é sincronizado para os nós pelas regras habituais. Se pelo menos um inbound tiver uma ordem diferente de 1, uma coluna compacta «**Ordem**» aparece na lista de Inbounds.

10.2. Configurações do servidor de assinaturas

Todos os parâmetros de assinatura estão na seção de configurações do painel, na aba «**Assinatura**». Abaixo, cada parâmetro é detalhado; entre parênteses estão a chave de configuração interna e o valor padrão.

A própria seção é dividida em abas: «**Configurações do painel**», «**Informações**», «**Perfil**», «**Certificados**», «**Happ**» e «**Clash / Mihomo**». Os campos de título da assinatura, URL de suporte, página de perfil, avisos e catálogo de temas estão na aba «**Perfil**»; as regras de roteamento Happ e Clash/Mihomo — nas abas correspondentes; o intervalo de atualização da assinatura — na aba «**Informações**».

Parâmetros principais

Campo (UI)	Chave	Valor padrão	Descrição
Habilitar assinatura	<code>subEnable</code>	<code>true</code> (habilitado)	Inicia o servidor de assinaturas separado. Dica: «Função de assinatura com configuração separada». Se desabilitado, o servidor de assinaturas não inicia e nenhum dos links funciona.
IP de escuta	<code>subListen</code>	vazio	Endereço IP no qual o servidor de assinaturas aceita conexões. Dica: «Deixe vazio por padrão para monitorar todos os endereços IP».

Campo (UI)	Chave	Valor padrão	Descrição
Porta da assinatura	subPort	2096	Porta TCP do servidor de assinaturas. Dica: «O número de porta para atender ao serviço de assinatura não deve estar em uso no servidor» – a porta deve estar livre e não conflitar com o painel ou Xray.
Caminho URI	subPath	/sub/	Caminho pelo qual as assinaturas comuns são entregues. Dica: «Deve começar com '/' e terminar com '/'».
Domínio de escuta	subDomain	vazio	Domínio pelo qual o acesso à assinatura é permitido (validação de Host). Dica: «Deixe vazio por padrão para escutar todos os domínios e endereços IP». Se definido, requisições com Host diferente são rejeitadas.

Importante sobre segurança: o caminho padrão `/sub/` (e `/json/` para JSON) é amplamente conhecido e facilmente detectável. O painel exibe o aviso: «O caminho de assinatura padrão `/sub/` é amplamente conhecido – altere-o.» e um aviso similar para JSON. Recomenda-se definir um caminho próprio e não óbvio.

TLS / certificado

Campo (UI)	Chave	Padrão	Descrição
Caminho do arquivo de chave pública do certificado de assinatura	subCertFile	vazio	Caminho completo para o arquivo de certificado (<code>.crt</code> / <code>fullchain</code>). Dica: «Insira o caminho completo começando com '/'».
Caminho do arquivo de chave privada do certificado de assinatura	subKeyFile	vazio	Caminho completo para o arquivo de chave privada. Dica: «Insira o caminho completo começando com '/'».

Se ambos os caminhos estiverem definidos e o certificado for carregado com sucesso, o servidor de assinaturas opera via **HTTPS**. Se os campos estiverem vazios ou o certificado não puder ser lido, o servidor volta para **HTTP** (o erro é registrado no log). A presença de TLS válido também afeta a formação da URL base: na porta 443 com TLS e na porta 80 sem TLS, o número da porta é omitido no link.

Intervalo de atualização

Campo (UI)	Chave	Padrão	Descrição
Intervalos de atualização da assinatura	subUpdates	12	Com que frequência (em horas) o aplicativo cliente deve re-solicitar a assinatura. Dica: «Intervalo entre atualizações no aplicativo cliente (em horas)». A partir da 3.5.0, o campo aceita 0-525600 e mostra o intervalo (o antigo limite de 168 da interface não coincidia com o servidor e, após o upgrade da 2.x, bloqueava o salvamento de quaisquer configurações).

O valor é transmitido ao cliente no cabeçalho HTTP `Profile-Update-Interval` ; os clientes modernos o utilizam como período de atualização automática da configuração.

Formato e informações na resposta

Campo (UI)	Chave	Padrão	Descrição
Codificar	subEncrypt	true	Dica: «Criptografar as configurações retornadas na assinatura». Tecnicamente, isso não é criptografia, mas sim codificação Base64 de todo o corpo da assinatura comum (formato esperado pela maioria dos clientes). Quando desabilitado, os links são entregues em texto simples, um por linha.
Mostrar informações de uso	subShowInfo	true	Dica: «Exibir o tráfego restante e a data de expiração após o nome da configuração». Quando habilitado, marcadores de tráfego restante () e prazo de validade (por exemplo, 5D, 3H) são adicionados ao nome (remark) de cada configuração; para clientes expirados/ indisponíveis, é exibido N/A .
Incluir e-mail no nome	subEmailInRemark	true	Dica: «Incluir o e-mail do cliente no nome do perfil de assinatura». Adiciona o e-mail do cliente ao remark do perfil.

Modelo de remark (Remark Template)

O nome exibido (remark) de cada configuração na assinatura é formado pelo **modelo de remark** – campo «**Modelo de observação**» (remarkTemplate) na aba «**Informações**» das configurações de assinatura. O antigo construtor de modelo de observação (seleção separada de partes inbound/email/proxy externo e símbolo separador) foi removido da interface; agora você escreve um formato de nome arbitrário e insere variáveis nele. O valor padrão é `{{INBOUND}}-{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` (ou seja, por padrão o nome do perfil contém o e-mail do cliente). Se o campo for deixado vazio, o modelo de remark anterior (não configurável pela interface) é utilizado como alternativa.

As variáveis são agrupadas nas seções **Client**, **Traffic** e **Time & status** e exibidas ao lado do campo como chips clicáveis `{{VAR}}` com dica ao passar o mouse; um clique insere o token no modelo, com uma prévia ao vivo disponível. Cada variável é substituída individualmente para o cliente específico no momento de geração da assinatura. A notação simplificada com chaves simples também é aceita (`{DATA_LEFT}` , `{EXPIRE_DATE}` , `{PROTOCOL}` , `{TRANSPORT}` , etc.) – o painel converte automaticamente para o formato interno `{{...}}` .

Variáveis disponíveis:

- **Identificação do cliente:** `{{EMAIL}}` (sinônimo – `{{USERNAME}}`), `{{INBOUND}}` (remark do próprio inbound), `{{HOST}}` (remark do host), `{{ID}}` (UUID), `{{SHORT_ID}}` (primeiros 8 caracteres do UUID), `{{SUB_ID}}` , `{{COMMENT}}` , `{{TELEGRAM_ID}}` , `{{PROTOCOL}}` , `{{TRANSPORT}}` .
- **Tráfego:** `{{TRAFFIC_USED}}` , `{{TRAFFIC_LEFT}}` , `{{TRAFFIC_TOTAL}}` (e suas variantes `*_BYTES` em bytes exatos), `{{UP}}` , `{{DOWN}}` , `{{USAGE_PERCENTAGE}}` .
- **Prazo e status:** `{{DAYS_LEFT}}` , `{{TIME_LEFT}}` , `{{EXPIRE_DATE}}` (AAAA-MM-DD), `{{JALALI_EXPIRE_DATE}}` (data no calendário jalali), `{{EXPIRE_UNIX}}` , `{{CREATED_UNIX}}` , `{{RESET_DAYS}}` , `{{STATUS}}` (active / expired / disabled / depleted), `{{STATUS_EMOJI}}` .
- **Conexão (Connection):** `{{PROTOCOL}}` – protocolo (VLESS, VMess, Trojan, etc.), `{{TRANSPORT}}` – rede de transporte (tcp, ws, grpc, etc.), `{{SECURITY}}` – segurança do transporte (TLS, REALITY, NONE; exibido em maiúsculas). Assim como as variáveis de consumo e prazo, essas três variáveis funcionam apenas no corpo da assinatura e são automaticamente removidas do remark nos links exibidos no painel (QR/«Informações») e na página de informações da assinatura.

O modelo pode ser dividido em segmentos com a barra vertical | . Um segmento no qual uma variável fornece um valor «ilimitado» (∞) – por exemplo `{{TRAFFIC_LEFT}}` ou `{{DAYS_LEFT}}` para um cliente sem restrições – é automaticamente ocultado. Além disso, o bloco com consumo de tráfego e prazo é exibido uma vez, no primeiro link do cliente, para não ser duplicado em cada configuração.

Exemplo. O modelo `{{EMAIL}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` para um cliente com 42 GB restantes e 7 dias resultará no nome `ivan@vpn 42.00GB 7D`, e para um cliente ilimitado – simplesmente `ivan@vpn` (segmentos com ∞ omitidos).

A partir da versão 3.4.2, a variável `{{EMAIL}}` (e o sinônimo `{{USERNAME}}`) é exibida apenas no **primeiro** link do cliente no corpo da assinatura – assim como o bloco de tráfego/prazo restante; nos demais links do mesmo cliente ela é omitida.

Nos links exibidos no painel (QR-code e janelas «Informações» na página «Clientes») e na página de informações da assinatura, o e-mail do cliente está presente no nome do perfil: no formato «inbound-host-email» quando um host está definido, ou «inbound-email» sem host. As informações de tráfego e prazo (bem como as variáveis do grupo «Conexão») não são substituídas nesses nomes exibidos – elas funcionam apenas no corpo da assinatura que o cliente VPN recebe.

Se a linha de estatísticas de tráfego do cliente ficou «órfã» após exclusão e recriação do inbound, a variável `{{TRAFFIC_USED}}` (e outros indicadores de consumo) não mostra mais `0.00B`: o painel busca adicionalmente as estatísticas pelo e-mail do cliente e substitui o tráfego utilizado correto. | Modelo de remark | `remarkTemplate|{{INBOUND}}|{{TRAFFIC_LEFT}}|{{DAYS_LEFT}}D` | Modelo livre do nome exibido (remark) de cada configuração com substituição de variáveis `{{VAR}}`. É substituído individualmente para cada cliente durante a geração da assinatura. O antigo construtor de «modelo de observação» (seleção de inbound/email/proxy externo e separador) foi removido da interface e é usado apenas como alternativa se o campo for deixado vazio. Para mais detalhes, consulte «Modelo de remark (Remark Template)» abaixo. |

Metadados do perfil (cabeçalhos de resposta)

Essas strings são transmitidas ao cliente nos cabeçalhos HTTP de resposta e exibidas no cliente VPN como metadados do perfil. Todas estão vazias por padrão.

Campo (UI)	Chave	Cabeçalho	Descrição
Título da assinatura	<code>subTitle</code>	<code>Profile-Title</code> (em Base64)	«Nome da assinatura visível pelo cliente no aplicativo VPN». Para Clash, também é usado como nome do perfil importado via <code>Content-Disposition</code> .
URL de suporte	<code>subSupportUrl</code>	<code>Support-Url</code>	«Link para suporte técnico, exibido no aplicativo VPN».
URL do perfil	<code>subProfileUrl</code>	<code>Profile-Web-Page-Url</code>	«Link para o seu site, exibido no aplicativo VPN». Se não definido, a URL real da requisição de assinatura é utilizada.
Aviso	<code>subAnnounce</code>	<code>Announce</code> (em Base64)	«Texto do aviso exibido no aplicativo VPN». A partir da 3.5.0, o texto também é exibido como banner informativo no topo da página de informações da assinatura (quando não vazio), e a variável <code>announce</code> fica disponível para os modelos personalizados.

Além disso, cada resposta inclui o cabeçalho `Subscription-Userinfo` com dados de tráfego agregados do cliente: `upload`, `download`, `total` e `expire` (momento de expiração em segundos). Com base nisso, o cliente exibe o tráfego restante e o prazo de validade.

Roteamento (apenas para o cliente Happ)

Campo (UI)	Chave	Padrão	Descrição
Habilitar roteamento	<code>subEnableRouting</code>	<code>false</code>	«Configuração global para habilitar o roteamento no aplicativo VPN. (Apenas para Happ)». Transmitido no cabeçalho <code>Routing-Enable</code> .
Regras de roteamento	<code>subRoutingRules</code>	vazio	«Regras de roteamento globais para o aplicativo VPN. (Apenas para Happ)». Transmitido no cabeçalho <code>Routing</code> .

| Ocultar configurações do servidor | `subHideSettings` | `false` | «Ocultar as configurações do servidor na assinatura (apenas para Happ)». Quando habilitado, o cliente Happ oculta a possibilidade de visualizar e alterar os parâmetros do servidor. A opção funciona apenas para o cliente Happ. |

Roteamento Incy (apenas para o cliente Incy)

Para o cliente VPN **Incy**, nas configurações de assinatura há uma aba separada **«Incy»** com dois campos: o seletor **«Habilitar roteamento»** (`subIncyEnableRouting`, desabilitado por padrão) e o campo de texto **«Regras de roteamento»** (`subIncyRoutingRules`) no formato `incy://routing/onadd/<base64>`. Quando o roteamento está habilitado e o campo está preenchido, essa string é acrescentada como uma linha separada no corpo da assinatura (formato raw) — assim, o perfil de roteamento é entregue ao cliente Incy sem conflitar com o cabeçalho `Routing` do cliente Happ. As configurações funcionam apenas para o cliente Incy.

URI de proxy reverso

Campo (UI)	Chave	Padrão	Descrição
URI de proxy reverso	<code>subURI</code>	vazio	«Alterar o URI base da URL de assinatura para uso atrás de servidores proxy».

Se o campo estiver vazio, o endereço base do link é construído pelo painel a partir do domínio e da porta da assinatura (levando em conta o TLS). Se a assinatura for distribuída via proxy reverso/CDN externo em outro domínio ou caminho, o URI base final é definido nesse campo e todos os links serão construídos a partir dele. Campos individuais equivalentes existem para JSON (`subJsonURI`) e Clash (`subClashURI`).

Se apenas o `subURI` geral estiver definido e os campos individuais para JSON e Clash estiverem vazios, os links desses formatos na página de assinatura herdam o esquema e o host do `subURI` (e não a porta do servidor `sub` e `http`) — assim, eles correspondem ao endereço do proxy reverso.

Exemplo: assinatura atrás de proxy reverso. A própria assinatura escuta na porta `2096`, mas externamente está disponível via nginx/CDN em `https://cfg.example.com/u/`. Para que os links na resposta sejam construídos a partir do endereço externo, e não do interno `domínio:2096`, o URI base final é definido no campo «Reverse proxy URI»:

Reverse proxy URI: `https://cfg.example.com/u`

Então o link final terá o formato `https://cfg.example.com/u/ivan2025`. Para os formatos JSON e Clash, se necessário, os campos individuais `subJsonURI` e `subClashURI` são preenchidos da mesma forma.

10.3. Formatos de saída

A assinatura pode ser entregue em três formatos independentes, cada um com seu próprio endpoint, que pode ser habilitado/desabilitado separadamente.

Endereço do servidor e nós na saída

O endereço do servidor nos links de assinatura é substituído pela mesma estratégia de endereço no link usada pelos links comuns e QR-codes no painel: «listen» – endereço de bind roteável, «custom» – endereço personalizado definido pelo usuário (`shareAddr`), «node» (padrão) – endereço do nó. Para inbounds sem uma estratégia explicitamente definida, a saída da assinatura não muda. Isso permite que um inbound de nó vinculado a um IP público específico entregue um endereço acessível aos clientes. A estratégia se aplica aos formatos raw, JSON e Clash.

O nome do nó (Node) não é adicionado ao nome (remark) do perfil na assinatura: no aplicativo cliente, apenas o remark do inbound definido pelo administrador é exibido, sem o sufixo interno no formato `@nome-do-nó`.

Para distinguir entradas de mesmo nome em uma assinatura multinó, defina remarks diferentes manualmente ou use hosts gerenciados (Hosts) com seus próprios Remark.

Se, devido a dessincronização entre nós, o mesmo cliente entrou duas vezes em um inbound JSON de serviço, a saída da assinatura elimina automaticamente esses duplicados por e-mail em todos os três formatos, de modo que perfis repetidos não aparecem na saída.

Hosts gerenciados (Hosts)

A seção **Hosts** (item do menu lateral; página de resumo com contagem Total/Enabled/Disabled e lista) define substituições de endereço para links de assinatura. Para cada inbound, é possível adicionar um ou mais **hosts** – endpoints que são substituídos nos links de assinatura entregues ao cliente **no lugar do endereço, porta e parâmetros TLS do próprio inbound**. Isso é conveniente para distribuir tráfego via CDN ou relay sem alterar o próprio inbound.

A partir da versão 3.5.0, um host é um «grupo»: um único registro abrange **vários inbound** e **vários endereços** de uma vez. As ações da lista (habilitar/desabilitar/excluir/reordenar) e a API trabalham com grupos (`groupId` string); surgiu o endpoint em massa `POST /panel/api/hosts/bulk/add ( {"inboundIds": [...], "hosts": [...], ...} )`, e `list / get / update / del / setEnable` operam sobre objetos de grupo.

Cada host (grupo) possui:

- **Remark** e descrição (Description), vinculação a **Inbounds** (seleção múltipla com pesquisa; no mínimo um), seletor **Enable** e atribuição a nós (**Nodes**).
- **Address** – a partir da 3.5.0 é uma **lista de endereços** (entrada por tags; separadores – vírgula, ponto e vírgula, espaço; placeholder `cdn.example.com`, `cdn2.example.com:443`). Cada entrada pode ter seu

próprio `:porta` embutido (inclusive IPv6 entre colchetes `[::1]:443`); a dica suspensa sugere endereços já usados por outros hosts. Lista vazia – herda o endereço do próprio inbound (na lista, isso é um rótulo laranja **Inherits**). **Port** (`0` – herda a porta do inbound) serve como porta padrão para as entradas sem porta embutida; **Tags** (consideradas apenas na assinatura RAW).

- Aba **Security** – `same` / `tls` / `none` / `reality` com SNI, fingerprint, ALPN, certificado fixado (pinned-cert), `allowInsecure` e ECH.
- Aba **Advanced** – cabeçalho Host, Path, rota VLESS, Mux, Sockopt, Final Mask e exclusão do host de formatos individuais de assinatura (raw / json / clash). A partir da 3.5.0, o **Final Mask do host entra também nos links raw** (`fm=` ; antes – apenas em JSON/Clash): as máscaras TCP/UDP do host são adicionadas às máscaras do inbound, e os parâmetros QUIC do host só são usados se o inbound não os tiver. O seletor **Allow insecure** agora também atua para **Hysteria/Hysteria2** (`insecure=1` no link, `skip-cert-verify: true` no Clash).
- Aba **Clash (mihomo)** – versão IP, Mihomo X25519, embaralhamento de hosts (Shuffle host).

Na lista, a coluna **Endpoint** mostra os endereços como chips (o primeiro visível, os demais – no popover «+N»), e a coluna **Inbounds** – chips de inbound coloridos por protocolo. A partir da 3.5.0, os hosts são ordenados **globalmente** (pela ordem de classificação, depois pela observação), e não dentro do inbound; habilitação, desabilitação e exclusão em massa são mantidas. Os hosts gerenciados substituem o antigo array External Proxy.

Rota VLESS (VLESS route). A partir da versão 3.4.2 este é um único número `0-65535` (e não uma lista de portas; a dica é – «um único valor VLESS route (0-65535), embutido no UUID, por exemplo 443; vazio – sem ele», placeholder `443`). O valor definido é realmente «embutido» no UUID de cada assinatura gerada (raw / JSON / Clash): o Xray lê os bytes 6-7 do UUID e os mascara antes da autenticação, por isso o cliente continua coincidindo. Um valor vazio ou inválido não altera o UUID.

Links comuns (SUB) – Base64 / texto simples

Formato base, endpoint `subPath` (padrão `/sub/`). Sempre habilitado (quando a assinatura está habilitada globalmente). Retorna uma lista de links Xray (`vless://` , `vmess://` , `trojan://` , `ss://` , etc.) – um por linha. Com a opção «Codificar» (`subEncrypt`) habilitada, toda a lista é codificada em Base64; quando desabilitada, é entregue em texto simples. Esse formato é compreendido por praticamente todos os clientes (v2rayNG, V2RayTun, Sing-box, NekoBox, Streisand, Shadowrocket, Happ, etc.).

Exemplo: corpo de resposta com «Codificar» desabilitado. Com `subEncrypt = false` , o endpoint `/sub/` entrega texto simples – um link por linha:

```
vless://3c8f...@a.example.com:443?security=reality&...#srvA-ivan
trojan://p4ss@b.example.com:443?security=tls&...#srvB-ivan
```

Com `subEncrypt = true` (padrão), a mesma lista é inteiramente codificada em Base64 e entregue em uma única string – é exatamente esse formato que a maioria dos clientes espera.

Assinatura JSON (sing-box e compatíveis)

Endpoint `subJsonPath` (padrão `/json/`), habilitado por uma opção separada.

Campo (UI)	Chave	Padrão	Descrição
Assinatura JSON	subJsonEnable	false	«Habilitar/desabilitar o endpoint JSON da assinatura de forma independente.».

Retorna a configuração JSON completa (formato compreendido pelo sing-box e clientes derivados – Podkop, OpenWRT sing-box, Karing, NekoBox). A partir da 3.5.0, os clientes de **inbound WireGuard nativos** também entram na assinatura JSON (`secretKey` , `address` , `peers[]` com `publicKey` / `endpoint` / `preSharedKey` / `keepAlive` / `allowedIPs` , `mtu`) – antes eles eram silenciosamente ignorados. Para esse formato, parâmetros adicionais estão disponíveis (aba `subFormats`):

- **Mux** (`subJsonMux` , padrão vazio) – configurações JSON de multiplexação (Mux) que são incorporadas ao outbound de cada stream da assinatura JSON. «Transmissão de múltiplos fluxos de dados independentes em uma única conexão.».
- **Final Mask** (`subJsonFinalMask` , padrão vazio) – «Máscaras finalmask xray (TCP/UDP) e configurações QUIC adicionadas a cada stream da assinatura JSON. Requer uma versão recente do xray no cliente.»
Configurado por subcampos: «Pacotes» (`packets`), «Comprimento» (`length`), «Intervalo» (`interval`), «Divisão máxima» (`maxSplit`), «Ruídos» (`noises` : «Tipo»/ `type` , «Pacote»/ `packet` , «Atraso (ms)»/ `delayMs` , «Aplicar a»/ `applyTo` , botão «+ Ruído»), além de «Paralelismo» (`concurrency`), «Paralelismo xudp» (`xudpConcurrency`) e «xudp UDP 443» (`xudpUdp443`).
- **Regras de roteamento** (`subJsonRules` , padrão vazio) – regras globais adicionadas à configuração JSON.

Assinatura Clash / Mihomo (YAML)

Endpoint `subClashPath` (padrão `/clash/`), habilitado por uma opção separada.

Campo (UI)	Chave	Padrão	Descrição
Assinatura Clash / Mihomo	subClashEnable	false	Habilita a geração de configuração YAML para clientes Clash e Mihomo.
Habilitar roteamento	subClashEnableRouting	false	«Adicionar regras de roteamento globais Clash/Mihomo às assinaturas YAML geradas.».
Regras de roteamento globais	subClashRules	vazio	«Regras Clash/Mihomo adicionadas ao início de cada assinatura YAML antes de MATCH,PROXY.».

A resposta é entregue com o tipo `application/yaml; charset=utf-8` . Se o «Título da assinatura» (`subTitle`) estiver definido, ele também é transmitido no cabeçalho `Content-Disposition` (`attachment; filename*=UTF-8'<title>`), para que o cliente Clash nomeie o perfil importado com esse nome.

O formato dos links e do YAML gerados é mantido atualizado para os clientes modernos: Shadowsocks-2022 (SS2022) não codifica mais userinfo em Base64; links Shadowsocks com ofuscação http são entregues no formato SIP002 com o plugin `obfs-local` ; para assinaturas Clash/Mihomo, um conjunto completo de campos XHTTP está implementado. A partir da 3.5.0, os clientes de **inbound WireGuard nativos** também entram na assinatura Clash/Mihomo (campos `private-key` , `public-key` , `pre-shared-key` , `persistent-keepalive` , `ip` / `ipv6` , `mtu` , `dns`) – antes eles eram silenciosamente ignorados. Isso não requer configurações separadas – os links são simplesmente reconhecidos de forma mais correta pelos clientes.

Observação: nesta versão, são suportados exatamente três formatos – links comuns (Base64/texto), JSON (compatível com sing-box) e Clash/Mihomo (YAML). Não há formato Outline separado no servidor de assinaturas.

10.4. Página de informações da assinatura e QR-codes

Se você abrir **qualquer um dos três links de assinatura** – raw, JSON ou Clash – em um navegador (ou adicionar explicitamente ao URL o parâmetro `?html=1` ou `?view=html`, ou enviar o cabeçalho `Accept: text/html`), o servidor, em vez da resposta «bruta», entrega uma **página visual de informações da assinatura** («Informações da assinatura»). Até a 3.5.0, apenas o link principal `/sub/` funcionava assim, enquanto `/json/` e `/clash/` entregavam ao navegador JSON/YAML bruto. Os clientes VPN ainda recebem a resposta legível por máquina, pois não solicitam HTML.

A página (aplicativo de página única construído com Vite) exhibe:

- **Informações da assinatura** (bloco Descriptions):
 - «ID da assinatura» – valor de `subId`;
 - «Status» – «Ativa», «Inativa» ou «Ilimitado». O status «inativa» é definido se o cliente estiver desabilitado, tiver esgotado o limite de tráfego ou o prazo de validade tiver expirado;
 - «Baixado» e «Enviado» – volumes de tráfego;
 - «Limite total» – limite de tráfego ou ∞ se não for limitado;
 - «Prazo de validade» – data de expiração ou «Sem prazo»;
 - tráfego restante e hora do último acesso online.
 - As datas são exibidas no calendário gregoriano ou jalali dependendo da configuração «Calendar Type» do painel (`datepicker`, padrão `gregorian`).
- **Links de assinatura**: para cada formato habilitado – uma linha separada com uma tag colorida (verde **SUB**, roxo **JSON**, dourado **CLASH**), botão de cópia e botão de **QR-code** (janela pop-up, tamanho 240 px). A linha com JSON e CLASH aparece apenas se o formato correspondente estiver habilitado nas configurações.
- **Links individuais** («Copiar link»): lista completa das configurações individuais incluídas na assinatura, cada uma com sua tag de protocolo, botão de cópia e QR-code (para links post-quantum, o QR não é gerado).
- **Botão «Copiar todas as configurações»** (acima da lista de links individuais): com um único clique, copia para a área de transferência todos os links de configuração (cada um em uma nova linha), sem precisar copiá-los um por um; após a conclusão, é exibida a notificação «Todas as configurações foram copiadas».
- **Botões de importação rápida em aplicativos** (menus suspensos por plataforma): para Android – v2box, v2rayNG (deep-link `v2rayng://install-config?url=...`), Sing-box, V2RayTun, NPV Tunnel, Happ (`happ://add/...`), Incy (`incy://add/...`); para iOS – Shadowrocket (via parâmetro `flag=shadowrocket`), v2box (`v2box://install-sub?url=...&name=...`), Streisand (`streisand://import/...`), V2RayTun, NPV Tunnel, Happ, Incy. Esses botões abrem o deep-link do aplicativo desejado com o endereço de assinatura já preenchido, ou copiam o link para a área de transferência.

A página de informações é entregue com cabeçalhos de proibição de cache (`Cache-Control: no-cache`), para que o cliente sempre veja os dados atualizados de tráfego e prazo de validade.

10.5. Modelos personalizados da página de assinatura

A partir da versão 3.3.0, é possível substituir a página de destino padrão da assinatura por um modelo HTML próprio. Por padrão, o endereço de assinatura entrega a página integrada, mas se um diretório com seu modelo for especificado, o painel irá renderizá-lo e inserir nele os dados atualizados do cliente (tráfego, prazo de validade, links, etc.).

Importante: o painel **não fornece** modelos prontos – o seu tema deve ser criado do zero. As instruções de criação e a lista de variáveis disponíveis estão em [docs/custom-subscription-templates.md](#).

Onde é habilitado

O diretório do tema é definido nas configurações do painel:

Configurações → **Assinatura** → seção «Informações da assinatura», campo «Diretório de temas da assinatura» (`subThemeDir`).

Descrição do campo na interface: «Caminho absoluto para a pasta com o modelo personalizado (index.html/ sub.html) para a página de assinatura (por exemplo, /etc/3x-ui/sub_templates/my-theme/). Deixe vazio para usar a página padrão.»

Na mesma seção estão as configurações relacionadas, cujos valores estão disponíveis no modelo:

Na descrição do campo «Diretório de temas da assinatura» há um link «**Guia de modelos** ↗» para a documentação sobre criação de modelos de estilo personalizados para a página de assinatura.

- «**Título da assinatura**» (`subTitle`) – nome visível pelo cliente;
- «**URL de suporte**» (`subSupportUrl`) – link para suporte técnico.

Parâmetro de configuração

Parâmetro	Valor padrão	Finalidade
<code>subThemeDir</code>	"" (vazio)	Caminho absoluto para o diretório com seu modelo HTML. Vazio = página padrão integrada.

Como definir seu modelo

1. Crie uma pasta para o tema no servidor (em qualquer lugar), por exemplo `/etc/3x-ui/sub_templates/my-theme/`.
2. Coloque dentro um arquivo HTML com o nome `index.html` ou `sub.html`.

Exemplo: caminho para o tema. Layout final no servidor e valor do campo nas configurações:

```
/etc/3x-ui/sub_templates/my-theme/
└─ index.html      (ou sub.html – tem prioridade)
```

Configurações → Assinatura → «Diretório de temas da assinatura»:
 /etc/3x-ui/sub_templates/my-theme/

O caminho deve ser **absoluto** (começar com /). Se a pasta não contiver nem `index.html` nem `sub.html`, o painel entregará a página integrada. 3. No painel, abra **Configurações** → **Assinatura** e insira o caminho **absoluto** para essa pasta no campo «Diretório de temas da assinatura». 4. Salve as configurações.

Comportamento de seleção de arquivo e renderização:

- Se o diretório contiver `sub.html`, ele será usado; caso contrário, `index.html` é utilizado. Ou seja, `sub.html` tem prioridade sobre `index.html`.
- O modelo é renderizado pelo mecanismo padrão do Go `html/template`.
- O modelo analisado é **armazenado em cache** e relido do disco somente quando o tempo de modificação do arquivo muda. Portanto, as edições no modelo são aplicadas sem reiniciar o painel, mas sem a sobrecarga de leitura/análise a cada requisição.
- A resposta é formada em buffer completo e somente então entregue ao cliente: se o modelo falhar durante a execução, uma página parcialmente gerada (corrompida) não será enviada ao usuário.

Comportamento padrão e fallback

- Campo vazio → a página SPA integrada é entregue (dados inseridos em `window.__SUB_PAGE_DATA__`).
- Caminho não existe ou não é um diretório → a página padrão é usada.
- O diretório não contém nem `index.html` nem `sub.html` → o aviso «subThemeDir set but no usable template found» é registrado no log, a página padrão é entregue.
- O arquivo de modelo existe, mas não pode ser analisado → o erro «custom template parse failed» é registrado no log, a página padrão é entregue.
- Erro durante a execução do modelo → «custom template execution failed» é registrado no log, a página padrão é entregue.

Ou seja, qualquer problema com o modelo personalizado não «quebra» a assinatura — o painel sempre recorre à página integrada. Todas as páginas de assinatura (tanto a personalizada quanto a padrão) são entregues com cabeçalhos de proibição de cache (`Cache-Control: no-cache, no-store, must-revalidate`), para que os clientes sempre recebam dados atualizados de tráfego e prazo.

Variáveis disponíveis no modelo

O contexto do modelo recebe um conjunto de dados do cliente da assinatura. Acesso via `{{ .nome }}`:

Variável	Tipo	Descrição
<code>{{ .sId }}</code>	string	ID da assinatura (UUID).
<code>{{ .enabled }}</code>	bool	Se o cliente/assinatura está habilitado.
<code>{{ .download }}</code>	string	Volume de download formatado (ex.: «2.5 GB»).
<code>{{ .upload }}</code>	string	Volume de upload formatado.
<code>{{ .total }}</code>	string	Limite total de tráfego formatado.

Variável	Tipo	Descrição
<code>{{ .used }}</code>	string	Tráfego utilizado formatado (download + upload).
<code>{{ .remained }}</code>	string	Tráfego restante formatado.
<code>{{ .expire }}</code>	int64	Prazo de validade – Unix time em segundos (<code>0</code> = sem prazo). Para <code>Date</code> no JS, multiplique por 1000.
<code>{{ .lastOnline }}</code>	int64	Hora do último acesso online – Unix time em milissegundos (<code>0</code> = nunca).
<code>{{ .downloadByte }}</code>	int64	Download em bytes exatos.
<code>{{ .uploadByte }}</code>	int64	Upload em bytes exatos.
<code>{{ .totalByte }}</code>	int64	Limite total em bytes exatos.
<code>{{ .subUrl }}</code>	string	URL da página de assinatura.
<code>{{ .subJsonUrl }}</code>	string	URL da configuração JSON da assinatura.
<code>{{ .subClashUrl }}</code>	string	URL da configuração Clash/Mihomo.
<code>{{ .subTitle }}</code>	string	Título da assinatura das configurações (pode estar vazio).
<code>{{ .subSupportUrl }}</code>	string	URL de suporte das configurações (pode estar vazio).
<code>{{ .links }}</code>	[]string	Lista de strings de configuração (VMess, VLESS, etc.). Iteração: <code>{{ range .links }} ... {{ end }}</code> .
<code>{{ .emails }}</code>	[]string	Lista de e-mails relacionados à assinatura.
<code>{{ .datepicker }}</code>	string	Formato de calendário atual do painel: <code>gregorian</code> ou <code>jalali</code> (obtido da configuração «Tipo de calendário»; se vazio – <code>gregorian</code>).

Exemplo mínimo do corpo do modelo usando algumas variáveis:

```
<h1>{{ .subTitle }}</h1>
<p>Utilizado: {{ .used }} de {{ .total }} (restante {{ .remained }})</p>
{{ range .links }}<div>{{ . }}</div>{{ end }}
```

Exemplo: data de expiração a partir de `expire`. O campo `{{ .expire }}` é Unix time em **segundos**, portanto para JavaScript deve ser multiplicado por 1000; o valor `0` significa «sem prazo»:

```
<script>
  var exp = {{ .expire }};
  document.write(exp === 0
    ? 'Sem prazo'
    : 'Até ' + new Date(exp * 1000).toLocaleDateString());
</script>
```

Observe que `{{ .lastOnline }}` já está em **milissegundos** – não é necessário multiplicá-lo por 1000.

11. Xray: roteamento, outbounds, DNS e extensões

A seção «**Configurações do Xray**» é um editor do modelo de configuração do Xray-core, com base no qual o painel gera o `config.json` final para iniciar o núcleo. A dica da seção do modelo: «*O arquivo de configuração do Xray é criado a partir do modelo.*» Ao contrário dos inbounds (que são armazenados separadamente no banco de dados e inseridos no modelo durante a geração da configuração), todo o restante – logs, roteamento, outbounds, DNS, política, estatísticas – é definido aqui.

Importante: o valor do modelo é armazenado no banco de dados sob a chave `xrayTemplateConfig`. Ao salvar, o painel o processa através de uma série de transformações automáticas (veja 11.11). Qualquer JSON sintaticamente inválido será rejeitado com o erro «*xray template config invalid*».

Localização no menu: «Saídas» e «Roteamento»

«**Saídas**» (Outbounds) e «**Roteamento**» (Routing) – são itens separados do menu lateral (logo abaixo de «Hosts», acima de «Configurações do painel»), cada um com seu endereço: `/outbound` e `/routing`. Links diretos para essas páginas e o recarregamento da página funcionam como esperado. No submenu «**Configurações do Xray**» permanecem apenas: Básico, Balanceador, DNS e Modelo Avançado. Na descrição abaixo, as seções 11.3 e 11.4 correspondem às páginas «Roteamento» e «Saídas».

11.1. Estrutura do editor: abas/modos

O editor oferece vários modos de exibição do modelo (filtros por seções JSON):

Modo	O que mostra
Básico	Seções básicas (Log, roteamento básico, configurações principais)
Modelo Avançado	JSON completo do modelo Xray
Todos	Todas as seções simultaneamente

Grupos lógicos de configurações dentro do editor:

- **Configurações gerais** (dica: «*Esses parâmetros descrevem as configurações gerais*»).
- **Log** (veja 11.10).
- **Conexões básicas**: bloqueios e rotas diretas.
- **Entradas** (dica: «*Alteração do modelo de configuração para conexão de determinados clientes*»).
- **Saídas** (veja 11.4).
- **Balanceador** (veja 11.5).
- **Roteamento** (dica: «*A prioridade de cada regra é importante!*», veja 11.3).
- **DNS / Fake DNS** (veja 11.6).

11.2. Configurações gerais (General)

Freedom Protocol Strategy

Campo	Rótulo	Descrição	Padrão
FreedomStrategy	Configuração da estratégia do protocolo Freedom	Estratégia de saída de rede para o outbound direto (freedom). Dica: « <i>Definir a estratégia de saída de rede no protocolo Freedom</i> ». Controla o campo <code>domainStrategy</code> dentro de <code>settings</code> do outbound com protocolo <code>freedom</code> .	No modelo de referência, <code>domainStrategy</code> para o outbound <code>freedom direct</code> é AsIs (o endereço não é resolvido, é transmitido como está).

`domainStrategy` para freedom (valores do Xray-core): **AsIs** (não resolver o domínio no lado do servidor), além da família `UseIP` / `UseIPv4` / `UseIPv6` e suas variantes «forçadas» `ForceIP*`, que fazem o servidor de saída resolver o domínio e conectar-se pelo IP obtido. Mude para `UseIPv4` se o servidor de saída não tiver IPv6 ou se precisar forçar apenas o uso de IPv4.

Freedom Happy Eyeballs (IPv4/IPv6)

Campo	Rótulo	Descrição
FreedomHappyEyeballs	Freedom Happy Eyeballs (IPv4/IPv6)	Dica: « <i>Conjunto de pilha dupla para saída direta (freedom) – útil em servidores de saída com IPv4 e IPv6.</i> » Ativa o algoritmo Happy Eyeballs (tentativa simultânea em ambas as famílias de endereços) para o outbound freedom.

Há um Happy Eyeballs separado também nas **configurações de dial (sockopt) de um outbound específico**: a partir da 3.5.0 ele é realmente ativado (antes, a configuração era serializada como desativada e o seletor «se perdia»). O «Try delay (ms)» padrão agora é **250**; um **0** definido explicitamente (= desativado) é preservado; também estão disponíveis `Prioritize IPv6`, `interleave (1)` e `maxConcurrentTry (4)`. | `try delay | (dica) | «Milissegundos antes de tentar outra família de endereços. 150–250 ms é um bom ponto de partida.»` Atraso antes de alternar para a família de endereços alternativa. O intervalo recomendado é de 150–250 ms. |

Overall Routing Strategy

Campo	Rótulo	Descrição	Padrão
RoutingStrategy	Configuração de roteamento de domínios	Estratégia geral de resolução DNS para roteamento. Dica: « <i>Definir a estratégia geral de roteamento de resolução DNS</i> ». Controla o campo <code>routing.domainStrategy</code> .	No modelo de referência, <code>routing.domainStrategy</code> = AsIs .

`routing.domainStrategy` define como as regras de roteamento de IP são correspondidas com solicitações de domínio: **AsIs** (apenas regras de domínio, sem resolução), **IPIfNonMatch** (se o domínio não correspondeu às regras – resolver e verificar regras de IP), **IPOnDemand** (resolver imediatamente ao encontrar uma regra de IP). Para que as regras de IP (por exemplo, `geoip:*`) funcionem com solicitações de domínio, normalmente é necessário **IPIfNonMatch**.

Outbound Test URL

Campo	Rótulo	Descrição	Padrão
outboundTestUrl	URL para teste de saída	URL para verificação de conectividade ao testar outbound. Dica: «URL para verificar a conectividade de saída». Armazenado separadamente do modelo, sob a chave xrayOutboundTestUrl.	https:// www.google.com/ generate_204

O valor passa por sanitização. No teste real do outbound, ele é verificado adicionalmente como uma URL pública – isso é proteção contra SSRF: o usuário não pode inserir uma URL arbitrária (incluindo interna) via cliente; a URL de teste sempre vem das configurações do servidor. Um valor vazio ao salvar/testar é substituído pelo padrão `generate_204`.

Default Outbound (a partir da 3.5.0)

O primeiro item da aba «Roteamento básico» é o seletor **Default Outbound**: «O tráfego que não corresponder a nenhuma regra de roteamento usa este outbound (o Xray pega o primeiro outbound da lista)». Na lista – `direct`, `blocked` e todas as tags existentes; o padrão é `direct`. A seleção **move o outbound indicado para a primeira posição** do modelo; `direct` / `blocked` ausentes são criados na hora e não são excluídos ao alternar.

Block BitTorrent

Campo	Rótulo	Descrição
Torrent	Bloquear BitTorrent	Adiciona a <code>routing.rules</code> uma regra que envia tráfego com <code>protocol: ["bittorrent"]</code> para o outbound <code>blocked</code> . No modelo de referência, essa regra está presente por padrão.

Limites de conexão (Connection Limits)

Dica: «Políticas de nível de conexão para usuários de nível 0. Deixe o campo vazio para usar o valor padrão do Xray.»
Esses parâmetros são gravados em `policy.levels.0`.

Campo	Rótulo	Descrição	Padrão
connIdle	Tempo limite de inatividade (segundos)	«Fecha a conexão após inatividade pelo número especificado de segundos. Reduzir o valor libera memória e descritores de arquivo mais rapidamente em servidores com alta carga (padrão no Xray: 300).»	vazio → padrão do Xray 300
bufferSize	Tamanho do buffer (KB)	«Tamanho do buffer interno por conexão em KB. Defina como 0 para minimizar o uso de memória em servidores com pouca RAM (o valor padrão do Xray depende da plataforma).» Placeholder: «auto».	vazio → depende da plataforma; 0 – minimizar

Exemplo (`policy.levels.0`). Os campos deste grupo são gravados na política de nível 0. Em um servidor com alta carga e pouca RAM, é possível acelerar a liberação de recursos assim:

```

"policy": {
  "levels": {
    "0": {
      "connIdle": 120,
      "bufferSize": 0
    }
  }
}

```

Aqui a conexão é encerrada após 120 s de inatividade (em vez dos 300 padrão), e `bufferSize: 0` minimiza o consumo de memória nos buffers. Um campo deixado vazio no formulário simplesmente não entra no JSON – e o Xray aplica seu valor padrão.

11.3. Regras de roteamento (routing)

Lista de regras `routing.rules`. **A ordem é crítica** («A prioridade de cada regra é importante!»): as regras são avaliadas de cima para baixo, a primeira correspondência é acionada. Dica: «Arraste para alterar a ordem». Botões de controle de ordem: **Primeiro**, **Último**, **Mover para cima**, **Mover para baixo**.

Cada regra tem `type: "field"`. Botões: **Criar regra**, **Editar regra**. Dica para campos de lista: «Elementos separados por vírgulas».

Na página «Roteamento», os botões «**Importar regras**» e «**Exportar regras**» estão agrupados no menu suspenso «**mais**» (more) – assim como na página «Saídas». O botão «**Exportar regras**» não baixa o arquivo imediatamente, mas abre uma janela modal com prévia do JSON e botões «**Copiar**» e «**Baixar**»: o conteúdo pode ser visualizado antes de salvar. A exportação de saídas na página «Saídas» funciona da mesma forma.

Route Tester (testador de rota)

Na aba Routing há uma subaba **Route Tester** – ela pergunta ao Xray em execução qual outbound processaria uma conexão específica, sem enviar tráfego real. Informe um domínio ou IP, porta, rede (TCP/UDP) e, se necessário, inbound e protocolo interceptado (`http / tls / quic / bittorrent`), depois clique em **Test Route**. A decisão vem diretamente do motor de roteamento ativo.

Na resposta, é mostrado o outbound selecionado e, ao usar um balanceador, também a tag do balanceador. Se nenhuma regra correspondeu, o testador informa que o tráfego vai para o outbound padrão (o primeiro na lista `outbounds`). Isso é útil para verificar a ordem das regras antes de confiar nelas.

Ativar e desativar uma regra individual

Uma regra de roteamento individual pode ser temporariamente **desativada** com uma chave, sem excluí-la. Na tabela de regras há uma coluna «**Ativar**» com uma chave (Switch), e no formulário da regra há o campo «**Ativar**» – também uma chave. Uma regra desativada não entra na configuração final do Xray, mas é mantida no modelo e pode ser reativada a qualquer momento.

A regra de serviço de estatísticas (`inboundTag: ["api"] → outboundTag: "api"`) não pode ser desativada – sua chave está bloqueada para não quebrar a contabilidade de tráfego do painel (veja 11.11).

Campos do formulário de regra

Campo do formulário	Rótulo	Campo JSON	Descrição
Origem	Origem	<code>source</code>	Endereços IP/sub-redes de origem. Lista separada por vírgulas.
Porta de origem	Porta de origem	<code>sourcePort</code>	Porta(s) de origem.
Destino	Destino	<code>domain + ip + port</code>	Domínios, IPs e portas de destino. Os domínios suportam prefixos <code>domain:</code> , <code>full:</code> , <code>regexp:</code> , <code>keyword:</code> , bem como <code>geosite:*</code> ; IPs – <code>geoip:*</code> e CIDR.
Rede	–	<code>network</code>	<code>tcp</code> , <code>udp</code> ou <code>tcp,udp</code> . A partir da 3.4.2 é gravado no config em minúsculas (<code>tcp / udp</code>) e exibido em maiúsculas na tabela de rotas (<code>TCP / UDP</code>); vazio – qualquer rede.
Protocolo	–	<code>protocol</code>	<code>http</code> , <code>tls</code> , <code>bittorrent</code> (detectado por sniffing).
Usuário	Usuário	<code>user</code>	Filtro por e-mail/identificador do usuário.
Atributos / Valor	Atributos / Valor	<code>attrs</code>	Atributos de cabeçalhos HTTP para correspondência.
VLESS route	VLESS route	–	Roteamento pelo campo route para VLESS.
Tags de entrada	Tags de entrada	<code>inboundTag</code>	Uma ou mais tags de inbound às quais a regra se aplica (incluindo o <code>api</code> interno e a tag DNS das configurações de DNS). Nas listas de inbound é exibido como «tag (remark)» se o inbound tiver uma observação separada, caso contrário apenas a tag; nas regras salvas continuam armazenadas apenas as tags.
Tag de saída	Tag de saída / Conexão de saída	<code>outboundTag</code>	Para onde direcionar o tráfego correspondente.
Tag do balanceador	Tag do balanceador / Balanceador	<code>balancerTag</code>	Dica: «Direciona o tráfego através de um dos balanceadores de carga configurados».

Exclusão mútua de `outboundTag` e `balancerTag`: «Não é possível usar `balancerTag` e `outboundTag` ao mesmo tempo. Se usados simultaneamente, apenas o `outboundTag` funcionará.» Em uma regra, defina ou a tag de saída ou a tag do balanceador.

Regras integradas do modelo de referência

No `config.json` padrão, a seção `routing` contém três regras (nessa ordem):

1. `inboundTag: ["api"]` → `outboundTag: "api"` – regra de serviço para a gRPC-API de estatísticas do painel.
2. `ip: ["geoip:private"]` → `outboundTag: "blocked"` – bloqueio de intervalos privados.
3. `protocol: ["bittorrent"]` → `outboundTag: "blocked"` – bloqueio do BitTorrent.

A regra `api → api` é sempre movida automaticamente para a posição 0 ao salvar (veja [11.11](#)), para que a solicitação de estatísticas não seja "capturada" por uma regra catch-all superior.

Exemplo de regra. Enviar todo o tráfego para sites russos e redes privadas diretamente (sem proxy), e o restante para um balanceador. A ordem importa: a regra «enviar diretamente» deve estar acima do catch-all. Em `routing.rules`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru", "domain:example.ru"],
  "ip": ["geoip:ru", "geoip:private"],
  "outboundTag": "direct"
}
```

Para que as regras de IP (`geoip:ru`) funcionem também para solicitações de domínio, normalmente é necessário `routing.domainStrategy: "IPIfNonMatch"` no nível superior do roteamento (veja [11.2](#)).

Grupos de roteamento pré-configurados (Conexões básicas)

No modo «Conexões básicas», o painel auxilia na criação de regras típicas a partir de listas prontas:

Grupo	Campos	Dica
Bloqueio por protocolos/sites	—	«Configure para que os clientes não tenham acesso a determinados protocolos»
Bloqueio por países	IPs bloqueados, Domínios bloqueados	«Esses parâmetros bloquearão o tráfego dependendo do país de destino.»
Conexões diretas	IPs diretos, Domínios diretos	«Conexão direta significa que determinado tráfego não será redirecionado por outro servidor.»
Regras IPv4	—	«Esses parâmetros permitirão que os clientes roteiem para domínios de destino apenas via IPv4»
Regras WARP	—	«Essas opções direcionarão o tráfego dependendo do destino específico via WARP.»
Roteamento NordVPN	—	«Essas opções direcionarão o tráfego dependendo do destino específico via NordVPN.»

MTPROTO-inbound: roteamento do tráfego do Telegram pelo Xray

O MTPROTO-inbound tem uma chave «**Route through Xray**» (desativada por padrão) e uma seleção opcional de **Outbound**. Quando ativada, o painel adiciona uma ponte SOCKS de loopback com a tag do próprio inbound à configuração do Xray, e o mtg direciona o tráfego do Telegram por ela. Após isso, o tráfego de saída do Telegram é gerenciado pelo roteador: ele pode ser correspondido com regras normais na aba Routing pela tag inbound ou forçado para o outbound ou balanceador selecionado através do campo **Outbound**. Deixe **Outbound** vazio para que as regras de roteamento tomem a decisão.

11.4. Outbounds (conexões de saída)

Lista `outbounds`. Botões: **Criar conexão de saída**, **Editar conexão de saída**. Dica: «*Alteração do modelo de configuração para definir as conexões de saída deste servidor*».

No modelo de referência, há dois outbounds obrigatórios:

- `protocol: "freedom"`, `tag: "direct"` — saída direta para a internet (com `domainStrategy: "AsIs"` e `finalRules: [{action: "allow"}]`);
- `protocol: "blackhole"`, `tag: "blocked"` — «buraco negro» para tráfego bloqueado.

Campos gerais do formulário de outbound

Campo	Rótulo	Descrição
Tag	Tag (dica: «Tag única»)	Identificador único do outbound. Placeholder: «tag-única». Validação: «A tag é obrigatória», «A tag já está sendo usada por outra saída».
Protocolo	—	Tipo de saída (veja abaixo).
Endereço / Porta	Endereço / Porta	Destino da conexão. Endereço e porta são obrigatórios.
Enviar através de	Enviar através de	Endereço IP local da interface de saída (<code>sendThrough</code>). Placeholder: «IP local».
Target Strategy	Target Strategy	Novo na 3.5.0. Como o domínio de destino é resolvido antes da conexão. 11 valores: <code>AsIs</code> (padrão, não resolver), <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> , <code>UseIPv6v4</code> , <code>UseIPv4v6</code> (resolução com fallback), <code>ForceIP</code> , <code>ForceIPv6v4</code> , <code>ForceIPv6</code> , <code>ForceIPv4v6</code> , <code>ForceIPv4</code> (exigem resolução bem-sucedida). Vazio = <code>AsIs</code> (não é gravado no config); para <code>freedom</code> , o valor é lido com fallback para o antigo <code>domainStrategy</code> (a chave legada continua sendo gravada para núcleos antigos).
Dialer proxy (cadeia)	—	Dica: «Conecte esta saída através de outra saída (por tag) para criar uma cadeia de proxy. Deixe vazio para conexão direta.» Placeholder: «Selecione a saída para encadeamento». Implementado via <code>streamSettings.sockopt.dialerProxy</code> .

A lista suspensa **Dialer Proxy** mostra não apenas outbounds locais, mas também tags de outbounds de assinaturas — assim é possível construir a cadeia também através de uma saída obtida por assinatura. O outbound `blackhole` e o próprio outbound em edição continuam excluídos da lista. Deixe o campo vazio para conexão direta.

Protocolos de outbound suportados

Protocolos suportados pelo formulário:

- **freedom** — saída direta. Campos `settings.domainStrategy`, `finalRules` (veja abaixo), Happy Eyeballs. Não é testável («*Outbound has no testable endpoint*»).
- **blackhole** — descarta o tráfego. Campo **Tipo de resposta**. Não é testável.
- **socks**, **http** — lista `settings.servers[]` com `address / port`; campo **Senha de autorização**. Para o protocolo **http**, abaixo dos campos **Username/Password** há um editor **Headers** (Cabeçalhos) — pares

chave/valor para cabeçalhos CONNECT enviados ao proxy HTTP upstream. Esses cabeçalhos são mantidos ao reabrir e salvar o outbound (antes eram perdidos). Atenção: apenas os cabeçalhos no nível de configurações (`settings.headers`) são aplicados; os cabeçalhos no nível de servidor individual são ignorados pelo xray-core.

- **vmess** — `settings.vnext[]` (`address / port`).
- **vless** — `settings.address / settings.port` .
- **trojan** , **shadowsocks** — `settings.servers[]` .
- **wireguard** — `settings.peers[]` com `endpoint` , mais chaves (veja [11.8](#)).
- **hysteria** — `settings.address / settings.port` (transporte UDP).

Para outbound do tipo **loopback**, está disponível o bloco **Sniffing** com os mesmos parâmetros do inbound: ativação, **destOverride**, **Metadata Only**, **Route Only** e lista de **domínios excluídos**.

Na máscara **UDP** (FinalMask) para **Hysteria2**, estão disponíveis modos adicionais. A máscara **Salamander** tem um seletor **Mode** com os valores **Salamander** e **Gecko**: o modo Gecko adiciona preenchimento aleatório de pacotes com campos **Min/Max** de tamanho (`packetSize` , intervalo 1–2048, padrão 512–1200) — isso protege contra fingerprinting por comprimento de pacotes. A máscara **Realm** (UDP hole-punching) ganhou um bloco opcional **TLS Config** com os campos **Server Name** (SNI), **ALPN** (`h3 / h2 / http/1.1`), **Fingerprint** (uTLS) e a chave **Allow Insecure**.

A partir da 3.5.0, foi adicionado um novo tipo de **máscara TCP — XMC (Minecraft)** (`xmc`): o fluxo é mascarado como protocolo Minecraft. Campos: **Hostname** (opcional; «endereço do servidor imitado no handshake»), **Usernames** (lista opcional; «nomes de jogadores oferecidos aos probers; por padrão o núcleo usa Dream») e o obrigatório **Password** (16 caracteres, geração automática com o botão 🔄; «senha de ofuscação»). Importante: **a combinação de Finalmask TCP com REALITY é rejeitada ao salvar** — ela derrubava o Xray-core na primeira conexão (erro: «Finalmask is not supported with REALITY security...»; veja Xray-core#6453). Uma máscara UDP com REALITY é permitida; combinações incorretas salvas anteriormente são automaticamente «curadas» (a máscara é descartada) na montagem do config.

Exemplo: cadeia via SOCKS upstream. O outbound `upstream` se conecta a um proxy SOCKS5 externo, e `chained` envia seu tráfego através dele (`dialerProxy`), formando uma cadeia. Em `outbounds` :

```
[
  {
    "tag": "upstream",
    "protocol": "socks",
    "settings": {
      "servers": [{ "address": "203.0.113.10", "port": 1080 }]
    }
  },
  {
    "tag": "chained",
    "protocol": "freedom",
    "streamSettings": {
      "sockopt": { "dialerProxy": "upstream" }
    }
  }
]
```

Agora uma regra de roteamento com `outboundTag: "chained"` enviará o tráfego para a internet através de `upstream`.

Importar outbound de link de compartilhamento

Um outbound pode ser importado a partir de um link de compartilhamento (`vless://`, `vmess://` etc.). Ao importar, as configurações do multiplexador **xmux** (XHTTP) transmitidas no bloco `extra=` do link também são salvas: após a importação, seus valores são inseridos no subformulário **XMUX** do outbound criado.

Campos Mux (multiplexação)

Paralelismo máximo, Conexões máximas, Reutilizações máximas, Solicitações máximas, Segundos máximos de reutilização, Período keep alive. Esses parâmetros configuram o comportamento mux/XUDP da saída.

Sockopts (configurações de socket)

Grupo **Sockopts**: **Intervalo keep alive, Mark (fwmark), Interface, Apenas IPv6, Aceitar proxy protocol, Proxy protocol, TCP user timeout (ms), TCP keep-alive idle (s).** O dialer-proxy da cadeia também é definido aqui.

Freedom finalRules (sobrescrita do bloqueio de IPs privados)

Para o outbound freedom, está disponível o grupo **Regras finais**:

Campo	Rótulo	Descrição
<code>overrideXrayPrivateIp</code>	Sobrescrever bloqueio padrão de IP privado no Xray	Remove a proibição interna do Xray para saídas para IPs privados.
<code>action</code>	Ação	<code>allow</code> (como no modelo de referência: <code>finalRules: [{action: "allow"}]</code>), <code>redirect</code> (Redirect) ou outros.
<code>blockDelay</code>	Atraso do bloqueio (ms)	Atraso antes de descartar a conexão.
<code>redirect / fragment</code>	Redirect / Fragment	Ações de redirecionamento e fragmentação de tráfego.

Máscara fragment: Lengths e Delays por fragmento

Na máscara **fragment** (tipo fragment no FinalMask, para TCP), os campos únicos Length e Delay são substituídos pelas listas **Lengths** e **Delays**: para cada segmento é possível definir um intervalo de comprimento separado (por exemplo `100-200`) e atrasos em milissegundos (por exemplo `10-20` ou `0`). As linhas das listas podem ser adicionadas e removidas; valores únicos salvos anteriormente são transferidos para um array de um elemento automaticamente.

Outros campos do formulário

- **UDP over TCP e Versão UoT** – para protocolos semelhantes ao shadowsocks.
- **Sem cabeçalho gRPC, Tamanho do chunk Uplink** – parâmetros de transporte gRPC.

- Campos TLS/uTLS: **Verificar nome do peer**, **Pinned SHA256**, **Short ID**, **Vision testpre**, placeholder «nome do servidor».

Testando saídas

Botões: **Testar**, **Testar todos**. Estados: **Testando conexão...**, **Teste bem-sucedido**, **Teste falhou**, **Não foi possível testar a conexão de saída**. Resultado: **Resultado do teste**, latência em milissegundos.

Dois modos (dica: «TCP: probe rápido apenas de dial. HTTP: solicitação completa via xray.»):

- **TCP** (mode=tcp) – dial simples para host:port , executado em paralelo em todos os endpoints, ~timeout de 5 s. Verifica apenas a acessibilidade TCP, não valida o protocolo proxy. Para freedom / blackhole /tag blocked retornará «Outbound has no testable endpoint».
- **HTTP** (mode=http ou vazio) – levanta uma instância temporária do Xray, executa uma solicitação HTTP real (URL de probe = outboundTestUrl do servidor), mede a latência real. Modo autoritativo, mas custoso: serializado por um bloqueio global («Another outbound test is already running, please wait»). Timeout de uma tentativa – 10 s, janela de espera do resultado – 15 s (aumentados para que outbounds saudáveis em canais lentos ou tunelados não sejam marcados como «Failed»). Em caso de falha, a causa real (erro de DNS, connection refused, expiração do deadline, erro TLS etc.) é gravada no log do painel/Xray, apontado pelas mensagens gerais de timeout.
- **Real delay (novo na 3.5.0)** – terceiro modo do seletor. Usa a mesma instância temporária do Xray, mas informa o **tempo completo de uma solicitação «fria», incluindo o estabelecimento do túnel** – um número próximo ao que os aplicativos clientes mostram. Dica do seletor: «TCP: probe rápido apenas de dial. HTTP: solicitação completa via xray. Real delay: tempo completo, incluindo o estabelecimento da conexão». O modo **HTTP** comum, a partir da 3.5.0, é medido em uma conexão «aquecida» (keep-alive), por isso seus números são menores e mais próximos da latência de uma única solicitação.

Protocolos UDP (wireguard , hysteria) e transportes UDP (kcp , quic , hysteria) **sempre** são testados no modo HTTP, mesmo que TCP tenha sido solicitado – um dial UDP simples não distingue um endpoint «vivo» de um «morto». Para wireguard na configuração de teste, noKernelTun: true é forçado.

Colunas Egress e Country (a partir da 3.5.0)

Após um teste **HTTP** ou **Real delay** bem-sucedido, o painel solicita pela mesma rota temporária os metadados do Cloudflare trace e mostra na lista de outbound duas colunas: **Egress** – o IP de saída (IPv4/IPv6, cada um oculto atrás de um «olho»; enquanto não houve teste – um traço com a dica «Execute o teste HTTP para mostrar o IP de saída e o país») e **Country** – a bandeira e o nome do país de saída; se a saída passa pelo Cloudflare WARP, é adicionado um rótulo laranja WARP. O teste TCP não preenche essas colunas; em dispositivos móveis, os dados são exibidos no card do outbound.

Verificação em lote e divisão por etapas

Testar e **Testar todos** no modo HTTP levantam uma instância temporária comum do Xray para um lote de outbounds, criam um inbound SOCKS de loopback com regra para cada um e enviam em paralelo uma solicitação HTTP real; **Testar todos** verifica outbounds em lotes. **Testar todos** também verifica os outbounds obtidos de assinaturas (tabela «de assinaturas», somente leitura) – suas linhas também são destacadas com o resultado do teste. Nesse caso, os outbounds `freedom` («direct») e `dns` não são testados em nenhum modo (não são proxies): o botão de teste não está disponível para eles, **Testar todos** os ignora, e a proteção do servidor proíbe seu teste HTTP mesmo em chamada direta de API. Além de sucesso/erro, o popup de resultado mostra o status HTTP da resposta e o detalhamento do tempo por etapas: **Proxy connect** (conexão com o proxy), **TLS via outbound** (TLS via outbound) e **First byte** (tempo até o primeiro byte) – isso ajuda a entender em qual etapa ocorreu o atraso ou a falha.

Estatísticas de tráfego de outbounds

O painel mantém contadores de tráfego por tags (`up` / `down` / `total`). O botão de reset zera os contadores para uma tag específica ou para todas (`tag = "-alltags-"`). Os campos **Informações da conta** e **Status da conexão de saída** mostram um resumo.

11.5. Balanceadores (Balancers)

Lista `routing.balancers`. Botões: **Criar balanceador**, **Editar balanceador**.

Na aba Balancers há colunas de estado ao vivo: **Live Target** mostra o alvo ativo atual do balanceador no Xray em execução, e **Override** permite substituir manualmente a escolha do alvo (o valor **Auto (strategy)** retorna a escolha por estratégia). O estado é atualizado por um botão separado. Se o balanceador ainda não estiver ativo no Xray em execução, o painel sugerirá salvar as alterações ou iniciar o Xray primeiro.

Campo	Rótulo	Descrição
Tag	Tag (dica: «Tag única»)	Identificador único. Placeholder: «tag única do balanceador». Validação: «A tag é obrigatória», «A tag já está sendo usada por outro balanceador».
Seletores	Seletores	Lista de tags de outbound (por substring) dentre os quais o balanceador seleciona a saída. Pelo menos um deve ser selecionado: «Selecione pelo menos uma saída».
Fallback	Fallback	Tag reserva se nenhum seletor correspondeu. A partir da 3.5.0, também é possível escolher outro balanceador (na lista, essas opções são marcadas com o rótulo azul «Balancer», dica «Selecione outro balanceador como fallback»). O Xray não suporta isso nativamente, por isso o painel constrói por conta própria um outbound loopback oculto <code>_bl_<alvo></code> e uma regra de roteamento (caminho: balanceador → loopback → servidor → balanceador alvo → outbound; um aviso sobre o «salto» extra é exibido no formulário). Há proteção contra ciclos (erro «não é possível selecionar – surgiria uma dependência cíclica»; a opção na lista é bloqueada com a dica do caminho do ciclo) e contra a exclusão de um balanceador em uso («Não é possível excluir – usado como fallback por: ...»). O prefixo de tag <code>_bl_</code> é reservado (proibido em tags); os objetos <code>_bl_</code> de serviço ficam ocultos das listas Outbounds/Routing, e as colunas Fallback/Live Target/Override mostram o nome do balanceador alvo, e não a tag do loopback.
Estratégia	Estratégia	Algoritmo de seleção (veja abaixo).

Estratégia e parâmetros de observação

A estratégia (`strategy.type`) define como o balanceador seleciona o outbound dentre os seletores. Valores do Xray-core: `random` (aleatório), `roundRobin` (round-robin), `leastPing` (latência mínima pelos resultados do observatory), `leastLoad` (carga mínima). Para `leastLoad` / `leastPing` , são usados parâmetros de `strategy.settings` :

Campo	Rótulo	Descrição
<code>expected</code>	Esperado	Placeholder: « <i>número ideal de nós</i> » – número alvo de nós ativos.
<code>maxRtt</code>	RTT máx.	Limite superior do RTT aceitável na seleção de candidatos.
<code>tolerance</code>	Tolerância	Tolerância ao comparar latências/cargas.
<code>baselines</code>	Baselines	Valores de latência de limiar para agrupamento de nós.
<code>costs</code>	Costs	Coefficientes de peso (cost) para tags individuais.

Exemplos de estratégias. O bloco `strategy` fica dentro do balanceador (no JSON – ao lado de `tag` e `selector`):

```
"strategy": { "type": "random" }      // seleção aleatória entre os seletores
"strategy": { "type": "roundRobin" }  // round-robin, alternadamente
"strategy": { "type": "leastPing" }   // latência mínima (requer observer)
```

Para `leastLoad` , os parâmetros são definidos em `settings` :

```
"strategy": {
  "type": "leastLoad",
  "settings": {
    "expected": 2,
    "maxRTT": "1s",
    "tolerance": 0.05,
    "baselines": ["500ms", "1s", "2s"],
    "costs": [
      { "regex": false, "match": "proxy-premium", "value": 0.1 },
      { "regex": true, "match": "^proxy-cheap-.+$$", "value": 5 }
    ]
  }
}
```

Como isso funciona (com exemplo). Suponha que o observer mediu as latências das saídas: `A = 250 ms` , `B = 280 ms` , `C = 700 ms` , `D = 1500 ms` . Com as configurações acima, a seleção ocorre assim:

1. **maxRTT: "1s"** – saídas com latência acima de 1 s são descartadas: `D` (1500 ms) é eliminado. Restam `A` , `B` , `C` .
2. **baselines + expected** – as saídas são agrupadas por limites de latência, e é escolhido o **menor** limiar que contenha pelo menos `expected` saídas. O limiar `500ms` já contém `A` e `B` – são 2 (= `expected`), portanto o grupo { `A` , `B` } é selecionado. `C` (700 ms) não entra na seleção enquanto houver rápidas suficientes (ele é uma «reserva quente»).

3. **tolerance: 0.05** — dentro do grupo selecionado, saídas com latências que diferem em no máximo 5% são consideradas equivalentes, e a carga é dividida igualmente entre elas. **A** (250) e **B** (280) diferem em ~12% (> 5%), portanto, em igualdade de condições, a preferência é pelo mais rápido **A**; se a diferença estivesse dentro de 5% — o tráfego iria tanto por **A** quanto por **B**.
4. **costs** — antes da comparação, ajustam o «custo» de saídas individuais: um **value** menor torna a saída mais atraente, um valor maior faz o oposto. No exemplo, **proxy-premium** recebe **0.1** (torna-se «mais barato» e é selecionado com mais frequência), e todos **proxy-cheap-*** (por expressão regular, **regexp: true**) — **5** (tornam-se «mais caros» e são usados por último). Assim é possível priorizar saídas suavemente, sem excluí-las rigidamente.

Resultado: o tráfego irá principalmente através de **A** (com latências próximas — igualmente com **B**), **C** permanece como reserva, **D** está excluído até que seu RTT caia abaixo de **maxRTT**.

Observer: **observatory** e **burstObservatory** (medições para **leastPing** / **leastLoad**)

As estratégias **leastPing** e **leastLoad** não medem nada por si mesmas — precisam de dados sobre a latência e disponibilidade de cada outbound. Esses dados são coletados pelo **observer** (observatory): ele «pinga» periodicamente cada outbound monitorado e registra o tempo de resposta e a disponibilidade. Os mesmos dados são mostrados na aba «**Observatório**» (estados **Ativo** / **Indisponível**, «**Última atividade**», «**Última tentativa**»).

A partir da versão **3.4.2**, a página «Balanceadores» foi dividida nas abas «**Configurações do balanceador**» (tabela e adição) e «**Observatory**», e o observer é editado por um **formulário estruturado** (o antigo editor de JSON bruto foi removido). O painel cria e remove observers conforme necessário: um **observatory** comum — para a estratégia **Least Ping**, um **burstObservatory** estendido — para **Least Load**, bem como para **Random/Round-robin com fallbackTag definido** (o xray-core exige isso; ao limpar o último **fallbackTag**, o observer é removido). Criar ou remover um observer exige um reinício completo do Xray.

Há duas variantes disponíveis:

- **observatory** — simples: **subjectSelector** + **probeURL** + **probeInterval**.
- **burstObservatory** — avançado, com configuração detalhada de ping via **pingConfig**; conveniente para várias saídas.

Exemplo de bloco **burstObservatory**:

```
{
  "subjectSelector": ["WS-SE", "WS-FR", "WS-PL"],
  "pingConfig": {
    "destination": "https://www.google.com/generate_204",
    "interval": "1m",
    "connectivity": "http://connectivitycheck.platform.hicloud.com/generate_204",
    "timeout": "5s",
    "sampling": 2
  }
}
```

Finalidade dos campos:

Campo	O que define
<code>subjectSelector</code>	Lista de prefixos de tags de outbound para monitoramento. O Xray pega todos os outbounds cujas tags começam com as strings indicadas. No exemplo, são monitoradas as saídas <code>WS-SE...</code> , <code>WS-FR...</code> , <code>WS-PL...</code> . Essas tags devem coincidir com as definidas nos Seletores do balanceador.
<code>pingConfig.destination</code>	URL solicitado através de cada outbound para medir a latência. Usa-se uma página «leve» com resposta <code>204</code> sem corpo – por exemplo <code>https://www.google.com/generate_204</code> . O tempo até a resposta é a latência medida.
<code>pingConfig.interval</code>	Com que frequência pingar cada outbound. String de duração: <code>"1m"</code> – uma vez por minuto, também <code>"30s"</code> , <code>"5m"</code> etc. Mais frequente – dados mais frescos, mas mais tráfego em segundo plano.
<code>pingConfig.connectivity</code>	(opcional) URL de verificação da conectividade básica do próprio servidor. Se inacessível – o problema está na rede do servidor, e o observer não marca o outbound como indisponível (proteção contra falsos positivos em caso de falha local). Normalmente também um endpoint com resposta <code>204</code> .
<code>pingConfig.timeout</code>	Quanto aguardar a resposta de um ping antes de considerar a tentativa malsucedida (por exemplo <code>"5s"</code>).
<code>pingConfig.sampling</code>	Quantas medições recentes armazenar e calcular a média para cada outbound. <code>2</code> – considerar os dois últimos pings (suaviza picos aleatórios).
<code>pingConfig.httpMethod</code>	Novo na 3.4.2. Método HTTP das sondas: <code>HEAD</code> (padrão) ou <code>GET</code> .

Na aba «**Observatory**», os mesmos parâmetros são definidos por formulário (e não por JSON). Para o `observatory` comum, são **Watched Outbounds** (`subjectSelector`, somente leitura), **Probe URL** (`probeURL`, padrão `https://www.google.com/generate_204`), **Probe Interval** (`probeInterval`, `1m`) e **Enable Concurrency** (`enableConcurrency`, habilitado por padrão). Para o `burst0bservatory` – os campos listados acima mais o novo «**Método HTTP**». Se o balanceador tiver ambos os observers, um interruptor segmentado alterna entre os formulários Observatory e Burst.

Como conectar tudo:

1. Na aba «**Observatory**», configure os parâmetros do observer (ele é criado automaticamente conforme a estratégia escolhida; o `subjectSelector` acompanha os seletores do balanceador).
2. Crie o balanceador: **Estratégia** = `leastPing`, em **Seletores** indique as mesmas tags de outbound (`WS-SE`, `WS-FR`, `WS-PL`).
3. Direcione o tráfego para ele com uma regra de roteamento (campo **Tag do balanceador**, veja 11.3).
4. Reinicie o Xray. Na aba «**Observatório**» aparecerão os estados das saídas, e o balanceador começará a escolher o mais rápido entre os ativos.

Em uma regra, não é possível definir `balancerTag` e `outboundTag` ao mesmo tempo – apenas `outboundTag` funcionará.

O que acontece ao excluir um outbound ou balanceador (a partir da 3.4.2)

Ao excluir um outbound ou balanceador, o painel, na mesma ação, **limpa as referências a ele no roteamento** e exibe uma **prévia das consequências** no diálogo de confirmação (título «Ao excluir, seu roteamento também será atualizado:», em seguida, por regra – «removida (não restou destino)» ou «mantida (agora usa ...)», e «Balanceador ... removido (não restaram alvos)»). Assim é evitada uma referência «pendente» (`balancerTag` / `outboundTag` / `dialerProxy`), por causa da qual antes o xray-core podia não iniciar no próximo reinício.

Exclusão de balanceador: as regras mantêm seu `outboundTag` , caso contrário são removidas. Exclusão de outbound: sua tag é retirada dos seletores e do `fallbackTag` dos balanceadores, o `dialerProxy` de outros outbounds é limpo, os balanceadores que ficaram vazios são removidos junto com as regras agora sem objeto, e os observers são reconstruídos.

11.6. DNS

Seção `dns` . Ativação: **Ativar DNS** (dica: «Ativar o servidor DNS integrado»).

A partir da 3.5.0, servidores DNS em **IPs privados** (por exemplo, seu próprio AdGuard/Pi-hole na mesma rede docker) funcionam «de fábrica»: o painel insere e mantém automaticamente uma regra allow gerenciada (direct, limitada à porta do servidor DNS) **antes** da regra bloqueadora `geoip:private` e a sincroniza quando `dns.servers` muda. Antes, as próprias consultas DNS do Xray eram silenciosamente bloqueadas por essa regra, adicionando ~4 s de atraso a cada novo domínio; regras manuais não são mais necessárias (as criadas manualmente não são tocadas).

Parâmetros gerais de DNS

Campo	Rótulo	JSON	Descrição / dica
<code>tag</code>	Nome da tag DNS	<code>dns.tag</code>	«Esta tag estará disponível como tag de entrada nas regras de roteamento.» Permite rotear as próprias solicitações DNS via <code>inboundTag</code> .
<code>clientIp</code>	IP do cliente	<code>dns.clientIp</code>	«Usado para notificar o servidor sobre a localização do IP especificado durante as consultas DNS» (EDNS Client Subnet).
<code>strategy</code>	Estratégia de consulta	<code>dns.queryStrategy</code>	«Estratégia geral de resolução de nomes de domínio.» Valores: <code>UseIP</code> , <code>UseIPv4</code> , <code>UseIPv6</code> .
<code>disableCache</code>	Desativar cache	<code>dns.disableCache</code>	«Desativa o cache de DNS».
<code>disableFallback</code>	Desativar DNS de fallback	<code>dns.disableFallback</code>	«Desativa as consultas DNS de fallback».

Campo	Rótulo	JSON	Descrição / dica
<code>disableFallbackIfMatch</code>	Desativar DNS de fallback ao corresponder	<code>dns.disableFallbackIfMatch</code>	«Desativa as consultas DNS de fallback quando a lista de domínios do servidor DNS corresponde».
<code>enableParallelQuery</code>	Ativar consultas paralelas	—	«Ativar consultas DNS paralelas a vários servidores para resolução mais rápida».
<code>useSystemHosts</code>	Usar Hosts do sistema	<code>dns.useSystemHosts</code>	«Usar o arquivo hosts do sistema instalado».

Exemplo de bloco `dns` . As consultas para domínios do Google são resolvidas via servidor DoH da Cloudflare, todo o resto — via `1.1.1.1` ; para consultas do Google, esperam-se apenas IPs não privados. No nível superior da configuração:

```
"dns": {
  "tag": "dns-inbound",
  "queryStrategy": "UseIPv4",
  "servers": [
    {
      "address": "https://cloudflare-dns.com/dns-query",
      "domains": ["geosite:google"],
      "expectIPs": ["geoip:!private"]
    },
    "1.1.1.1"
  ]
}
```

O servidor em formato de string (`"1.1.1.1"`) sem campos é o servidor padrão para todos os outros domínios. A tag `dns-inbound` pode então ser usada como `inboundTag` nas regras de roteamento para direcionar as próprias consultas DNS pelo outbound adequado.

Cache de registros desatualizados

Campo	Rótulo	Descrição
<code>serveStale</code>	Usar desatualizados	«Retornar resultados desatualizados do cache durante a atualização em segundo plano».
<code>serveExpiredTTL</code>	TTL de desatualizados	«Validade (segundos) dos registros de cache desatualizados; 0 = sem limite».

Servidores DNS (lista `dns.servers`)

Botões: **Criar DNS**, **Editar DNS**, **Excluir todos** (confirmação: «*Todos os servidores DNS serão removidos da lista. Esta ação não pode ser desfeita.*»). Modelos: **Usar modelo**, janela **Modelos DNS**, incluindo predefinição **Família**.

Ao clicar em **Editar DNS** em um registro de servidor DNS (assim como em um registro Fake DNS), a janela de edição preenche os valores salvos do servidor, não os valores padrão.

Campos do servidor DNS:

Campo	Rótulo	Descrição
address	—	Endereço DNS (IP, URL DoH, <code>localhost</code> , <code>fakedns</code> etc.).
domains	Domínios	Lista de domínios para os quais este servidor é usado.
expectIPs	IPs esperados	Aceitar resposta somente se o IP estiver na lista.
unexpectedIPs	IPs não esperados	Descartar respostas com os IPs indicados.
skipFallback	Ignorar Fallback	Não usar este servidor como fallback.
finalQuery	Consulta final	Marca o servidor como final na cadeia.
timeoutMs	Timeout (ms)	Timeout da consulta ao servidor.

Hosts (registros estáticos)

Grupo **Hosts** (`dns.hosts`). Botão **Adicionar Host**; estado vazio **Hosts não definidos**. Campos: domínio (placeholder: «*Domínio (ex. domain:example.com)*») e valores (placeholder: «*IP ou domínio – insira e pressione Enter*»).

Logs de DNS

Veja 11.10: flag **Logs DNS** (`dnsLog`) na seção de logs.

11.7. Fake DNS

Seção `fakedns`. Botões: **Criar Fake DNS**, **Editar Fake DNS**.

Campo	Rótulo	Descrição
ipPool	Sub-rede do pool de IPs	Intervalo CIDR do qual são emitidos IPs fictícios (por exemplo <code>198.18.0.0/15</code>).
poolSize	Tamanho do pool	Quantos endereços manter no pool circular.

O Fake DNS é usado em conjunto com o sniffing no inbound: o núcleo emite um IP fictício ao cliente, memoriza a correspondência domínio↔IP e restaura o domínio no roteamento. Para que o Fake DNS funcione, o servidor DNS com endereço `fakedns` deve ser adicionado à lista de servidores DNS.

Exemplo: combinação Fake DNS + servidor DNS. Primeiro definimos o pool de endereços fictícios, depois adicionamos o servidor DNS `fakedns` para que as consultas de domínio recebam IPs deste pool:

```
"fakedns": [
  { "ipPool": "198.18.0.0/15", "poolSize": 65535 }
],
"dns": {
  "servers": [
    { "address": "fakedns", "domains": ["geosite:geolocation-!cn"] },
    "1.1.1.1"
  ]
}
```

Adicionalmente, no inbound é necessário ativar o sniffing com `destOverride: ["fakedns"]`, caso contrário o núcleo não terá como obter o domínio real para restauração.

11.8. WireGuard / WARP / NordVPN

Campos WireGuard (`wireguard`)

Campo	Rótulo	Descrição
<code>secretKey</code>	Chave secreta	Chave privada da interface local.
<code>publicKey</code>	Chave pública	Chave pública do peer.
<code>psk</code>	Chave compartilhada	PreShared Key (opcional).
<code>allowedIPs</code>	Endereços IP permitidos	Intervalos roteados para o túnel.
<code>endpoint</code>	Ponto de extremidade	<code>host:port</code> do peer.
<code>domainStrategy</code>	Estratégia de domínio	Estratégia de resolução para o outbound WireGuard.

Cloudflare WARP (`warp`)

A integração usa a API `https://api.cloudflareclient.com/v0a4005` (client-version `a-6.30-3596`). Ações do controlador (`/xray/warp/:action`): `config`, `reg`, `license`, `data`, `del`.

Passo a passo:

1. **Criar conta WARP** → `reg` : o painel gera/recebe chaves privada (`privateKey`) e pública (`publicKey`), registra o dispositivo na Cloudflare e salva `access_token`, `device_id`, `license_key`, `private_key` (e também `client_id`) na configuração `warp`.
2. **Chave de licença WARP / WARP+** → `license` : instalação de uma chave WARP+ de 26 caracteres (placeholder: «Chave WARP+ de 26 caracteres»). Em caso de erro: «Falha ao definir a licença WARP.» Se a configuração ainda não foi obtida: «Obtenha a configuração WARP primeiro.»
3. **Informações da conta: Nome do dispositivo, Modelo do dispositivo, Dispositivo ativo, Tipo de conta, Função, WARP+ data, Cota, Uso.**
4. **Adicionar saída** – cria um outbound WireGuard com as chaves e o endpoint da Cloudflare obtidos.
5. **Excluir conta** → `del` : limpa os dados WARP salvos.

NordVPN (nord / nordvpn)

A integração usa NordLynx (= WireGuard). Ações do controlador (/xray/nord/:action): countries , servers , reg , setKey , data , del .

Passo a passo:

1. **Token de acesso** → reg : o painel solicita as credenciais NordLynx de api.nordvpn.com e extrai nordlynx_private_key . Salva private_key e token na configuração nord . Alternativa – setKey : inserir a **Chave privada** diretamente (não pode estar vazio).
2. **País** → countries carrega a lista de países; **Cidade** (ou **Todas as cidades**).
3. **Servidor** → servers carrega os servidores do país selecionado (countryId é validado como número – proteção contra injeções). Filtro: são mostrados apenas servidores com **Carga** > 7%. Se não houver servidores: «*Nenhum servidor encontrado para o país selecionado*». Se o servidor não tiver chave pública NordLynx: «*O servidor selecionado não informa a chave pública NordLynx*».
4. Criação/atualização de saída: toasts «*Saída NordVPN adicionada*» / «*Saída NordVPN atualizada*».

Prioridade IPv4 e TUN em espaço de usuário

Os outbounds WireGuard gerados pelos assistentes WARP e NordVPN usam domainStrategy: "ForceIPv4v6" (prioridade IPv4 com fallback para IPv6 em hosts somente-v6) em vez de ForceIP – isso elimina o «travamento» do handshake em hosts com IPv6 parcialmente configurado, quando um registro AAAA do endpoint da Cloudflare é selecionado. Além disso, o TUN em espaço de usuário (noKernelTun: true) é ativado em vez do kernel TUN: o último requer permissões e roteamento fwmark e falha silenciosamente em muitos VPS, enquanto a verificação de conectividade integrada do painel sempre testa via TUN em espaço de usuário – agora o tráfego real e a verificação seguem o mesmo caminho. A alteração se aplica apenas a outbounds recém-adicionados ou redefinidos; os modelos já salvos mantêm suas configurações.

11.9. Reverse-proxy e TUN

Reverse (proxy reverso)

Seção reverse da configuração do Xray. No formulário de outbound, há uma chave para o tipo **Proxy reverso**. Botões: **Criar proxy reverso**, **Editar proxy reverso**.

Campo	Rótulo	Descrição
Tipo	Tipo	Bridge ou Portal – dois papéis do proxy reverso do Xray.
Domínio	Domínio	Domínio-rótulo de serviço para o par bridge[?]portal.
Tag / Conexão	Tag / Conexão	Tags para vincular bridge e portal.
Reverse Tag	Tag do proxy reverso	Dica: « <i>Tag de conexão de saída para proxy reverso VLESS simples. Deixe vazio para desativar.</i> » Placeholder: « <i>tag de saída (vazio = desativado)</i> ». Implementa o proxy reverso VLESS simplificado.

No formulário de outbound, há também campos de fluxo reverso: **Sniffing reverso**, **Workers**, **Reservado**, **Intervalo mínimo de upload (ms)**, **Tamanho máximo de upload (bytes)**.

TUN (tun)

Campo	Rótulo	Descrição	Padrão
name	—	«Nome da interface TUN.»	xray0
mtu	—	«Unidade máxima de transmissão. Tamanho máximo dos pacotes de dados.»	1500
userLevel	Nível do usuário	«Todas as conexões estabelecidas por este fluxo de entrada usarão este nível de usuário.»	0

11.10. Logs e estatísticas (Stats, metrics)

Log (log)

Dica: «Os logs podem diminuir a velocidade do servidor. Ative apenas os tipos de log necessários quando precisar!»

Seção log do modelo de referência: access: "none", error: "", loglevel: "warning", dnsLog: false, maskAddress: "".

Campo	Rótulo	JSON	Descrição	Padrão
logLevel	Nível de logs	loglevel	«Nível de log para logs de erros...» Valores: debug, info, warning, error, none.	warning
accessLog	Logs de acesso	access	«Caminho para o arquivo de log de acesso. O valor especial «none» desativa os logs de acesso.»	none
errorLog	Logs de erros	error	«Caminho para o arquivo de logs de erros. O valor especial «none» desativa os logs de erros.»	"" (padrão)
dnsLog	Logs DNS	dnsLog	«Ativar logs de consultas DNS»	false
maskAddress	Mascaramento de endereço	maskAddress	«Quando ativado, o endereço IP real é substituído por um mascarado nos logs.»	"" (desativado)

Estatísticas (stats / policy)

Grupo **Estatísticas**. Ativa contadores em policy.system e policy.levels. No modelo de referência: statsInboundUplink: true, statsInboundDownlink: true, statsOutboundUplink: false, statsOutboundDownlink: false; para o nível 0 — statsUserUplink: true, statsUserDownlink: true.

Campo	Rótulo	Descrição	Padrão
statsInboundUplink	Estatísticas de uplink de entrada	«Ativa a coleta de estatísticas para o tráfego de saída de todos os proxies de entrada.»	true

Campo	Rótulo	Descrição	Padrão
<code>statsInboundDownlink</code>	Estatísticas de downlink de entrada	«Ativa a coleta de estatísticas para o tráfego de entrada de todos os proxies de entrada.»	true
<code>statsOutboundUplink</code>	Estatísticas de uplink de saída	«Ativa a coleta de estatísticas para o tráfego de saída de todos os proxies de saída.»	false
<code>statsOutboundDownlink</code>	Estatísticas de downlink de saída	«Ativa a coleta de estatísticas para o tráfego de entrada de todos os proxies de saída.»	false

As estatísticas de clientes e inbounds (uplink/downlink) são a base da exibição de tráfego no painel e nos clientes; não é recomendado desativá-las. As estatísticas de outbound estão desativadas por padrão e são necessárias apenas se você monitorar o tráfego por tags de saída.

Metrics

No modelo de referência há uma seção `metrics` (`listen: "127.0.0.1:11111"` , `tag: "metrics_out"`) e a API correspondente `metrics_out` . O painel usa esse listener para coletar métricas e snapshots do observatory: ele analisa `metrics.listen` do modelo, consulta `/debug/vars` e agrega o histórico de latências por tags. Se você alterar o endereço/porta de `metrics.listen` , o painel passará a usar o novo endereço; a remoção da seção `metrics` desativará a coleta de gráficos do observatory.

O teste de outbound no modo HTTP levanta uma instância temporária **separada** do Xray com seu próprio listener `metrics` em uma porta aleatória — não é o mesmo listener do config principal.

11.11. Salvamento, reinicialização e transformações automáticas

Botões

Botão	Ação
Salvar	<code>POST /xray/update</code> : valida e salva o modelo + <code>outboundTestUrl</code> .
Reiniciar Xray	Recarrega o serviço com a configuração salva. Confirmação: «Reiniciar xray?» / «Recarrega o serviço xray com a configuração salva.»

Toasts: sucesso — «Xray reiniciado com sucesso», «Xray parado com sucesso»; erros — «Ocorreu um erro ao reiniciar o Xray.», «Ocorreu um erro ao parar o Xray.» A janela **Saída do reinício do Xray** mostra a saída de diagnóstico do núcleo.

Aplicação a quente de alterações (sem reinicialização completa)

As alterações em inbounds, outbounds e regras de roteamento são aplicadas «ao vivo»: ao clicar em **Salvar**, o painel calcula a diferença entre a configuração antiga e a nova e aplica apenas as partes alteradas via gRPC-API do Xray (HandlerService/RoutingService), sem reiniciar o processo. A reinicialização completa é executada

automaticamente apenas quando são alteradas seções sem API de recarga a quente (`log` , `dns` , `policy` , `observatory` etc.). Por isso, na página do Xray, não é necessário um botão separado «Reiniciar» — **Salvar** já aplica as alterações. A reinicialização do núcleo, quando necessário, ainda é executada automaticamente (veja também a recarga automática ao atualizar assinaturas e a rotação do WARP).

Restauração do modelo padrão

O endpoint `GET /xray/getDefaultJsonConfig` retorna o modelo de referência (`config.json` , embutido no binário). Com ele é possível redefinir a configuração para os padrões de fábrica.

Transformações automáticas ao salvar

Ao salvar as configurações do Xray, o painel executa (nesta ordem):

1. **Remoção de wrappers** — remove wrappers do tipo `{ "xraySetting": <config>, "inboundTags": ..., "outboundTestUrl": ... }` se eles entraram acidentalmente no valor (caso contrário as camadas se acumulariam a cada salvamento). São removidas até 8 camadas.
2. **Verificação da configuração** — o JSON é analisado na estrutura de configuração do Xray; em caso de erro — rejeição com «*xray template config invalid*».
3. **Garantia da regra de estatísticas** — a regra `inboundTag: ["api"]` → `outboundTag: "api"` é movida forçosamente para a posição 0 em `routing.rules` (ou adicionada, se ausente). Isso garante que a solicitação gRPC de estatísticas do painel não seja interceptada por uma regra catch-all superior (caso contrário, os clientes podem aparecer offline com tráfego zero mesmo com o proxy funcionando).

Devido ao item 3, não tente remover ou mover a regra `api` → `api` — o painel a restaurará ao salvar novamente. Ela é a infraestrutura de serviço de estatísticas, não uma rota de usuário.

11.12. Outbound de assinatura (com atualização automática)

A partir da versão 3.3.0, o painel pode importar `outbound s` diretamente de uma URL de assinatura — no mesmo formato que os provedores de VPN fornecem para aplicativos clientes. As assinaturas são relidas periodicamente em segundo plano, portanto o conjunto de `outbound s` no servidor é mantido atualizado sem edição manual do modelo de configuração. A partir da 3.5.0, nos links `ss://` (SIP002) com parâmetros de query (`?plugin=` , `?type=`) ou `/` final, a porta é analisada corretamente (antes ela se tornava silenciosamente `0`); um link com uma porta realmente inválida é ignorado, em vez de ser importado quebrado.

Na interface, a seção é chamada «**Assinaturas de saídas**», descrição: «Importar saídas de URLs de assinaturas remotas (vmess/vless/trojan/ss/...). As tags permanecem inalteradas para uso em balanceadores e regras de roteamento. A atualização é realizada automaticamente.» A seção está localizada na página Xray, acima do painel de configuração de `outbound s`.

Como funciona

As assinaturas são armazenadas separadamente do modelo de configuração do Xray. O modelo **nunca é sobrescrito**: os `outbound`s obtidos das assinaturas são adicionados à configuração final dinamicamente a cada geração da configuração do Xray.

Adicionar uma assinatura

No formulário «Adicionar assinatura», estão disponíveis os seguintes campos:

Campo	Chave	Padrão	Finalidade
URL da assinatura	<code>url</code>	– (obrigatório)	Endereço da assinatura. Placeholder: «https://... (lista de links em base64)». Aceita apenas HTTP(S); o endereço é verificado quanto à segurança.
Observação	<code>remark</code>	vazio	Rótulo arbitrário (placeholder «ex. nós HK»).
Prefixo de tag	<code>tagPrefix</code>	<code>subN-</code>	Prefixo com o qual começam as tags dos <code>outbound</code> s importados. Se deixado vazio, o painel escolherá o menor número disponível no formato <code>sub1-</code> , <code>sub2-</code> etc.
Intervalo de atualização	<code>updateInterval</code>	600 segundos (10 minutos)	Com que frequência a assinatura é relida. Na interface é definido em horas/minutos.
Ativo	<code>enabled</code>	sim (<code>true</code>)	Apenas as assinaturas ativas entram na configuração e são atualizadas automaticamente.
Permitir endereços privados	<code>allowPrivate</code>	não (<code>false</code>)	Permite URLs em localhost, LAN e IPs privados. Desativado por padrão como proteção contra SSRF – ative apenas para fontes locais confiáveis.
Antes das saídas manuais	<code>prepend</code>	não (<code>false</code>)	Se ativado, os <code>outbound</code> s desta assinatura são colocados antes dos <code>outbound</code> s manuais do modelo, e um deles pode se tornar o <code>outbound</code> padrão. Caso contrário, são adicionados depois .

O botão «**Prévia**» (`POST /outbound-subs/parse`) permite baixar e analisar a URL antes de salvar e ver quais `outbound`s e tags serão obtidos; nada é gravado no banco de dados. Se nada for reconhecido pela URL, é exibido «Nenhuma saída encontrada nesta URL.»

A ordem de várias assinaturas na lista geral de `outbound`s é definida por prioridade (`priority`) e alterada com as setas para cima/baixo (`POST /outbound-subs/:id/move`).

Formatos de assinatura aceitos

O corpo da resposta da URL é processado assim:

- O conteúdo é primeiro tentado como **base64** (variantes padrão e URL-safe, com preenchimento automático de padding e remoção de espaços/quebras de linha). Se for base64 – é decodificado; caso contrário, é usado como está.

- Em seguida, o corpo é dividido em linhas. Cada linha não vazia que não comece com `#` é analisada como link. Linhas não reconhecidas (comentários, protocolos não suportados) são silenciosamente ignoradas.
- Esquemas de links suportados: `vmess://`, `vless://`, `trojan://`, `ss://` (Shadowsocks), `hysteria2://` / `hy2://`, `wireguard://` / `wg://`.

Ou seja, é compatível com uma assinatura comum no formato «lista de links codificada em base64», como na maioria dos provedores.

Tags estáveis

Para cada link é calculada uma «identidade» estável (URI principal sem o fragmento de observação; para vmess – JSON interno sem o campo `ps`). A correspondência «identidade → tag» é preservada, e na próxima atualização o mesmo servidor recebe a mesma tag, mesmo que a observação ou parâmetros secundários tenham mudado. Isso foi feito especificamente para que balanceadores e regras de roteamento continuem funcionando após atualizações:

- A tag exata no balanceador/regra continuará apontando para o mesmo servidor.
- Um seletor de prefixo/wildcard (por exemplo, `hk-*`) pegará automaticamente novos servidores que a assinatura retornar depois – essa é a forma recomendada de «assinar um pool».
- Se um servidor desaparecer da assinatura, sua tag simplesmente some do array final de `outbound s`; com `fallbackTag` no balanceador, o Xray o utiliza.
- Se o provedor mudou o UUID/host/credenciais do servidor, a identidade muda – isso é considerado um novo `outbound` com nova tag.

Dentro de uma única exportação, as tags são deduplicadas com o sufixo `-N`. As tags de assinaturas preservam caracteres não-ASCII (por exemplo, cirílico) e permanecem legíveis: letras e dígitos Unicode são mantidos no slug, e a pontuação é substituída por hífen – as tags de nomes cirílicos não são mais reduzidas apenas a números.

Como funciona a atualização automática

- A tarefa de atualização de assinaturas em segundo plano é executada de acordo com o cronograma **a cada 5 minutos**.
- Em cada execução, ela percorre todas as assinaturas ativas e atualiza apenas aquelas cujo próprio intervalo expirou: uma assinatura é atualizada se nunca foi atualizada antes ou se desde a última atualização já passou pelo menos seu `updateInterval`. Assim a tarefa verifica as assinaturas com frequência, mas cada assinatura específica é relida não mais do que o seu `updateInterval` (padrão de 10 minutos). Isso está refletido na dica correspondente na interface.
- Atualização: a URL é verificada novamente quanto à segurança como pública (endereços privados são bloqueados se a assinatura não tiver `allowPrivate` definido), a solicitação vai pelo cliente proxy do painel com o cabeçalho `User-Agent: 3x-ui-outbound-sub/1.0`. A cadeia de redirecionamentos é limitada a 10 saltos, e cada salto também é verificado quanto à privacidade (proteção contra SSRF). Espera-se HTTP 200; caso contrário, um erro é registrado.
- Após a análise bem-sucedida, o resultado é salvo, o horário da última atualização é definido e o erro é limpo. Em caso de erro, seu texto é visível na interface como «Último erro», e os `outbound s` obtidos anteriormente permanecem válidos.

- Se pelo menos uma assinatura for realmente atualizada, a tarefa marca o Xray para reinicialização e envia uma invalidação de interface para que a UI busque os novos `outbound`s. A recarga real do Xray ocorre no próximo ciclo de 30 segundos do gerenciador.

A atualização manual de uma assinatura — botão **«Atualizar agora»** (`POST /outbound-sub/:id/refresh`); ele também marca o Xray para reinicialização. Adicionar, alterar ou excluir uma assinatura também aciona o flag de reinicialização do Xray (ao excluir, seus `outbound`s saem da configuração na próxima recarga). A interface informa: «Após adicionar ou atualizar, reinicie o Xray (ou aguarde a próxima recarga automática) para que as saídas fiquem ativas.»

Como isso chega à configuração do Xray

A cada geração da configuração do Xray, os `outbound`s de assinaturas ativas são divididos em dois grupos — `prepend` (flag «Antes das saídas manuais») e os demais — e são costurados com o modelo: `[prepend das assinaturas] + [outbound's do modelo] + [demais assinaturas]`. Dentro de cada grupo, as assinaturas seguem por prioridade. Os `outbound`s manuais do modelo não são afetados; se o array de `outbound`s do modelo por alguma razão não puder ser analisado, os `outbound`s das assinaturas não são misturados a ele (para não perder os manuais).

Os `outbound`s importados também são exibidos no próprio painel de `outbound`s em um bloco separado **«Das assinaturas de saídas (somente leitura)»** — não é possível editá-los lá, o gerenciamento é apenas pela seção «Assinaturas de saídas».

11.13. Rotação de IP no WARP

No 3X-UI é possível criar um `outbound` WARP — uma conexão WireGuard de saída para o Cloudflare WARP (tag `warp` na configuração do Xray). O painel registra por conta própria nos servidores da Cloudflare um dispositivo/conta, obtém as chaves e endereços WireGuard e os insere no `outbound` com a tag `warp`. Por esse `outbound` o tráfego acessa a internet sob o endereço IP do Cloudflare WARP. A novidade da versão 3.3.0 é a possibilidade de alterar esse IP de saída manualmente ou por agendamento, sem recriar a conta WARP manualmente.

O gerenciamento está na seção **Xray** no cartão WARP (após clicar em «Criar conta WARP» e obter a configuração; antes disso, as ações estão indisponíveis — o painel informará «Obtenha a configuração WARP primeiro»).

O que acontece ao trocar o IP

O botão **«Trocar IP»** inicia a troca de IP. A lógica:

1. Um novo par de chaves WireGuard é gerado.
2. Com a nova chave, o dispositivo WARP é registrado novamente nos servidores da Cloudflare (novo `device_id`, `access_token`, endereços e dados do peer).
3. Os novos dados são gravados no `outbound` WARP da configuração do Xray: são atualizados `secretKey`, `address` (v4 /32 e v6 /128), `reserved` (de `client_id`), bem como `publicKey` e `endpoint` do peer.

4. Se uma chave de licença WARP+ havia sido definida anteriormente (com pelo menos 26 caracteres), ela é reinstalada automaticamente na nova conta. Em caso de falha, isso é apenas um aviso nos logs – a troca de IP não é cancelada.
5. Após a troca bem-sucedida, o Xray é marcado como necessitando de reinicialização para que o novo outbound entre em vigor.

Em caso de sucesso, a interface mostra «Endereço IP WARP alterado com sucesso!».

Rotação automática por agendamento

No cartão WARP há uma chave «**Atualização automática do endereço IP**» e o campo «**Intervalo (dias)**». Dica: «0 – desativar. Altera automaticamente o endereço IP.»

Parâmetro	Valor
Configuração no banco de dados	<code>warpUpdateInterval</code> (inteiro, ≥ 0)
Valor padrão	0 (rotação automática desativada)
Unidade de medida	dias
0	desativa a troca automática
> 0	trocar o IP a cada N dias

Salvar o intervalo armazena `warpUpdateInterval`, e com valor maior que 0 redefine o «horário da última atualização» para o momento atual – caso contrário o agendador trocava o IP já no próximo tick.

O agendamento é executado por uma tarefa em segundo plano iniciada uma vez por hora – ou seja, o painel verifica a cada hora se é hora de rotar. Algoritmo de verificação:

- se o intervalo ≤ 0 – não faz nada;
- se o «horário da última atualização» for 0 (por exemplo, o intervalo foi definido editando o banco de dados diretamente) – é a primeira execução: a tarefa apenas registra o horário base e NÃO troca o IP imediatamente;
- se desde a última atualização passaram pelo menos `intervalo × 24 × 3600` segundos – é executada a mesma troca de IP, o horário é atualizado e a reinicialização do Xray é agendada.

Detalhe importante: a troca manual pelo botão «Trocar IP» também redefine o horário da última atualização. Portanto, após uma rotação manual, a contagem do intervalo automático começa novamente e a troca programada não ocorrerá imediatamente a seguir.

«Via proxy do painel»

Alterado na versão 3.3.1. A configuração separada «Proxy de rede do painel» (`panelProxy`) foi removida. O tráfego de saída do próprio painel (incluindo solicitações à API WARP) agora é direcionado pelo **outbound de tráfego do painel** selecionado – um outbound Xray ou balanceador (veja a seção 13). A descrição abaixo se aplica às versões anteriores à 3.3.1.

Todas as solicitações à API Cloudflare WARP (registro, obtenção de configuração, definição de licença, troca de IP) não vão diretamente, mas pelo cliente HTTP do painel com timeout de 15 segundos. Esse cliente respeita a configuração **«Proxy de rede do painel»** (`panelProxy`) das configurações do painel.

Da descrição da configuração: o proxy roteia as próprias solicitações de saída do painel (atualizações de bases geo, verificações de versão do Xray/painel, Telegram, e agora também as chamadas ao WARP) — para contornar a filtragem do servidor. São aceitos endereços do tipo `socks5://` ou `http(s)://`, por exemplo um inbound SOCKS local do próprio Xray. Se o campo estiver vazio ou o proxy estiver incorretamente definido — é usada a conexão direta (o comportamento não é interrompido).

Benefício para o WARP: se o servidor não conseguia acessar diretamente `api.cloudflareclient.com`, o registro e a rotação falhavam anteriormente. Agora, definindo em `panelProxy` um proxy funcional (inclusive o próprio inbound Xray), é possível garantir a disponibilidade da API WARP e o funcionamento tanto do botão manual quanto da rotação programada.

Quando isso é útil

- Troca regular do IP de saída para o outbound que usa WARP — reduz o risco de bloqueios e rastreamento por um único endereço.
 - «Renovar» o IP manualmente, se o endereço atual da Cloudflare foi incluído em listas negras ou está funcionando lentamente.
 - Servidores sem acesso direto à API Cloudflare WARP: o roteamento de solicitações via `panelProxy` torna o registro e a rotação funcionais.
-

12. Nós (multipainel, master/slave)

A seção **Nós** transforma uma instalação comum do 3X-UI em um **painel central (master)**, que monitora e gerencia remotamente outros painéis 3X-UI (filhos). Cada nó é uma instalação separada do 3X-UI em seu próprio servidor; o master acessa-o pela sua própria API HTTP, consulta seu estado e sincroniza com ele os inbounds e clientes que lhe foram atribuídos. Essa é a funcionalidade de **multipainel**: em vez de acessar cada painel individualmente, você vê todos os servidores em uma única lista e os gerencia de forma centralizada.

Princípio importante: **um nó não é um agente, mas um painel 3X-UI completo**. O master não "instala" nada nele – apenas se conecta à sua API via token. Remover um nó da lista interrompe apenas o monitoramento; o painel remoto em si não é afetado (dica: «Isso interromperá o monitoramento do nó. O painel remoto em si não será afetado»).

12.1. Resumo no topo da lista

Acima da tabela de nós são exibidos contadores agregados:

Campo	Descrição
Total de nós	Número total de nós na lista.
Online	Quantos nós têm o status <code>online</code> .
Offline	Quantos nós têm o status <code>offline</code> .
Latência média	Latência média (ping) até os nós, em milissegundos.

12.2. Adicionando e editando um nó

Os botões **Adicionar nó** e **Editar nó** abrem um formulário com os campos do nó.

São obrigatórios (dica: «Nome, endereço, porta e token de API são obrigatórios») os campos **Nome**, **Endereço**, **Porta** e **Token de API**.

Ao clicar em «Salvar» (tanto ao adicionar quanto ao editar), o painel **primeiro verifica a acessibilidade do nó** com um tempo limite de 6 segundos. Se o nó não responder, o registro não é salvo e um erro é exibido. Ou seja, não é possível adicionar um nó que esteja comprovadamente inacessível.

Campos do formulário

Campo	Padrão	Valores permitidos	Descrição
Nome	– (obrigatório)	string não vazia, única	Nome interno do nó. A coluna de nome tem restrição de unicidade – não é possível criar dois nós com o mesmo nome. Texto de exemplo no placeholder: <code>ex: de-frankfurt-1</code> . Ao salvar, espaços nas bordas são removidos.
Observação	vazio	qualquer string	Nota/descrição opcional do nó. Não afeta o funcionamento.

Campo	Padrão	Valores permitidos	Descrição
Esquema	<code>https</code>	<code>http</code> / <code>https</code>	Protocolo de conexão com o painel remoto. Se deixado vazio ou com valor inválido, a normalização definirá <code>https</code> . Se o nó responde por HTTP comum mas o esquema está como <code>https</code> , o painel retornará uma dica clara: «the server speaks HTTP, not HTTPS; set the node scheme to http».
Endereço	– (obrigatório)	host ou IP	Endereço do painel remoto. Placeholder: <code>panel.example.com</code> ou <code>1.2.3.4</code> . O endereço é normalizado; por padrão, endereços privados/locais são proibidos como proteção contra SSRF – veja «Permitir endereço privado».
Porta	– (obrigatório)	inteiro 1–65535	Porta do painel web do nó remoto. Valores fora do intervalo são rejeitados («node port must be 1-65535»).
Caminho base	<code>/</code>	string de caminho	Caminho base (web base path) do painel remoto, se configurado. É normalizado: garante que começa e termina com <code>/</code> (valor vazio → <code>/</code>). O painel acrescenta a ele <code>panel/api/server/status</code> ao fazer consultas.
Token de API	– (obrigatório)	token do painel remoto	Bearer token para acesso à API do nó. É enviado no cabeçalho <code>Authorization: Bearer <token></code> . Placeholder: «Token da página de Configurações do painel remoto». Dica: «O painel remoto exige seu token de API na seção Configurações → Token de API». Ou seja, o token deve ser criado no próprio nó (Configurações → Token de API) e colado aqui.
Habilitado	<code>true</code>	sim/não	Habilita o monitoramento e a sincronização do nó. Nós desabilitados não são consultados pelas tarefas em segundo plano (heartbeat e traffic-sync os ignoram) e não participam da atualização em massa do painel.
Permitir endereço privado	<code>false</code>	sim/não	Remove a proteção SSRF e permite conectar-se ao nó por endereço privado/local. Dica: «Habilitar apenas para nós em rede privada ou VPN». Habilite apenas quando o nó estiver realmente em uma rede privada ou acessível via VPN.

Obtenção e regeneração do token no lado do nó

O token é obtido no painel remoto na seção **Configurações → Token de API**. Lá também é possível reemitir-lo: o botão **Gerar novo token** exibe o aviso: «Regenerar o token invalidará o token atual. Qualquer painel central que o utilize perderá o acesso até que seja atualizado. Continuar?». Após a regeneração, o token antigo no painel master deixará de funcionar – é necessário atualizá-lo no formulário do nó.

Conexão de saída (Connection outbound)

O campo **Connection outbound** (Conexão de saída, `outboundTag`) define como o tráfego das chamadas do master à API deste nó sai do servidor. Se você selecionar uma tag de Xray-outbound, as chamadas do painel ao nó não passarão diretamente, mas pelo outbound especificado; o painel adicionará automaticamente à configuração em execução um inbound de ponte no loopback e aplicará a alteração em tempo real, sem reinicialização. Dica: «Route this node's panel API traffic through the selected Xray outbound. A loopback bridge inbound is added to the running config automatically and applied live. Leave empty for a direct connection».

O seletor funciona como o seletor de outbound do painel: as tags são agrupadas em **Outbounds** (saídas comuns) e **Balancers** (balanceadores); outbounds do tipo blackhole são ocultados da lista. Valor vazio (placeholder «Direct connection») = conexão direta com o nó.

Importação de inbound (seleção de inbounds para sincronização)

No formulário do nó há uma configuração **Importar inbound** (`inboundSyncMode`) com dois modos: **Todos os inbounds** (`all` , padrão) e **Selecionados** (`selected`). Por padrão, o master sincroniza com o nó todos os inbounds nos quais esse nó está selecionado; nós existentes continuam funcionando no modo «Todos os inbounds».

No modo **Selecionados**, aparece abaixo do campo uma seleção múltipla de tags de inbound. Clique em **Carregar inbounds** – o master usará os parâmetros de conexão informados (ainda não salvos) para solicitar ao nó a lista de seus inbounds (endpoint `POST /panel/api/nodes/inbounds`) e exibirá suas tags; marque as desejadas. O painel sincronizará e implantará no nó apenas as tags marcadas, enquanto os demais inbounds existentes diretamente no nó permanecerão intocados – o master não os exclui nem os gerencia.

Exemplo: solicitar a lista de inbounds do nó para importação seletiva. O corpo contém os parâmetros de conexão ainda não salvos; a resposta contém as tags dos inbounds disponíveis no nó:

```
POST /panel/api/nodes/inbounds
Content-Type: application/json

{ "name": "de-fra-1", "scheme": "https", "address": "node1.example.com",
  "port": 2053, "basePath": "/", "apiToken": "abcdef..." }
```

12.3. Verificação TLS (para nós https)

O grupo de campos define como o master verifica o certificado HTTPS do nó. Essas configurações **são relevantes apenas para o esquema https** ; para nós `http` são ignoradas.

Verificação TLS – lista suspensa, dica: «Como o painel verifica o certificado HTTPS do nó. Fixação ou Ignorar – para certificados autoassinados (apenas nós https)».

Modo	Valor	Padrão	Descrição
Verificar (CA padrão)	<code>verify</code>	sim (padrão)	Verificação normal da cadeia de certificados por CA confiável. Adequado para nós com certificado público/Let's Encrypt. Também usado para todos os nós <code>http</code> .
Fixar certificado (SHA-256)	<code>pin</code>	–	A cadeia CA não é verificada, mas o SHA-256 do certificado folha do nó é comparado com a impressão digital armazenada (comparação em tempo constante). Mantém a proteção contra MITM para certificados autoassinados . Requer o preenchimento do campo de impressão digital.
Ignorar verificação	<code>skip</code>	–	A verificação do certificado é completamente desabilitada. Aviso: «Ignorar a verificação remove a proteção contra ataques "homem no meio" – o token de API pode ser interceptado. É melhor fixar o certificado».

Aos três modos acima, na versão 3.4.0 foi adicionado um quarto – **Mutual TLS (certificado de cliente)** (`mtls`), disponível, assim como os demais, apenas para o esquema `https` .

Modo	Valor	Padrão	Descrição
Mutual TLS (certificado de cliente)	mtls	—	Além de verificar o certificado do nó, o master também se autentica perante o nó com um certificado de cliente emitido por sua própria CA. Para o nó nesse modo, o token de API torna-se opcional — o nó reconhece o master pelo certificado. Ao selecionar esse modo, é exibida a dica: «This node authenticates the panel with a client certificate. Copy this panel's CA from the Node mTLS section onto the node, set its Trusted parent CA, then restart it».

Para habilitar o TLS mútuo para um nó: no lado do nó, defina o modo **Mutual TLS**, copie a CA do painel gerenciador da seção **Node mTLS** (veja abaixo), configure-a no nó como **CA pai confiável** e reinicie o nó.

Se qualquer valor diferente de `skip`, `pin` ou `mtls` for selecionado, a normalização forçará `verify`.

Fixação de certificado

Ao selecionar **Fixar certificado**, aparecem:

- **SHA-256 do certificado fixado** — campo de entrada. Aceita a impressão digital em **base64** (formato `pinnedPeerCertSha256` do Xray) ou em **hex** com dois pontos ou sem (estilo `openssl -fingerprint`). Dica: «SHA-256 do certificado do nó em base64 ou hex. Clique em "Obter" para lê-lo do nó agora». Placeholder: «SHA-256 em base64 ou hex». Ao selecionar `pin`, uma impressão digital vazia ou inválida gera um erro de validação ao salvar.

Exemplo: a mesma impressão digital em dois formatos. O campo aceita qualquer um dos formatos — ambos representam o mesmo certificado:

```
# base64 (formato pinnedPeerCertSha256 do Xray)
607TNg3l2k0pq8R1sT2uV3wX4yZ5a6B7c8D9e0F1g2=

# hex com dois pontos (estilo openssl x509 -fingerprint -sha256)
E8:E2:D3:60:DE:5D:9A:4D:29:AB:CF:11:B2:7C:34:...
```

Se a impressão digital ainda não for conhecida, clique em **Obter** — o master a lerá automaticamente do nó via HTTPS e preencherá o campo.

- Botão **Obter** — conecta-se ao nó via HTTPS sem verificar o certificado e lê o SHA-256 do certificado folha atual (endpoint `POST /certFingerprint`), preenchendo-o no campo. Após o sucesso — «Certificado atual do nó obtido»; em caso de falha — «Não foi possível obter o certificado». Disponível apenas para nós https.

Node mTLS (autenticação TLS mútua entre painéis)

Na página **Nós** há uma seção separada **Node mTLS** — configuração de autenticação TLS mútua que adiciona um segundo fator (certificado de cliente) ao token de API para chamadas «painel → nó». O TLS mútuo é opcional; se os campos da seção estiverem vazios, os nós operam pelo esquema anterior — **apenas com o token de API** (dica: «Mutual TLS adds a client-certificate factor on top of the API token for node-to-node calls. It is opt-in: leave it empty to keep token-only auth»). A seção tem duas operações:

- **Copiar CA deste painel** (`POST /panel/api/nodes/mtls/ca`) — copia o certificado raiz (CA) deste painel para a área de transferência. Essa CA deve ser enviada aos nós gerenciados para que eles confiem no

certificado de cliente do painel; nos próprios nós, depois disso, define-se o modo de verificação TLS como **Mutual TLS** (dica: «Hand this CA to the nodes this panel manages, then set their TLS verification to Mutual TLS»). Após copiar – «CA certificate copied to clipboard».

- **CA pai confiável** (`Trusted parent CA` , `POST /panel/api/nodes/mtls/trustCA`) – campo usado quando este próprio painel funciona como nó para um painel superior (gerenciador). Cole aqui a CA do painel gerenciador para exigir seu certificado de cliente e clique em **Save trust CA**. A alteração requer **reinicialização do painel** (dica: «When this panel is itself a node, paste the managing panel's CA here to require its client certificate. Restart the panel to apply»).

12.4. O que é exibido para cada nó

Colunas da tabela e campos do cartão do nó (estado observado, preenchido a cada consulta de heartbeat):

Campo	Descrição
Status	<code>online</code> / <code>offline</code> / <code>unknown</code> – veja abaixo.
CPU	Carga do processador do servidor remoto em porcentagem.
Memória	Uso de RAM em porcentagem (calculado como <code>current/total*100</code>).
Uptime	Tempo de funcionamento contínuo do servidor (em segundos).
Latência	Tempo de resposta do nó na última consulta (ms).
Último ping	Hora do último heartbeat bem-sucedido (segundos unix; <code>0</code> = «nunca»; valor recente é exibido como «agora mesmo»).
Versão do Xray	Versão do Xray-core em execução no nó.
Versão do painel	Versão do 3X-UI no nó – comparada com a atual para o indicador de atualização.
(inbounds)	Quantos inbounds estão fisicamente hospedados neste nó.
(clientes)	Número de clientes nos inbounds do nó.
(online)	Quantos clientes do nó estão atualmente conectados.
(esgotados)	Quantos clientes do nó expiraram ou esgotaram o limite de tráfego . Clientes desabilitados manualmente não entram nesse contador.
(velocidade)	Velocidade de transferência atual (ao vivo) nos inbounds hospedados no nó.

Os contadores de inbounds/clientes/online são vinculados ao nó pelo seu GUID estável (`panelGuid`), e não pelo id local – para que um cliente em um subnó seja contabilizado exatamente sob o subnó, e não sob o nó intermediário pelo qual ele é sincronizado.

Para inbounds hospedados no nó, a página exibe clientes online, contadores e **velocidade de transferência atual**. A vinculação pelo GUID estável distingue corretamente também os nós «clonados» com o mesmo `panelGuid` .

Status do nó

Status	Quando é definido
online	O nó respondeu <code>success=true</code> à consulta <code>panel/api/server/status</code> .
offline	O nó não respondeu, retornou erro HTTP, <code>success=false</code> ou uma resposta irreconhecível.
unknown	Valor inicial, enquanto o nó ainda não foi consultado nenhuma vez.

Em caso de consulta malsucedida, o texto do erro é salvo e exibido de forma clara, o que ajuda a diagnosticar a causa do «offline».

12.5. Ações sobre um nó

- **Testar conexão** (`POST /test`) – no formulário do nó, verifica a conexão usando os parâmetros informados (ainda não salvos) com um tempo limite de 6 s. Resultado: «Conexão OK ({ms} ms)» ou «Não foi possível conectar». Útil para depurar endereço/porta/token/TLS antes de salvar.
- **Verificar agora** (botão «Verificar agora», `POST /probe/:id`) – consulta não planejada de um nó já salvo; atualiza imediatamente o status e as métricas (CPU/memória/uptime/latência/versões) e registra o heartbeat. Em caso de falha – «Verificação falhou».

Exemplo: testar e consultar um nó via API do master. «Testar conexão» verifica parâmetros ainda não salvos do formulário:

```
POST /panel/api/nodes/test
Content-Type: application/json

{ "scheme": "https", "address": "de-frankfurt-1.example.com", "port": 2053,
  "basePath": "/", "apiToken": "eyJhbGciOi...", "tlsMode": "verify" }
```

Consulta não planejada de um nó já salvo com id 7:

```
POST /panel/api/nodes/probe/7
```

- **Atualizar painel** (`POST /updatePanel` com corpo `{ids:[...]}`) – inicia no nó seu atualizador automático nativo: o nó baixa a última versão do 3X-UI e reinicia com ela. O botão **Atualizar selecionados ({count})** executa isso para vários nós marcados de uma vez. Ao lado do nó é exibido um indicador: **Atualização disponível** ou **Atualizado**, com base na comparação da versão do painel do nó com a mais recente.

Exemplo: atualizar vários nós com uma única solicitação. O corpo contém os ids dos nós marcados; serão atualizados apenas os habilitados e `online` ; os demais serão retornados como ignorados.

```
POST /panel/api/nodes/updatePanel
Content-Type: application/json

{ "ids": [3, 7, 12] }
```

Resposta do tipo «Atualização iniciada em 2 nós, 1 falhou»: o nó 12, por exemplo, pode ter estado offline e, portanto, sido ignorado.

- Confirmação: «Atualizar {count} nós para a versão mais recente? Cada nó selecionado baixará a última versão e será reiniciado. Apenas nós habilitados e online serão atualizados».
- **Apenas nós habilitados com status online são atualizados.** Um nó desabilitado nos resultados é marcado como «node is disabled», offline – como «node is offline». Resultado: «Atualização iniciada em {ok} nós, {failed} falharam». Se nenhum nó adequado for selecionado – «Selecione pelo menos um nó habilitado e online».

No diálogo de confirmação de atualização (tanto para um único nó quanto em massa) há uma caixa de seleção **Atualizar para o canal de desenvolvimento (último commit)**. Se marcada, os nós selecionados instalarão a build rolling dev-latest (último commit do branch main) em vez da versão estável; com a caixa desmarcada, o nó é atualizado pelo seu canal habitual. Com a caixa marcada, é exibido um aviso abaixo dela: «Builds de desenvolvimento acompanham cada commit no main e não são versões estáveis – não há rollback automático». O sinalizador dev é transmitido via `POST /panel/api/nodes/updatePanel` ao nó, e este inicia a atualização pelo canal dev.

- **Set Cert from Panel** (auxiliar, `GET /webCert/:id`) – ao criar um inbound no nó, permite preencher os caminhos para o certificado TLS **do próprio** painel web do nó (e não do painel central), para que os arquivos existam exatamente no nó. Requer que o nó esteja habilitado e acessível.
- **Excluir nó** (`POST /del/:id`) – confirmação: «Excluir o nó "{name}"? Isso interromperá o monitoramento do nó. O painel remoto em si não será afetado». Exclui o registro do nó e suas estatísticas de tráfego acumuladas; o painel remoto continua funcionando normalmente. **Um nó só pode ser excluído após remover todos os inbounds dele.** Se pelo menos um inbound ainda estiver vinculado ao nó (via `node_id`), o painel rejeitará a exclusão com um erro do tipo «cannot delete node: N inbound(s) still attached to it; detach or delete them first» – primeiro desvincula ou exclui esses inbounds, depois exclui o nó. Isso evita inbounds «órfãos» com referência pendente a um nó excluído.

12.6. Histórico de métricas

O botão/gráfico de histórico acessa `GET /history/:id/:metric/:bucket`. Métricas disponíveis: **cpu** e **mem** – são acumuladas a cada heartbeat bem-sucedido. O tamanho do intervalo de agregação (`bucket`, em segundos) é limitado por uma lista de permissões:

Exemplo: solicitação de histórico. Gráfico de carga de CPU do nó 7 com agregação por intervalos de 60 segundos (retorna até 60 pontos):

```
GET /panel/api/nodes/history/7/cpu/60
```

Para memória e modo «tempo real» (2 s) – respectivamente `.../7/mem/60` e `.../7/cpu/2`. Valores fora da lista de permissões são rejeitados («invalid metric» / «invalid bucket»).

Bucket (s)	Finalidade
2	Modo tempo real
30	Intervalos de 30 s

Bucket (s)	Finalidade
60	Intervalos de 1 min
120	Intervalos de 2 min
180	Intervalos de 3 min
300	Intervalos de 5 min

São retornados até 60 pontos. Métrica ou bucket inválidos são rejeitados («invalid metric» / «invalid bucket»).

12.7. Como os inbounds e clientes são sincronizados

Um inbound «pertence» a um nó pelo campo `node_id` (o nó é selecionado no editor de inbound):

Exemplo: token no formulário do nó. O token é obtido no painel filho (Configurações → Token de API) e colado no campo **Token de API** do master. A cada consulta, o master o envia no cabeçalho:

```
GET https://panel.example.com:2053/panel/api/server/status
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.abc123...
```

Se o painel filho tiver um **caminho base** (web base path) configurado, por exemplo `/secret/`, o master o inserirá automaticamente antes de `panel/api/server/status` → `https://panel.example.com:2053/secret/panel/api/server/status`.

1. **Implantação de configuração (reconcile).** A qualquer alteração em um inbound/cliente vinculado a um nó, o nó é marcado como «sujo». A tarefa em segundo plano para cada nó habilitado **com status online**, se houver alterações, implanta nele seus inbounds (por `node_id`) e então limpa o sinalizador de «estado sujo». Um nó que está desabilitado, offline ou «sujo» é considerado «pendente» – a implantação nele é adiada até que a conexão seja restabelecida.
2. **Coleta de tráfego.** A mesma tarefa solicita ao nó um snapshot de tráfego e o mescla nas estatísticas locais. Com base no tráfego mesclado, é feita a verificação de esgotamento de limites/prazos e, se necessário, a desabilitação de clientes; o contador «esgotados» por nó reflete exatamente isso. Se o nó estiver inacessível, seus clientes online são limpos.

Para um cliente vinculado a vários painéis ao mesmo tempo, o master na mesma tarefa também envia aos nós o consumo de tráfego **total de todos os painéis** daquele cliente (em uma tabela separada no nó, com chave – GUID do master; sobrescrita a cada envio, portanto a redefinição no lado do master também é propagada). No nó, o tráfego do cliente exibe o maior dos dois valores – o local ou o recebido –, e ao ultrapassar a cota total, o cliente é desabilitado **localmente no próprio nó** (pelo mesmo mecanismo de reinicialização do Xray no desligamento automático, que encerra conexões já estabelecidas). Isso elimina a situação em que o nó via apenas sua parcela do tráfego, subestimava o consumo e continuava a atender um cliente que já havia esgotado o limite total. Ao redefinir o tráfego, renovar automaticamente ou excluir o cliente, os contadores enviados são zerados.

Quando ocorre a **primeira** sincronização de um inbound hospedado em um nó (adição de um novo nó ou reimportação de inbound), o master inicializa os contadores de tráfego dos clientes com os valores reais do nó. Antes, nessa situação, o contador total do inbound era transferido corretamente, mas os contadores individuais

dos clientes eram zerados, e o master subestimava o consumo dos clientes por todo o histórico acumulado antes da conexão do nó. Agora, se o inbound for criado na mesma sincronização, a nova linha `client_traffics` herda o valor do contador do nó (a linha de base é definida igual a ele, portanto o próximo delta é zero e o tráfego não é contabilizado duas vezes). A sementeira do contador é aplicada apenas para o inbound criado nessa mesma passagem: um cliente que reaparecer sob um inbound já existente ainda começa do zero (proteção contra tráfego «fantasma»), e um cliente recém-excluído não «ressuscita» ao recriar seu inbound.

3. **Heartbeat.** Uma tarefa em segundo plano separada consulta periodicamente todos os nós **habilitados** (com limite de paralelismo) via `panel/api/server/status`, atualiza o status/métricas/versões e, se houver clientes web, distribui a árvore de nós atualizada via WebSocket.

12.8. Cadeias de nós (subnós / nós transitivos)

A topologia pode não ser plana: um nó pode ser master para seus próprios nós. Esses painéis subordinados aparecem para você como **Subnós** — são **projeções somente leitura**, obtidas do nó direto.

- Dica: «Somente leitura: nó subordinado acessível via {parent}. Gerencie-o a partir do painel próprio de {parent}». Ou seja, o subnó não pode ser editado, excluído ou atualizado aqui — todas as operações com ele são realizadas a partir do painel de seu pai direto.
- A identidade do subnó é determinada por seu GUID; graças a isso, os clientes online e os inbounds são contabilizados exatamente sob o nó físico que os hospeda, mesmo em uma cadeia `Node1 → Node2 → Node3` (o master «percorre» um nível mais fundo por meio de cada nó direto).
- Se o nó direto ficar inacessível, seu cache de subnós é limpo e os subnós desaparecem da árvore até que a conexão seja restabelecida.

12.9. Correções da 3.5.0 (estabilidade dos nós)

Pacote de correções na 3.5.0 perceptíveis para o operador de nós:

- **Salvar um cliente sem alterações não derruba mais o tráfego do nó.** Antes, qualquer salvamento de cliente (mesmo «abriu — salvou») enviava ao nó uma atualização completa do inbound com recriação do handler do Xray — o nó constava online, mas não passava tráfego até um reinício manual. Agora salvamentos sem alterações não fazem nada, e edições reais seguem em uma única atualização leve, sem recriação.
- **As sobrescritas de Host do nó são aceitas no master:** na primeira aceitação de um inbound do nó, suas linhas de Hosts (SNI, fingerprint, ALPN etc.) são importadas, e as assinaturas carregam os parâmetros TLS corretos.
- **A renovação automática no nó abre uma janela de cota nova** (um pacote do tipo «100 GB a cada 30 dias» volta a entregar o volume completo).
- **A exclusão de um cliente no master o exclui completamente nos nós** (registro + tráfego), e não apenas o desvincula; restos de exclusões em massa anteriores podem ser removidos com a ação «Excluir clientes não vinculados» no nó.
- **Os inbound do nó não são varridos antes da primeira aceitação:** salvar um nó recém-adicionado não pode mais excluir seus inbound ativos.
- **Um único inbound incorreto não interrompe a sincronização de tráfego do nó** (de quebra, inbound legados `socks` nos nós são renomeados para `mixed`).
- **A verificação de conflito de portas é limitada ao próprio nó** — a edição do endereço de escuta de um inbound de nó não é mais rejeitada por causa da mesma porta em outro nó («port ... already used by inbound ...»).

12.10. Nós: novidades na versão 3.3.0

Na versão 3.3.0, a seção **Nós** recebeu três melhorias notáveis: atribuição correta de tráfego e clientes online em topologias multinível (multi-hop), sincronização de client-IP entre nós e um indicador de status separado para o caso em que o painel do nó está ativo, mas o núcleo Xray nele caiu.

1. Multi-hop: atribuição correta de tráfego na cadeia de subnós

Antes, os contadores (número de inbounds, clientes online, esgotados) eram calculados no nível do nó «direto». Se você tivesse uma cadeia do tipo `Master → Node1 → Node2 → Node3`, tudo o que estava fisicamente em `Node2 / Node3` era erroneamente atribuído a `Node1`, pelo qual chegava ao master. Na versão 3.3.0, a atribuição é feita pela fonte real.

Como funciona:

- **Os subnós tornam-se visíveis como linhas separadas.** Cada painel publica a lista de seus nós diretos; são incluídos apenas nós com `Guid` conhecido – a identidade estável é necessária para atribuir o nó a um «salto» acima. O master periodicamente (a partir da tarefa de heartbeat) busca essas listas e as armazena em cache, e então adiciona aos nós diretos os subnós «transitivos».
- **Nós transitivos são somente leitura.** Na interface, eles são marcados como «**Subnó**» com a dica: «*Somente leitura: nó subordinado acessível via {pai}. Gerencie-o a partir do painel próprio de {pai}.*» Não há botões de controle nessa linha – o nó é gerenciado a partir do painel de seu pai imediato.
- **Hierarquia via GUID.** O `ParentGuid` de um nó direto é o GUID do próprio master; o de um nó transitivo é o GUID de seu nó pai. Assim é construída a árvore.
- **A fonte de verdade para os contadores é `origin_node_guid` no inbound.** Este é o `panelGuid` do nó que fisicamente hospeda esse inbound. Ele é definido durante a sincronização do inbound com o nó e é **preservado como está nos saltos subsequentes**, portanto um inbound profundamente aninhado é atribuído ao nó real, e não ao intermediário. Por esse GUID são recalculados os contadores de número de inbounds, clientes online e clientes esgotados. Lógica de seleção da chave:

Estado do inbound	Atribuído a
<code>origin_node_guid</code> definido	esse GUID (nó fonte real)
vazio, mas <code>node_id</code> definido	GUID sintético do nó (build antigo, ainda não informou seu <code>panelGuid</code>)
vazio e <code>node_id</code> vazio	GUID próprio do master (inbound no Xray local)

Os clientes online também são agrupados por GUID, portanto cada linha de nó exibe apenas os que estão realmente conectados a ele.

O que o usuário vê: em uma topologia plana (nós diretamente sob o master), nada muda – os contadores por GUID e por `id` coincidem. Mas assim que uma cadeia de nós aparece, linhas-«Subnós» surgem na lista, e os números de inbounds/online/esgotados de cada nó passam a refletir exatamente sua própria carga, e não a soma de tudo que passou por ele em trânsito.

2. Sincronização de client-IP do access.log entre nós

O limite por IP (`limitIp` do cliente) depende dos endereços que o Xray registra em seu access.log. Antes, cada nó via apenas as conexões consigo mesmo, portanto a restrição «no máximo N IPs por cliente» não funcionava no cluster: um cliente podia conectar-se a diferentes nós e contornar o limite. Na versão 3.3.0, os IPs observados são sincronizados em todo o cluster.

Como funciona:

- Em cada nó, uma tarefa em segundo plano analisa o access.log, extraíndo de cada linha o IP, o e-mail do cliente e o timestamp, e os armazena em uma tabela local (um registro por e-mail; os IPs são armazenados como array JSON `{ip, timestamp}`). Endereços locais `127.0.0.1` e `::1` são descartados.
- A sincronização **a cada 10 segundos** realiza uma troca bidirecional para cada nó habilitado e online: puxa os IPs do nó e os mescla na tabela local, e então envia ao nó o panorama consolidado do master.
- A mesclagem combina observações antigas e recebidas **sem duplicação** de um mesmo IP visto em vários nós, e **sem ressurreição de registros desatualizados**: é aplicado o mesmo limiar de idade que na tarefa local — **30 minutos**. Para cada IP, é armazenado o timestamp mais recente. Registros de outros nós recebem um novo id local (os espaços de id dos nós são independentes); a inserção concorrente do mesmo e-mail é protegida contra duplicatas.
- Ao calcular o limite, um IP é considerado «ativo» se foi observado na varredura local atual ou tem um timestamp muito recente da base sincronizada (**dentro de 2 minutos**). É exatamente isso que faz o limite funcionar em escala de todo o cluster, mesmo que o endereço tenha sido observado em outro nó. Ao exceder o limite, os IPs «ativos» mais antigos são enviados para o log do fail2ban e as conexões são encerradas forçosamente (remove/re-add do cliente via API do Xray).

O que o usuário vê: a restrição por número de IPs agora funciona para todo o cluster, e não para cada nó individualmente; no painel, por cliente, são visíveis os IPs observados em qualquer nó (dentro da janela de 30 minutos). Não há botão/configuração separada para isso — a sincronização ocorre automaticamente em segundo plano, desde que o access.log do nó esteja habilitado e acessível (para o próprio limite, o Fail2Ban também é necessário no nó).

3. Indicador de status separado: painel do nó online, mas Xray caiu

Antes, o status do nó era essencialmente «online / offline». Se o painel do nó respondia, o nó era considerado online — mesmo quando o núcleo Xray nele não estava funcionando e os clientes, na prática, não conseguiam se conectar. Na versão 3.3.0, a saúde do painel e a saúde do núcleo Xray são separadas.

Como funciona:

- Ao consultar o nó, o master extrai da resposta do `/panel/api/server/status` remoto os campos `xray.state` e `xray.errorMsg` e os armazena no nó. Esses campos são preenchidos mesmo quando o ping do painel é bem-sucedido mas o núcleo está com problemas — exatamente para distinguir a acessibilidade do painel do estado do Xray.
- Valores de `xray.state`: `"running"` (em execução), `"stop"` (parado), `"error"` (erro).
- Esses valores são traduzidos em status do nó. Aos conhecidos foram adicionados novos:

Chave de status	Legenda	Quando é exibido
online	«Online»	o painel responde, Xray está em execução (running)
offline	«Offline»	o painel está inacessível / ping falhou
unknown	«Desconhecido»	estado ainda não determinado
xrayError	«Erro do Xray»	painel online, mas o núcleo Xray está em estado error (há errorMsg)
xrayStopped	«Parado»	painel online, mas Xray está parado (stop)

- Para esse tipo de estado, a interface usa um **indicador roxo separado** (cor diferente do verde «online» e vermelho «offline»). O roxo sinaliza diretamente: o nó pode ser alcançado, o problema está no próprio núcleo Xray, e não na rede ou no painel em si.

O que o usuário vê: em vez de um enganoso «verde» com o núcleo caído, o nó é destacado em **roxo** com o status «**Erro do Xray**» ou «**Parado**». Isso mostra imediatamente que é preciso corrigir o Xray no nó (reiniciar o núcleo, verificar errorMsg), e não investigar a acessibilidade do próprio nó. O mesmo xrayState / xrayError é propagado também para os subnós transitivos (veja item 1), portanto o estado incorreto do núcleo é visível em toda a cadeia.

13. Configurações do Pannel

A seção "Configurações" (título da página — **Configurações**, em inglês *Panel Settings*) controla o comportamento da própria interface web 3X-UI: em qual endereço e porta ela escuta, como é protegida, como interage com o bot do Telegram e serviços externos, e em qual fuso horário executa as tarefas agendadas. Cada parâmetro é armazenado na tabela `settings` do banco de dados como um par "chave — valor"; se o valor não estiver no banco, o valor padrão é aplicado.

Importante — aplicação das alterações. Qualquer alteração nesta página deve ser salva pelo botão **Salvar** (*Save*) e, em seguida, o painel deve ser reiniciado para que as alterações entrem em vigor. A dica exibida é: "Salve as alterações e reinicie o painel para aplicá-las." Ao salvar, é exibida a notificação "Configurações alteradas".

13.1. Salvar e reiniciar o painel

Elemento	Finalidade
Salvar (<i>Save</i>)	Grava todos os campos do formulário no banco de dados (<code>POST /panel/setting/update</code>). Antes de gravar, os valores são validados — endereços, portas ou caminhos incorretos serão rejeitados e o painel retornará um erro.
Reiniciar painel (<i>Restart Panel</i>)	Reinicia o servidor web do painel (<code>POST /panel/setting/restartPanel</code>). A reinicialização ocorre com um atraso de 3 segundos. Dica: "Tem certeza de que deseja reiniciar o painel? Confirme e o painel será reiniciado em 3 segundos. Se o painel ficar indisponível, verifique o log do servidor." Em caso de sucesso — "Painel reiniciado com sucesso."
Restaurar configurações padrão (<i>Reset to Default</i>)	Exclui todas as configurações salvas no banco de dados; em seguida, o painel passa a usar os valores padrão. As credenciais do administrador não são redefinidas por esta operação.

A reinicialização é feita enviando ao processo do painel o sinal `SIGHUP` (ou por meio de um hook de reinicialização registrado). No Windows, a reinicialização automática via sinal não é suportada. **As alterações nos parâmetros de escuta (IP, porta, caminho, domínio, certificados, fuso horário) são aplicadas somente após reiniciar o painel.**

13.2. Configurações gerais (aba "Painel" / *General*)

Idioma da interface (*Language*)

Idioma da interface web do painel. Idiomas disponíveis: `en-US` (inglês), `ru-RU` (russo), `zh-CN`, `zh-TW`, `fa-IR`, `ar-EG`, `es-ES`, `id-ID`, `ja-JP`, `pt-BR`, `tr-TR`, `uk-UA`, `vi-VN`. Esta é uma configuração de exibição e não afeta o funcionamento do Xray.

Tipo de calendário (*Calendar Type*)

- **Chave:** `datepicker`
- **Valor padrão:** `gregorian` (gregoriano).

- **Finalidade:** tipo de calendário utilizado na seleção de datas (por exemplo, ao definir a data de validade dos clientes). Dica: "As tarefas agendadas serão executadas de acordo com este calendário." O valor alternativo é o calendário persa (jalali), muito utilizado pelo público iraniano do painel.

Tamanho da paginação (*Pagination Size*)

- **Chave:** `pageSize`
- **Valor padrão:** `25`
- **Valores permitidos:** inteiro de `0` a `1000`.
- **Finalidade:** número de linhas por página nas tabelas (listas de conexões/inbound). Dica: "Define o tamanho da página para a tabela de conexões. Defina 0 para desativar" — quando `0`, a paginação é desativada e todos os registros são exibidos em uma única lista.
- **Reinicialização do painel não é necessária** (configuração de exibição).

Reiniciar Xray após desativação automática (*Restart Xray After Auto Disable*)

- **Chave:** `restartXrayOnClientDisable`
- **Valor padrão:** `true`
- **Finalidade:** quando um cliente é desativado automaticamente (por expiração do prazo de validade ou por atingir o limite de tráfego), o Xray é reiniciado para encerrar as conexões já estabelecidas desse cliente. Dica: "Quando um cliente é desativado automaticamente por expiração do prazo ou limite de tráfego, reiniciar o Xray." A função em si não mudou — o interruptor apenas reside na aba "Painel" (*General*), junto com as demais configurações gerais.

Modelo de observação e caractere separador (*Remark Model & Separation Character*)

- **Chave:** `remarkModel`
- **Valor padrão:** `-ieo`
- **Finalidade:** define como o nome (remark) da configuração é formado na assinatura. A string consiste no **primeiro caractere** — o separador — seguido de uma **sequência de letras de ordem**:
 - `i` — observação do inbound (*inbound remark*);
 - `e` — e-mail do cliente;
 - `o` — rótulo adicional (*extra*).

Com o valor padrão `-ieo`, o separador é `-` e a ordem das partes é: inbound → e-mail → extra (por exemplo, `MyInbound-user@mail-extra`). Partes vazias são omitidas. O campo "Exemplo de observação" (*Sample Remark*) na interface mostra uma pré-visualização do nome gerado. A inclusão do e-mail no nome depende adicionalmente do parâmetro "Incluir e-mail no nome" nas configurações de assinatura (seção sobre assinaturas).

Exemplo: como o valor de `remarkModel` afeta o nome da configuração. Suponha que o inbound se chame `VLESS-Reality`, o e-mail do cliente seja `alex@vpn` e o rótulo adicional seja `RU`. Então:

Valor do campo	Nome resultante (remark)
-ieo (padrão)	VLESS-Reality-alex@vpn-RU
_ie	VLESS-Reality_alex@vpn
-ei	alex@vpn-VLESS-Reality
o (espaço como separador, apenas rótulo)	RU

O primeiro caractere da string é sempre o separador; as demais letras definem quais partes e em que ordem comporão o nome.

13.3. Acesso ao painel: IP, porta, caminho, domínio, certificado

Este grupo define o ponto de entrada de rede do painel. **Todas as alterações aqui são aplicadas somente após reiniciar o painel.**

Campo	Chave	Valor padrão	Descrição
Endereço IP para gerenciar o painel (<i>Listen IP</i>)	webListen	"" (vazio)	IP em que a interface web escuta. Vazio = escutar em todos os IPs. Dica: "Deixe em branco para conexão de qualquer IP". Se definido, deve ser um endereço IP válido (caso contrário, o salvamento é rejeitado).
Domínio do painel (<i>Listen Domain</i>)	webDomain	"" (vazio)	Nome de domínio do painel para verificação de requisição por domínio. Vazio = aceitar conexões de quaisquer domínios e IPs. Dica: "Deixe em branco para conexão de quaisquer domínios e IPs."
Porta do painel (<i>Listen Port</i>)	webPort	2053	Porta em que o painel opera. Dica: "Porta em que o painel opera". Permitido de 1 a 65535. A porta deve estar livre; o painel e o serviço de assinatura não podem usar simultaneamente o mesmo par IP:porta.
Caminho URI (<i>URI Path</i>)	webBasePath	/	Caminho base de URL do painel (basePath). Dica: "Deve começar com '/' e terminar com '/'". Ao salvar, o painel adiciona automaticamente as barras inicial e final, caso estejam ausentes. Caracteres proibidos no caminho são rejeitados.

Certificado do painel (TLS / HTTPS)

Campo	Chave	Valor padrão	Descrição
Caminho para o arquivo de chave pública do certificado do painel (<i>Public Key Path</i>)	webCertFile	""	Caminho completo para o arquivo de certificado (cadeia). Dica: "Insira o caminho completo começando com '/'".
Caminho para o arquivo de chave privada do certificado do painel (<i>Private Key Path</i>)	webKeyFile	""	Caminho completo para o arquivo de chave privada. Dica: "Insira o caminho completo começando com '/'".

Se **ao menos um** dos caminhos de certificado/chave for definido, o painel tenta carregar o par "certificado + chave" ao salvar; em caso de erro (arquivo inexistente, incompatibilidade entre chave e certificado), o

salvamento é rejeitado. Quando ambos os caminhos corretos são definidos, o painel passa a usar HTTPS. Ambos os campos vazios = o painel opera via HTTP simples.

Avisos de segurança (*Security warnings*). O painel exibe o bloco "Seu painel pode estar exposto:" com avisos quando detecta uma configuração insegura:

- operação via HTTP simples — "configure o TLS para produção";
- porta padrão 2053 — "altere-a para uma porta aleatória";
- caminho base padrão / — "altere-o para um valor aleatório";
- caminho de assinatura padrão /sub/ e assinatura JSON /json/ — "altere-o". São recomendações, não bloqueios.

13.4. Sessão, proxy do painel e proxies confiáveis (aba "Proxy e servidor" / *Proxy and Server*)

Duração da sessão (*Session Duration*)

- **Chave:** `sessionMaxAge`
- **Valor padrão:** 360 (minutos, ou seja, 6 horas).
- **Valores permitidos:** de 1 a 525600 minutos (1 ano).
- **Finalidade:** por quanto tempo o administrador permanece autenticado sem precisar fazer login novamente. A unidade é **minuto**. Dica: "Duração da sessão no sistema (valor: minuto)".

Outbound para tráfego do painel (*Panel Traffic Outbound*)

- **Chave:** `panelOutbound`
- **Valor padrão:** "" (vazio = conexão direta).
- **Finalidade:** define o **outbound** Xray pelo qual o painel envia **suas próprias requisições** — verificações de versão e download do painel/Xray, chamadas ao Telegram, atualização regular de arquivos geo — para contornar a filtragem de servidor do GitHub/Telegram. O campo é uma **lista suspensa**: nela estão listados os outbounds do template de configuração do Xray, os outbounds de assinaturas de outbound, bem como os **balanceadores** de rota (em grupo separado). Outbounds do tipo `blackhole` são excluídos da lista — roteá-los para um "buraco negro" não faz sentido. Dica literal: "Roteia as próprias requisições do painel — verificações de versão e downloads do painel/Xray, Telegram e atualização regular de arquivos geo — por este outbound Xray para contornar a filtragem de servidor do GitHub/Telegram. Um inbound de bridge loopback local é adicionado automaticamente à configuração em execução e aplicado em tempo real. A atualização automática de Geodata integrada ao Xray não é afetada; ela tem seu próprio outbound para download. Deixe em branco para conexão direta."

Como funciona. Ao selecionar um outbound, o painel adiciona automaticamente à configuração em execução um inbound loopback de serviço (bridge SOCKS com a tag `panel-egress`) e uma regra de roteamento que direciona o próprio tráfego HTTP do painel para o outbound selecionado. Se um balanceador for selecionado, o `balancerTag` é inserido na regra e o tráfego do painel é distribuído entre seus participantes. O bridge e a regra são aplicados **em tempo real**, sem reinicialização completa do

painel. Deixe o campo vazio para conexão direta. A atualização automática de geo-dados integrada ao Xray **não é afetada** por esta configuração – ela tem seu próprio outbound dentro do roteamento do Xray.

- **Formato:** `socks5://` (ou `socks5h://`) ou `http(s)://`, com autenticação se necessário no formato `socks5://user:pass@host:port`. Os esquemas suportados são estritamente: `socks5`, `socks5h`, `http`, `https` – outros esquemas são considerados inválidos e o painel volta à conexão direta. O exemplo típico é um inbound SOCKS local do próprio Xray.
- Dica literal: "Roteia as requisições de saída do próprio painel (atualizações de geo, verificações de versão do Xray/painel, Telegram) por este proxy para contornar a filtragem de servidor do GitHub/Telegram. Aceita `socks5://` ou `http(s)://`, ex.: inbound SOCKS local do Xray. Deixe em branco para conexão direta."
- Um proxy inválido não gera erro ao salvar – o painel simplesmente usa conexão direta e registra um aviso no log.

Exemplo de valores do campo. Se o servidor já possui um inbound SOCKS local do Xray na porta `10808`, direcione as próprias requisições do painel por ele:

```
socks5://127.0.0.1:10808
```

Para um proxy HTTP externo com autenticação:

```
http://user:pass@proxy.example.com:8080
```

Após salvar e reiniciar, o painel buscará atualizações de geo-bases, verificará versões e acessará o Telegram pelo proxy especificado.

CIDRs de proxy confiáveis (*Trusted proxy CIDRs*)

- **Chave:** `trustedProxyCIDRs`
- **Valor padrão:** `127.0.0.1/32, ::1/128` (apenas host local).
- **Formato:** lista de endereços IP ou sub-redes CIDR separados por vírgula (por exemplo, `10.0.0.0/8, 192.168.1.5`). Cada elemento é validado como IP ou CIDR – um valor incorreto é rejeitado ao salvar.
- **Finalidade:** lista as origens com permissão para definir os cabeçalhos `X-Forwarded-Host`, `X-Forwarded-Proto` e o cabeçalho de IP real do cliente. Dica literal: "IP/CIDR separados por vírgula com permissão para definir os cabeçalhos de host encaminhado, proto e IP do cliente." Deve ser configurado quando o painel opera por trás de um proxy reverso (nginx, Caddy, etc.) para identificar corretamente os IPs dos clientes e o esquema.

Exemplo: painel atrás de um proxy reverso. Se o nginx está no mesmo host e encaminha requisições ao painel, mantenha a confiança apenas ao host local (valor padrão):

```
127.0.0.1/32, ::1/128
```

Se o proxy está em um servidor separado na rede interna `10.0.0.0/8`, adicione sua sub-rede; caso contrário, o painel ignorará os cabeçalhos enviados por ele e verá o IP do proxy em vez do cliente real:

```
127.0.0.1/32,::1/128,10.0.0.0/8
```

Exemplo do bloco nginx correspondente, que encaminha o IP real e o esquema:

```
proxy_set_header X-Real-IP      $remote_addr;
proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
proxy_set_header X-Forwarded-Proto $scheme;
proxy_set_header X-Forwarded-Host $host;
```

13.5. Bot do Telegram (aba "Bot do Telegram" / *Telegram Bot*)

Ativar bot do Telegram (*Enable Telegram Bot*)

- **Chave:** `tgBotEnable`
- **Tipo/padrão:** booleano, `false`.
- **Finalidade:** ativa o funcionamento do bot do Telegram. Dica: "Acesso às funções do painel pelo bot do Telegram".

Token do Telegram (*Telegram Token*)

- **Chave:** `tgBotToken`
- **Padrão:** `""`.
- **Finalidade:** token do bot. Dica: "É necessário obter o token junto ao gerenciador de bots do Telegram @botfather".
- **Característica de segurança:** o token é um valor secreto. Na resposta do painel à leitura das configurações, ele não é retornado (o campo é limpo e apenas o flag "configurado/não configurado" é enviado). Se o campo for deixado em branco ao salvar, o token anteriormente salvo **é mantido** (não é apagado).

Idioma do bot do Telegram (*Telegram Bot Language*)

- **Chave:** `tgLang`
- **Padrão:** `en-US`.
- **Finalidade:** idioma das mensagens do bot (independentemente do idioma da interface web). A lista de idiomas disponíveis coincide com os idiomas do painel.

ID de usuário do administrador do bot (*Admin Chat ID*)

- **Chave:** `tgBotChatId`
- **Padrão:** `""`.
- **Formato:** um ou mais Telegram User IDs numéricos **separados por vírgula**.
- **Finalidade:** destinatários de notificações e administradores com permissão para gerenciar o painel pelo bot. Dica: "Um ou mais User IDs do(s) administrador(es) do bot do Telegram. Para obter o User ID, use @userinfobot ou o comando '/id' no bot."

Frequência de notificações (*Notification Time*)

- **Chave:** `tgRunTime`
- **Padrão:** `@daily` (uma vez por dia).
- **Formato:** string no formato **Crontab** (são suportadas tanto expressões cron padrão quanto abreviações como `@daily`, `@hourly`, `@every 1h`). Dica: "Especifique o intervalo de notificações no formato Crontab". Controla os relatórios periódicos do bot.

Exemplos de valores do campo.

Valor	Quando o bot envia o relatório
<code>@daily</code>	uma vez por dia à meia-noite (padrão)
<code>@hourly</code>	a cada hora
<code>@every 6h</code>	a cada 6 horas
<code>0 9 * * *</code>	diariamente às 09:00
<code>30 8 * * 1</code>	toda segunda-feira às 08:30

O horário é calculado no fuso horário definido na configuração "Fuso horário" (item 13.6).

Proxy SOCKS (*SOCKS Proxy*)

- **Chave:** `tgBotProxy`
- **Padrão:** `""`.
- **Finalidade:** proxy SOCKS5 específico para a conexão do bot ao Telegram. Dica: "Se você precisa de um proxy Socks5 para se conectar ao Telegram, configure seus parâmetros conforme o guia." Aplica-se especificamente ao tráfego do bot (diferente do "Proxy de rede do painel" geral do item 13.4).

Servidor de API do Telegram (*Telegram API Server*)

- **Chave:** `tgBotAPIServer`
- **Padrão:** `""` (usar o servidor padrão `api.telegram.org`).
- **Formato:** URL `http(s)://...`; ao salvar, passa por validação de URL – endereços inválidos são rejeitados. Dica: "Servidor de API do Telegram utilizado. Deixe em branco para usar o servidor padrão." Necessário para um servidor de Telegram Bot API implantado de forma independente.

Notificações do bot (grupo "Notificações" / *Notifications*)

Campo	Chave	Padrão	Descrição
Backup do banco de dados (<i>Database Backup</i>)	<code>tgBotBackup</code>	<code>false</code>	Enviar ao Telegram o arquivo de backup do banco de dados junto com o relatório. Dica: "Enviar notificação com o arquivo de backup do banco de dados".
Notificação de login (<i>Login Notification</i>)	<code>tgBotLoginNotify</code>	<code>true</code>	Notificar quando houver tentativa de login no painel. Dica: "Exibe o nome de usuário, endereço IP e horário quando alguém tenta acessar seu painel."

Campo	Chave	Padrão	Descrição
Antecedência da notificação de expiração (<i>Expiration Date Notification</i>)	<code>expireDiff</code>	<code>0</code>	Com quantos dias de antecedência ao vencimento do cliente enviar a notificação. <code>0</code> – desativado. Permitido <code>>= 0</code> . Dica: "Receber notificação de expiração da sessão antes de atingir o valor limite (valor: dia)".
Limite de tráfego para notificação (<i>Traffic Cap Notification</i>)	<code>trafficDiff</code>	<code>0</code>	Limite de tráfego restante para notificação. Dica: "Receber notificação de esgotamento de tráfego antes de atingir o limite (valor: GB)". Permitido <code>0–100</code> .
Limite de carga de CPU (<i>CPU Load Notification</i>)	<code>tgCpu</code>	<code>80</code>	Notificar os administradores se o uso de CPU ultrapassar o limite (em %). Permitido <code>0–100</code> . Dica: "Notificar os administradores no Telegram se a carga de CPU ultrapassar este limite (valor: %)".

13.6. Data e hora (aba "Data e hora" / *Date and Time*)

Fuso horário (*Time Zone*)

- **Chave:** `timeLocation`
- **Valor padrão:** `Local` (fuso horário do sistema do servidor).
- **Formato:** nome de zona da base IANA tz (por exemplo, `Europe/Moscow`, `UTC`, `Asia/Tehran`).
- **Finalidade:** fuso horário no qual o painel executa as tarefas agendadas (relatórios do bot, redefinição/verificação de tráfego, expiração de prazos). Dica: "As tarefas agendadas são executadas de acordo com o horário neste fuso horário".
- **Validação:** ao salvar, a zona é verificada – uma zona inexistente é rejeitada. Se posteriormente um valor incorreto estiver no banco de dados, o painel em tempo de execução voltará para `Local` e, se este também estiver indisponível, para `UTC`.

13.7. Tráfego externo e comportamento do Xray (aba "Tráfego externo" / *External Traffic*)

Campo	Chave	Padrão	Descrição
Informar tráfego externo (<i>External Traffic Inform</i>)	<code>externalTrafficInformEnable</code>	<code>false</code>	Notificar uma API externa a cada atualização de tráfego. Dica: "Notificar uma API externa a cada atualização de tráfego."
URI de informação de tráfego externo (<i>External Traffic Inform URI</i>)	<code>externalTrafficInformURI</code>	<code>""</code>	URL para o qual o painel envia atualizações de tráfego. Passa por validação de URL ao salvar. Dica: "As atualizações de tráfego são enviadas para este URI".
Reiniciar Xray após desativação automática (<i>Restart Xray After Auto Disable</i>)	<code>restartXrayOnClientDisable</code>	<code>true</code>	Reiniciar o Xray quando um cliente é desativado automaticamente por expiração ou por exceder o limite de tráfego. Dica: "Quando um cliente é desativado automaticamente por expiração do prazo ou limite de tráfego, reiniciar o Xray." O interruptor está na aba "Painel" (General) – veja o item 13.2; aqui é apresentado por completude.

13.8. Outros: template de configuração do Xray e URL de verificação

Template de configuração do Xray (*xrayTemplateConfig*)

- **Chave:** `xrayTemplateConfig`
- **Padrão:** template JSON integrado (embedded), fornecido com a build.
- **Finalidade:** template JSON base da configuração do Xray-core, sobre o qual o painel constrói inbound/outbound. Este valor **não é retornado** na resposta comum de todas as configurações e é editado em uma página de configuração separada do Xray, não na lista geral de campos de configurações do painel. O template padrão está disponível via `GET /panel/setting/getDefaultJsonConfig`.

URL de verificação de saída (*xrayOutboundTestUrl*)

- **Chave:** `xrayOutboundTestUrl`
- **Padrão:** `https://www.google.com/generate_204`
- **Finalidade:** URL utilizado ao verificar a funcionalidade das conexões de saída (outbound). Ao ser definido, passa por sanitização como URL HTTP(S).

13.9. Conta do administrador e tokens de API

Esses parâmetros estão na aba adjacente ("Conta" / *Authentication*) e são detalhados na seção sobre segurança; aqui — um breve resumo das chaves.

- **Alteração de credenciais** (campos "Login atual", "Senha atual", "Novo login", "Nova senha") é salva por uma requisição separada `POST /panel/setting/updateUser`. O login e a senha atuais corretos são necessários; o novo login e a nova senha não devem estar em branco. Mensagens: "Você alterou com sucesso as credenciais do administrador." / "Nome de usuário ou senha incorretos".
- **Autenticação de dois fatores (2FA)** — chaves `twoFactorEnable` (padrão `false`) e o secreto `twoFactorToken`. O token é secreto: com 2FA ativado, deixar o campo em branco ao salvar não apaga o token existente. Na **primeira** ativação do 2FA, o painel invalida as sessões atuais (a "época de login" é incrementada).
- **Tokens de API** são gerenciados por endpoints separados (`/panel/setting/apiTokens...`): listagem, criação (`apiTokens/create`), exclusão, ativação/desativação. O próprio token é exibido **apenas uma vez na criação** e não é armazenado em formato legível: "Copie este token agora. Por razões de segurança, ele não é armazenado em formato legível e não será exibido novamente."

Os detalhes sobre 2FA, senhas, sincronização LDAP e formatos de assinatura (JSON/Clash, fragmentation, noises, mux) estão nas seções específicas correspondentes do manual.

13.10. Alterações de API na versão 3.3.0 (importante para integrações)

Na versão 3.3.0, a estrutura dos caminhos da API do servidor foi alterada. Se você possui integrações externas (scripts, bots, painéis centrais, tarefas de CI) que acessam o painel via HTTP, elas **precisam ser atualizadas**, caso contrário deixarão de funcionar.

⚠ BREAKING CHANGE: os endpoints `/panel/setting/*` e `/panel/xray/*` foram movidos para `/panel/api`

Anteriormente, o gerenciamento de configurações do painel e da configuração do Xray ficava separado, nos caminhos `/panel/setting/*` e `/panel/xray/*`. Agora ambos os conjuntos estão registrados dentro do grupo de API comum `/panel/api`. Os caminhos antigos foram **removidos completamente** – uma requisição a eles retornará 404.

Por que isso foi feito: todo o grupo `/panel/api` passa por uma verificação de acesso unificada, ou seja, esses endpoints agora aceitam o mesmo cabeçalho `Authorization: Bearer <token>` que o restante da API. O token de API representa acesso completo de administrador, e assim toda a superfície da API se tornou uniforme.

O que NÃO mudou: as páginas da interface web (rotas SPA) `/panel/settings` e `/panel/xray` permanecem no lugar – a mudança é apenas nos endpoints de API do servidor.

Tabela de correspondência de caminhos (antigo → novo)

O prefixo para todos os caminhos abaixo – simplesmente `api/` foi adicionado após `/panel/`.

Antes (≤ 3.2.x)	Depois (3.3.0)	Método
<code>/panel/setting/all</code>	<code>/panel/api/setting/all</code>	POST
<code>/panel/setting/defaultSettings</code>	<code>/panel/api/setting/defaultSettings</code>	POST
<code>/panel/setting/update</code>	<code>/panel/api/setting/update</code>	POST
<code>/panel/setting/updateUser</code>	<code>/panel/api/setting/updateUser</code>	POST
<code>/panel/setting/restartPanel</code>	<code>/panel/api/setting/restartPanel</code>	POST
<code>/panel/setting/getDefaultJsonConfig</code>	<code>/panel/api/setting/getDefaultJsonConfig</code>	GET
<code>/panel/setting/apiTokens</code>	<code>/panel/api/setting/apiTokens</code>	GET
<code>/panel/setting/apiTokens/create</code>	<code>/panel/api/setting/apiTokens/create</code>	POST
<code>/panel/setting/apiTokens/delete/:id</code>	<code>/panel/api/setting/apiTokens/delete/:id</code>	POST
<code>/panel/setting/apiTokens/setEnabled/:id</code>	<code>/panel/api/setting/apiTokens/setEnabled/:id</code>	POST
<code>/panel/xray/</code>	<code>/panel/api/xray/</code>	POST
<code>/panel/xray/update</code>	<code>/panel/api/xray/update</code>	POST
<code>/panel/xray/getDefaultJsonConfig</code>	<code>/panel/api/xray/getDefaultJsonConfig</code>	GET
<code>/panel/xray/getXrayResult</code>	<code>/panel/api/xray/getXrayResult</code>	GET
<code>/panel/xray/getOutboundsTraffic</code>	<code>/panel/api/xray/getOutboundsTraffic</code>	GET
<code>/panel/xray/resetOutboundsTraffic</code>	<code>/panel/api/xray/resetOutboundsTraffic</code>	POST

Antes (≤ 3.2.x)	Depois (3.3.0)	Método
<code>/panel/xray/testOutbound</code>	<code>/panel/api/xray/testOutbound</code>	POST
<code>/panel/xray/warp/:action</code>	<code>/panel/api/xray/warp/:action</code>	POST
<code>/panel/xray/nord/:action</code>	<code>/panel/api/xray/nord/:action</code>	POST
<code>/panel/xray/outbound-subs</code> (e <code>/outbound-subs/*</code>)	<code>/panel/api/xray/outbound-subs</code> (e <code>/outbound-subs/*</code>)	GET/POST/ DELETE

Os sub-caminhos, corpos de requisição e formatos de resposta não foram alterados — apenas o **prefixo** mudou.

Como corrigir integrações existentes

1. Encontre em seus scripts/configurações todas as ocorrências de `/panel/setting/` e `/panel/xray/`.
2. Substitua o prefixo: adicione `api/` imediatamente após `/panel/` (por exemplo, `/panel/setting/all` → `/panel/api/setting/all`).
3. Corpos de requisição, parâmetros e formato de resposta não precisam ser alterados — apenas a URL muda.
4. Como as configurações e a configuração do Xray agora estão sob `/panel/api`, elas podem (e devem) ser acessadas com o mesmo token de API `Authorization: Bearer <token>` que `/panel/api/inbounds/*` e demais endpoints. Não se esqueça do CSRF-middleware, que está ativo em todo o grupo `/panel/api`.

Exemplo: leitura de todas as configurações via API. Antes (≤ 3.2.x):

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/setting/all" \
-H "Authorization: Bearer <token>"
```

Agora (3.3.0) — adicionado `api/` após `/panel/`:

```
curl -sk -X POST "https://panel.example.com:2053/MyPath/panel/api/setting/all" \
-H "Authorization: Bearer <token>"
```

Da mesma forma para reinicialização do painel: `POST /panel/api/setting/restartPanel`. O caminho antigo `/panel/setting/restartPanel` agora retornará 404.

API tipada: esquemas e documentação (Swagger / OpenAPI)

Na versão 3.3.0, a especificação OpenAPI passou a ser totalmente tipada. Antes, as respostas tipadas eram descritas por um objeto vazio `{}`; agora os componentes e esquemas (`components.schemas`) são gerados diretamente a partir dos modelos de dados. Com isso:

- O Swagger UI exibe os modelos de dados reais, e não stubs sem conteúdo.
- Geradores externos (`openapi-generator` e similares) podem gerar clientes prontos na linguagem desejada a partir da especificação.
- Cada resposta tipada possui um `$ref` para um modelo específico e exemplos de resposta incluídos.

Onde consultar a documentação da API:

- **Página Swagger integrada.** No menu do painel – item **"Documentação da API"** (rota SPA `/panel/api-docs`). Aqui todos os endpoints estão listados de forma interativa, com descrições, corpos de requisição e exemplos de resposta.
 - **Especificação OpenAPI 3.0 bruta** disponível no endereço `/panel/api/openapi.json`. Esta URL pode ser inserida diretamente no Postman, Insomnia ou `openapi-generator`. A especificação está integrada ao binário na etapa de build; quando o painel opera com um `webBasePath` não padrão, o campo `servers` na especificação é automaticamente reescrito para o caminho base atual, de modo que o botão "Try it out" e os geradores externos apontem para o prefixo correto.
-

14. Bot do Telegram

O painel 3X-UI possui um bot do Telegram integrado, por meio do qual é possível receber notificações sobre o estado do servidor e dos clientes, bem como gerenciar clientes individuais diretamente pelo mensageiro. O bot funciona com a tecnologia de long polling (consulta contínua ao Telegram), portanto não requer um domínio externo nem uma porta aberta — basta ter acesso de saída aos servidores do Telegram.

O bot distingue dois tipos de interlocutores:

- **Administrador** — usuário cujo Telegram User ID está indicado nas configurações do bot (campo «User ID do administrador do bot»). Tem acesso a todas as funções: estatísticas do servidor, backup, gerenciamento de clientes, reinicialização do Xray.
- **Cliente** — qualquer outro usuário cujo Telegram User ID esteja vinculado a um cliente específico de inbound (campo `tgId` do cliente). Vê apenas as informações de suas próprias assinaturas.

Exemplo: vinculação de um cliente ao Telegram. Para que o usuário receba estatísticas de sua assinatura, seu Telegram User ID numérico é registrado no campo `tgId` do cliente. Nas configurações JSON do cliente, fica assim:

```
{
  "email": "ivan",
  "id": "6f1e6b1a-0c3d-4f2a-9b7e-1a2b3c4d5e6f",
  "tgId": "123456789",
  "enable": true,
  "limitIp": 2,
  "totalGB": 53687091200,
  "expiryTime": 0
}
```

Após isso, o usuário com User ID `123456789` poderá solicitar ao bot `/usage ivan` e ver suas estatísticas. O mesmo ID pode ser definido pelo administrador através do botão « Definir usuário do Telegram» no cartão do cliente — não é necessário editar o JSON manualmente.

14.1. Ativação e configuração do bot

Todos os parâmetros do bot são definidos no painel, na seção **Configurações** → **Bot do Telegram**. Após alterar as configurações, basta salvá-las — o painel as aplica imediatamente, sem necessidade de reinicialização. Se o sinalizador de ativação (`tgBotEnable`), o token, o User ID dos administradores ou o endereço do servidor de API forem alterados, o painel interrompe e reinicia automaticamente o bot com os novos parâmetros. A regra anterior de reinicialização do painel após a troca de token não se aplica mais.

Campo (UI)	Chave de configuração	Valor padrão	Descrição
Ativar bot do Telegram	<code>tgBotEnable</code>	<code>false</code>	Chave principal. Dica: «Acesso às funções do painel pelo bot do Telegram». Enquanto desativado, o bot não é iniciado e as tarefas de notificação não são agendadas.

Campo (UI)	Chave de configuração	Valor padrão	Descrição
Token do Telegram	tgBotToken	(vazio)	Token do bot. Dica: «É necessário obter o token do gerenciador de bots do Telegram @botfather». Sem um token válido, o bot não inicia.
Proxy SOCKS	tgBotProxy	(vazio)	Proxy para conexão ao Telegram. Dica: «Se precisar de um proxy Socks5 para se conectar ao Telegram, configure os parâmetros conforme o guia».
Servidor de API do Telegram	tgBotAPIServer	(vazio)	Servidor de API alternativo do Telegram. Dica: «Servidor de API do Telegram utilizado. Deixe em branco para usar o servidor padrão».
User ID do administrador do bot	tgBotChatId	(vazio)	Um ou mais Telegram User IDs de administradores separados por vírgula. Dica: «Para obter o User ID, use @userinfobot ou o comando /id no bot».
Frequência de notificações para administradores do bot	tgRunTime	@daily	Agendamento do relatório periódico no formato crontab. Dica: «Especifique o intervalo de notificações no formato Crontab».
Backup do banco de dados	tgBotBackup	false	Dica: «Enviar notificação com o arquivo de backup do banco de dados». Anexa o backup ao relatório periódico.
Notificação de login	tgBotLoginNotify	true	Dica: «Exibe o nome de usuário, endereço IP e horário quando alguém tenta acessar seu painel».
Limite de carga da CPU para notificação	tgCpu	80	Limite de carga da CPU em porcentagem (validação 0–100). Dica: «Notificar administradores no Telegram se a carga da CPU exceder este limite (valor: %)». Com valor 0, a verificação da CPU é desativada.
Idioma do bot do Telegram	–	–	Idioma no qual o bot formata todas as mensagens.

Obtenção do token pelo @BotFather

1. Abra no Telegram o diálogo com @BotFather.
2. Envie o comando /newbot e siga as instruções (nome do bot e username único terminando em bot).
3. O BotFather fornecerá um token no formato 123456789:AA... . Copie-o para o campo **Token do Telegram**.

Obtenção do User ID do administrador

O User ID é o identificador numérico da conta (não o username). Você pode obtê-lo de duas formas:

- Escrever ao bot @userinfobot.
- Iniciar o bot já configurado e enviar o comando /id – ele retornará o seu ID.

Digite o número obtido no campo **User ID do administrador do bot**. Para designar vários administradores, liste os IDs separados por vírgula (por exemplo, 11111111,22222222). Cada ID é validado como número inteiro; um valor inválido causará erro na inicialização do bot.

Exemplo: valor do campo «User ID do administrador do bot». Um administrador – apenas um número:

123456789

Dois administradores separados por vírgula (espaços são opcionais):

123456789,987654321

Cada valor deve ser um número inteiro. Entradas como `@username` ou `123 456` (com espaço dentro do número) não são aceitas – o bot não iniciará.

Proxy

São suportados os esquemas `socks5://`, `http://` e `https://`. Se o campo de proxy estiver vazio, o bot tenta usar o proxy geral do painel (se configurado e com esquema suportado). Um URL com esquema não suportado ou sintaxe incorreta é ignorado – o bot se conecta diretamente. O proxy é útil quando o acesso direto à API do Telegram a partir do servidor está bloqueado.

Notificações por e-mail (SMTP)

Além do Telegram, os mesmos eventos podem ser recebidos por e-mail. O canal é configurado na seção **Configurações** → **Email**, na aba **SMTP Settings**:

Campo (UI)	Chave de configuração	Valor padrão	Descrição
Enable Email Notifications	<code>smtpEnable</code>	<code>false</code>	Chave principal de notificações por e-mail via SMTP.
SMTP Host	<code>smtpHost</code>	(vazio)	Host do servidor SMTP (por exemplo, <code>smtp.gmail.com</code>).
SMTP Port	<code>smtpPort</code>	<code>587</code>	Porta do servidor SMTP.
SMTP Username	<code>smtpUsername</code>	(vazio)	Nome de usuário para autenticação SMTP. Também usado como endereço do remetente (From).
SMTP Password	<code>smtpPassword</code>	(vazio)	Senha para autenticação SMTP. Armazenada de forma oculta; se a senha já estiver definida, o campo mostra o indicador «configurado», e pode ser deixado em branco para manter a senha atual.
Recipients	<code>smtpTo</code>	(vazio)	Lista de destinatários separados por vírgula (por exemplo, <code>admin@example.com, ops@example.com</code>).
Encryption	<code>smtpEncryptionType</code>	<code>starttls</code>	Tipo de criptografia da conexão: <code>none</code> (sem criptografia), <code>starttls</code> (STARTTLS) ou <code>tls</code> (TLS implícito).

O botão **Send Test Email** envia um e-mail de teste e mostra o resultado por etapas: **Connection** (conexão), **Authentication** (autenticação) e **Send** (envio). Se algo der errado, o diagnóstico indica em qual etapa ocorreu o erro (por exemplo, «Authentication failed – check username and password» ou «Server requires STARTTLS – change encryption type»), facilitando o ajuste dos parâmetros.

Na segunda aba (**Notifications**), são selecionados os eventos sobre os quais serão enviados e-mails – com os mesmos grupos de cards que o Telegram (veja «Barramento de eventos e seleção de notificações» na seção 14.5).

Servidor de API do Telegram

Por padrão, o bot acessa a API oficial do Telegram. No campo **Servidor de API do Telegram**, é possível indicar o endereço de um servidor Bot API próprio (`telegram-bot-api`). O URL é verificado quanto à segurança; um endereço bloqueado ou inválido é descartado e o servidor padrão é utilizado.

14.2. Menu principal e botões

O menu é acionado pelo comando `/start` . Os botões são um teclado inline anexado à mensagem; o conjunto de botões depende de você ser administrador ou cliente.

Menu do administrador

Botão	Ação
 Relatório classificado de uso de tráfego	Lista todos os clientes ordenados por tráfego, com o consumo de cada um; e-mails sem dados são marcados com « Sem resultados».
 Estado do servidor	Resumo do servidor (veja a seção 14.5). O botão « Atualizar» atualiza os dados.
Redefinir todo o tráfego	Zera os contadores de tráfego de todos os clientes. Solicita confirmação («Você tem certeza?»), depois exibe « Sucesso» ou « Falha» para cada cliente e, ao final, « Redefinição de tráfego concluída para todos os clientes».
 Backup do BD	Envia o arquivo do banco de dados e o <code>config.json</code> (veja a seção 14.6).
 Log de banimentos	Envia os arquivos de log de endereços IP banidos por exceder o limite de IP.
 Conexões de entrada	Resumo de todos os inbounds: Remark, porta, tráfego, número de clientes, data de expiração.
 Expirando em breve	Lista de inbounds e clientes cujo tráfego ou prazo está prestes a se esgotar (veja a seção 14.5).
 Comandos	Exibe a ajuda dos comandos do administrador.
 Online	Quantidade e lista de clientes online; clicar no e-mail abre o cartão do cliente. Botão « Atualizar».
 Todos os clientes	Abre a seleção de inbound e, em seguida, a lista de seus clientes – para visualização e gerenciamento.
 Novo cliente	Inicia o assistente de adição de cliente (seleção de inbound → rascunho → confirmação).
Configurações de assinatura / links individuais / QR code	Seleção de inbound e cliente para obter o link de assinatura, links individuais ou QR codes.

Menu do cliente

O cliente tem acesso a um conjunto limitado de botões:

Botão	Ação
Estatísticas do cliente	Exibe os dados de todas as assinaturas vinculadas ao Telegram User ID do cliente.
Comandos	Exibe a ajuda dos comandos do cliente.
Configurações de assinatura	Seleção do próprio cliente → link de assinatura.
Links individuais	Seleção do próprio cliente → links individuais.
QR code	Seleção do próprio cliente → QR codes.

Se o usuário não tiver nenhum cliente com seu Telegram User ID, o bot responde: « Sua configuração não foi encontrada! Por favor, peça ao administrador para usar seu Telegram User ID na configuração. Seu User ID: ...». Esse ID deve ser fornecido ao administrador para que ele o registre no campo do cliente.

14.3. Comandos do bot

A partir da versão 3.5.0, **oito** comandos estão registrados no menu «/» do Telegram:

Comando	Descrição (do menu)	Acesso	O que faz
/start	Mostrar o menu principal	todos	Saudação; para o administrador, exibe adicionalmente «  Bem-vindo ao bot de gerenciamento !» e o menu principal.
/help	Ajuda do bot	todos	Exibe a saudação geral e a sugestão de escolher um item do menu.
/status	Verificar o status do bot	todos	Responde «  O bot está funcionando normalmente».
/id	Mostrar seu Telegram ID	todos	Retorna «  Seu User ID: ... ». Útil para obter o próprio User ID.
/usage	Mostrar o consumo do cliente: /usage email	todos (veja abaixo)	Adicionado ao menu na 3.5.0.
/inbound	Buscar inbound: / inbound remark (admin)	administrador	Adicionado ao menu na 3.5.0.
/restart	Reiniciar o núcleo Xray (admin)	administrador	Adicionado ao menu na 3.5.0.
/clearall	Reiniciar o tráfego de todos os clientes (admin)	administrador	Novo na 3.5.0: solicita confirmação (botões «Cancelar» / «Confirmar redefinição de tráfego») e zera o tráfego de todos os clientes.

Detalhes dos comandos com argumentos:

- **/usage [Email]** — busca de cliente por e-mail.
 - Para o **administrador**, exibe o cartão completo do cliente (com botões de gerenciamento).
 - Para o **cliente**, exibe apenas sua própria assinatura com o e-mail indicado (pela vinculação do Telegram User ID). Sem argumento, o bot solicita o e-mail: « Por favor, informe o e-mail para pesquisa».

- **/inbound [nome da conexão]** – somente para o administrador. Busca o inbound pelo Remark e exibe seus parâmetros com estatísticas de todos os clientes. Sem argumento (ou para um cliente) – «Comando desconhecido».
- **/restart** – somente para o administrador. Reinicia o Xray Core. Respostas possíveis: « Xray Core reiniciado com sucesso», « O Xray Core não está em execução» (se o núcleo não estiver rodando), « Erro ao reiniciar o Xray Core.». Qualquer argumento após **/restart** resulta em mensagem de comando desconhecido com a dica **/restart**.

Em chats em grupo, um comando no formato **/comando@botusername** é processado apenas se o username corresponder ao nome do bot atual.

Mais três mudanças da 3.5.0: na lista de **clientes online**, os botões são rotulados como **email** – observação do **inbound** (nomes iguais em inbound diferentes não se confundem mais); as mensagens de **backup** e do **log de banimentos** começam com a linha **Hostname==...** – útil quando um único bot atende vários painéis; a busca do cartão do cliente por **Telegram ID** funciona independentemente da formatação do JSON das configurações (antes, registros «compactos» vindos de nós ou de importação não eram encontrados).

Ajuda do administrador (botão «Comandos»):

 Para reiniciar o Xray Core: **/restart**
 Para buscar um cliente por e-mail: **/usage [Email]**
 Para buscar conexões de entrada (com estatísticas de clientes): **/inbound [nome da conexão]**
 Seu Telegram User ID: **/id**

Ajuda do cliente:

\$
 Para ver informações sobre sua assinatura: **/usage [Email]**
 Seu Telegram User ID: **/id**

14.4. Gerenciamento de clientes (somente administrador)

Ao abrir o cartão de um cliente (via «Todos os clientes», «Online», «Expirando em breve» ou **/usage**), o administrador vê os dados do cliente (e-mail, inbounds vinculados, status «Ativo», status de conexão, data de expiração, consumo de tráfego) e botões inline de gerenciamento:

Botão	Finalidade
 Atualizar	Recarregar o cartão do cliente.
 Redefinir tráfego	Zerar o contador de tráfego do cliente. Requer confirmação «  Confirmar redefinição de tráfego?».
 Limite de tráfego	Definir o limite de tráfego. Valores pré-definidos:  Ilimitado (0), 1/5/10/20/30/40/50/60/80/100/150/200 GB ou «  Personalizado» – inserção de número  pelo teclado numérico integrado (botões 0–9, «  » – redefinir para 0, «  » – apagar o último dígito, «  Confirmar: N»). O valor é definido em gigabytes.
 Alterar data de expiração	Opções pré-definidas:  Ilimitado, «  Personalizado», adicionar 7/10/14/20 dias, 1/3/6/12 meses. Um número positivo prorroga o prazo (soma dias à data de expiração atual ou a «agora», se o prazo já expirou); 0 remove a restrição de prazo.

Botão	Finalidade
 Log de IP	Exibe os endereços IP registrados do cliente (com marcações de tempo, se houver). No log estão disponíveis « Atualizar » e « Limpar IP » (com confirmação « Confirmar limpeza de IP? »).
 Limite de IP	Limite de IPs simultâneos. Opções: ☹ Ilimitado (0), 1–10 ou « Personalizado » (teclado numérico).
 Definir usuário do Telegram	Exibe o Telegram User ID vinculado ao cliente; permite remover a vinculação (« Remover usuário do Telegram » com confirmação). A vinculação de um novo usuário é realizada pelo seletor de contato do Telegram do sistema.
 Ativar/Desativar	Ativa ou desativa o cliente. Requer confirmação « Confirmar ativação/desativação do usuário? ».

Todas as operações que alteram a configuração (limite de tráfego/IP, data de expiração, vinculação/desvinculação de usuário do Telegram, ativar/desativar) marcam o Xray para reinicialização quando necessário, para que as alterações entrem em vigor. Após uma operação bem-sucedida, o bot exibe uma confirmação no formato «  : ... » e reexibe o cartão.

Qualquer entrada numérica nos assistentes é limitada a valores < 999999.

14.5. Notificações e relatórios

As notificações são enviadas a todos os administradores (todos os User IDs de `tgBotChatId`).

Barramento de eventos e seleção de notificações

As notificações são construídas sobre um barramento de eventos unificado, com dois canais de entrega – **Telegram** e **e-mail (SMTP)**. Para cada canal, é possível selecionar separadamente quais eventos serão notificados. Em **Configurações** → **Telegram**, isso é feito na aba **Notifications**; em **Configurações** → **Email** – na aba de mesmo nome.

Os eventos são agrupados em cards; cada grupo tem um botão mestre com contador de eventos ativados (n/total) e estado intermediário quando apenas alguns estão selecionados. Os grupos disponíveis são:

- **Outbound** – «Down» (`outbound.down`) e «Up» (`outbound.up`): queda e recuperação de outbound.
- **Xray Core** – «Crash» (`xray.crash`): encerramento inesperado do núcleo Xray.
- **Nodes** – «Down» (`node.down`) e «Up» (`node.up`): nó ficou inacessível ou foi restaurado.
- **System** – «CPU high (%)» (`cpu.high`) e «Memory high (%)» (`memory.high`): alta carga do processador e da memória RAM. Ambos os eventos possuem um campo inline de limite em porcentagem.
- **Security** – «Login attempt» (`login.attempt`): tentativa de acesso ao painel.

O conjunto de eventos ativados é armazenado separadamente: para Telegram – em `tgEnabledEvents`, para Email – em `smtpEnabledEvents`. Por padrão, em ambos os canais estão ativados «Login attempt» e «CPU high» (valor `login.attempt,cpu.high`).

Notificação de acesso ao painel

Controlada pela opção **Notificação de login** (`tgBotLoginNotify` , ativada por padrão). A cada tentativa de acesso ao painel web, os administradores recebem uma mensagem:

- Em caso de sucesso: «  Login bem-sucedido no painel.» + host, nome de usuário, IP, horário.
- Em caso de falha: «  Erro de login no painel.» + host, **motivo** (por exemplo, «Erro 2FA» em caso de segundo fator incorreto), nome de usuário, IP, horário.

Sobrecarga de CPU e memória

A cada minuto, o painel verifica a carga do processador e da memória RAM. Se o limite `tgCpu` > 0 e a carga média da CPU por minuto o ultrapassar, os administradores recebem: «  A carga da CPU está em N%, o que excede o limite de M%». Da mesma forma, a carga da RAM é verificada contra o limite `tgMemory` (padrão 80%) – evento «Memory high (%)».

Ambos os limites são definidos nos campos inline ao lado dos eventos «CPU high (%)» e «Memory high (%)» no grupo **System** da aba Notifications (veja «Barramento de eventos e seleção de notificações» abaixo). Para o canal Email, existem chaves separadas `smtpCpu` e `smtpMemory` . Com o valor do limite em 0, a verificação correspondente é desativada.

Relatório periódico (agendado)

É agendado pela expressão cron do campo **Frequência de notificações** (`tgRunTime` , padrão `@daily`). Se o valor estiver vazio ou inválido, é utilizado `@daily` . O relatório inclui:

Construtor de agendamento

O campo **Frequência de notificações para administradores do bot** é definido não por digitação direta, mas por meio de um construtor de agendamento. Primeiro, seleciona-se o modo em um menu suspenso:

- **@every – repetir com intervalo** – aparecem um campo numérico e a seleção de unidade (**Segundos / Minutos / Horas**); o resultado é montado em uma expressão como `@every 6h` .
- **@hourly – a cada hora, @daily – todos os dias às 00:00, @weekly – toda semana, @monthly – todo mês** – presets prontos, salvos como o macro correspondente (`@hourly` , `@daily` , `@weekly` , `@monthly`).
- **Personalizado (crontab)** – campo para expressão crontab própria. O agendador do painel trabalha com segundos habilitados, portanto a expressão personalizada é composta de **6 campos**: segundo, minuto, hora, dia do mês, mês, dia da semana (por exemplo, `0 30 8 * * *` – todos os dias às 08:30:00). Ao mudar para este modo, o campo é preenchido com o equivalente crontab da seleção atual, para servir de ponto de partida.

Exemplo: valores do campo «Frequência de notificações» (`tgRunTime`). São suportadas tanto as abreviações prontas quanto o formato crontab completo:

Valor	Quando dispara
<code>@daily</code>	uma vez por dia à meia-noite (valor padrão)

Valor	Quando dispara
@hourly	a cada hora
@every 6h	a cada 6 horas
0 9 * * *	todos os dias às 09:00
0 9 * * 1	toda segunda-feira às 09:00
0 */12 * * *	a cada 12 horas (às 00:00 e 12:00)

Ordem dos campos no crontab: minuto, hora, dia do mês, mês, dia da semana.

1. A linha « Relatórios agendados: » com a data e hora atuais.
2. **Estado do servidor** (veja abaixo).
3. Bloco «Expirando em breve» por inbounds e clientes.
4. Notificações pessoais para clientes com Telegram User ID vinculado – cada cliente não administrador recebe a lista de suas assinaturas com tráfego ou prazo prestes a se esgotar (considerando as desativadas).
5. Se **Backup do banco de dados** (`tgBotBackup`) estiver ativado – backup do BD para os administradores.

Estado do servidor contém: host, versão do 3X-UI e do Xray, IPv4/IPv6, tempo de atividade (em dias), carga média (Load1/2/3), RAM (atual/total), número de clientes online, contadores de conexões TCP/UDP, tráfego de rede total (↑/↓) e status do Xray.

«Expirando em breve» exibe:

- por inbounds: número de desativados e número de «prestes a se esgotar», seguido da listagem desses inbounds (Remark, porta, tráfego, data de expiração);
- por clientes: o mesmo, mais os cartões dos clientes e botões com seus e-mails (clique abre o cartão do cliente).

Os limites de «prestes a se esgotar» são retirados das configurações gerais do painel: margem de tráfego (em GB) e margem de prazo (em dias). Um inbound/cliente é considerado «a se esgotar» se o tráfego restante até o limite for menor que a margem OU se os dias restantes até a data de expiração forem menores que a margem.

14.6. Backup e logs

- **Backup do BD** (botão « Backup do BD» ou opção no relatório periódico): o bot envia a hora do backup, o arquivo do banco de dados (`x-ui.db` , ou `x-ui.dump` para PostgreSQL) e o arquivo de configuração do Xray `config.json` .

O nome do arquivo de backup enviado pelo bot é formado com base no endereço do servidor: é utilizado o valor de **webDomain** e, se não estiver definido, o IP público do servidor. Isso ajuda a identificar de qual servidor veio o arquivo quando os backups são coletados de vários painéis. Se o endereço não puder ser determinado, é usado um nome genérico.

- **Log de banimentos** (botão « Log de banimentos»): envia os arquivos de log atual e anterior dos endereços IP banidos por exceder o limite de IP. Arquivos vazios não são enviados.

14.7. Particularidades de funcionamento

- **Mensagens longas** são divididas em partes (limite ~2000 caracteres); o teclado inline é anexado à última parte.
 - **Paralelismo**: comandos e cliques em botões são processados de forma concorrente (pool de até 10 manipuladores simultâneos).
 - **Confiabilidade de envio**: em caso de erros de conexão, as mensagens são reenviadas com atraso exponencial (1s/2s/4s, até 3 tentativas).
 - **Cache**: os dados de «Estado do servidor» são armazenados em cache para que cliques frequentes em «Atualizar» não sobrecarreguem o sistema.
 - **Reinicialização do bot**: ao salvar configurações que afetam o bot (sinalizador de ativação, token, User IDs dos administradores ou endereço do servidor de API), o painel interrompe automaticamente o ciclo de polling anterior e inicia um novo com os parâmetros atuais – não é necessário reiniciar o painel para isso. Apenas uma instância de recebimento de atualizações funciona por vez.
-

15. Bases geográficas (geoip / geosite e personalizadas)

As bases geográficas são arquivos binários `.dat` que o Xray-core utiliza para roteamento e filtragem de tráfego com base na localização geográfica (faixas de IP) ou na categoria de domínios. O painel é capaz de baixar e atualizar tanto o conjunto padrão de arquivos geo quanto fontes personalizadas arbitrárias especificadas por URL. Todos os arquivos são armazenados no diretório `bin` ao lado do binário do Xray (caminho padrão `bin`, substituível pela variável de ambiente `XUI_BIN_FOLDER`).

15.1. O que são geoip.dat e geosite.dat

- **geoip.dat** — base de correspondências "endereço IP → código de país/região". É usada em regras de roteamento na forma `geoip:<código>`, por exemplo `geoip:ru`, `geoip:cn`, bem como para marcadores especiais `geoip:private` (redes privadas/locais). Em essência, responde à pergunta "em qual país este IP está localizado".
- **geosite.dat** — base de correspondências "domínio → categoria/lista". É usada na forma `geosite:<categoria>`, por exemplo `geosite:category-ads-all` (domínios de anúncios), `geosite:google`, `geosite:ru`. Em essência, são listas agrupadas de domínios.

Esses arquivos são necessários para construir regras do tipo "todo o tráfego para IPs/domínios russos — diretamente, o restante — pelo outbound" e similares. As próprias regras são definidas na seção de roteamento do Xray; as bases geográficas apenas fornecem dados para elas. Sem arquivos geo atualizados, as regras que referenciam `geoip:` / `geosite:` não funcionarão ou dependerão de listas desatualizadas.

Exemplo: regra "domínios e IPs russos — diretamente". Essa regra na seção de roteamento direciona todo o tráfego para recursos russos para o outbound com a tag `direct`:

```
{
  "type": "field",
  "domain": ["geosite:category-ru"],
  "ip": ["geoip:ru"],
  "outboundTag": "direct"
}
```

15.2. Arquivos geo padrão e sua atualização

O painel contém uma lista de permissões (allowlist) fixa com seis arquivos padrão com fontes de download definidas de forma rígida. A atualização é realizada via

POST `/panel/api/server/updateGeofile/:fileName` (ou sem nome de arquivo — para atualizar todos de uma vez).

Exemplo: atualização de um arquivo e de todos ao mesmo tempo via API. Atualizar apenas `geoip_RU.dat`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile/
geoip_RU.dat' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Atualizar todos os seis arquivos padrão com uma única requisição (sem indicar o nome do arquivo):

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/updateGeofile' \
-H 'Cookie: 3x-ui=<session-cookie>'
```

Resposta de sucesso:

```
{ "success": true, "msg": "Geofile updated successfully", "obj": null }
```

Nome do arquivo	Fonte (repositório de releases)
geoup.dat	github.com/Loyalsoldier/v2ray-rules-dat (geoup.dat)
geosite.dat	github.com/Loyalsoldier/v2ray-rules-dat (geosite.dat)
geoup_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geoup.dat)
geosite_IR.dat	github.com/chocolate4u/Iran-v2ray-rules (geosite.dat)
geoup_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geoup.dat)
geosite_RU.dat	github.com/runetfreedom/russia-v2ray-rules-dat (geosite.dat)

Particularidades da atualização dos arquivos padrão:

- **Botão de atualização de um único arquivo.** Antes do download, é exibida uma confirmação: "Você realmente deseja atualizar o arquivo geo?" com a explicação "Isso atualizará o arquivo #filename#." (em inglês *Do you really want to update the geofile? This will update the #filename# file.*). Em caso de sucesso, aparece a notificação "Arquivos geo atualizados com sucesso" (em inglês *Geofile updated successfully*).
- **Botão "Atualizar todos"** (em inglês *Update all*) baixa todos os seis arquivos. Confirmação: "Isso atualizará todos os arquivos geo." (em inglês *This will update all geofiles.*).
- **Download condicional.** Se o arquivo local já existir, o cabeçalho `If-Modified-Since` com o horário de modificação do arquivo é adicionado à requisição. A resposta `304 Not Modified` do servidor significa que o arquivo não foi alterado – ele não é baixado novamente, apenas o carimbo de tempo do arquivo é atualizado.
- **Segurança do nome do arquivo.** Somente nomes da allowlist são aceitos; o nome é verificado quanto à ausência de `..`, separadores de caminho `/` e `\`, caminhos absolutos, e deve corresponder ao padrão `^[a-zA-Z0-9._-]+\..dat$`. Qualquer nome fora da lista é rejeitado com o erro "Invalid geofile name".
- **Reinicialização do Xray.** Após o download dos arquivos geo, o Xray-core é reiniciado para que ele releia as bases atualizadas. Se a reinicialização falhar, a mensagem de erro incluirá uma linha correspondente.

Atualização das bases geo pela linha de comando (x-ui)

As bases geo também podem ser atualizadas sem o painel – pelo menu interativo `x-ui` (item de atualização de arquivos geo) ou pelo comando não interativo `x-ui update-all-geofiles`. Para cada arquivo do conjunto (geoup/geosite, incluindo os conjuntos IR e RU) é exibido um status separado: "atualizado", "já está atualizado" ou "erro de download". Em caso de falha no download, nenhuma mensagem de sucesso falsa é exibida. A reinicialização do Xray (e consequentemente a interrupção das conexões ativas) ocorre somente se pelo menos um arquivo foi de fato atualizado; se nenhum arquivo foi alterado (todos retornaram `304 Not Modified`), o painel e o Xray não são reiniciados.

15.3. Atualização automática de geo-dados pelo Xray (Geodata Auto-Update)

Fontes `.dat` adicionais por URL arbitrário não são adicionadas pelas ferramentas do painel, mas sim pela seção nativa `geodata` do Xray-core. A seção correspondente está disponível na janela modal de atualizações do Xray (Dashboard → atualizações do Xray, `xrayUpdates`) – esta é a aba "Atualização Automática de Geodata" (em inglês *Geodata Auto-Update*). O painel aqui apenas edita a chave `geodata` no template de configuração do Xray; o download, a verificação e o recarregamento a quente dos arquivos são realizados pelo próprio núcleo do Xray.

Na parte superior da seção é exibida uma dica: "O Xray baixa esses arquivos de acordo com o agendamento e os recarrega sem reinicialização. As URLs devem ser HTTPS. O arquivo já deve existir na pasta bin antes que o Xray possa atualizá-lo." (em inglês *Xray downloads these files on schedule and hot-reloads them without a restart. URLs must be HTTPS. Each file must already exist in the bin folder once before Xray can update it.*).

Campos da seção

- **Agendamento (cron)** (em inglês *Schedule (cron)*) – string cron com 5 campos; valor padrão `0 4 * * *` (diariamente às 04:00). Ao salvar, é verificado que a string contém exatamente 5 campos, caso contrário é exibido o erro "O cron deve conter 5 campos, ex.: `0 4 * * *`".
- **Baixar via outbound (opcional)** (em inglês *Download through outbound (optional)*) – lista suspensa com as tags dos outbounds disponíveis (incluindo outbounds de inscrições), pelo qual o Xray baixará os arquivos; outbounds com protocolo `blackhole` não aparecem na lista. O campo pode ser deixado vazio – nesse caso é usada a conexão direta. Esta seleção é independente do outbound para as próprias requisições do painel (ver §11): a atualização automática do geodata tem seu próprio outbound separado para download.
- **Lista de arquivos** – cada linha define um par "URL + Nome do arquivo" (em inglês *File name*). A URL deve começar com `https://` (caso contrário "Uma URL HTTPS é necessária para cada arquivo."). O nome do arquivo é informado de forma simples, sem caminhos ou separadores – apenas caracteres `^[A-Za-z0-9._-]+$` (caso contrário "O nome do arquivo deve ser simples, ex.: `geosite_custom.dat` (sem caminhos)"). Ao inserir a URL, o painel tenta preencher o nome do arquivo automaticamente a partir do último segmento do caminho. O botão "Adicionar arquivo" (em inglês *Add file*) adiciona uma linha, o botão de lixeira a remove.

Se a lista estiver vazia, é exibida a dica: "Nenhum arquivo configurado. Referencie os arquivos nas regras de roteamento como `ext:geosite_custom.dat:categoria`." (em inglês *No files configured. Reference files in routing rules as `ext:geosite_custom.dat:category`.*).

Salvamento

O botão "Salvar e reiniciar o Xray" (em inglês *Save & Restart Xray*) exibe a confirmação "Salvar configurações de geodata?" com a explicação "O template de configuração do Xray será atualizado e o Xray será reiniciado." (em inglês *Save geodata settings? This updates the Xray config template and restarts Xray*). Após salvar, a chave `geodata` é gravada no template de configuração (`POST /panel/api/xray/update`) e o Xray é reiniciado (`POST /panel/api/server/restartXrayService`). Se a lista de arquivos estiver vazia, a chave `geodata` é removida do template.

Particularidades importantes:

- **O arquivo já deve existir em `bin`**. O Xray atualiza apenas os arquivos `.dat` que já estão presentes na pasta `bin` no momento da inicialização. Portanto, um novo arquivo personalizado deve primeiro ser colocado em `bin` manualmente (ou pelo menos uma versão vazia/desatualizada deve ser criada com o nome correto), e só então o Xray começará a mantê-lo atualizado de acordo com o agendamento.
- **Recarregamento a quente**. Após o download programado, o Xray relê as bases atualizadas sem reinicialização completa do processo.
- **Compatibilidade**. Os arquivos geo baixados anteriormente (tanto os padrão quanto os personalizados) continuam funcionando nas regras de roteamento com a sintaxe `ext:` sem alterações.

Se a lista estiver vazia, é exibida a dica: "Nenhuma fonte geo personalizada ainda – clique em 'Adicionar' para criar uma" (em inglês *No custom geo sources yet – click Add to create one*).

Colunas da tabela e campos da fonte

Campo (UI)	JSON	Valor padrão	Descrição
Tipo (<i>Type</i>)	<code>type</code>	– (obrigatório)	Tipo do recurso: apenas <code>geosite</code> ou <code>geoip</code> . Determina o nome do arquivo resultante.
Apelido (<i>Alias</i>)	<code>alias</code>	– (obrigatório)	Identificador curto da fonte. O nome do arquivo é construído a partir dele e do tipo.
URL (<i>URL</i>)	<code>url</code>	– (obrigatório)	Link direto para o arquivo <code>.dat</code> (http/https).
Ativado (<i>Enabled</i>)	–	–	Indicador de atividade da fonte na lista.
Atualizado (<i>Last updated</i>)	<code>lastUpdatedAt</code>	<code>0</code>	Horário da última atualização bem-sucedida (tempo Unix; <code>0</code> – ainda não atualizado).
Roteamento (ext:...) (<i>Routing (ext:...)</i>)	–	–	String pronta para regras de roteamento: <code>ext:<arquivo.dat>:tag</code> .
Ações (<i>Actions</i>)	–	–	Botões "Editar", "Excluir", "Atualizar agora".

Adicionalmente, o banco de dados armazena campos de serviço: `localPath` (caminho real do arquivo no diretório `bin`), `lastModified` (valor do cabeçalho `Last-Modified` retornado pelo servidor, usado para download condicional), `createdAt` e `updatedAt`.

Nomenclatura dos arquivos

O nome do arquivo resultante é formado automaticamente a partir do tipo e do apelido:

- tipo `geoip` → `geoip_<alias>.dat`;
- tipo `geosite` → `geosite_<alias>.dat`.

Por exemplo, uma fonte com tipo `geosite` e apelido `myads` criará o arquivo `geosite_myads.dat`.

Exemplo: adição de uma fonte via API. Adicionar uma lista personalizada de domínios de anúncios como recurso `geosite` com o apelido `myads`:

```
curl -X POST 'https://panel.example.com:2053/panel/api/server/customGeo/add' \
-H 'Cookie: 3x-ui=<session-cookie>' \
-H 'Content-Type: application/json' \
-d '{
  "type": "geosite",
  "alias": "myads",
  "url": "https://example.com/lists/myads.dat"
}'
```

O painel baixará o arquivo no diretório `bin` como `geosite_myads.dat`, salvará o registro e reiniciará o Xray.

Botões e ações

- **Adicionar** (em inglês *Add*) – abre o formulário "Adicionar fonte" (em inglês *Add custom geo*). O botão de salvamento é "Salvar" (em inglês *Save*). API: `POST /add`.
- **Editar** (em inglês *Edit*) – formulário "Editar fonte" (em inglês *Edit custom geo*). API: `POST /update/:id`. Ao alterar o tipo ou o apelido, o arquivo antigo é excluído e o novo é baixado novamente.
- **Excluir** (em inglês *Delete*) – confirmação "Excluir esta fonte personalizada?" (em inglês *Delete this custom geo source?*). Remove o registro do banco de dados e o próprio arquivo `.dat`. API: `POST /delete/:id`. Em caso de sucesso: "Arquivo geo personalizado " excluído".
- **Atualizar agora** (em inglês *Update now*) – baixa novamente a fonte específica e atualiza o carimbo de tempo. API: `POST /download/:id`. Em caso de sucesso: "Arquivo geo " atualizado".
- **Atualizar todos** – atualiza todas as fontes personalizadas de uma vez. API: `POST /update-all`. Em caso de sucesso total: "Todas as fontes geo personalizadas foram atualizadas" (em inglês *All custom geo sources updated*). Se pelo menos uma fonte não for atualizada, a operação é considerada parcialmente malsucedida com a mensagem "Falha ao atualizar uma ou mais fontes geo personalizadas" (em inglês *One or more custom geo sources failed to update*), e a resposta lista as fontes bem-sucedidas e as com falha.

Após qualquer uma das ações (adição, edição, exclusão, atualização, atualização de todos com pelo menos um sucesso) o Xray-core é reiniciado.

Passo a passo: adição de uma fonte

1. Clique em "Adicionar".
2. No campo "Tipo", selecione `geosite` ou `geoip`.
3. No campo "Apelido", insira um identificador (apenas letras latinas minúsculas, dígitos, `-` e `_`; dica de placeholder: `a-z 0-9 _ -`).
4. No campo "URL", informe o link direto para o arquivo `.dat` (deve começar com `http://` ou `https://`).
5. Clique em "Salvar". O painel baixará imediatamente o arquivo no diretório `bin`, salvará o registro e reiniciará o Xray.

15.4. Validação e restrições

Ao criar e editar uma fonte, verificações rigorosas são realizadas. Mensagens de erro:

Condição	Mensagem (PT)	Mensagem (EN)
Tipo diferente de <code>geosite</code> / <code>geoip</code>	O tipo deve ser geosite ou geoip	<i>Type must be geosite or geoip</i>

Condição	Mensagem (PT)	Mensagem (EN)
Apelido vazio	Informe o apelido	<i>Alias is required</i>
Caracteres inválidos no apelido (não corresponde a <code>^[a-z0-9_-]+\$</code>)	O apelido contém caracteres inválidos	<i>Alias must match allowed characters</i>
Apelido reservado	Este apelido está reservado	<i>This alias is reserved</i>
URL vazia	Informe a URL	<i>URL is required</i>
URL inválida (não pode ser analisada)	URL inválida	<i>URL is invalid</i>
Esquema diferente de http/https	A URL deve usar http ou https	<i>URL must use http or https</i>
Host vazio/inválido ou bloqueado pela proteção SSRF	Host da URL inválido	<i>URL host is invalid</i>
Duplicata "tipo + apelido"	Este apelido já está em uso para este tipo	<i>This alias is already used for this type</i>
Fonte não encontrada	Fonte não encontrada	<i>Custom geo source not found</i>
Erro de download	Falha no download	<i>Download failed</i>

Dicas no formulário (validação no cliente): "Apelido: apenas a-z, dígitos, - e _" (*Alias may only contain lowercase letters, digits, - and _*) e "A URL deve começar com http:// ou https://" (*URL must start with http:// or https://*).

Restrições técnicas adicionais:

- **Apelidos reservados.** Não é possível usar apelidos que conflitem com os arquivos padrão. São reservados (comparação sem distinção de maiúsculas/minúsculas, hífen equiparado a sublinhado): `geoip`, `geosite`, `geoip_ir`, `geosite_ir`, `geoip_ru`, `geosite_ru`. Por exemplo, `geosite-ru` será rejeitado como `geosite_ru`.
- **Proteção SSRF.** O host da URL é resolvido para um IP, e se ele apontar para um endereço privado/interno, o download é bloqueado (o usuário verá "Host da URL inválido"). Isso impede o uso do painel para acessar serviços internos.
- **Proteção contra path traversal.** O caminho final do arquivo deve estar dentro do diretório `bin` (com resolução de links simbólicos); qualquer tentativa de sair desse diretório é rejeitada.
- **Tamanho mínimo do arquivo.** O arquivo baixado é considerado válido apenas se tiver no mínimo 64 bytes; um arquivo muito pequeno é rejeitado com erro de download.
- **Proxy e download condicional.** Se um proxy estiver configurado nas definições do painel, o download é feito por meio dele; caso contrário, é usada a conexão direta com transporte seguro contra SSRF. Assim como para os arquivos padrão, é aplicado `If-Modified-Since / 304 Not Modified` (um arquivo não alterado não é baixado novamente). O timeout de download é de 10 minutos; a verificação de disponibilidade da URL (HEAD, e se necessário – GET parcial) é de 12 segundos.

15.5. Verificação automática na inicialização do painel

Ao iniciar, o painel percorre todas as fontes personalizadas e verifica a existência e integridade do arquivo local para cada uma (arquivo ausente, sendo um diretório, ou menor que 64 bytes). Se o arquivo estiver ausente ou corrompido, a fonte é testada e uma tentativa de novo download é realizada. Isso garante que, após reinstalação ou perda do diretório `bin`, os arquivos geo personalizados sejam restaurados automaticamente.

15.6. Uso das bases geográficas nas regras de roteamento

Nas regras de roteamento do Xray, as bases geográficas são usadas em campos como `domain` / `ip` por meio de prefixos:

- **geoip**: para bases de IP — `geoip:<código>`. Exemplos: `geoip:ru`, `geoip:cn`, `geoip:private`. Obtido de `geoip.dat` (ou `geoip_RU.dat` etc., se a regra aponta para um arquivo específico).
- **geosite**: para bases de domínios — `geosite:<categoria>`. Exemplos: `geosite:category-ads-all`, `geosite:google`, `geosite:ru`. Obtido de `geosite.dat`.

Exemplo: bloqueio de anúncios via geosite. Regra que envia todos os domínios de anúncios para um "buraco negro" (pressupõe-se um outbound com a tag `blocked` e protocolo `blackhole`):

```
{
  "type": "field",
  "domain": ["geosite:category-ads-all"],
  "outboundTag": "blocked"
}
```

Para arquivos **personalizados**, é usada a sintaxe de arquivo externo `ext:`. A dica na UI diz: "Nas regras de roteamento, use o valor como `ext:arquivo.dat:tag` (substitua tag)." (em inglês *In routing rules use the value column as `ext:file.dat:tag` (replace tag)*). Formato:

```
ext:<nome_do_arquivo.dat>:<tag>
```

onde `<nome_do_arquivo.dat>` é `geoip_<alias>.dat` ou `geosite_<alias>.dat`, e `<tag>` é a lista/categoria específica dentro do arquivo. O painel na coluna "Roteamento (ext:...)" sugere um template pronto no formato `ext:geosite_myads.dat:tag` — basta substituir `tag` pela tag desejada. O nome desse arquivo é definido na seção "Atualização Automática de Geodata" (ver §15.3) no campo "Nome do arquivo" — por exemplo `geosite_custom.dat`; é referenciado nas regras como `ext:geosite_custom.dat:category`.

Exemplo: regra baseada em arquivo personalizado. Se uma fonte do tipo `geosite` com apelido `myads` foi adicionada, e a lista dentro do arquivo `.dat` está marcada com a tag `ads`, a regra de roteamento fica assim:

```
{
  "type": "field",
  "domain": ["ext:geosite_myads.dat:ads"],
  "outboundTag": "blocked"
}
```

Para uma fonte de IP (tipo `geoip`, apelido `mycorp`, tag `office`), o campo será `"ip": ["ext:geoip_mycorp.dat:office"]`.

16. Operação: backups, logs, atualização, CLI

Esta seção aborda a manutenção cotidiana do painel: criação e restauração de backups do banco de dados, visualização dos logs (registros) do painel e do Xray, reinicialização e parada de serviços, atualização do Xray e do próprio painel, tarefas periódicas (cron) e remoção do painel. Parte das operações é realizada pela interface web (abas na página «Dashboard» e «Configurações do painel»), parte — pelo menu de console `x-ui` no servidor.

16.1. Backup e restauração do banco de dados

Todos os dados do painel (inbound, clientes, grupos, nós, configurações) são armazenados em um único banco de dados. O gerenciamento de backups está disponível na página «**Dashboard**» na aba «**Backup**», com o título do bloco «**Backup e restauração**».

O painel suporta dois mecanismos de banco de dados, e o comportamento do backup depende disso:

- **SQLite** (padrão) — os dados ficam no arquivo `x-ui.db`.
- **PostgreSQL** — se o painel estiver configurado para PostgreSQL, o bloco exibe uma dica:

«Este painel está rodando em PostgreSQL. "Backup" baixa um arquivo `pg_dump` (.dump), e "Restauração" o carrega de volta via `pg_restore`. A restauração também aceita um banco SQLite (.db) ou um dump de migração SQLite e importa seus dados no PostgreSQL. As ferramentas de cliente PostgreSQL (`pg_dump` e `pg_restore`) devem estar instaladas no servidor.»

Exportação (criação de cópia)

O botão «**Exportar banco de dados**» (`Back Up`) baixa o arquivo de backup para o seu dispositivo.

Mecanismo do BD	Nome do arquivo	O que acontece no servidor
SQLite	<code>{host}_AAAA-MM-DD_HHMMSS.db</code>	Primeiro é executado um checkpoint WAL para que o arquivo contenha os registros mais recentes, em seguida o arquivo inteiro é lido e disponibilizado para download
PostgreSQL	<code>{host}_AAAA-MM-DD_HHMMSS.dump</code>	O <code>pg_dump</code> é executado e o arquivo é disponibilizado para download

A partir da versão 3.4.2, o nome do arquivo de backup baixado contém o endereço pelo qual você abriu o painel e a **data-hora**: `{host}_AAAA-MM-DD_HHMMSS` com a extensão `.db` (SQLite) ou `.dump` (PostgreSQL) — por exemplo `panel.example.com_2026-06-27_000000.db`. Assim os arquivos são agrupados por servidor e ordenados por tempo, e várias cópias do mesmo dia não se sobreescrevem. O mesmo nome é usado nos backups enviados pelo bot do Telegram (`host` vem do domínio/IP do painel, com `x-ui` como alternativa).

Dicas na interface:

- **SQLite**: «Clique para baixar o arquivo `.db` contendo o backup do seu banco de dados atual para o seu dispositivo.»

- PostgreSQL: «Clique para baixar o dump PostgreSQL (.dump) do banco de dados atual para o seu dispositivo.»

Tecnicamente, a exportação é uma requisição `GET /panel/api/server/getDb`. O nome do anexo é formado pelo servidor (`Content-Disposition`) de acordo com o mecanismo utilizado.

O nome do arquivo de backup é formado com base no endereço do servidor, e não com o nome fixo `x-ui.db` / `x-ui.dump`. Ao baixar pelo navegador, ele é obtido a partir do endereço do painel na barra de endereços (host da requisição), caso contrário – do domínio web configurado, e na ausência deste – do IP público do servidor (primeiro IPv4, depois IPv6), com fallback para `x-ui`. Assim, backups de diferentes servidores são facilmente distinguíveis. A extensão permanece `.db` para SQLite e `.dump` para PostgreSQL; backups via Telegram são nomeados pelo mesmo domínio/IP.

Exemplo: baixar o backup via API. A mesma exportação pode ser obtida por uma requisição no console – por exemplo, para um script de backup automático. É necessária uma sessão autenticada (cookie de login):

```
# 1) Fazemos login e salvamos o cookie de sessão
curl -s -c cookies.txt \
      -d 'username=admin&password=admin' \
      https://panel.example.com:2053/panel/login

# 2) Baixamos o arquivo do banco (o nome é definido pelo servidor: x-ui.db ou x-ui.dump)
curl -s -b cookies.txt -OJ \
      https://panel.example.com:2053/panel/api/server/getDb
```

Se o painel estiver aberto com um caminho base (Web Base Path), ele deve ser adicionado à URL: `...:2053/<base_path>/panel/api/server/getDb`.

Importação (restauração)

O botão «**Importar banco de dados**» (`Restore`) abre a seleção de arquivo e o envia ao servidor para restauração (`POST /panel/api/server/importDB`, campo do formulário `db`).

Dicas na interface:

- SQLite: «Clique para selecionar e carregar um arquivo .db ou um dump de migração (.dump) do seu dispositivo para restaurar o banco de dados.»
- PostgreSQL: «Clique para selecionar e carregar um backup PostgreSQL (.dump), um banco SQLite (.db) ou um dump de migração SQLite para restaurar o banco de dados. Isso substituirá todos os dados atuais.»

A partir da 3.5.0, o tipo do arquivo é detectado pelo conteúdo (`PGDMP` – arquivo `pg_dump`; cabeçalho «SQLite format 3» – banco `.db`; `PRAGMA / BEGIN TRANSACTION` iniciais – dump SQL em texto), e não pela extensão; o diálogo de seleção aceita `.dump`, `.db` em ambos os SGBDs. Arquivos SQLite enviados a um painel PostgreSQL são importados no PostgreSQL em uma única transação (veja 3.14); antes da importação, o arquivo é verificado como um BD de painel genuíno, e esquemas antigos são migrados automaticamente. Antes de restaurar um arquivo `pg_dump`, o painel verifica antecipadamente (antes de parar o Xray) sua legibilidade: se o dump foi criado por um PostgreSQL mais novo, é exibido um erro com o comando exato – por exemplo, «...run 'x-ui pgclient 17'...».

Processo de importação para SQLite (importante entender que é atômico e com rollback):

1. O arquivo carregado é verificado quanto ao formato – deve ser um banco SQLite válido; caso contrário, é retornado o erro «Invalid db file format».
2. O arquivo é salvo como `x-ui.db.temp` temporário e passa por uma verificação de integridade.
3. **O Xray é parado** antes da substituição do BD.
4. O banco atual é renomeado para o backup `x-ui.db.backup` (fallback).
5. O arquivo temporário é movido para o lugar do banco ativo, é realizada a inicialização e as migrações de esquema, depois a migração de inbound.
6. **Se alguma etapa falhar** – é executado o rollback: o banco anterior é restaurado a partir de `x-ui.db.backup`, e o Xray é reiniciado com os dados antigos.
7. Em caso de sucesso, o arquivo de fallback é excluído e **o Xray é reiniciado automaticamente** com os dados restaurados.

Mensagens da interface pelo resultado:

Resultado	Texto
Sucesso	«Banco de dados importado com sucesso»
Erro na importação	«Ocorreu um erro ao importar o banco de dados»
Erro ao ler o arquivo	«Ocorreu um erro ao ler o banco de dados»

A restauração substitui completamente os dados atuais. Como o Xray para brevemente durante o processo, as conexões existentes dos clientes são interrompidas durante a importação.

Arquivo de migração entre mecanismos (SQLite ⇌ PostgreSQL)

Separado do backup comum, existe a função **«Baixar arquivo de migração»** (Download Migration, requisição `GET /panel/api/server/getMigration`). Ela gera um arquivo portátil para migração para outro mecanismo de BD:

A partir da 3.5.0, o botão é exibido **apenas em painéis PostgreSQL** (no SQLite, um backup comum `.db` agora é restaurado diretamente no PostgreSQL – um export separado não é necessário):

Mecanismo atual	O que é baixado	Nome do arquivo	Finalidade
PostgreSQL	Banco SQLite construído a partir dos dados do PostgreSQL	<code>x-ui.db</code>	Reverter o painel para SQLite

Dica: «Clique para baixar o banco de dados SQLite (.db), construído a partir dos seus dados PostgreSQL e pronto para rodar o painel em SQLite.»

A conversão `.db ⇌ .dump` para SQLite também pode ser feita via CLI com o comando `x-ui migrateDB [file]` (veja a seção 16.7). Além disso, a partir da 3.5.0 a migração entre mecanismos tornou-se **transacional e sem perdas** (são transferidos inclusive os grupos de clientes e os contadores globais de tráfego; uma importação malsucedida não altera o BD de destino), e para atualizar as ferramentas de cliente do PostgreSQL

surgiram o comando `x-ui pgclient [versão]` e o item **10. Install/Upgrade client tools (pg_dump/pg_restore)** no menu PostgreSQL (o repositório oficial do PostgreSQL é adicionado se necessário).

Backup via bot do Telegram

Se um bot do Telegram estiver configurado (veja a seção sobre notificações), ele pode enviar o backup diretamente para o chat do administrador. O backup via Telegram inclui **dois arquivos**: o próprio banco de dados (`x-ui.db`, ou `x-ui.dump` no PostgreSQL) e a configuração do Xray `config.json`. A mensagem é precedida pela linha « Hora do backup: ...».

Há duas formas de obter o backup no Telegram:

1. **Sob demanda.** O botão « **Backup do BD**» no menu do bot – o bot envia imediatamente os arquivos no chat atual.
2. **Automaticamente com o relatório.** Nas configurações do bot há um interruptor «**Backup do banco de dados**» (Database Backup) com a descrição «Enviar notificação com o arquivo de backup do banco de dados». Quando ativado, a cada envio periódico do relatório, o bot envia o backup a todos os administradores após o relatório. O período de envio do relatório é definido pelo agendamento cron do bot (veja a seção 16.6). Entre os arquivos e entre os administradores, o bot faz pausas para não exceder os limites do Telegram.

O backup via bot é enviado apenas se o bot estiver ativo; no PostgreSQL, também requer que `pg_dump` esteja instalado no servidor.

16.2. Visualização de logs

O painel tem dois visualizadores de logs independentes, ambos acessados pela aba «**Logs**» no «Dashboard». Cada janela pode ser atualizada (ícone «atualizar» no cabeçalho) e permite baixar o conteúdo exibido para o arquivo `x-ui.log` (botão com ícone de download).

Logs do painel (aplicação / syslog)

Janela de logs do painel (`POST /panel/api/server/logs/{count}`). Controles:

Elemento	Valor padrão	Descrição
Número de linhas	20	Lista suspensa: 20 / 50 / 100 / 500 / 1000
Nível	Info	Nível mínimo: Debug / Info / Notice / Warning / Error
SysLog (checkbox)	desativado	De onde obter os logs: do buffer da aplicação ou do journal do sistema
Atualização automática (checkbox)	desativado	Reler o log a cada 5 segundos (veja abaixo)

O comportamento depende do checkbox **SysLog**:

- **Desativado (padrão):** os logs são obtidos do buffer circular interno do painel, filtrados pelo nível selecionado. Os registros são exibidos com nível (DEBUG / INFO / NOTICE / WARNING / ERROR) e fonte: **X-UI:** – mensagens do próprio painel, **XRAY:** – mensagens encaminhadas do Xray.

Notificações simples sem carimbo de data/hora e nível (por exemplo, a mensagem de sistema «Syslog is not supported» no Windows) agora são exibidas integralmente, como estão. O formato `YYYY/MM/DD LEVEL – corpo` é reconhecido estritamente; todo o restante é exibido sem análise, portanto essas linhas não são mais truncadas (antes, as três primeiras palavras eram tratadas incorretamente como data/hora/nível).

- **Ativado:** o painel executa no servidor `journalctl -u x-ui --no-pager -n <count> -p <level>`, ou seja, exibe o journal do sistema do serviço `x-ui`. O número de linhas permitido é de 1 a 10000; o nível aceita valores syslog (`emerg/0`, `alert/1`, `crit/2`, `err/3`, `warning/4`, `notice/5`, `info/6`, `debug/7`). No Windows, o modo SysLog não é suportado – será exibido um aviso para desmarcar o checkbox e usar os logs da aplicação. Se `systemd` /serviço não estiver disponível, aparecerá uma mensagem de erro ao iniciar o `journalctl`.

Exemplo: o mesmo journal no console do servidor. Quando o painel está inacessível (por exemplo, não inicia), o journal do sistema pode ser lido diretamente – é exatamente o comando que o painel executa no modo SysLog:

```
# últimas 100 linhas de nível warning e acima
journalctl -u x-ui --no-pager -n 100 -p warning

# acompanhar o journal em tempo real
journalctl -u x-ui -f
```

O nível nesta janela filtra a **saída**. O nível mínimo que é efetivamente gravado no console/syslog é determinado pelo nível de logging do painel (variável de ambiente, padrão `Info` ; no arquivo, o painel sempre grava no nível `DEBUG`).

Logs de acesso do Xray (journal de acesso)

Janela separada para o access-log do Xray (`POST /panel/api/server/xraylogs/{count}`). Ela analisa as linhas do journal de acesso do Xray e as exibe em tabela: **Date, From, To, Inbound, Outbound, Email**.

A partir da versão 3.4.1, esta janela e o botão de abertura no cartão de status do Xray são denominados **«Logs de acesso»** (`Access Logs`) – antes, eram chamados simplesmente de «Logs». A renomeação foi feita para não confundir o visualizador de access-log do Xray com o visualizador de logs do próprio painel, que antes tinha o mesmo nome.

Elemento	Valor padrão	Descrição
Número de linhas	20	20 / 50 / 100 / 500 / 1000

Elemento	Valor padrão	Descrição
Filtro	vazio	Busca textual por substring (aplicada ao pressionar Enter)
Atualização automática (checkbox)	desativado	Rer o log a cada 5 segundos (veja abaixo)
Direct (checkbox)	ativado	Exibir conexões diretas (tráfego pelo outbound freedom)
Blocked (checkbox)	ativado	Exibir conexões bloqueadas (tráfego para o outbound blackhole)
Proxy (checkbox)	ativado	Exibir tráfego proxiado

O tipo de evento é determinado automaticamente pela tag da conexão de saída na linha de log: correspondência com tags freedom → «DIRECT» (verde), blackhole → «BLOCKED» (vermelho), todo o restante → «PROXY» (azul). Linhas `api -> api` e linhas vazias são ignoradas.

Atualização automática. Em ambas as janelas de logs («Logs» e «Logs de acesso»), há um checkbox «**Atualização automática**» (Auto Update). Se ativado, o conteúdo do log é relido automaticamente a cada 5 segundos, mantendo todas as configurações atuais da janela – número de linhas, nível/filtro e checkboxes Direct / Blocked / Proxy. A consulta é interrompida assim que a janela é fechada ou o checkbox é desmarcado.

Para que esta janela exiba registros, o Xray deve ter o **journal de acesso** ativado com um caminho para o arquivo (não none) – veja abaixo. Se o access-log estiver desativado ou o arquivo inacessível, a janela ficará vazia («No Record...»).

16.3. Nível e configuração de logging do Xray

Os parâmetros de logging do próprio Xray são definidos na página «**Configurações do Xray**» no bloco «**Log**» (Log) com o aviso:

«Os logs podem deixar o servidor mais lento. Ative apenas os tipos de logs necessários quando precisar!»

Campo	Tradução	Valor padrão	Descrição
Nível de logs (logLevel)	Log Level	warning	Nível de detalhe do log de erros do Xray. Valores válidos: debug , info , notice , warning , error . Dica: «Nível de log para os logs de erros, indicando quais informações devem ser registradas.»
Logs de acesso (accessLog)	Access Log	none	Caminho para o arquivo de journal de acesso. O valor especial none desativa os logs de acesso. Dica: «Caminho para o arquivo de log de acesso. O valor especial "none" desativa os logs de acesso.»
Logs de erros (errorLog)	Error Log	vazio (caminho padrão)	Caminho para o arquivo de logs de erros; none os desativa. Dica: «Caminho para o arquivo de logs de erros. O valor especial "none" desativa os logs de erros.»
Logs DNS (dnsLog)	DNS Log	false (desat.)	Ativar logging de requisições DNS. Dica: «Ativar logs de requisições DNS.»

Campo	Tradução	Valor padrão	Descrição
Mascaramento de endereço (<code>maskAddress</code>)	Mask Address	vazio (desat.)	Quando ativado, o endereço IP real é substituído automaticamente por um endereço mascarado nos logs. Dica: «Quando ativado, o endereço IP real é substituído por um endereço mascarado nos logs.»

Como por padrão «**Logs de acesso**» = **none** , a janela «Logs do Xray» (seção 16.2) fica inicialmente vazia. Para que ela funcione, defina aqui um caminho para o access-log e reinicie o Xray.

Note que um access-log vazio afeta apenas essa janela. A lista de clientes online no «Dashboard» e o limite de quantidade de IPs no formulário do cliente **não dependem** do access-log – o painel determina os clientes online e conta seus endereços IP via a API de online-stats do núcleo Xray (estatísticas de conexões). Em versões antigas do núcleo onde essa API não existe, o painel retorna automaticamente ao método anterior (leitura do access-log), e nesse caso o caminho para o access-log aqui ainda é necessário para o limite de IP.

Limite de quantidade de IPs e fail2ban. A própria restrição de quantidade de IPs por cliente (campo «IP Limit» no formulário do cliente e no cadastro em massa) só é aplicada no servidor se o **fail2ban** estiver instalado – é ele que bane os endereços que excedem o limite. O painel verifica a presença do fail2ban (`GET /panel/api/server/fail2banStatus`); se não estiver instalado, o campo «IP Limit» fica indisponível com uma dica explicativa (no Windows – com uma mensagem separada), e os limites definidos anteriormente nesses servidores são automaticamente zerados, pois não tinham efeito de qualquer forma. O bloqueio do fail2ban se aplica tanto ao TCP quanto ao UDP. A partir da 3.5.0, uma conexão «morta» (o cliente desapareceu sem o fechamento correto do TCP) é registrada **uma única vez**, e não a cada varredura de 10 segundos: a linha `[LIMIT_IP]` e o ciclo de desativação só se repetem em uma reconexão real, por isso o contador do fail2ban não é inflado e não é mais necessário aumentar o `maxretry` (o formato do log e o failregex não mudaram). Em servidores comuns, o fail2ban agora é instalado automaticamente durante a instalação e atualização do painel (veja a seção 16.5).

Exemplo: bloco `Log` que fará a janela «Logs do Xray» começar a exibir registros. Na configuração JSON do Xray, fica assim:

```
{
  "log": {
    "loglevel": "warning",
    "access": "./access.log",
    "error": "",
    "dnsLog": false,
    "maskAddress": ""
  }
}
```

O principal é substituir `"access": "none"` por um caminho para o arquivo (por exemplo, `"./access.log"`). Após salvar, reinicie o Xray e a tabela na janela «Logs do Xray» será preenchida com registros.

16.4. Gerenciamento do Xray: parada e reinicialização

O estado do Xray é gerenciado a partir do cartão do Xray no «Dashboard». O estado atual é exibido com um dos seguintes valores: **Iniciado** (Running), **Parado** (Stopped), **Desconhecido** (Unknown), **Erro** (Error). Em caso de erro, aparece a dica «Erro ao iniciar o Xray».

Botão	Tradução	Endpoint	Ação
Parar	Stop	POST /panel/api/server/stopXrayService	Para o processo do Xray. Em caso de sucesso – notificação de aviso «Xray service has been stopped».
Reiniciar	Restart	POST /panel/api/server/restartXrayService	Reinicia (ou inicia) o Xray com a configuração atual. Em caso de sucesso – notificação «Xray service has been restarted successfully».

Após qualquer uma das operações, o painel envia o novo estado via WebSocket, portanto o status no «Dashboard» é atualizado sem recarregar a página. Se a operação falhar, o estado do Xray se torna «Erro» e o texto do erro aparece na notificação.

Além da reinicialização manual, o painel verifica automaticamente se o Xray precisa ser reiniciado (tarefa em segundo plano a cada 30 s) e se o processo caiu (verificação a cada segundo) – veja a seção 16.6.

Monitor de saúde do túnel (reinicialização automática do Xray)

Na versão 3.4.1 foi introduzido o **monitor de saúde do túnel** opcional. Se ativado, o painel verifica periodicamente a acessibilidade de uma URL definida e, após várias verificações consecutivas com falha, reinicia automaticamente o núcleo do Xray – isso ajuda a recuperar um túnel que parou de passar tráfego. Por padrão, o monitor está **desativado** e é configurado **apenas por variáveis de ambiente do serviço** (não há configurações dele na interface web – isso foi uma decisão dos autores).

O monitor é ativado pela variável `XUI_TUNNEL_HEALTH_MONITOR=true`. A variável `XUI_TUNNEL_HEALTH_PROXY` deve apontar para um inbound local do Xray (por exemplo `socks5://127.0.0.1:1080`) – assim a sonda passa pelo próprio Xray e verifica exatamente o túnel; sem ela, verifica-se apenas a conectividade do host, e a reinicialização não resolverá um problema de conexão de rede do servidor. As demais variáveis definem os parâmetros da verificação:

Variável	Finalidade	Padrão
<code>XUI_TUNNEL_HEALTH_MONITOR</code>	Ativar o monitor (lig/desl)	<code>false</code>
<code>XUI_TUNNEL_HEALTH_PROXY</code>	Proxy pelo qual a sonda passa (aponte para um inbound local do Xray)	vazio
<code>XUI_TUNNEL_HEALTH_URL</code>	URL a ser verificada	<code>https://www.cloudflare.com/cdn-cgi/trace</code>
<code>XUI_TUNNEL_HEALTH_INTERVAL</code>	Intervalo entre verificações	<code>30s</code>
<code>XUI_TUNNEL_HEALTH_TIMEOUT</code>	Timeout de uma única verificação	<code>10s</code>

Variável	Finalidade	Padrão
XUI_TUNNEL_HEALTH_FAILURES	Número de falhas consecutivas antes de reiniciar	3
XUI_TUNNEL_HEALTH_COOLDOWN	Pausa mínima entre reinicializações	5m

A reinicialização do Xray interrompe as conexões de todos os clientes conectados, portanto faz sentido manter o intervalo e o limite de falhas suficientemente grandes para que uma falha casual em uma única sonda não provoque reinicializações desnecessárias.

16.5. Reinicialização e atualização do painel

Reinicialização do painel

Na página «**Configurações do painel**» há a ação «**Reiniciar painel**» (`Restart Panel`, `POST /panel/api/setting/restartPanel`). Após confirmação, o painel é reiniciado **em 3 segundos**.

Mensagens:

- Confirmação: «Tem certeza de que deseja reiniciar o painel? Confirme e a reinicialização ocorrerá em 3 segundos. Se o painel ficar inacessível, verifique o log do servidor.»
- Sucesso: «Painel reiniciado com sucesso».

Tecnicamente, no Linux a reinicialização é feita enviando o sinal `SIGHUP` ao processo do painel (ou via hook registrado). No Windows, o envio de `SIGHUP` não é suportado.

Atualização automática do painel (Update Panel)

No «Dashboard» está disponível a função «**Atualizar painel**» (`Update Panel`) – atualização do 3X-UI para o último release diretamente pela interface web.

Antes da atualização, o painel compara as versões (`GET /panel/api/server/getPanelUpdateInfo`), consultando o último release do 3x-ui no GitHub:

Campo	Tradução
Versão atual do painel	Current panel version
Última versão do painel	Latest panel version
Painel atualizado / «Atualizado»	Panel is up to date / Up to date – exibido se não há nova versão

Iniciar a atualização – `POST /panel/api/server/updatePanel` . Diálogo de confirmação:

- «Você realmente quer atualizar o painel?»
- «Isso atualizará o 3X-UI para a versão #version# e reiniciará o serviço do painel.»

Após iniciar – mensagem pop-up «Atualização do painel iniciada» (`Panel update started`); em caso de falha na verificação de versão – «Falha ao verificar atualização do painel» (`Panel update check failed`).

O que acontece no servidor: a atualização automática é suportada **apenas no Linux** (em outros sistemas operacionais será retornado o erro «panel web update is supported only on Linux installations»). O painel baixa o script oficial `update.sh` do GitHub (raw.githubusercontent.com/MHSanaei/3x-ui/main/update.sh) e o executa em um processo separado: de preferência via `systemd-run` em uma unidade separada (`x-ui-web-update-<timestamp>`), e na ausência do `systemd` – como um processo separado e desvinculado. Ao terminar, o script atualiza os componentes e reinicia o serviço do painel. O `bash` é necessário para a execução.

Se durante a atualização o script gerar um novo caminho base aleatório da interface web (Web Base Path), o serviço `x-ui` é reiniciado automaticamente para que o novo caminho funcione imediatamente. (Sem a reinicialização, o servidor continuaria servindo o caminho antigo, enquanto a interface exibiria o novo, e o novo endereço ficaria inacessível até uma reinicialização manual.)

Canal Dev (rolling), a partir da 3.4.2. O botão de versão do painel no «Dashboard» agora sempre abre a janela de atualização (antes, em uma build estável atual, ele levava à página de releases do GitHub), e o interruptor «Canal Dev» (`devChannelEnable`) é exibido sempre – mesmo em uma build estável. Ao ativá-lo, na próxima atualização é possível passar para o canal dev «rolante» (builds a cada commit na `main`). Ao ativar, é exibido um aviso: «Os builds dev acompanham cada commit na main e não são releases estáveis – não há rollback automático». Sem uma ação explícita do usuário, nada é atualizado.

Canal de atualização Dev (builds rolling por commit)

Além da atualização comum para o release estável, existe o «**Canal de desenvolvimento**» (`Dev`) opcional. O interruptor aparece na janela de atualização do painel **apenas em builds dev** (builds de CI, compilados a partir de um commit específico); em releases estáveis ele não está visível. Quando ativado, o painel será atualizado para o build rolling `dev-latest` , que acompanha cada commit da branch `main` e não é um release estável – é exibido um aviso de que os builds dev são instáveis e não há rollback automático. No modo dev, a janela exibe «Commit atual» / «Último commit» em vez de números de versão. A função está disponível apenas no Linux com `systemd`.

Em builds dev, o painel exibe sua versão como `dev+<short-commit>` em vez do número de versão estável enganoso – no badge da barra lateral, no cartão do «Dashboard», na janela de atualização, no relatório de status do bot do Telegram e na saída do comando `x-ui -v` . Em releases estáveis, a exibição da versão não muda.

Nos nós, o painel do mesmo 3x-ui é atualizado centralmente via `POST /panel/api/nodes/updatePanel` – veja a seção sobre nós.

Instalação automática do fail2ban

Para que o limite de quantidade de IPs por cliente (seção 16.3) funcione imediatamente, durante a instalação e atualização do painel em um servidor comum o `fail2ban` agora é instalado e configurado automaticamente (antes, isso acontecia apenas na imagem Docker). O comportamento é controlado pela variável de ambiente `XUI_ENABLE_FAIL2BAN` : a configuração é executada se a variável não estiver definida ou for igual a `true` . A execução manual está disponível com o comando `x-ui setup-fail2ban` . A falha na configuração do `fail2ban` não interrompe a instalação ou atualização do painel.

Instalação e atualização em hosts somente IPv6

Os scripts `install.sh` e `update.sh` agora funcionam corretamente em servidores com apenas IPv6: o download do release, do script `x-ui.sh` e dos arquivos de serviço não usa mais IPv4 forçado (`curl -4`), mas sim o protocolo disponível. Portanto, o painel pode ser instalado e atualizado também em um host sem endereço IPv4.

Correções de scripts na 3.5.0

- **Família RHEL** (Rocky/Alma/RHEL/Oracle): a instalação local do PostgreSQL pelo menu agora funciona (autenticação por senha em vez de ident), e o fail2ban para o limite de IP é instalado do **EPEL**.
- **Arch/Manjaro**: a instalação e a atualização não executam mais o upgrade completo do sistema `pacman -Syu` — apenas a atualização da base de pacotes.
- **ARM de 32 bits**: o binário do Xray baixado é nomeado `xray-linux-arm32` — exatamente como o painel o executa (antes, a atualização colocava o arquivo com outro nome e o painel continuava rodando o núcleo antigo).
- **Certificado de IP**: o IPv4 público autodetectado é exibido para confirmação (Enter — aceitar) antes da emissão; em caso de recusa — entrada manual validada.
- **Escolha da porta ACME**: Enter (aceitar a porta 80 padrão) não exibe mais o falso «Your input is invalid».

Substituição da porta do painel pela variável XUI_PORT

A porta de escuta da interface web do painel pode ser substituída pela variável de ambiente `XUI_PORT` — ela age apenas durante a execução do processo atual e **não altera** o valor salvo de `webPort` no banco de dados. Valores válidos são de `1` a `65535` ; valores vazios, incorretos ou fora do intervalo são ignorados (usa-se `webPort`) com um aviso no log. Isso é conveniente na implantação, principalmente no Docker: ao usar rede bridge, a porta publicada do contêiner deve corresponder a `XUI_PORT` — por exemplo, `XUI_PORT=8080` e `ports: "8080:8080"` .

Atualização e alternância de versão do Xray-core

Nessa mesma área do «Dashboard», é possível gerenciar a versão do Xray-core separadamente do painel.

- **Atualizações do Xray** (`Xray Updates`) / **Seleção de versão** (`Version`) — lista suspensa de versões disponíveis. Dicas: «Selecione a versão desejada» e aviso «Importante: versões antigas podem não suportar as configurações atuais».
- Instalação/troca de versão — `POST /panel/api/server/installXray/{version}` . Diálogo: «Trocar versão do Xray» / «Você realmente quer trocar a versão do Xray?». Em caso de sucesso — «Xray atualizado com sucesso».

Exemplo: trocar a versão do Xray-core via API. A versão é indicada pela tag de release do XTLS/Xray-core (com prefixo `v`). Por exemplo, trocar para `v1.8.24` :

```
curl -s -b cookies.txt -X POST \
  https://panel.example.com:2053/panel/api/server/installXray/v1.8.24
```

(`cookies.txt` — arquivo com cookie do exemplo na seção 16.1.) Após a instalação, o Xray será reiniciado automaticamente com a versão selecionada.

No servidor, ao trocar de versão, o Xray é primeiro parado, o arquivo da versão desejada é baixado do GitHub (XTLS/Xray-core), o binário é descompactado e substituído, e então o Xray é reiniciado com verificação dos tamanhos do arquivo/binário.

16.6. Tarefas periódicas (cron)

O painel registra uma série de tarefas em segundo plano ao iniciar. Seus agendamentos são fixos (não configuráveis na interface, exceto o agendamento do relatório do Telegram e a sincronização LDAP). Abaixo estão as tarefas relacionadas à operação.

Tarefa	Agendamento	Finalidade
Verificação de funcionamento do Xray	a cada 1 s	Controle de que o processo Xray está em execução
Verificação de necessidade de reinicialização do Xray	a cada 30 s	Reinicialização se a configuração foi marcada como alterada
Coleta de tráfego do Xray	a cada 5 s (início 5 s após o arranque)	Contabilização de tráfego de inbound/clientes
Verificação de IPs de clientes	a cada 10 s	Controle do limite de IP pelo log
Heartbeat e sincronização de tráfego dos nós	a cada 5 s	Troca com os nós
Limpeza de logs	diariamente (@daily)	Limpa os logs de limite de IP e o access-log persistente, rotacionando o log atual para <code>*.prev.log</code> . A partir da 3.5.0, a limpeza diária também abrange o error-log do Xray (antes — apenas o access)
Limitação do crescimento dos logs do Xray	a cada 10 minutos (@every 10m)	Novo na 3.5.0. Trunca o access-log e o error-log do Xray quando qualquer um deles excede 64 MiB ; logs desativados (<code>none</code> /vazio) não são tocados
Reset de tráfego por período	<code>@hourly</code> , <code>@daily</code> , <code>@weekly</code> , <code>@monthly</code>	Zera os contadores de tráfego dos inbound (e seus clientes) que têm o período de reset automático correspondente configurado
Relatório do Telegram	definido nas configurações do bot (padrão <code>@daily</code>)	Envio de relatório aos administradores; quando a opção está ativada — com o backup do BD anexado (seção 16.1)
Reset do armazenamento hash do Telegram	a cada 2 m	Apenas quando o bot está ativado
Controle de carga de CPU para o Telegram	a cada 10 s	Apenas se o limiar de CPU > 0 estiver definido

Adicionalmente:

- **Reset periódico de tráfego** é acionado apenas para os inbound com o modo de reset automático correspondente selecionado (a cada hora/dia/semana/mês). A tarefa zera o tráfego do próprio inbound e de todos os seus clientes.
- **Verificação de expiração e esgotamento.** A desativação de clientes por expiração do prazo e esgotamento do limite de tráfego é realizada no âmbito da contabilização de tráfego: clientes com `expiry_time` expirado ou volume esgotado são marcados e desativados; quando necessário, é calculado o próximo prazo (para limites cíclicos e modo «contagem a partir do primeiro uso»). No «Dashboard» e nas listas, isso é refletido pelos status «Expirado»/«Esgotado»/«Expira em breve».
- **Backup automático no Telegram** — é um efeito colateral da tarefa de relatório; não há um agendamento cron separado apenas para o backup. Portanto, a frequência do backup automático é igual à frequência do relatório do bot.

16.7. Menu de console e CLI (`x-ui`)

No servidor, o painel é gerenciado pelo comando `x-ui`. Sem argumentos, abre o menu interativo «3X-UI Panel Management Script»; com argumento, executa um subcomando específico. Itens do menu relacionados à operação:

Nº no menu	Item	Ação
1	Install	Instalação do painel (baixa e executa <code>install.sh</code>)
2	Update	Atualização de todos os componentes do <code>x-ui</code> para a última versão sem perda de dados; depois — reinicialização automática
3	Update to Dev Channel (latest commit)	Atualização para o build rolling <code>dev-latest</code> (último commit da branch <code>main</code>) com confirmação (veja 16.5)
4	Update Menu	Atualização apenas do script de menu <code>x-ui</code>
5	Legacy Version	Instalação de uma versão específica (antiga) do painel pelo número informado (por exemplo, <code>2.4.0</code>)
6	Uninstall	Remoção completa do painel e do Xray (veja 16.8)
7	Reset Username & Password	Redefinição do login/senha do administrador
8	Reset Web Base Path	Redefinição do caminho base da interface web
9	Reset Settings	Redefinição das configurações para os valores padrão
10	Change Port	Alteração da porta do painel
11	View Current Settings	Visualização das configurações atuais
12–14	Start / Stop / Restart	Iniciar, parar, reiniciar o serviço do painel
15	Restart Xray	Reiniciar apenas o Xray
16	Check Status	Status atual do serviço
17	Logs Management	Visualização e limpeza de logs (veja abaixo)

Nº no menu	Item	Ação
18-19	Enable / Disable Autostart	Ativar/desativar inicialização automática do serviço ao iniciar o sistema operacional
27	Update Geo Files	Atualização dos arquivos geo (GeoIP/GeoSite)
25	PostgreSQL Management	Gerenciamento do PostgreSQL

A numeração dos itens do menu mudou na versão 3.4.1: com a adição do item 3 «Update to Dev Channel», todos os itens subsequentes foram deslocados em uma unidade. O total passou a ser 28 itens, com seleção no intervalo [0-28] .

Gerenciamento de logs no CLI (item 16)

O submenu «Logs Management» agora é aberto pelo item **17** (antes – 16):

- **Debug Log** – visualização em streaming do journal do serviço: `journalctl -u x-ui -e --no-pager -f -p debug` (no Alpine – `grep` em `/var/log/messages`).
- **Clear All logs** – limpeza do journal do sistema: `journalctl --rotate + journalctl --vacuum-time=1s`, após o qual o serviço é reiniciado. (Indisponível no Alpine.)

Subcomandos diretos do `x-ui`

Todos os subcomandos disponíveis:

Comando	Descrição
<code>x-ui</code>	Abrir o menu administrativo
<code>x-ui start</code>	Iniciar o painel
<code>x-ui stop</code>	Parar o painel
<code>x-ui restart</code>	Reiniciar o painel
<code>x-ui restart-xray</code>	Reiniciar o Xray
<code>x-ui status</code>	Status atual
<code>x-ui settings</code>	Exibir as configurações atuais
<code>x-ui enable</code>	Ativar inicialização automática ao iniciar o sistema operacional
<code>x-ui disable</code>	Desativar inicialização automática
<code>x-ui log</code>	Visualizar logs
<code>x-ui banlog</code>	Visualizar logs de banimentos do Fail2ban
<code>x-ui setup-fail2ban</code>	Instalar e configurar o fail2ban para limite de IP (veja 16.5)
<code>x-ui update</code>	Atualizar o painel

| `x-ui update-dev` | Atualizar o painel para o canal de desenvolvimento (build rolling `dev-latest`) || `x-ui update-all-geofiles` | Atualizar todos os arquivos geo (com reinicialização subsequente) || `x-ui migrateDB [file]` | Conversão do banco `.db` para `.dump` (SQLite) || `x-ui legacy` | Instalar uma versão legada || `x-ui install` | Instalar o painel || `x-ui uninstall` | Remover o painel |

O comando `x-ui update` baixa e executa o `update.sh` oficial (o mesmo que a atualização web da seção 16.5), solicitando confirmação: «This function will update all x-ui components to the latest version, and the data will not be lost.» Ao terminar, o painel é reiniciado automaticamente.

Flags `-webCert` / `-webCertKey` no subcomando `setting`. Os caminhos para o certificado e a chave privada da interface web do painel podem ser definidos diretamente no subcomando `x-ui setting -webCert <caminho> -webCertKey <caminho>` — informar qualquer uma dessas flags salva o caminho correspondente (assim como o subcomando separado `cert`), e o painel muda imediatamente para HTTPS.

Obtenção do token de API via CLI

O comando de obtenção do token de API via CLI (item de menu/comando `x-ui`) não exibe um token emitido anteriormente. Os tokens de API são armazenados apenas como hashes, portanto o token existente não pode ser obtido em texto simples. Se tokens já estiverem configurados, o comando informa a quantidade deles, sugere gerenciá-los no painel (**Settings** → **API Tokens**, veja a seção sobre tokens de API) e imediatamente gera um **novo token de reserva** com o nome no formato `cli-fallback-<timestamp>`, exibindo-o para que o CLI permaneça útil sem precisar acessar a interface.

16.8. Remoção do painel

A remoção é feita pelo CLI — item de menu **5 (Uninstall)** ou comando `x-ui uninstall`. Antes da remoção, é solicitada uma confirmação (padrão «não»): «Are you sure you want to uninstall the panel? xray will also be uninstalled!».

Após confirmação, o script:

1. Para o serviço e desativa sua inicialização automática (`systemctl stop/disable x-ui`, ou no Alpine — `rc-service / rc-update`), remove o arquivo de unidade do serviço e recarrega a configuração do `systemd`.
2. Remove os diretórios de dados e da aplicação (`/etc/x-ui/`, diretório de instalação) e o arquivo de variáveis de ambiente do serviço (`/etc/default/x-ui`, `/etc/conf.d/x-ui` ou `/etc/sysconfig/x-ui` — dependendo da distribuição).
3. Remove o próprio script `x-ui` e exibe a mensagem «Uninstalled Successfully.», bem como o comando para reinstalação.

Se o painel estava usando PostgreSQL (no arquivo de variáveis de ambiente `XUI_DB_TYPE=postgres`), após a remoção dos arquivos do painel o script pergunta adicionalmente se o servidor PostgreSQL também deve ser removido junto com todos os seus bancos de dados: «Also purge PostgreSQL and delete all of its data?». A solicitação requer confirmação explícita (padrão — recusa) e é acompanhada de um aviso: a remoção afetará

TODOS os bancos de dados PostgreSQL na máquina, incluindo os pertencentes a outras aplicações, e é irreversível. Ao recusar, o PostgreSQL e seus dados permanecem intactos.

A remoção é irreversível: junto com o painel, são removidos o Xray e todos os dados (incluindo o banco de dados). Se os dados puderem ser necessários, faça a exportação do banco com antecedência (seção 16.1).

16.9. Comando `x-ui migrateDB`

A partir da versão 3.3.0, o script de gerenciamento `x-ui.sh` recebeu o subcomando `migrateDB` — um wrapper em torno do binário interno `x-ui` (`x-ui migrate-db`) para converter o banco de dados do painel SQLite entre dois formatos: o binário `.db` e o dump de texto portátil `.dump` (texto SQL simples).

O que o comando faz

O comando opera em duas direções, e a direção é determinada **automaticamente** pelo arquivo de entrada:

Direção	Como é chamado	O que acontece
<code>.db → .dump</code>	dump (exportação)	o banco SQLite binário é exportado para um arquivo SQL de texto
<code>.dump → .db</code>	restore (restauração)	a partir do arquivo SQL de texto, um banco SQLite binário é reconstruído

Por baixo do capô, o script chama o binário do painel:

- exportação: `x-ui migrate-db --src <entrada> --dump <saída>`
- restauração: `x-ui migrate-db --restore <entrada> --out <saída>`

Sintaxe de chamada

```
x-ui migrateDB [file.db|file.dump] [output]
```

- **[file.db|file.dump]** — arquivo de entrada (primeiro argumento). Se não for especificado, é usado o banco padrão instalado do painel: `/etc/x-ui/x-ui.db`.
- **[output]** — caminho para o arquivo de saída (segundo argumento). Opcional: se ausente, o nome é escolhido automaticamente junto ao arquivo de entrada (veja abaixo).

Exemplos:

```
x-ui migrateDB                               # exportar /etc/x-ui/x-ui.db -> /etc/x-ui/x-
ui.dump
x-ui migrateDB /etc/x-ui/x-ui.db backup.dump
x-ui migrateDB backup.dump restored.db       # reconstruir .db a partir do dump
```

Como a direção é determinada

O script verifica a extensão do arquivo de entrada:

- *.db, *.sqlite, *.sqlite3 → modo **dump** (exportação para texto);
- *.dump, *.sql → modo **restore** (reconstrução do banco).

Se a extensão não for reconhecida, o script lê os primeiros 16 bytes do arquivo: a assinatura SQLite format 3 indica um banco binário (modo dump), caso contrário o arquivo é considerado um dump (modo restore).

Nome do arquivo de saída, se o segundo argumento não for informado:

- na exportação — o mesmo nome do arquivo de entrada, com extensão `.dump`;
- na restauração — o mesmo nome com extensão `.db`.

Verificações de proteção e comportamento

- **Existência do binário.** Se o binário `x-ui` não for encontrado ou não for executável — é exibido o erro «x-ui binary not found ... Is the panel installed?».
- **Suporte à função na build.** O script verifica se o binário suporta `migrate-db --dump/--restore` (via `x-ui migrate-db -h`). Se não suportar — é sugerido atualizar primeiro o painel com `x-ui update`.
- **Existência do arquivo de entrada.** Se o arquivo de entrada não existir, é exibido um erro e a linha com a sintaxe de chamada.
- **Substituição da saída.** Se o arquivo de saída já existir, é solicitada uma confirmação (padrão «não»); sem confirmação, a operação é cancelada. Na restauração, o arquivo de saída antigo é excluído previamente.
- **Proteção do banco «ativo».** Na restauração para o banco padrão `/etc/x-ui/x-ui.db` enquanto o painel está em execução, a operação é recusada com a exigência de primeiro parar o painel (`x-ui stop`) ou escolher outro caminho de saída. Isso evita a sobrescrita do banco ativo de um serviço em funcionamento.
- Em caso de falha na reconstrução do banco, o arquivo de saída incompleto é excluído.

Para que serve

- **Backup.** O `.dump` de texto é legível por humanos, conveniente para armazenar em sistemas de controle de versão e para visualização diferencial do conteúdo do banco.
- **Migração.** O dump é portátil entre máquinas e resistente às diferenças nas versões do formato do arquivo SQLite — em um novo servidor, um `.db` funcional é reconstruído a partir dele.
- **Diagnóstico.** A partir do `.dump`, é possível inspecionar visualmente a estrutura e os dados do painel sem ter ferramentas SQLite disponíveis.

Modo interativo

Além da chamada direta, a conversão está disponível no menu interativo. No submenu PostgreSQL (`x-ui` → seção de gerenciamento do PostgreSQL) há o item **9. Convert SQLite .db <--> .dump**: ele pergunta o caminho do arquivo de entrada (padrão `/etc/x-ui/x-ui.db`) e do arquivo de saída (pode ser deixado vazio para nomeação automática), e a direção, como no modo CLI, é determinada automaticamente.

O documento foi preparado com base no código-fonte do 3X-UI. Se algum item da interface na sua versão for diferente – o comportamento do painel e as dicas na própria interface têm prioridade.